Python Handbook — Vol 1

the language — types, control flow, strings, collections, functions, modules, errors and files, OOP, iterators and generators (Ch 1–11)

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Getting Started with Python**
   - 1.1 Who this book is for, and how it is organised
   - 1.2 What Python is
   - 1.3 Where Python is used
   - 1.4 Installing Python and creating a project
   - 1.5 Two ways to run code: the REPL and scripts
   - 1.6 IDEs and editors
   - 1.7 Your first program
   - 1.8 How CPython runs your code
   - 1.9 Basic syntax
   - 1.10 Comments and docstrings
   - 1.11 PEP 8 and PEP 20 — writing Pythonic code
   - Chapter 1 TL;DR
2. **Chapter 2 — Variables, Data Types and Operators**
   - 2.1 Variables
   - 2.2 Identity, equality and mutability
   - 2.3 Dynamic vs strong typing
   - 2.4 Built-in data types
   - 2.5 Numbers
   - 2.6 Booleans and truthiness
   - 2.7 Type conversion
   - 2.8 Operators
   - Chapter 2 TL;DR
3. **Chapter 3 — Control Flow: Decisions and Loops**
   - 3.1 Conditional statements — overview
   - 3.2 `if`, `elif`, `else`
   - 3.3 Conditional expressions (the "ternary")
   - 3.4 `match` / `case` — structural pattern matching
   - 3.5 Loops — overview and the iteration protocol
   - 3.6 The `for` loop
   - 3.7 The `while` loop
   - 3.8 Loop control: `break`, `continue`, `pass`
   - 3.9 `for-else` and `while-else`
   - 3.10 Common loop idioms
   - Chapter 3 TL;DR
4. **Chapter 4 — Strings**
   - 4.1 What a string is — creating, indexing, looping
   - 4.2 Immutability — and how to "update" a string
   - 4.3 String operators
   - 4.4 Slicing
   - 4.5 Escape sequences and raw strings
   - 4.6 String formatting: f-strings, the format-spec mini-language, `str.format`, `%`
   - 4.7 String methods
   - 4.8 Joining strings efficiently
   - 4.9 Unicode vs bytes, encodings and UTF-8
   - 4.10 Regular expressions — a practical introduction
   - Chapter TL;DR
5. **Chapter 5 — Lists, Tuples and Arrays**
   - 5.1 The `array` module — typed, compact arrays
   - 5.2 Lists — the everyday sequence
   - 5.3 List internals — a dynamic array — and the big-O table
   - 5.4 Adding and removing items
   - 5.5 List methods (complete table) and slicing in depth
   - 5.6 List comprehensions
   - 5.7 Sorting
   - 5.8 Copying — aliasing, shallow vs deep, and the nested-list trap
   - 5.9 Tuples — immutable sequences and records
   - 5.10 Unpacking tuples (and any iterable)
   - 5.11 Joining tuples
   - 5.12 Tuple methods
   - 5.13 `namedtuple` and `typing.NamedTuple` — tuples with field names
   - 5.14 Choosing: `list` vs `tuple` vs `array.array` vs NumPy
   - Chapter TL;DR
6. **Chapter 6 — Dictionaries, Sets and the `collections` Toolbox**
   - 6.1 Dictionaries — key → value
   - 6.2 Hashing — how dicts and sets find things in O(1)
   - 6.3 Nested dictionaries
   - 6.4 Dictionary methods and view objects
   - 6.5 Dict and set comprehensions
   - 6.6 Sets — unique, unordered collections
   - 6.7 Set operations
   - 6.8 Set methods (complete table) and `frozenset`
   - 6.9 The `collections` module
   - 6.10 `heapq` — priority queues on a plain list
   - 6.11 `bisect` — binary search on sorted lists
   - 6.12 When to use what — the decision table
   - Chapter TL;DR
7. **Chapter 7 — Functions**
   - 7.1 Defining and calling functions
   - 7.2 Function arguments — every kind
   - 7.3 Scope and namespaces — LEGB, `global`, `nonlocal`
   - 7.4 Closures
   - 7.5 First-class and higher-order functions, `lambda`, `map` / `filter`, `key=`
   - 7.6 Decorators and `functools`
   - 7.7 Recursion
   - 7.8 Docstrings and annotations

- Chapter TL;DR

8. **Chapter 8 — Modules, Packages and Environments**
   - 8.1 Modules and `import`
   - 8.2 How `import` really works
   - 8.3 Packages, `__init__.py`, namespace packages, `__all__`
   - 8.4 Absolute vs relative imports
   - 8.5 `if __name__ == "__main__":` and `python -m`
   - 8.6 Common import errors and circular imports
   - 8.7 `pip` and virtual environments
   - 8.8 `requirements.txt` vs `pyproject.toml`; `uv` and Poetry
   - 8.9 Project layout and publishing basics
   - Chapter TL;DR
9. **Chapter 9 — Errors, Exceptions and Files**
   - 9.1 Exceptions and the hierarchy
   - 9.2 `try` / `except` / `else` / `finally`
   - 9.3 Raising, re-raising and custom exceptions
   - 9.4 Exception chaining and notes
   - 9.5 EAFP vs LBYL
   - 9.6 `ExceptionGroup` and `except*` (Python 3.11+)
   - 9.7 `assert`
   - 9.8 Context managers and `with`
   - 9.9 File I/O — modes, text vs binary, encodings
   - 9.10 `pathlib`, `shutil`, `tempfile`
   - 9.11 `csv`, `json`, `pickle`
   - Chapter TL;DR
10. **Chapter 10 — Object-Oriented Programming in Python**
    - 10.1 Classes and instances, `__init__` and `self`
    - 10.2 The data model: dunder methods
    - 10.3 Operator overloading
    - 10.4 Inheritance and `super()`
    - 10.5 Multiple inheritance and the MRO (C3 linearization)
    - 10.6 Mixins
    - 10.7 Abstract base classes, duck typing and Protocols
    - 10.8 Composition vs inheritance
    - 10.9 Encapsulation conventions: `_single` and `__double` underscores
    - 10.10 dataclasses
    - 10.11 Enums
    - 10.12 `__slots__`
    - 10.13 Pythonic design patterns (brief)
    - 10.14 Chapter self-check
    - Chapter TL;DR
11. **Chapter 11 — Iterators, Generators and Functional Tools**
    - 11.1 Iterable vs iterator: the iteration protocol
    - 11.2 Generators and `yield`
    - 11.3 `yield from`, `send()` and `close()` (brief)
    - 11.4 Lazy evaluation and memory
    - 11.5 `itertools` — the iterator toolkit
    - 11.6 Comprehensions — best practices
    - 11.7 `map` / `filter` / `reduce`, `any` / `all`, `zip(strict=True)` — the functional toolbox
    - 11.8 Chapter self-check
    - Chapter TL;DR

Chapter 1 — Getting Started with Python

From "never run Python" to "I understand what happens when I press Enter": install Python, run code two ways, see how the CPython interpreter (the standard program that reads and runs Python code) turns text into executed instructions, and learn the layout rules (indentation, lines, quotes, comments) every later chapter relies on.

#	Topic	You will be able to…
1.1	Who this book is for	pick your path through the chapters
1.2	What Python is, history, design traits	explain Python in one minute
1.3	Where Python is used	name real products built with it
1.4	Installing Python, virtual environments	set up a clean project on any OS
1.5–1.6	REPL vs scripts, IDEs	choose the right tool
1.7	First program	write and understand `print("Hello, World!")`
1.8	How CPython runs code	describe source → bytecode → virtual machine
1.9	Basic syntax	identifiers, indentation, continuation, quotes, `input()`, `;`
1.10	Comments and docstrings	document code the way tools expect
1.11	PEP 8 and PEP 20	write professional-looking Python

All code lives in `code/` and was run with CPython 3.14: `python code/<file>.py`.

1.1 Who this book is for, and how it is organised

What it is

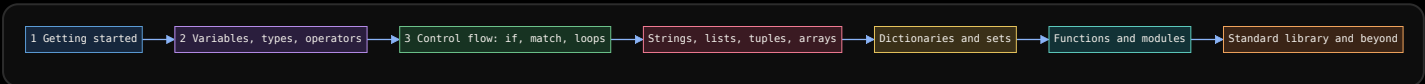
One self-contained path through core Python, from first program to intermediate topics.

Who should read it

Reader	What you get
Complete beginners (students, career changers)	A first language from zero; every term defined on first use
Developers from Java, C#, JS, Go…	The "Python way" plus traps that differ from your language
Data analysts / scientists	A language base before NumPy, pandas, scikit-learn
Teachers and trainers	Ready-made explanations, examples and self-checks
Hobbyists and automators	Enough Python to script your own tasks

Prerequisites: a computer (Windows, macOS, Linux), ability to install software, willingness to type code yourself. Only arithmetic-level maths.

How the book is organised



Each chapter builds on the previous one. Topics follow one rhythm: *what it is → how it works → code with real output → trade-offs → common mistakes → interview questions → self-check → TL;DR* ("too long; didn't read" summary).

1.2 What Python is

What it is

A **high-level, general-purpose programming language**. *High-level*: you say *what* to do without managing memory addresses or CPU registers. *General-purpose*: the same language builds websites, trains neural networks and renames files.

Python is **multi-paradigm**: procedural (a list of steps), object-oriented (an *object* bundles data with the functions acting on it; a *class* is the blueprint for objects), or functional (composing small functions). You don't have to pick one.

A short history

Year	Event
Dec 1989	Guido van Rossum starts Python at CWI (Dutch national maths/CS institute) as a successor to the ABC teaching language. Named after *Monty Python*, not the snake.
1991	First public release, 0.9.0.
2000	Python 2.0: list comprehensions and a garbage collector (automatic memory clean-up) that reclaims reference cycles (objects pointing at each other in a loop).
2008	Python 3.0: deliberate backward-incompatible clean-up (`print` became a function; text became Unicode by default).
2018–19	Guido steps down as "Benevolent Dictator For Life" (July 2018); since 2019 an elected five-person **Steering Council** governs.
2020	Python 2 end of life (1 January 2020). Only Python 3 is maintained.
2020 → now	One feature release per year, every October (since 3.9; PEP 602). 3.10 added `match`; 3.11 made CPython markedly faster; 3.13 added a new interactive shell and experimental free-threading (threads running Python truly in parallel); 3.14 is current.

Language changes are proposed in **PEPs** (*Python Enhancement Proposals*), numbered design documents — e.g. PEP 8 (style), PEP 20 (philosophy).

Why it matters — the design traits

Trait	In practice	Caveat
Readable syntax	Indentation shows structure; English keywords (`and`, `not`, `in`).	Whitespace errors are syntax errors (code can't run at all).
Dynamically typed	No declared variable types; each *value* carries its type, checked at run time.	Type mistakes surface at run time unless you add type hints and a checker such as mypy (reports mismatches without running code).
Strongly typed	No silent mixing: `"3" + 4` is an error, not `"34"` or `7`.	You convert explicitly (Chapter 2).
Automatic memory management	Reference counting (counting names pointing at each object) plus a cycle collector; no `free()`.	Less control over memory layout than C.
"Interpreted"	You run source directly; internally CPython compiles to bytecode first (§1.8).	Pure-Python loops are slower than C/Java; heavy numeric work goes to C libraries.
Extensible	Hot spots (where most time is spent) can be written in C, C++ or Rust and imported like modules (this is how NumPy is fast).	Adds build complexity.
Embeddable	A C/C++ program can host the interpreter as a scripting language (Blender, GIMP, games).	—
"Batteries included"	Large standard library (files, JSON, HTTP, dates, maths, testing, SQLite) plus 500 000+ third-party packages on PyPI (Python Package Index).	Third-party quality varies.

Common mistakes

- Believing "interpreted" means "read one line, run it". The whole file is compiled first; a syntax error on line 200 stops line 1 from running (§1.8).
- Confusing *dynamic* with *weak* typing. Python is dynamic **and** strong.
- Learning from Python 2 material (`print "hi"` is obsolete).

In the interview

> **30-second answer:** "Python is a high-level, dynamically but strongly typed, garbage-collected, multi-paradigm language. The reference implementation, CPython, compiles source to bytecode and runs it on a stack-based virtual machine. It trades raw speed for developer productivity and pushes heavy computation into C extensions like NumPy."

- *Compiled or interpreted?* → Both: compiled to bytecode, then interpreted by a VM. "Interpreted" describes the user experience.
- *Why is Python slow, and how do people cope?* → Every operation goes through dynamic type checks in the VM. Teams vectorise with NumPy (one operation over a whole array, in C), write C/Rust extensions, use PyPy (a JIT — *just-in-time* — implementation that compiles hot code to machine code while running), or rely on CPython's speed-ups since 3.11.
- *What is a PEP?* → A design document proposing a change or standard.

Self-check

1. Is `"3" + 4` allowed? → No, `TypeError` (wrong type for an operation): strong typing.
2. Who governs Python? → The elected five-member Steering Council.
3. When did Python 2 support end? → 1 January 2020.

TL;DR

Readable, dynamic-but-strong, garbage-collected, batteries-included. Guido van Rossum (1989–91); Python 3 (2008) is the only live line; changes go through PEPs.

1.3 Where Python is used

What it is

Python wins wherever glue code (connecting other programs and libraries) is needed and a mature library exists.

Area	Key tools	Real-world examples
Web back-ends	Django (batteries-included: ORM — maps tables to classes — admin, auth), Flask (minimal), FastAPI (async, type-hint driven)	Instagram runs one of the largest Django deployments; Reddit was rewritten in Python in 2005; Pinterest and Mozilla use Django/Flask
Data analysis	pandas, NumPy, Jupyter notebooks (code cells + output + notes)	Finance, analytics, research; Jupyter is the default data-science notebook
Machine learning & AI	scikit-learn, PyTorch, TensorFlow/Keras, Hugging Face Transformers	PyTorch (Meta) and TensorFlow (Google) are Python-first; most LLM tooling ships Python SDKs first
Scientific computing	SciPy, NumPy, Matplotlib, Astropy	NumPy was used in the first black-hole image (Event Horizon Telescope) and LIGO's gravitational-wave detection
Automation & scripting	`pathlib`, `subprocess`, `shutil`, Ansible	Sysadmin scripts, CI pipelines (automatic build-and-test on every change); Ansible is written in Python
Networking	`socket`, `asyncio`, Twisted, `requests` / `httpx`	Network tooling, scrapers, async servers
Game development & scripting	Pygame, embedded Python	*EVE Online* (Stackless Python servers); *Civilization IV* game logic; Blender and Maya scripting APIs
Desktop GUIs (windows, buttons, menus)	Tkinter (bundled), PyQt/PySide, wxPython	Internal tools; the IDLE editor itself (Tkinter)
Education	IDLE, Thonny, turtle	Common first language (e.g. MIT 6.100L)
IoT & embedded (Internet of Things: small networked devices)	MicroPython, CircuitPython	Microcontrollers (tiny one-chip computers) such as Raspberry Pi Pico and ESP32, reading sensors and talking to networks

When not to use it

Situation	Better fit	Why
Hard real-time (guaranteed deadlines) or latency-critical (trading engines, game-engine inner loops)	C, C++, Rust	Predictable performance, no interpreter overhead
Browser front-end code	JavaScript / TypeScript	Browsers run JS natively (Python via WebAssembly is niche)
Native mobile apps	Swift, Kotlin	Platform SDKs and tooling
CPU-bound (limited by processor speed, not by waiting) multi-threaded code on standard CPython	Go, Rust, Java — or Python multiprocessing / free-threaded builds	The Global Interpreter Lock (GIL) lets only one thread run Python bytecode at a time in the default build

Self-check

1. Django vs Flask? → Django: full-featured with ORM/admin; Flask: minimal core.
2. Why is Python fast enough for ML? → Heavy maths runs in C/C++/CUDA (NVIDIA GPU toolkit) inside NumPy/PyTorch; Python orchestrates.

TL;DR

Python dominates data, ML, scripting and web back-ends because of its libraries; avoid it for hard real-time, browser front-ends and native mobile.

1.4 Installing Python and creating a project

What it is

To run Python you need an **interpreter**. The official one is **CPython** (written in C) from python.org. Others exist (PyPy, MicroPython, GraalPy); "Python" here means CPython.

How it works — install per OS

OS	Recommended	Check
Windows	python.org installer (tick **"Add python.exe to PATH"** — PATH is the folder list searched for commands) or Microsoft Store. Also installs the `py` launcher: `py -3.14`.	`py --version` or `python --version`
macOS	python.org installer or Homebrew: `brew install python`. Don't rely on an old system copy.	`python3 --version`
Linux	Package manager: `sudo apt install python3 python3-venv`. **Never remove** the system Python — the OS uses it.	`python3 --version`

On macOS/Linux the terminal command is usually `python3` (plain `python` may be missing). Inside an activated venv, `python` always means that environment's interpreter.

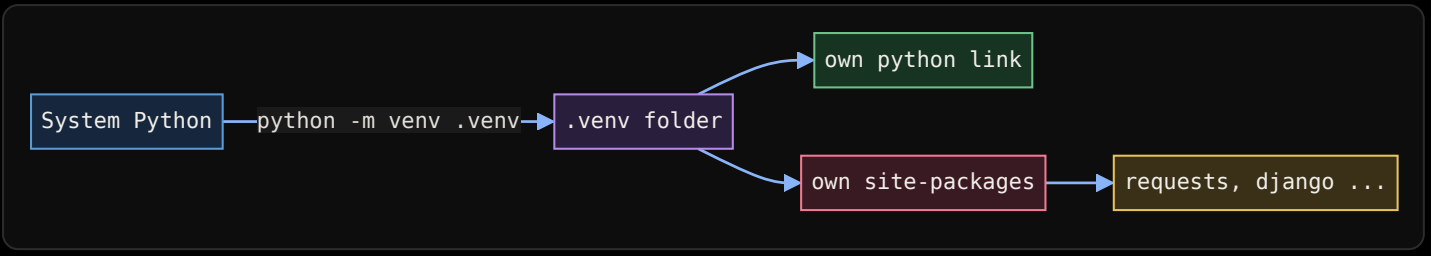
Virtual environments (intro)

What it is: a *virtual environment* ("venv") is a private folder with its own `python` link and its own `site-packages` folder for installed libraries — one toolbox per project.

The problem it solves: project A needs Django 4, project B Django 5; global installs can't satisfy both and can break OS tools. Modern Linux distributions and Homebrew refuse global `pip` installs (`error: externally-managed-environment`, PEP 668).

How it works:

1. Create: `python3 -m venv .venv` (Windows: `py -m venv .venv`).
2. Activate: `source .venv/bin/activate` (macOS/Linux) or `.venv\Scripts\activate` (Windows); prompt shows `(.venv)`.
3. Install: `python -m pip install requests` → goes into `.venv/…/site-packages` only.
4. Record: `python -m pip freeze > requirements.txt`; recreate with `python -m pip install -r requirements.txt`.
5. Leave: `deactivate`. Delete by deleting `.venv`.



`python -m pip` (not bare `pip`) guarantees you install into the interpreter you actually run. **uv** and **Poetry** manage venvs and dependencies for you on the same idea.

Common mistakes

- Not ticking "Add to PATH" on Windows → `'python' is not recognized`.
- `sudo pip install` globally → breaks OS tools.
- Committing `.venv/` to Git. Commit `requirements.txt` / `pyproject.toml`; add `.venv/` to `.gitignore`.
- `pip` from one Python, `python` from another → "installed but import fails". Use `python -m pip`.

In the interview

30-second answer: "One virtual environment per project with `python -m venv .venv`, install with `python -m pip`, and pin dependencies in `requirements.txt` or `pyproject.toml`. That isolates versions between projects and keeps the system interpreter clean."

- *Where do packages go?* → The venv's own `site-packages`; `sys.prefix` points into the venv (check with `python -c "import sys; print(sys.prefix)"`).
- *Does a venv copy the interpreter?* → It links to (or copies) the base binary and adds its own `site-packages`; the standard library is shared.

TL;DR

Install CPython from python.org or your OS, give every project its own `.venv`, install with `python -m pip`.

1.5 Two ways to run code: the REPL and scripts

What it is

- **REPL** (*Read–Eval–Print Loop*): `python3` with no arguments gives a `>>>` prompt; each line is read, evaluated, its result printed. Continuation lines show `...`.
- **Script**: a `.py` file run with `python3 hello.py`. Nothing is echoed — you must `print()`.

```
$ python3
>>> 2 + 3
5
>>> _ * 10          # _ holds the last printed result (REPL only)
50
>>> name = "Ada"
>>> name            # a bare expression is echoed in the REPL...
'Ada'
>>> exit()          # or Ctrl-D (macOS/Linux), Ctrl-Z then Enter (Windows)
```

Since 3.13 the default REPL has colour, multi-line editing, and `exit`/`quit` without parentheses. `help(len)` shows docs; `help()` opens a help browser.

Ways to run Python

Command	What it does	Use it for
`python3`	Start the REPL	Quick experiments
`python3 script.py`	Run a file	Real programs
`python3 -i script.py`	Run, then drop into the REPL with its variables	Debugging / exploring
`python3 -m module`	Run a module from the import path	`-m venv`, `-m pip`, `-m http.server`, `-m pytest`
`python3 -c "print(1+1)"`	Run a one-liner	Shell one-liners
Jupyter notebook	Code + output + text cells	Data exploration, teaching

On macOS/Linux a **shebang** line `#!/usr/bin/env python3` plus `chmod +x script.py` lets you run `./script.py`.

The `if __name__ == "__main__":` guard

Every module has `__name__`: `"__main__"` when **run directly**, the module's name when **imported**. Start-up code under the guard makes a file both a runnable script and an importable library — every script in this book does it.

Trade-offs

	REPL	Script
Feedback	Instant	After each run
Keeps your work	No (history only)	Yes, versionable
Auto-prints values	Yes	No — use `print()`
Best for	Trying one idea	Anything you'll run twice

Common mistakes

- Typing `python3 hello.py` *inside* `>>>` → `SyntaxError`; it's a shell command.
- Expecting a bare `2 + 3` in a script to print. Only the REPL echoes.
- Naming your file after a standard module (`random.py`) — `import random` imports *your* file.

TL;DR

REPL for trying, scripts for keeping (`_` = last REPL result; `-i` = run then inspect). Guard start-up code with `if __name__ == "__main__":`.

1.6 IDEs and editors

What it is

An **IDE** (*Integrated Development Environment*) combines an editor with running, debugging (pausing to inspect values), autocompletion and error checking.

Tool	Best for	Notes
VS Code + Python extension	Most people	Free; Pylance completion/type checking; debugger, Jupyter
PyCharm	Large projects	Deep refactoring and inspections; core free, advanced features paid (Pro)
Jupyter / JupyterLab	Data analysis, teaching	Cells, inline charts; not for large apps
IDLE	First hour	Ships with Python
Thonny	Absolute beginners	Visualises variables and step-by-step execution
Vim / Neovim / Emacs / Zed / Sublime	Keyboard-driven devs	Add a language server (Pyright, pylsp) for completion and go-to-definition

Pair any editor with a **formatter** (Black or Ruff format — rewrites layout to PEP 8), a **linter** (Ruff or Flake8 — flags bugs and style issues) and optionally a **type checker** (mypy or Pyright).

Common mistakes

- Editor using a different interpreter than your venv → imports "missing". Select the `.venv` interpreter.
- Hand-formatting code; let the formatter run on save.

TL;DR

VS Code or PyCharm for projects, Jupyter for data, IDLE/Thonny to start; always add a formatter and linter.

1.7 Your first program

What it is

```
# Print "Hello, World!" to the console
print("Hello, World!")
```

- `print` is a **built-in function** (always available). Calling it = name + parentheses containing **arguments**.
- `"Hello, World!"` is a **string** (`str`): characters between quotes.
- `print` writes to **standard output** (the console) followed by a newline. `# …` is a **comment**, ignored by Python.

Two keyword arguments (passed as `name=value`): `sep` (between arguments, default space) and `end` (after, default `"\n"`). From `code/first_program.py`:

```
print("Hello, World!")
print("Python", sys.version_info.major, sys.version_info.minor, sep=".", end="  <- running version\n")
```

Output:

```
Hello, World!
Python.3.14  <- running version
```

Common mistakes

- `Print("hi")` → `NameError` (unknown name): Python is case-sensitive.
- `print "hi"` → `SyntaxError` (Python 2 syntax).
- Curly quotes pasted from a word processor (`“hi”`) → `SyntaxError`.

Self-check

1. Print `a-b-c` from three arguments? → `print("a", "b", "c", sep="-")`.
2. Print without a trailing newline? → `print(x, end="")`.

TL;DR

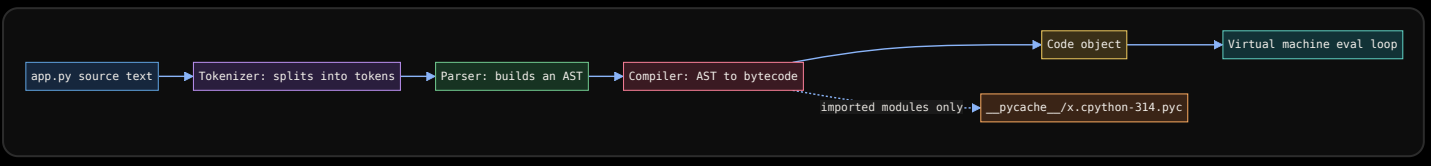
`print(...)` sends text to the console; quoted text is a `str`; everything is case-sensitive.

1.8 How CPython runs your code

What it is

CPython does **not** execute your text line by line. It first translates the *entire file* into **bytecode** — compact instructions for an imaginary stack-based computer (a *stack*: last value in is first out) — then its **virtual machine** (VM) executes them.

How it works



1. **Tokenize** — split into names, numbers, operators, `INDENT`/`DEDENT` markers. Comments are discarded here.
2. **Parse** — tokens become an **Abstract Syntax Tree (AST)** describing the program's structure. Since 3.9 CPython uses a PEG parser (*Parsing Expression Grammar*; PEP 617). Syntax errors are found here, for the whole file, before anything runs.
3. **Compile** — the AST becomes a **code object**: bytecode plus tables of constants and names.
4. **Cache** — for *imported* modules, the code object is saved as `.pyc` in `__pycache__/` so later imports skip steps 1–3 if the source is unchanged. The main script is not cached.
5. **Execute** — the VM's evaluation loop pushes/pops values on a stack and calls C code for the real work. Since 3.11 it *specialises* hot instructions for the types it actually sees (PEP 659) — one reason 3.11+ is noticeably faster than 3.10.

From `code/how_cpython_runs.py`:

```
SOURCE = 'price = 20\nqty = 3\ntotal = price * qty\nprint("total =", total)\n'
code = compile(SOURCE, "<demo>", "exec")       # steps 1-3
dis.dis(code)                                  # show the bytecode
exec(code, {})                                 # step 5
```

Output (CPython 3.14 — instruction names change between versions):

```
implementation: cpython 3.14.7
code object: code | constants: (20, 'total =', None) | names: ('price', 'qty', 'total', 'print')
--- bytecode ---
  1           LOAD_SMALL_INT          20
              STORE_NAME               0 (price)
  2           LOAD_SMALL_INT           3
              STORE_NAME               1 (qty)
  3           LOAD_NAME                0 (price)
              LOAD_NAME                1 (qty)
              BINARY_OP                5 (*)
              STORE_NAME               2 (total)
  4           LOAD_NAME                3 (print)
              PUSH_NULL
              LOAD_CONST               1 ('total =')
              LOAD_NAME                2 (total)
              CALL                     2
              ...
--- run ---
total = 60
SyntaxError at line 2 - so line 1 never ran
cached bytecode file: __pycache__/shop.cpython-314.pyc
cache tag for this interpreter: cpython-314
```

Line 3: push `price`, push `qty`, `BINARY_OP *` pops both and pushes the product, `STORE_NAME` pops it into `total` — a stack machine.

Other implementations

Implementation	Runs on	Notable for
CPython	C	The reference; what python.org ships
PyPy	RPython, JIT-compiled	Often several times faster on long-running pure-Python code
MicroPython	Microcontrollers	Tiny memory footprint
GraalPy, Jython	JVM	Calling Java code directly

CPython added an experimental JIT compiler (3.13, off by default) and an optional **free-threaded** build without the GIL (experimental in 3.13, officially supported but still optional in 3.14).

Trade-offs

Bytecode + VM is portable (same `.py` on any OS) and quick to start, at the cost of per-instruction overhead versus native machine code.

Common mistakes

- Thinking `.pyc` files protect source — they are trivially decompiled and version-specific.
- Committing `__pycache__/` to Git.

In the interview

> **30-second answer:** "CPython tokenizes and parses the whole file into an AST, compiles it to bytecode in a code object, caches bytecode for imported modules in `__pycache__`, and a stack-based virtual machine executes it. Since 3.11 it specialises instructions based on observed types."

- *What's in a `.pyc`?* → A header (magic number for the bytecode version, plus source timestamp/size or hash) followed by the serialised code object.
- *What is the GIL?* → A lock in default CPython allowing only one thread to execute bytecode at a time; simplifies memory management but limits CPU-bound threading.
- *How can you see bytecode?* → `dis.dis(func)`.
- *Why can a syntax error on line 50 stop line 1 running?* → The whole file is parsed and compiled before execution.

TL;DR

Source → tokens → AST → bytecode → VM. "Interpreted" really means "compiled to bytecode, then interpreted". Only imported modules are cached as `.pyc`.

1.9 Basic syntax

Identifiers

An **identifier** names a variable, function, class or module. Rules (breaking them is a `SyntaxError`):

1. Starts with a letter or `_`, then letters, digits, underscores; any length.
2. No spaces or symbols (`-`, `@`, `$`, `!`, `#`, `%`).
3. Case-sensitive: `total`, `Total`, `TOTAL` differ.
4. Not a **keyword**. Python 3.14 has 35 keywords (`if`, `for`, `class`, `True`, `None`, …) plus four **soft keywords** — `match`, `case`, `type`, `_` — special only in certain positions, so `match = 3` is legal.
5. Unicode letters are allowed (`café = 1`), but use ASCII in shared code.

From `code/first_program.py` (`str.isidentifier()` and `keyword.iskeyword()`):

```
'total'      valid identifier? True
'_private'   valid identifier? True
'user2'      valid identifier? True
'2user'      valid identifier? False
'user-name'  valid identifier? False
'class'      valid identifier? False
'café'       valid identifier? True
```

Lines and indentation

A **block** (body of an `if`, loop, function, class) is marked by *indentation*, not braces. A line ending in `:` opens it; all its lines share one indent; it ends when indentation returns to the outer level.

```
if temperature > 30:
    label = "hot"      # inside the if-block
else:
    label = "mild"     # inside the else-block
print(label)           # back at the outer level: always runs
```

- PEP 8: **4 spaces per level**, never tabs.
- Inconsistent tabs/spaces → `TabError`. Unexpected indent → `IndentationError: unexpected indent`; missing after `:` → `IndentationError: expected an indented block`.
- An empty block is illegal — use `pass` (Chapter 3).

Why it matters: visible layout *is* the structure. In brace languages misleading indentation hides bugs (Apple's 2014 "goto fail" SSL bug (SSL/TLS is the encryption behind `https`): a duplicated, indented line looked like part of an `if`). In Python that mismatch cannot exist.

Multi-line statements

Method	Example	Verdict
Implicit — inside `()`, `[]`, `{}`	`total = (1` ⏎ `+ 2` ⏎ `+ 3)`	Preferred by PEP 8
Explicit — `\` at line end	`total = 1 + \` ⏎ `2 + \` ⏎ `3`	Works, but a stray space after `\` breaks it

Both give `6` in `line_continuation_demo()`. PEP 8 suggests breaking *before* binary operators (`+ 2` starts the next line).

Quotation marks

Form	Use	Example
`'...'`	Strings containing `"`	`'It said "hi"'`
`"..."`	Strings containing `'`	`"It's sunny"`
`'''...'''` / `"""..."""`	Multi-line text; docstrings	`"""Line one⏎Line two"""`

Single and double quotes mean the same; be consistent (Black standardises on `"`). Output of `quotes_demo()`:

```
It said "hi" | It's sunny | ['Line one', 'Line two']
```

Blank lines

Ignored by the interpreter (in the REPL a blank line ends a multi-line block). PEP 8: **two** around top-level functions/classes, **one** between methods.

Reading user input

`input(prompt)` prints the prompt, waits for Enter, and returns **a `str` — always**, even for digits.

```
user_name = input("Please enter your name: ")
print("Hello, " + user_name + "!")
```

The runnable script stubs `input()`; a user typing `Ada` sees:

```
Please enter your name: Ada
Hello, Ada!
```

For maths, convert: `age = int(input("Age: "))` (Chapter 2).

Several statements on one line

```
import sys; x = "foo"; sys.stdout.write(x + "\n")      # prints: foo
```

Legal but discouraged by PEP 8 (harder to read, diff and debug). `sys.stdout.write` writes exactly the string given, with no automatic newline.

Common mistakes

- Missing colon: `if x > 3` → `SyntaxError: expected ':'`.
- Pasting tab-indented code into a space-indented file.
- `input() + 1` → `TypeError`.
- `user-name` means "user minus name".

In the interview

> **30-second answer:** "Python uses indentation to define blocks — four spaces per PEP 8 — so visual structure equals logical structure. Statements end at the newline unless you're inside brackets, the preferred way to continue a line. Identifiers are case-sensitive and can't be keywords."

- *What's a soft keyword?* → Reserved only in context, like `match`/`case`, so old code using `match` as a variable still works.
- *Why brackets over backslashes?* → A backslash breaks if followed by whitespace; brackets let editors auto-indent.

TL;DR

Names: letters/digits/underscore, no leading digit, not keywords, case-sensitive. Blocks = `:` + 4-space indent. Continue lines inside brackets. `input()` returns `str`. Avoid `;`.

1.10 Comments and docstrings

What it is

- A **comment** (`#` to end of line) is discarded by the tokenizer; it never reaches bytecode, so it costs nothing at run time.
- A **docstring** is a string literal that is the *first statement* of a module, function, class or method. It is kept in the object's `__doc__` attribute and read by `help()`, IDE tooltips and Sphinx.

Single-line and inline comments

```
# A comment explains WHY; the code already says WHAT.
return width * height  # inline comment: two spaces before the '#'
```

PEP 8: `# `(hash + space); at least two spaces before an inline `#`; keep comments current — a wrong comment is worse than none.

"Multi-line comments"

Python has **no** multi-line comment syntax.

1. **Consecutive `#` lines** — the correct way (editors toggle them with one shortcut).
2. **An unassigned triple-quoted string** — an *expression statement* whose value is created and discarded (CPython usually optimises it away). Not a comment: as the first statement in a function it silently becomes the docstring, and it must obey indentation. Reserve triple quotes for docstrings.

Docstrings

From `code/comments_docstrings.py`:

```
def area(width: float, height: float) -> float:
    """Return the area of a rectangle.

    Args:
        width: size along x, in metres.
        height: size along y, in metres.
    """
    return width * height
```

Output:

```
module doc first line: Chapter 1 - comments vs docstrings, PEP 20, and keywords.
function doc first line: Return the area of a rectangle.
area(3, 4) = 12
```

PEP 257: triple double quotes; a one-line imperative summary ("Return…", not "Returns…") ending in a period; for longer ones a blank line then details. Common detail layouts: Google style (above), NumPy style, reStructuredText.

Comments vs docstrings

	`#` comment	Docstring
Kept at run time?	No	Yes, in `__doc__`
Audience	Maintainers	Users (via `help()`, IDEs)
Content	*Why* a tricky line exists	*What* it does, arguments, return, errors
Position	Anywhere	First statement of module/class/function

Common mistakes

- Comments repeating code (`x += 1 # add one to x`).
- Leaving commented-out code — version control keeps history.
- A docstring after the first statement is just an ignored string.

In the interview

> **30-second answer:** "Comments start with `#`, are discarded by the tokenizer and explain *why*. Docstrings are the first string literal in a module, class or function, stored in `__doc__`, describing *what* for users. There's no block-comment syntax; use several `#` lines."

- *Is a triple-quoted string a comment?* → No: a string expression evaluated and discarded — unless first in a body, then a docstring.
- *Read a docstring at runtime?* → `obj.__doc__` or `help(obj)`.

TL;DR

`#` = note for maintainers, gone at run time. Docstring = first-statement string in `__doc__`. No real multi-line comments.

1.11 PEP 8 and PEP 20 — writing Pythonic code

What it is

- **PEP 8**: the official **style guide** — how code should *look*.
- **PEP 20**, *The Zen of Python* (Tim Peters): the **philosophy** in 19 aphorisms. `import this` prints it.

PEP 8 essentials

Rule	Example
4 spaces per indent, no tabs	`····return x`
Lines ≤ 79 chars (72 for comments/docstrings); many teams use 88–100	—
`snake_case` variables, functions, modules	`user_age`, `load_data()`
`CapWords` (CamelCase) classes	`UserProfile`
`UPPER_SNAKE_CASE` constants	`MAX_RETRIES = 3`
Leading `_` for internal names	`_cache`
Spaces around `=` in assignments and comparisons; none inside brackets	`x = f(a, b)` not `x=f( a,b )`
No spaces around `=` in keyword arguments	`print(x, end="")`
Imports at top, one per line, grouped: stdlib, third-party, local	`import os` ⏎ `import sys`
Two blank lines around top-level defs	—
Compare to `None` with `is`	`if x is None:`
Don't compare booleans to `True`	`if ready:`

PEP 8 itself says project consistency beats rigid rules. Automate: **Black**/**Ruff** format; **Ruff**/**Flake8** lint.

PEP 20 — the aphorisms that matter most

From `code/comments_docstrings.py`, which captures `import this`:

```
Zen of Python has 19 aphorisms, e.g.:
   Beautiful is better than ugly.
   Explicit is better than implicit.
   Simple is better than complex.
   Readability counts.
35 hard keywords, e.g. ['False', 'None', 'True', 'and', 'as', 'assert']
soft keywords (only special in some places): ['_', 'case', 'match', 'type']
```

Aphorism	In practice
Explicit is better than implicit	`from math import sqrt`, not `from math import *`
Readability counts	Descriptive names; one statement per line
Errors should never pass silently	No `except: pass`
There should be one obvious way to do it	`for item in items`, not index juggling
Flat is better than nested	Early returns over deep `if` pyramids (Chapter 3)
In the face of ambiguity, refuse the temptation to guess	Why `"3" + 4` raises

"**Pythonic**" means code that follows these idioms.

Common mistakes

- Arguing style in code review — let the formatter decide.
- `camelCase` functions (a Java habit).

In the interview

> **30-second answer:** "PEP 8 is the style guide — 4-space indents, snake_case functions, CapWords classes, 79-char lines, grouped imports — enforced with Ruff or Black. PEP 20 is the Zen of

> Python; I lean on 'explicit is better than implicit', 'readability counts' and 'errors should never pass silently'."

- *What does "Pythonic" mean?* → Using the language's idioms (iteration, `with` context managers — Chapter 9 — comprehensions) instead of translating another language line by line.

TL;DR

PEP 8 = how code looks (automate it). PEP 20 = why Python feels the way it does. Together they define "Pythonic".

Chapter 1 TL;DR

- Python: high-level, dynamically **and** strongly typed, garbage-collected; CPython is the reference; only Python 3 is supported.
- Used for web back-ends, data, ML, science, automation, game scripting, GUIs, education, IoT — because of its libraries.
- Install from python.org or your OS; one `.venv` per project; install with `python -m pip`.
- REPL for experiments, `.py` scripts for real work, `if __name__ == "__main__":` for start-up code.
- CPython: source → tokens → AST → bytecode (`.pyc` for imports) → stack VM; the whole file compiles before any line runs.
- Syntax: `:` + 4-space blocks; case-sensitive identifiers; continue inside brackets; `input()` returns `str`; avoid `;`.
- `#` comments vanish at compile time; docstrings live in `__doc__`.
- Follow PEP 8 (via a formatter) and the spirit of PEP 20.

Chapter 2 — Variables, Data Types and Operators

How Python stores data (names pointing at objects), which built-in types exist (available without `import`), how numbers really behave (why `0.1 + 0.2 != 0.3`), how to convert between types, and every operator family with its precedence. The model from §2.1 — *names are labels on objects* — is the most important idea in the book: most "weird" Python behaviour stops being weird once you have it.

#	Topic	Key ideas
2.1	Variables	creating, deleting, multiple assignment, naming, names vs objects
2.2	Identity, equality, mutability	`is` vs `==`, `id()`, mutable vs immutable, aliasing
2.3	Dynamic vs strong typing	why `x` can change type but `"3" + 4` fails
2.4	Built-in data types	text, numeric, sequence, mapping, set, boolean, binary, `None`
2.5	Numbers	unbounded `int`, `float` pitfalls, `Decimal`, `Fraction`, `complex`, `random`
2.6	Booleans and truthiness	falsy values, `bool` is an `int`
2.7	Type conversion	implicit widening, explicit constructors, failure cases
2.8	Operators	all families, walrus `:=`, chained comparisons, precedence

Code: `code/` — every output below is real, from CPython 3.14.

2.1 Variables

What it is

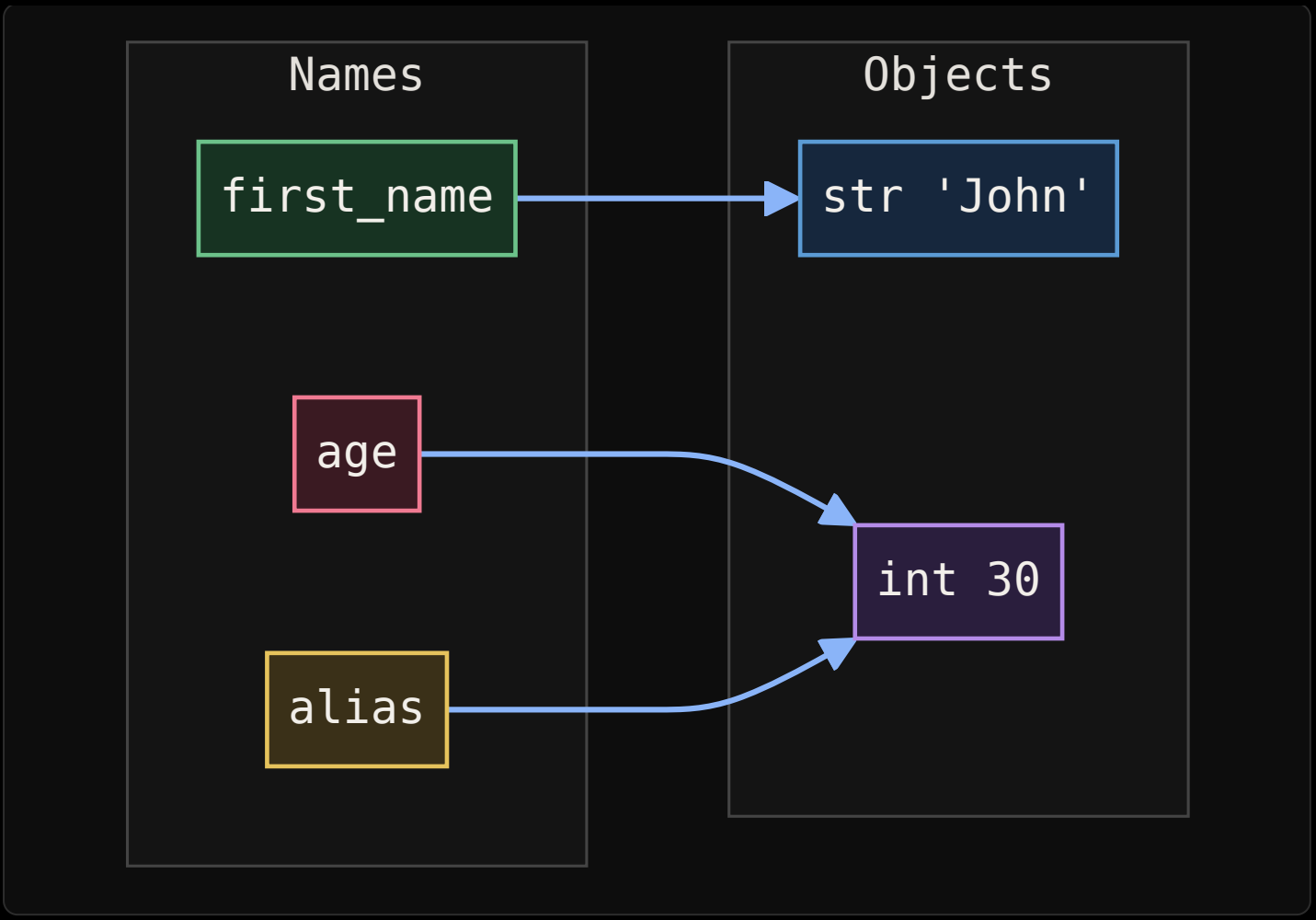
A **variable** is a **name that refers to an object**. An **object** is a chunk of memory holding a value, its **type**, and a reference count (how many names and containers point at it). The name stores no data — it is a label. Objects are parcels; names are sticky labels: several labels can go on one parcel, a label can move, peeling one off doesn't affect the parcel.

No declaration step: a variable exists from its first **assignment** with `=`.

How it works — creating and printing

```
first_name = "John"      # name first_name -> str object "John"
age = 30                 # name age        -> int object 30
height = 5.11            # name height     -> float object 5.11
print(first_name, age, height)          # John 30 5.11
```

`name = expression`: (1) evaluate the right side → an object; (2) bind the name to it in the current **namespace** (a dictionary-like table of names → objects).



After `alias = age`, both names point at the **same** object. Plain assignment never copies.

Multiple assignment

From `code/names_and_objects.py`:

```
first_name, age, height = "John", 30, 5.11   # tuple unpacking: one name per value
a = b = c = 0                                # chained: three names, ONE object
x, y = 1, 2
x, y = y, x                                  # swap: right side is built first, then unpacked
head, *rest = [10, 20, 30, 40]               # starred: rest collects the leftovers as a list
```

Output:

```
John 30 5.11
swap: 2 1 | chained: 0 0 0
head: 10 rest: [20, 30, 40]
```

Unpacking spreads a sequence's items (here a *tuple* — Python's fixed sequence written with commas) into names; counts must match: `x, y = 1, 2, 3` → `ValueError: too many values to unpack` (`ValueError` = right type, unusable content). Beware `a = b = []` — one shared list (§2.2).

Deleting variables — `del`

`del name` removes the **name**; the object is freed only when no references remain (CPython frees it immediately at refcount zero).

```
player_score = 120
alias = player_score
del player_score
print(player_score)     # NameError - the name is gone
print(alias)            # 120 - the object survives, alias still refers to it
```

Output (the script runs the snippet inside a function, then at module level with a name `player`):

```
UnboundLocalError (a NameError subclass, inside a function): cannot access local variable 'player_score' where it is not associated with a value
object still reachable via alias: 120
NameError at module level: name 'player' is not defined
```

A deleted name **always** raises `NameError` (in a function, its subclass `UnboundLocalError`). If an online playground shows nothing, it isn't displaying *stderr* (the error output stream) — run locally to see the traceback. `del` is more common for dict keys (`del d["k"]`) and list items (`del items[0]`).

Case sensitivity

`Age`, `age`, `AGE` are three unrelated names:

```
Age/age/AGE: 25 30 35
```

Naming rules (enforced) and conventions (PEP 8)

Rules — else `SyntaxError`	Conventions — else reviewers complain
Start with a letter or `_`	`snake_case` variables/functions: `user_age`
Then letters, digits, `_` only	`CapWords` classes: `UserProfile`
Not a keyword (`class`, `if`, `None` …)	`UPPER_SNAKE_CASE` constants: `MAX_USERS = 100`
Case-sensitive	Descriptive: `username` not `usrnm`; leading `_` = internal
	Don't shadow (reuse and hide) built-ins: `list`, `str`, `sum`, `id`, `input`

Valid: `username`, `_user_id`, `user2name`. Invalid: `2user`, `user-name` (minus), `class`. After `list = [1, 2]`, `list("abc")` fails: `TypeError: 'list' object is not callable`.

Common mistakes

- Assuming `b = a` copies a list. Use `a.copy()`, `list(a)` or `a[:]` (shallow copy — new outer list, shared items), or `copy.deepcopy(a)` for nested data.
- Shadowing built-ins (`list`, `dict`, `sum`, `max`, `id`, `type`).
- Using a variable before assigning it → `NameError`.

In the interview

> **30-second answer:** "Python variables are names bound to objects, not boxes holding values. Assignment never copies; it binds a name to the object on the right. `del` unbinds a name, and CPython frees the object when its reference count reaches zero."

- *`a = b = []`?* → One list, two names; appending via `a` shows via `b`.
- *How does `x, y = y, x` work?* → The right side builds the tuple `(y, x)` first, then unpacks it.
- *Does `del x` free memory?* → It removes one reference; the object is freed only if that was the last.

TL;DR

Variables are labels on objects; `=` binds, never copies; `del` removes a label; names are case-sensitive, snake_case, not keywords, don't shadow built-ins.

2.2 Identity, equality and mutability

What it is

Property	Question	How to see it	Can it change?
Identity	*Which* object?	`id(obj)` (CPython: memory address)	Never, for its lifetime
Type	*What kind?*	`type(obj)`	No (normally)
Value	*What* does it contain?	`==`, `print`	Only if the type is **mutable**

- `a == b` — **equality**: same value? (calls `a.__eq__(b)` — a *dunder* (double-underscore) method, the hook behind an operator)
- `a is b` — **identity**: the very same object? (compares `id`s; cannot be overridden)

How it works

```
list_a = [1, 2, 3]
list_b = [1, 2, 3]      # a second, separate list with equal contents
list_c = list_a         # another name for the first list
```

```
a == b: True | a is b: False | a is c: True
id(a) == id(c): True
```

Mutable types change in place, so every alias sees the change: `list`, `dict`, `set`, `bytearray`, most user classes. **Immutable**: `int`, `float`, `complex`, `bool`, `str`, `tuple`, `frozenset`, `bytes`, `None`. "Changing" an immutable creates a new object and rebinds:

```
n = 10
m = n
m += 1          # builds a new int 11 and rebinds m; n still refers to 10
```

```
team: ['ann', 'bob', 'cy'] | same object? True     # list mutated through an alias
n: 10 m: 11                                        # int "changed" = rebinding
str is immutable: 'str' object does not support item assignment
tuple holding a list: ([1, 99], [2])
```

The last line: the tuple always holds *the same two list objects*, but those lists are mutable.

Why it matters

- Passing a list to a function passes a reference: the function can mutate the caller's list.
- Only **hashable** objects (able to produce a fixed *hash* — an integer fingerprint used for fast lookup), typically immutable ones, can be dict keys or set members.

When to use `is`

Only for **singletons** (exactly one exists): `None`, `True`, `False`, `Ellipsis`, and your own sentinel objects (unique placeholders meaning e.g. "no value given"). `x == None` can be fooled by a custom `__eq__`.

Common mistakes

- `if x is 5:` / `if name is "bob":` — sometimes "works" because CPython caches small ints (−5 to 256) and some strings, an implementation detail. 3.8+ emits `SyntaxWarning: "is" with 'int' literal`. Use `==`.
- `grid = [[0] * 3] * 3` → **three references to one row**. Use `[[0] * 3 for _ in range(3)]`.
- Mutable default arguments (`def f(items=[])`) share one list across calls (functions chapter).

In the interview

> **30-second answer:** "`==` compares values via `__eq__`; `is` compares identity. I use `is` only for singletons like `None`. Mutable objects like lists and dicts change in place, so aliases see the change; immutable ones like ints, strings and tuples can't, so 'modifying' them rebinds the name to a new object."

- *Why `256 is 256` True but `1000 is 1000` maybe not?* → Small-int cache −5..256; implementation detail.
- *Can a tuple change?* → Its slots can't be reassigned, but mutable items inside can be mutated; such a tuple is unhashable.
- *Why must dict keys be immutable?* → Keys are stored by hash; a changing value would change the hash and lose the key.

Self-check

1. `[] == []` and `[] is []`? → `True`, `False`.
2. Mutable among `str`, `list`, `tuple`, `set`, `frozenset`? → `list`, `set`.

TL;DR

`==` = same value, `is` = same object (only for `None`/`True`/`False`/sentinels). Mutable objects change in place (aliases see it); immutable ones are replaced.

2.3 Dynamic vs strong typing

What it is

Axis	Python	Meaning
Static ↔ dynamic	**Dynamic**	Types belong to *objects*, checked at run time; a *name* can hold an `int` now, a `str` later.
Weak ↔ strong	**Strong**	Incompatible types raise errors instead of silently converting.

```
value = 42              # name -> int
value = "forty-two"     # same name -> str: fine (dynamic)
"Age: " + 30            # TypeError: can only concatenate str (not "int") to str  (strong)
```

```
int -> str
TypeError: can only concatenate str (not "int") to str
```

JavaScript is dynamic **and** weak (`"Age: " + 30` → `"Age: 30"`, `"5" - 2` → `3`); C is static but fairly weak (implicit int/pointer conversions); Java is static and strong. Python *does* widen numbers (`1 + 2.5` → `3.5`) along the numeric tower bool → int → float → complex (§2.7).

Type hints

`def area(w: float, h: float) -> float:` (function annotations, 3.0+; `typing` module 3.5) and `age: int = 30` (variable annotations, 3.6+). The interpreter **ignores** hints at run time; static checkers (mypy, Pyright — they read code without running it) and IDEs use them. Large codebases (Dropbox, where mypy was developed; Instagram) adopted hints to keep dynamic code safe at scale.

Trade-offs

Dynamic typing gives	Dynamic typing costs
Less boilerplate (repetitive set-up code); fast prototyping	Type errors appear at run time
Duck typing: "has `.read()`? treat it as a file"	IDEs know less without hints
Generic code (works for many types) for free	Slower execution (type checks on every operation)

Self-check

1. Why does `"3" * 2` work but `"3" + 2` fail? → `str * int` is defined (repetition → `"33"`); `str + int` isn't.
2. Does `x: int = "hi"` raise? → No; hints aren't enforced; mypy would flag it.

TL;DR

Dynamic = names can re-point to any type; strong = no silent cross-type coercion (except numeric widening). Hints + mypy add static checking.

2.4 Built-in data types

What it is

A **data type** defines what values an object holds and which operations make sense.

Category	Types	Mutable?	Literal / constructor
Text	`str`	No	`"hi"`, `'hi'`, `"""hi"""`
Numeric	`int`, `float`, `complex`	No	`10`, `10.5`, `2 + 3j`
Sequence	`list`, `tuple`, `range`	list only	`[1, 2]`, `(1, 2)`, `range(5)`
Mapping	`dict`	Yes	`{"name": "Alice"}`
Set	`set`, `frozenset`	set only	`{1, 2}`, `frozenset([1, 2])`
Boolean	`bool`	No	`True`, `False`
Binary	`bytes`, `bytearray`, `memoryview`	bytearray yes; bytes no; memoryview reflects its buffer	`b"abc"`, `bytearray(b"abc")`, `memoryview(b"abc")`
None	`NoneType`	No	`None`

From `code/data_types_tour.py`:

```
text       -> str        mutable=False
int        -> int        mutable=False
float      -> float      mutable=False
complex    -> complex    mutable=False
list       -> list       mutable=True
tuple      -> tuple      mutable=False
range      -> range      mutable=False
dict       -> dict       mutable=True
set        -> set        mutable=True
frozenset  -> frozenset  mutable=False
bool       -> bool       mutable=False
bytes      -> bytes      mutable=False
bytearray  -> bytearray  mutable=True
memoryview -> memoryview mutable=False
None       -> NoneType   mutable=False
```

`type(x)` gives the exact type; prefer `isinstance(x, int)` (accepts subclasses). `isinstance(True, int)` is `True` because `bool` subclasses `int`.

Each category in brief

Strings, lists, tuples, dicts and sets get full chapters later.

Text — `str`**:** immutable sequence of Unicode characters: `len(s)`, `s.upper()`, `s[0]`.

Numeric: `int` (counts, IDs), `float` (measurements), `complex` (engineering) — §2.5.

```
int + 5: 15 | float + 0.5: 11.0 | complex: (3+1j)
```

Sequences keep order and support indexing `s[i]` (from 0), slicing `s[i:j]`, `len()`, `in`:

- `list` — mutable everyday container.
- `tuple` — immutable; fixed records, coordinates, dict keys. Slightly smaller and faster to create than lists.
- `range` — immutable, **lazy**: stores only start/stop/step, so `range(10**12)` uses almost no memory. `range(5)` = 0..4 (stop exclusive).

```
range(5) as list: [0, 1, 2, 3, 4] | 3 in r: True
```

Mapping — `dict`**:** key → value pairs with average O(1) lookup; keys must be hashable; insertion-ordered since 3.7.

> **Key term — Big-O notation:** how running time grows with data size. **O(1)** (constant): same cost for 10 or 10 million items; **O(n)** (linear): grows in proportion to *n*. It describes the trend, not seconds.

```
user = {"name": "Alice", "age": 25}
user["age"] += 1                # modify
user["city"] = "Paris"          # add
user.get("email", "n/a")        # safe lookup with default; user["email"] would raise KeyError
```

```
dict: {'name': 'Alice', 'age': 26, 'city': 'Paris'} | get missing key: n/a
```

Sets: unordered, **unique** hashable items, fast membership. `set` is mutable; `frozenset` is immutable and hashable (usable as a key). Duplicates are silently ignored. `{}` is an empty **dict**; empty set is `set()`.

```
set size: 4 | sorted: ['basketball', 'soccer', 'tennis', 'volleyball']
frozenset usable as dict key: palette
```

Binary types hold raw bytes (0–255: files, network packets, images):

- `bytes` — immutable: `bytes([50, 100, 150])`, `b"abc"`.
- `bytearray` — mutable: `data[1] = 105`.
- `memoryview` — zero-copy *window* onto a buffer; slicing doesn't copy and later changes show through.

```
data = bytearray([50, 100, 150])
data[1] = 105
view = memoryview(data)[1:3]     # window on items 1..2, no copy
data[2] = 200                    # change the buffer after creating the view...
```

```
bytes: [50, 100, 150] | bytearray: [50, 105, 200] | view: [105, 200]
bytes range 0-255: bytes must be in range(0, 256)
text <-> bytes: b'caf\xc3\xa9' café
```

The view shows `200` — shared memory. Text ↔ bytes goes through an **encoding** (UTF-8 is standard): `"café".encode("utf-8")`, `b"...".decode("utf-8")`.

None — `NoneType`**:** the single object meaning "no value": missing optional argument, not-yet-computed result, return value of a function without `return`. Test with `is None`.

```
No value provided
function without return gives: None
```

Common mistakes

- `{}` for an empty set.
- List as dict key → `TypeError: unhashable type: 'list'`; use a tuple.
- `b"a" + "b"` → `TypeError`; encode/decode explicitly.
- `type(x) == int` rejects subclasses; prefer `isinstance`.

Real-world examples

- JSON from a web API maps onto `dict`, `list`, `str`, `int`/`float`, `bool`, `None` (`json.loads`).
- `bytes`/`memoryview` are used by `asyncio` streams and NumPy's buffer protocol to share memory without copies.
- `frozenset` keys appear in caches keyed by a set of options (see System Design Fundamentals).

In the interview

> **30-second answer:** "Built-ins are str; int/float/complex; list/tuple/range; dict; set/frozenset; bool; bytes/bytearray/memoryview; and None. The key split is mutable — list, dict, set, bytearray — versus immutable; only hashable, typically immutable, objects can be dict keys or set members."

- *List vs tuple?* → List mutable, for homogeneous collections that change; tuple immutable, for fixed records, hashable if its items are.
- *Why is `range(10**9)` cheap?* → Lazy; values computed on demand.
- *memoryview for?* → Slicing large binary buffers without copying.

TL;DR

Eight categories; memorise the mutable ones (list, dict, set, bytearray). `range` is lazy, `dict` ordered (3.7+), `{}` is a dict, `None` = "no value".

2.5 Numbers

What it is

Three built-in numeric types — `int`, `float`, `complex` — plus two exact standard-library types: `decimal.Decimal` and `fractions.Fraction`.

`int` — arbitrary precision

No fixed size, no overflow (C's 32/64-bit wraparound). From `code/numbers_demo.py`:

```
sum: 13
2**100 = 1267650600228229401496703205376 | bits: 101
factorial(25) = 15511210043330985984000000
readable literal 1_000_000 = 1000000 | hex/bin/oct: 255 5 15
int->str digit limit (CPython 3.11+): 4300
```

- **How:** CPython stores an int as an array of 30-bit "digits"; huge ints are slower but correct.
- **Literals:** `1_000_000`, `0xFF`, `0b101`, `0o17`.
- **Limit:** since 3.11, converting an int of more than 4 300 decimal digits to/from `str` raises `ValueError` by default (denial-of-service protection against huge numeric strings). Raise with `sys.set_int_max_str_digits()`.

`float` — IEEE-754 double precision

A 64-bit **binary** floating-point number (IEEE-754 standard): 1 sign, 11 exponent, 52 fraction bits → about **15–17 significant digits** (`sys.float_info.dig` is 15), range up to ~1.8 × 10^{308}.

The pitfall: most decimal fractions have no exact binary form (like 1/3 in decimal). `0.1` is stored as the nearest binary fraction:

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3? False | isclose? True
exact value stored for 0.1: 0.1000000000000000055511151231257827021181583404541015625
max float: 1.7976931348623157e+308 | epsilon: 2.220446049250313e-16
digits trusted (dig): 15
2**53 + 1 as float: 9007199254740992.0 (lost the +1)
inf/nan: True False
round(2.5), round(3.5): 2 4 | round(2.675, 2): 2.67
1/3 + 1/6 = 1/2 | 0.1 added ten times: 0.9999999999999999 | ten Fraction(1, 10) == 1: True
sum([0.1]*10) (3.12+ uses compensated summation): 1.0 | math.fsum: 1.0
```

1. **Never compare floats with `==`.** Use `math.isclose(a, b)` (default relative tolerance 1e-9) or `abs(a - b) < tol`.
2. **Errors accumulate** (`0.1` ten times ≠ `1.0`). `math.fsum` — and since 3.12 the built-in `sum()` for floats — use compensated summation (tracking and correcting each addition's rounding error).
3. **Integers above 2^{53} lose precision** as floats. Keep IDs and money-in-cents as `int`.
4. **`round()` is "round half to even"** (banker's rounding): `round(2.5) == 2`, `round(3.5) == 4`. `round(2.675, 2)` is `2.67` because 2.675 is stored as 2.67499999…
5. `float("inf")`, `float("-inf")`, `float("nan")` exist; **NaN ("Not a Number", e.g. `inf - inf`) is not equal to anything, including itself**.

The gap between neighbouring floats grows with magnitude:

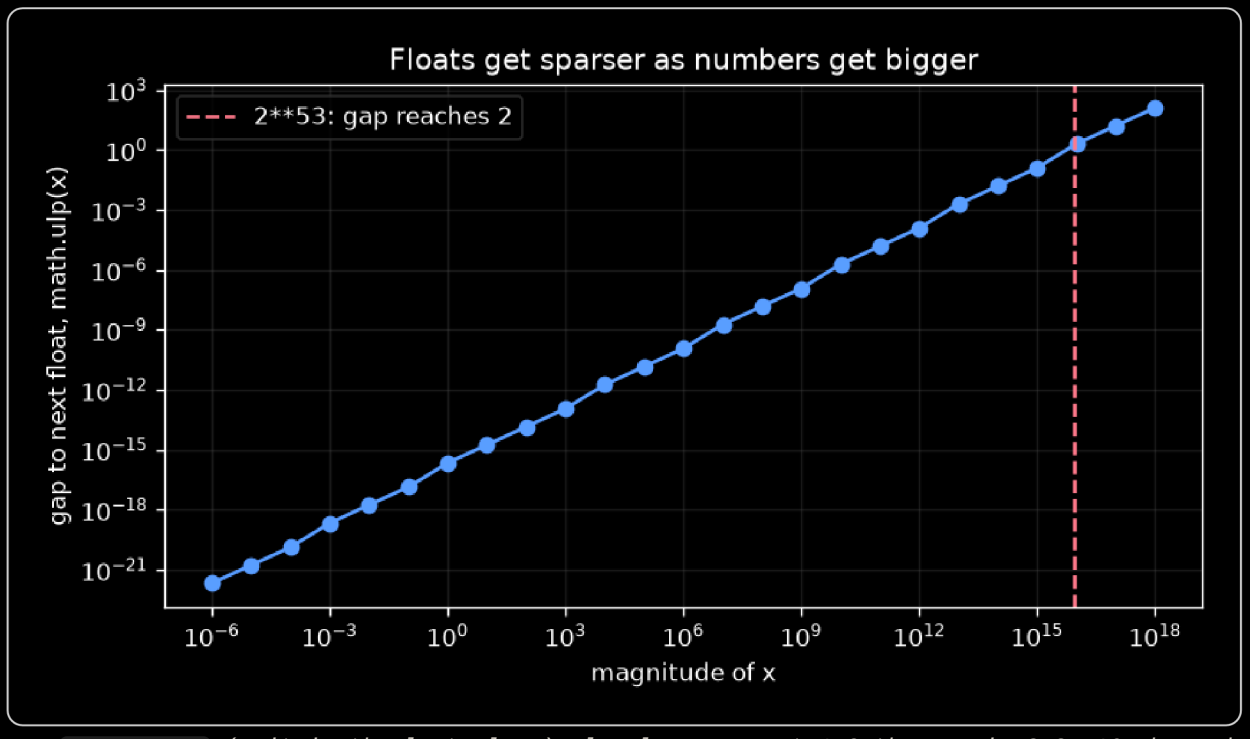


Figure — `math.ulp(x)` (unit in the last place), log-log axes: at 1.0 the gap is 2.2e-16; beyond $2^{53} \approx 9\times10^{15}$ consecutive floats are 2 apart, so odd integers can't be represented.

`Decimal` — exact base-10 arithmetic

Base-10 numbers with controllable precision (28 significant digits default) and rounding. Use for **money** and anything that must match pen-and-paper.

```
Decimal 0.10+0.20 = 0.30 | == 0.30? True
Decimal from float (wrong way): 0.1000000000000000055511151231257827021181583404541015625
round half up 2.675 -> 2.68
```

- Construct from a **string**: `Decimal("0.10")`; `Decimal(0.1)` copies the float's binary error.
- `quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)` rounds to cents like invoices.
- Much slower than float (fine for business logic).

`Fraction` — exact rationals

`Fraction(1, 3)` stores numerator/denominator as ints, so `1/3 + 1/6` is exactly `1/2`. Good for exact geometry, probability, music ratios; slow for heavy work as numerators/denominators grow.

`complex`

`a + bj` (engineers' `j`). `.real`, `.imag` (floats), `.conjugate()`, `abs(z)` = magnitude; `cmath` has complex `sqrt`, `exp`, `phase`.

```
z.real: 1.0 z.imag: 2.0 |z|: 5.0 conjugate: (1-2j)
```

Used in signal processing (FFT), electrical engineering (impedance), quantum simulators.

The division family

```
7/2 = 3.5 | 7//2 = 3 | -7//2 = -4 | -7%2 = 1 | divmod(17,5) = (3, 2)
int(-2.7) truncates: -2 | math.floor(-2.7): -3
```

- `/` is **true division**, always float (`4 / 2 == 2.0`).
- `//` is **floor division**, toward **negative infinity**: `-7 // 2 == -4` (C and Java truncate to −3).
- `%` satisfies `a == (a // b) * b + a % b`, so it takes the **divisor's sign**: `-7 % 2 == 1`, `7 % -3 == -2`.
- `divmod(a, b)` = `(a // b, a % b)`.
- `int(x)` **truncates** toward zero; `math.floor`/`math.ceil` round down/up.
- Division by zero raises `ZeroDivisionError` for ints and floats.

Random numbers

`random` is **pseudo-random** (Mersenne Twister formula): deterministic given a seed. `random.seed(42)` makes tests reproducible.

```
random.seed(42)
random.randint(1, 10)          # int in [1, 10], BOTH ends inclusive
random.random()                # float in [0.0, 1.0)
random.choice(["red", "green", "blue"])
```

```
randint(1,10): 2 | random(): 0.025 | choice: green
```

Also `uniform(a, b)`, `shuffle(list)`, `sample(pop, k)`, `gauss(mu, sigma)`. **Never** use `random` for passwords or tokens — use `secrets`, which draws from the OS's cryptographic generator.

Trade-offs

Type	Exact?	Speed	Use for
`int`	Yes	Fast (small), slower when huge	Counts, IDs, money in cents
`float`	No (≈15–17 digits)	Fastest, hardware-backed	Science, ML, measurements
`Decimal`	Yes in base 10 (to set precision)	Slow	Money, tax
`Fraction`	Yes	Slow, can blow up	Exact rational maths
`complex`	Float-based	Fast	Signals, AC circuits

Common mistakes

`total == 0.3` with floats · `Decimal(0.1)` · expecting `round(2.5) == 3` · floats for currency · assuming `-7 // 2 == -3` · `random` for security tokens.

Real-world examples

- Big ints power cryptography and RSA teaching code with no bignum library.
- Financial back-ends store amounts as `Decimal` (Django's `DecimalField`).
- NumPy/PyTorch use float32/float64 arrays; ML training even uses 16-bit floats, trading precision for speed.

In the interview

> **30-second answer:** "Python ints are arbitrary precision, so they never overflow. Floats are IEEE-754 doubles, so decimal fractions like 0.1 are approximations — compare with `math.isclose`, and use `Decimal` built from strings for money. `//` floors toward negative infinity and `%` takes the divisor's sign."

- *Why* `0.1 + 0.2 != 0.3`*?* → Neither has an exact binary form; the rounded sum differs from rounded 0.3 in the last bit.
- `round()` *rule?* → Half to even, avoiding upward bias in large sums.
- *Reproducible randomness?* → `random.seed(n)` or a `random.Random(n)` instance.

Self-check

1. `2 ** 64` overflow? → No: `18446744073709551616`.
2. `-9 // 4` and `-9 % 4`? → `-3` and `3`.
3. Can `random.randint(1, 10)` return 10? → Yes, both ends inclusive.
4. Why `float(2**53 + 1) == float(2**53)`? → Above 2^{53} float spacing is 2.

TL;DR

int = exact, unbounded; float = fast, approximate (`isclose`, never `==`); `Decimal` (from strings) for money; `Fraction` for exact ratios; `//` floors, `%` follows the divisor; seed `random` for repeatability, `secrets` for security.

2.6 Booleans and truthiness

What it is

`bool` has exactly `True` and `False` (keywords). Comparisons produce them; `if`/`while` consume them. `bool` is a **subclass of** `int`: `True == 1`, `True + True == 2`.

Truthiness — how any object becomes a boolean

In a boolean context (`if x:`, `while x:`, `and`/`or`/`not`, `bool(x)`) every object is truthy **except** these **falsy** values:

Falsy	Examples
The constants	`None`, `False`
Numeric zeros	`0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`
Empty collections	`""`, `()`, `[]`, `{}`, `set()`, `range(0)`, `b""`

Truthy includes `"0"`, `"False"`, `" "`, `[0]`, `-1`.

How it works: `bool(x)` calls `x.__bool__()`; failing that `x.__len__()` (non-zero = true); otherwise truthy. Your classes can hook in.

Idioms

```
if not cart:          # instead of: if len(cart) == 0
if name:              # instead of: if name != ""
if x is None:         # when 0 or "" are valid values, test for None explicitly
```

Common mistakes

- `if x == True:` → `if x:` (PEP 8).
- `if value:` when `0` is legitimate data. Use `if value is not None:`.
- `bool("False")` is `True`.
- `True == 1`, so `{1: "a", True: "b"}` has **one** key (`{1: 'b'}`).

In the interview

30-second answer: "Falsy values are None, False, numeric zeros and empty containers; everything else is truthy. `bool` subclasses `int`. I use truthiness for emptiness checks but `is None` when zero or empty string are valid values."

- *Custom truthiness?* → Implement `__bool__`, or `__len__` as fallback.

Self-check

1. `bool([0])`? → `True`. 2. `True + True + False`? → `2`.

TL;DR

Falsy = `None`, `False`, zeros, empties; everything else truthy. `bool` is an `int`.

2.7 Type conversion

What it is

Type conversion (casting) builds a new object of another type, **implicitly** (Python does it) or **explicitly** (you call a constructor — the type called like a function — such as `int()`).

Implicit conversion

Only **within the numeric tower**, widening: `bool → int → float → complex`. From `code/conversions.py`:

```
6 + 1.5 = 7.5 float
True + 2 = 3 | 2 + 1j = (2+1j)
```

Never between text and numbers (`"6" + 1.5` → `TypeError`).

Explicit conversion

```
str(10)                          -> '10'
int("20")                        -> 20
float(4)                         -> 4.0
int(True), int(False)            -> (1, 0)
int(2.8)                         -> 2
float("3.5")                     -> 3.5
bool(0)                          -> False
complex(2, 3)                    -> (2+3j)
list("abc")                      -> ['a', 'b', 'c']
tuple("abc")                     -> ('a', 'b', 'c')
sorted(set("abac"))              -> ['a', 'b', 'c']
dict([(1,'one'),(2,'two')])      -> {1: 'one', 2: 'two'}
bin(3), hex(255), oct(8)         -> ('0b11', '0xff', '0o10')
int("ff", 16), int("0b11", 0)    -> (255, 3)
int("  42\n"), int("1_000")      -> (42, 1000)
ord('A'), chr(66)                -> (65, 'B')
```

Function	Converts…	Notes
`int(x)`	number or numeric string → int	Truncates floats toward zero; strips whitespace; accepts `_`; optional `base` (2–36, or 0 = read the prefix)
`float(x)`	number or string → float	Accepts `"1e3"`, `"inf"`, `"nan"`
`str(x)`	anything → readable text	`repr(x)` gives the unambiguous developer form (`'hi\n'`)
`bool(x)`	anything → truthiness	§2.6
`complex(re, im)`	numbers or string → complex	`complex("2+3j")` works
`list()`, `tuple()`, `set()`	any iterable → that collection	`set` dedupes and loses order
`dict()`	key/value pairs or keyword args → dict	`dict(a=1)`
`bin()`, `hex()`, `oct()`	int → prefixed string	Reverse with `int(s, 0)`
`ord()`, `chr()`	character ↔ Unicode code point	

When conversion fails

```
int("3.5")   -> ValueError: invalid literal for int() with base 10: '3.5'
int("abc")   -> ValueError: invalid literal for int() with base 10: 'abc'
float("1,5") -> ValueError: could not convert string to float: '1,5'
int(None)    -> TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
bool("False") = True | bool("") = False
```

- `ValueError` = right type, bad content; `TypeError` = wrong type.
- `"3.5"` → int: `int(float("3.5"))` → 3 (decide whether truncation is intended).
- Wrap user-input conversions in `try/except ValueError` (exceptions chapter).
- Parse booleans explicitly: `text.strip().lower() in {"true", "1", "yes"}`.
- Locale formats fail: `float("1,5")`, `float("1,000")`.

Building strings from numbers

`"Age: " + str(age)` works, but **f-strings** (3.6+; literals prefixed with `f` that embed `{expressions}`) are clearer and format too: `f"Age: {age}"`, `f"pi ~ {3.14159:.2f}"` → `pi ~ 3.14`.

In the interview

> **30-second answer:** "Python only converts implicitly up the numeric tower — bool, int, float, complex. Everything else is explicit via constructors like `int()`, `float()`, `str()`, `list()`. Bad content raises `ValueError`, a wrong type raises `TypeError`. `int()` truncates toward zero."

- *`int("0x1A", 0)`? →* 26; base 0 reads the prefix.
- *`str` vs `repr`? →* str for users; repr for developers, unambiguous.
- *`int(-3.9)`? →* `-3`. *`"1010"` binary → 10? →* `int("1010", 2)`.

TL;DR

Implicit only for numbers (widening); explicit via constructors; `ValueError` for bad text, `TypeError` for wrong types; `input()` returns `str`; prefer f-strings.

2.8 Operators

What it is

An **operator** computes a result from **operands**. Each calls a dunder method — `a + b` calls `a.__add__(b)` — which is how `+` adds numbers, joins strings and concatenates lists. Examples from `code/operators_demo.py` with `a, b = 10, 3`.

Arithmetic

Op	Meaning	`10 ? 3`
`+` `-` `*`	add, subtract, multiply	13, 7, 30
`/`	true division (always float)	3.3333333333333335
`%`	modulus (remainder)	1
`**`	exponentiation	1000
`//`	floor division	3
`@`	matrix multiplication (NumPy; no built-in type implements it)	—

Also on other types: `"ab" * 3 == "ababab"`, `[1] + [2] == [1, 2]`.

Comparison

`==`, `!=`, `>`, `<`, `>=`, `<=` → `bool`.

```
compare: False True True False True False
```

Strings compare lexicographically by code point (`"Z" < "a"` is True). Unrelated types with `<` raise `TypeError` (`1 < "2"`); `==` just returns `False`.

Chained comparisons: `18 <= age < 65` = `18 <= age and age < 65`, with `age` evaluated once.

```
18 <= age < 65: True
```

Assignment and augmented assignment

`x += 3` ≈ `x = x + 3`; every arithmetic and bitwise operator has a form:

```
v += 3  -> 13
v -= 2  -> 11
v *= 2  -> 22
v /= 2  -> 11.0
v %= 4  -> 3.0
v **= 2 -> 9.0
v //= 2 -> 4.0
v &= 3  -> 0
v |= 8  -> 8
v ^= 6  -> 14
v <<= 1 -> 28
v >>= 2 -> 7
```

`/=` made it a float through `//=`; the script converts back with `int(v)` before the bitwise steps (they need ints). Bits: `4 & 3` → `100 & 011 = 000`; `0 | 8` → `1000`; `8 ^ 6` → `1110` (14); `14 << 1` → 28; `28 >> 2` → 7.

Subtlety: for **mutable** objects augmented operators mutate in place: `nums += [3]` extends the existing list (aliases see it); `nums = nums + [3]` builds a new one.

```
list += mutates in place: [1, 2, 3]
```

No `++`/`--`: `x++` is a `SyntaxError`, `++x` is `+(+x)`.

Logical: `and`, `or`, `not`

Expression	Result
`x and y`	`x` if `x` is falsy, else `y`
`x or y`	`x` if `x` is truthy, else `y`
`not x`	always a bool

1. **Short-circuit:** the right operand is evaluated only if needed — `0 and 1/0` returns `0`.
2. **They return an operand:** `"" or "guest"` → `"guest"` (the defaults idiom; use `x if x is not None else default` when `0`/`""` are valid).

```
True and False: False | True or False: True | not True: False
'' or 'guest' -> guest | 0 and 1/0 -> 0
```

Bitwise

Op	Name	`2 ? 3` (`0010`, `0011`)
`&`	AND — 1 where both are 1	2 (`0010`)
`|`	OR — 1 where either is 1	3 (`0011`)
`^`	XOR — 1 where bits differ	1 (`0001`)
`~`	NOT; `~x == -x - 1` (two's complement)	`~2 == -3`
`<<`	shift left n = ×2^n	`2 << 1 == 4`
`>>`	shift right n = floor ÷2^n	`2 >> 1 == 1`

```
bitwise: 2 3 1 -3 4 1 | 0010 & 0011 = 0010
```

Uses: permission flags (`READ | WRITE`, test with `perms & WRITE`), `enum.Flag`, network masks, hashing. On sets: `|` union, `&` intersection, `-` difference, `^` symmetric difference; on dicts (3.9+) `|` merges.

Membership: `in`, `not in`

List, tuple, string (substring), dict (**keys**), set.

```
membership: True True True True
```

`x in list` is O(n); `x in set` / `dict` average O(1). For repeated lookups convert to a set.

Identity: `is`, `is not`

Same object (§2.2).

```
identity: False True True
```

Walrus operator `:=` (assignment expression, 3.8+)

Assigns **and** returns the value, so you can bind inside a condition and avoid computing twice.

```
for line in log:
    if (m := re.match(r"error: (.*)", line)) is not None:
        print("walrus captured:", m.group(1))
if (count := len(data)) > 2:
    print(f"walrus: {count} items")
```

```
walrus captured: disk full
walrus: 3 items
```

Usually needs parentheses. Classic uses: `while (chunk := f.read(8192)):` and filters in comprehensions (compact `[expr for x in data]` collection builders, Chapter 5): `[y for x in data if (y := f(x)) > 0]`. Don't use it where a separate assignment reads better.

Operator precedence

Precedence decides which operator binds first; **associativity** orders equal-precedence ones (left-to-right except `**`, right-to-left). Highest to lowest (language reference):

#	Operators	Notes
1	`(…)`, `[…]`, `{…}`	parenthesised expressions and displays
2	`x[i]`, `x[i:j]`, `f(…)`, `x.attr`	subscription, slicing, call, attribute
3	`await x`	asyncio
4	`**`	right-associative: `2 ** 3 ** 2 == 512`
5	`+x`, `-x`, `~x`	unary; weaker than `**` on its left, so `-2 ** 2 == -4`
6	`*`, `@`, `/`, `//`, `%`	
7	`+`, `-`	
8	`<<`, `>>`	
9	`&`	
10	`^`	
11	`|`	
12	`in`, `not in`, `is`, `is not`, `<`, `<=`, `>`, `>=`, `!=`, `==`	**one level**, chainable
13	`not x`	
14	`and`	
15	`or`	
16	`x if c else y`	conditional expression
17	`lambda`	Chapter 7
18	`:=`	assignment expression

Beginners hit two: `&`, `^`, `|` bind **tighter** than comparisons (in pandas write `(df.a > 1) & (df.b < 2)`), and `and` binds tighter than `or`.

Worked example (`a, b, c = 10, 20, 30`):

```
result = a + b * c ** 2 / 10 - 5 <= b or b % a == 0 and c > b
```

1. `c ** 2` → 900
2. `b * 900` → 18000; `/ 10` → 1800.0 (same level, left to right)
3. `a + 1800.0 - 5` → 1805.0
4. `1805.0 <= b` → `False`
5. `b % a == 0` → `True`; `c > b` → `True`
6. `and` first: `True and True` → `True`
7. `False or True` → **`True`**

```
precedence: True True | left side value: 1805.0
-2 ** 2 = -4 | (-2) ** 2 = 4 | 2 ** 3 ** 2 = 512
not 1 == 2 -> True | 1 < 2 == 2 -> True
```

The second `True` is the fully parenthesised version. `not 1 == 2` is `not (1 == 2)`; `1 < 2 == 2` chains as `1 < 2 and 2 == 2`. **Rule of thumb:** if you have to think about precedence, add parentheses.

Common mistakes

- `x++` doesn't exist; `if a = b:` is invalid (use `==`, or `:=` deliberately).
- `x == 1 & y == 2` parses as the chain `x == (1 & y) == 2`. Write `(x == 1) & (y == 2)` or use `and`.
- `if x == 1 or 2:` is always truthy (`(x == 1) or 2`). Write `if x in (1, 2):`.

- Comparing floats with `==`:

```
0.1*3 == 0.3: False | abs diff: True
```

Real-world examples

- `timeout = config.get("timeout") or 30`.
- `while (chunk := sock.recv(4096)):` in socket code.
- Bit flags: `os.open(path, os.O_WRONLY | os.O_CREAT)`, `re.IGNORECASE | re.MULTILINE`.
- `y = X @ w` in NumPy/PyTorch.

In the interview

> **30-second answer:** "Python has arithmetic, comparison, assignment/augmented, logical, bitwise, membership and identity operators, plus the walrus `:=`. `and` / `or` short-circuit and return an operand, comparisons chain like maths, `**` is right-associative and binds tighter than unary minus, and bitwise operators bind tighter than comparisons."

- *`-3 ** 2`?* → `-9`.
- *Why does `if x == 1 or 2` always pass?* → `(x == 1) or 2`; `2` is truthy.
- *`a += [1]` vs `a = a + [1]`?* → First mutates in place (aliases see it); second creates a new list.
- *`in` on list vs set?* → O(n) vs average O(1).

Self-check

1. `10 // 4`, `10 % 4`, `10 / 4`? → `2`, `2`, `2.5`.
2. `None or "x"`; `0 and "x"`? → `"x"`; `0`.
3. `5 & 3`, `5 | 3`, `5 ^ 3`? → `1`, `7`, `6`.
4. `1 < 3 > 2`? → `True` (chain).
5. `2 ** -1`? → `0.5`.

TL;DR

Know `/` vs `//` vs `%`; `and`/`or` short-circuit and return operands; comparisons chain; `+=` mutates mutables in place; precedence: `**` > unary > `* /` > `+ -` > shifts > `&` > `^` > `|` > comparisons > `not` > `and` > `or`. When in doubt, parenthesise.

Chapter 2 TL;DR

- A variable is a **name bound to an object**; `=` never copies; `del` removes a name.
- `==` compares values, `is` identity — `is` only for `None`/singletons.
- Mutable: `list`, `dict`, `set`, `bytearray`. Immutable: numbers, `str`, `tuple`, `frozenset`, `bytes`, `None`.
- **Dynamically** (names re-point) and **strongly** (no silent coercion) typed.
- `int` unbounded; `float` approximate (`isclose`); `Decimal("…")` for money; `Fraction` for exact ratios.
- Falsy: `None`, `False`, zeros, empty containers; `bool` is an `int`.
- Implicit conversion only up the numeric tower; else `int()`, `float()`, `str()`… with `ValueError`/`TypeError` on bad input.
- `//` floors, `%` follows the divisor, `and`/`or` short-circuit, comparisons chain, `:=` assigns inside expressions; parenthesise rather than memorise.

Chapter 3 — Control Flow: Decisions and Loops

Control flow is how a program chooses *which* statements run (`if`, `match`) and *how many times* (`for`, `while`). With Chapter 2's truthiness rules (empty and zero values count as false) these constructs express any algorithm.

#	Topic	Key ideas
3.1	Conditionals overview	branching and where it's used
3.2	`if` / `elif` / `else`	order of tests, guard clauses
3.3	Conditional expressions	`a if cond else b`
3.4	`match` / `case`	literal, OR, capture, wildcard, sequence, mapping, class patterns, guards, `as`
3.5	Loops and the iteration protocol	iterables, iterators, `for` vs `while`
3.6	`for` loops	dicts, nesting, `range`, `enumerate`, `zip`
3.7	`while` loops	unknown counts, infinite loops
3.8	Loop control	`break`, `continue`, `pass`
3.9	`for-else` / `while-else`	"no break happened"
3.10	Loop idioms	`sum` / `any` / `all`, counting, safe modification

Code: `code/conditionals.py`, `code/pattern_matching.py`, `code/loops.py` — outputs are real (CPython 3.14).

3.1 Conditional statements — overview

What it is

A **conditional statement** evaluates a **condition** (any expression, read by truthiness) and runs a block only when it holds. Two constructs:

- `if` / `elif` / `else` — test conditions one after another.
- `match` / `case` (3.10+) — compare one value against **patterns** describing its shape; a far more powerful cousin of C/Java `switch`.

Why it matters — real-world uses

Typical uses: input validation (reject a short password), navigation (logged in → dashboard), games (health ≤ 0 → game over), alerts (heart rate out of range), web routing by HTTP method and path (404 if none matches).

3.2 `if`, `elif`, `else`

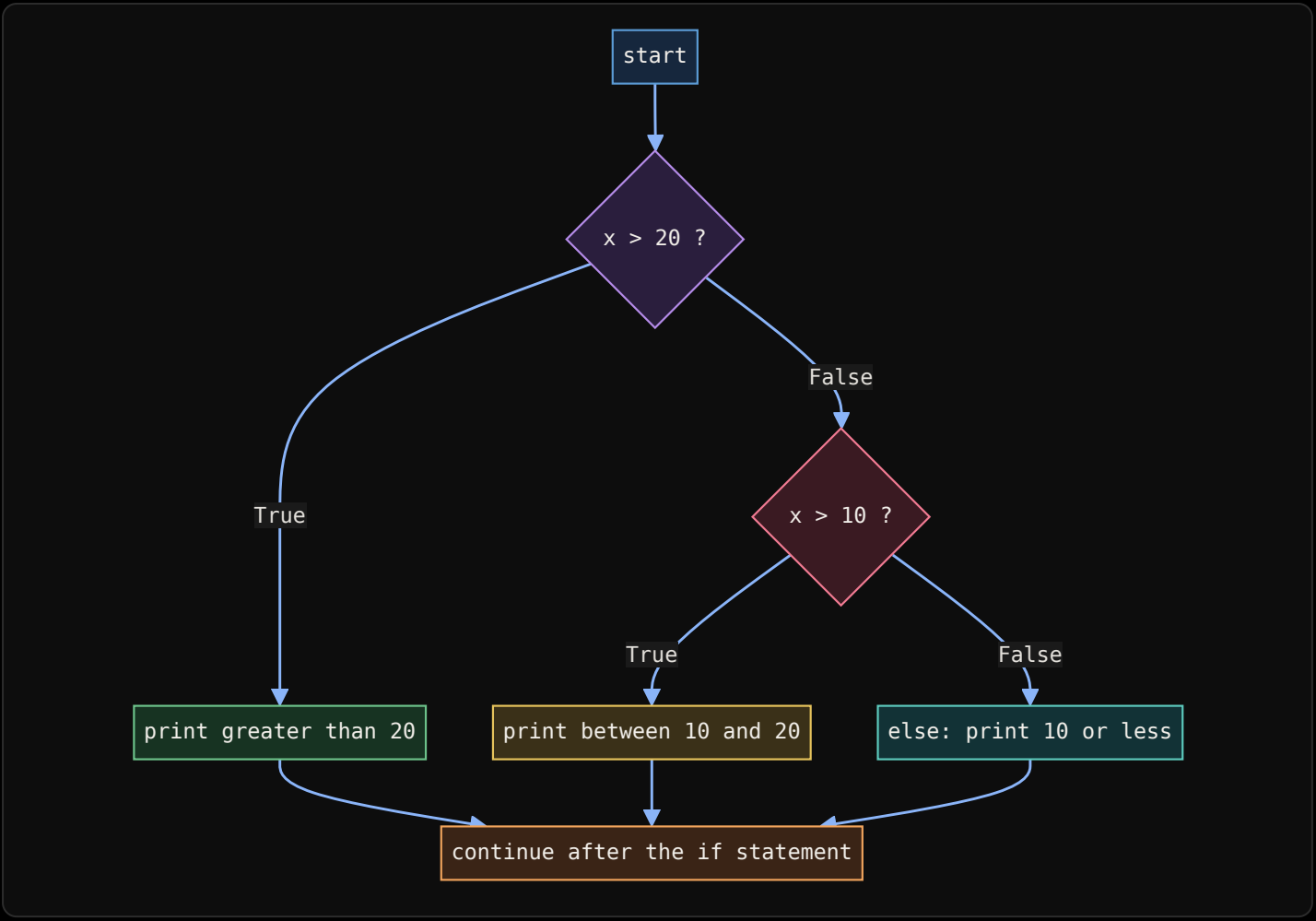
What it is

```
if condition1:
    ...            # runs if condition1 is truthy
elif condition2:
    ...            # runs if condition1 was falsy and condition2 is truthy
else:
    ...            # runs if every condition above was falsy
```

- `if` required; any number of `elif`; optional `else` last. Headers end with `:`.

- **At most one** branch runs: the first truthy one. Later conditions are not evaluated.

How it works



From `code/conditionals.py`:

```
x = 10
if x > 5:
    print("x is greater than 5")          # runs

x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")      # runs

def describe(x: int) -> str:
    if x > 20:
        return "x is greater than 20"
    elif x > 10:
        return "x is greater than 10 but not more than 20"
    else:
        return "x is 10 or less"
```

Output:

```
x is greater than 5
x is not greater than 5
25 -> x is greater than 20
15 -> x is greater than 10 but not more than 20
5 -> x is 10 or less
```

Order matters

The first true branch wins, so test the **most specific** condition first:

```python
def grade(score: int) -> str:
    if not 0 <= score <= 100:
        raise ValueError(f"score out of range: {score}")
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    return "F"
```

```
grades: ['A', 'B', 'C', 'F']
```

If `score >= 70` came first, every A and B student would get a C.

Nesting vs guard clauses

Deep nesting ("arrow code") is hard to read. A **guard clause** handles an edge case early with `return`/`raise` (`raise` signals an error — Chapter 9), keeping the main path flat — "flat is better than nested":

```python
def shipping_cost(total: float, is_member: bool) -> float:
    if total <= 0:
        return 0.0
    if is_member or total >= 50:
        return 0.0
    return 4.99
```

```
shipping: 4.99 0.0 0.0
```

Writing good conditions

- Truthiness: `if not cart:` rather than `if len(cart) == 0:`.
- Test `None` explicitly when `0`/`""` are valid — `discount = 0` looks "missing" to a truthiness test:

```
cart is empty
hello, guest
falsy check says missing: True | is-None check says missing: False
```

- Chained ranges: `if 18 <= temp <= 26 and not raining:`; membership instead of `or` chains: `if day in {"sat", "sun"}:`.

```
weekend
good weather for a walk
```

Trade-offs

Approach	Good for	Weak at
`if/elif/else`	Arbitrary conditions, ranges, few branches	Many equality tests on one value
`match/case`	One value, many shapes; destructuring (pulling parts into names)	Plain boolean conditions
dict lookup `{"a": f, "b": g}[key]()`	Constant keys → action (a *dispatch table*)	Conditions that aren't exact keys

Common mistakes

- `=` instead of `==` → `SyntaxError`; `else if` → must be `elif`.
- `if x == 1 or 2:` — always true; write `if x in (1, 2):`.
- `if flag == True:` → `if flag:`.
- Overlapping ranges in the wrong order (grading example).

Real-world examples

- Django views: `if request.method == "POST": … else: …`.
- Feature flags: `if flags.is_enabled("new_checkout", user): …`.

In the interview

> **30-second answer:** "`if` evaluates conditions top-down by truthiness and runs only the first matching branch; `elif` chains alternatives and `else` catches the rest. I order conditions from most specific to most general, use guard clauses to avoid deep nesting, and `is None` when zero or empty values are legitimate."

- *Does Python have `switch`?* → `match` (3.10+) covers it and more; dicts serve as dispatch tables.
- *Are later `elif`s evaluated after a match?* → No.

Self-check

1. With `x = 20`, which branch of `describe` runs? → The `elif` (`20 > 20` false, `20 > 10` true).
2. "Between 1 and 10 inclusive"? → `1 <= n <= 10`.

TL;DR

First truthy branch wins; order specific → general; flat guard clauses; use `in`, chained comparisons and truthiness.

3.3 Conditional expressions (the "ternary")

What it is

The **expression** form of `if/else` (three parts, hence "ternary"):

```
value_if_true if condition else value_if_false
```

```
parity = "even" if n % 2 == 0 else "odd"          # 7 is odd
print("Access granted." if has_access else "Access denied.")
```

Only the chosen branch is evaluated; `else` is required. Very low precedence (just above `lambda`), so parenthesise inside bigger expressions. Example: `abs` = `x if x >= 0 else -x`.

When not to use it

- Long branches or side effects (printing, changing data) — use an `if` statement.
- Nested ternaries (`a if x else b if y else c`) — legal but hard to read.
- For defaults `x or default` is shorter — only when all falsy `x` should be replaced.

TL;DR

`a if cond else b` picks a value; keep it short and un-nested.

3.4 `match` / `case` — structural pattern matching

What it is

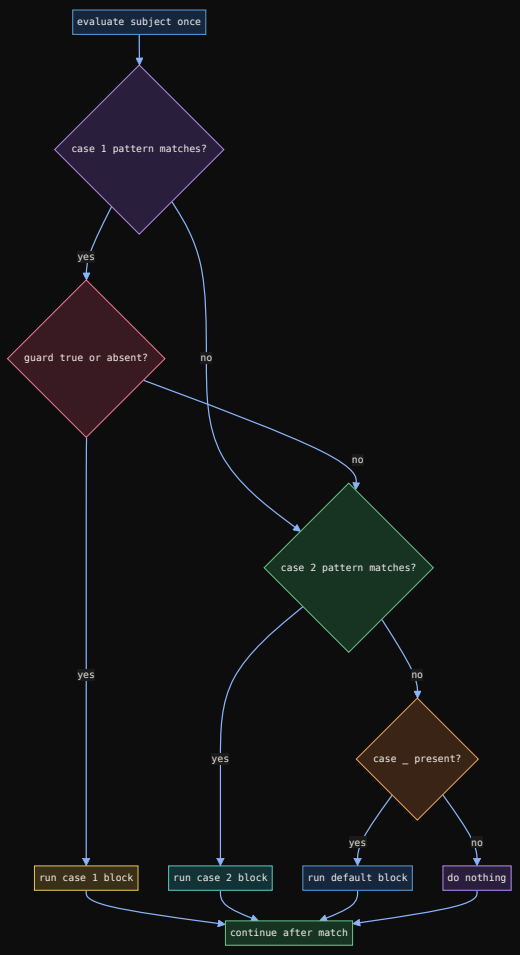
`match` (3.10, PEPs 634–636) evaluates a **subject** once and tries each `case` **pattern** in order; the first that matches (and whose optional guard is true) runs. No match → nothing happens, no error.

Unlike C/Java `switch`, a pattern describes the **structure** of the data — "a list starting with `"go"` plus one item", "a dict whose `type` is `"click"`", "a `Point` with `y == 0`" — and **binds** parts to names in one step (*destructuring*).

```
match subject:
    case pattern1:
        ...
    case pattern2 if guard:
        ...
    case _:
        ...          # wildcard: the default
```

`match`/`case` are **soft keywords** — `match` still works as a variable name.

How it works



No fall-through (C cases without `break` run into the next); you never write `break`.

The pattern kinds

Pattern	Looks like	Matches when…	Binds
Literal	`200`, `"Monday"`, `None`, `True`	subject `==` literal (`None`/`True`/`False` via `is`)	—
Capture	`x` (bare name)	always	subject → `x`
Wildcard	`_`	always	nothing
Value	`Color.RED`, `http.HTTPStatus.OK` (dotted)	subject `==` that value	—
OR	`p1 | p2 | p3`	any alternative matches	all alternatives must bind the same names
AS	`pattern as name`	pattern matches	matched part → `name`
Sequence	`[a, b]`, `(a, b)`, `[first, *rest]`	subject is a sequence (**not** `str`/`bytes`) of matching length and items	items
Mapping	`{"type": "click", "pos": p}`	mapping has those keys with matching values; **extra keys ignored**	values; `**rest` collects the others
Class	`Point(x=0, y=0)`, `Point(x, 0)`, `int(n)`, `str()`	`isinstance(subject, Point)` and listed attributes match	attributes
Guard	`case p if cond:`	pattern matches **and** `cond` truthy	(pattern's bindings)

Literal and OR patterns

From `code/pattern_matching.py`:

```python
def http_status(code: int) -> str:
    match code:
        case 200:
            return "Success"
        case 404:
            return "Not Found"
        case 500:
            return "Server Error"
        case 301 | 302 | 307 | 308:          # OR pattern
            return "Redirect"
        case _:                              # wildcard
            return "Unknown status code"

def day_kind(day: str) -> str:
    match day:
        case "Monday" | "Friday":
            return "Start/end of the work week"
        case "Tuesday" | "Wednesday" | "Thursday":
            return "Midweek"
        case _:
            return "Weekend"
```

```
['Success', 'Not Found', 'Server Error', 'Redirect', 'Unknown status code']
Midweek | Start/end of the work week | Weekend
```

Sequence patterns

```python
def first_color(colors: list[str]) -> str:
    match colors:
        case []:
            return "empty list"
        case ["red", *others]:        # first item "red"; others = the rest (a list)
            return f"red first, {len(others)} others"
        case ["blue", *_]:            # *_ = any number of remaining items, not bound
            return "blue first"
        case [single]:                # exactly one item
            return f"only one color: {single}"
        case _:
            return "different first color"
```

```
[] -> empty list
['red', 'blue', 'green'] -> red first, 2 others
['blue', 'red'] -> blue first
['pink'] -> only one color: pink
['pink', 'red'] -> different first color
```

Without a star the length must match exactly; at most one star per sequence pattern.

Command parsing, with an OR sub-pattern captured by `as` and a **guard**:

```python
def command(text: str) -> str:
    match text.split():
        case ["go", ("north" | "south" | "east" | "west") as direction]:
            return f"moving {direction}"
        case ["take", item]:
            return f"picked up {item}"
        case ["drop", *items] if items:          # guard: need at least one item
            return f"dropped {', '.join(items)}"
        case ["quit" | "exit"]:
            return "bye"
        case _:
            return f"unknown command: {text!r}"
```

```
'go north'       -> moving north
'take lamp'      -> picked up lamp
'drop key coin'  -> dropped key, coin
'drop'           -> unknown command: 'drop'
'quit'           -> bye
'dance'          -> unknown command: 'dance'
```

`"drop"` alone matches `["drop", *items]` with `items == []`, but the guard fails, so matching continues to `_`.

Tuples work the same way:

```python
match person:                                  # person = (25, "Engineer")
    case (age, "Engineer") if age > 20:
        return f"Adult Engineer aged {age}"
    case (age, profession):
        return f"{profession} aged {age}"
```

```
Adult Engineer aged 25 | Engineer aged 19
```

Strings are not sequences for matching (deliberate, so `"ab"` doesn't match `[a, b]`):

```
'ab' is matched as a str, not as a sequence of characters
```

Mapping patterns

Ideal for JSON-like events and API payloads:

```python
match event:
    case {"type": "click", "pos": (x, y)}:          # nested sequence pattern
        return f"click at {x},{y}"
    case {"type": "key", "key": str(k)} if len(k) == 1:
        return f"key {k!r}"
    case {"type": "resize", **rest}:                # rest = remaining key/values
        return f"resize with {rest}"
    case {"type": other}:
        return f"unhandled event type {other!r}"
    case _:
        return "not an event"
```

```
{'type': 'click', 'pos': (3, 4), 'button': 1} -> click at 3,4
{'type': 'key', 'key': 'q'} -> key 'q'
{'type': 'resize', 'w': 800, 'h': 600} -> resize with {'w': 800, 'h': 600}
{'type': 'scroll'} -> unhandled event type 'scroll'
['click'] -> not an event
```

The first event's extra `"button"` key doesn't prevent the match.

Class patterns

> **Key term — class:** a blueprint for objects of your own type (Chapter 10); `Point(1, 2)` is an *instance* of `Point`. `isinstance(obj, Point)` asks "was `obj` made from `Point` or a subclass?".

`ClassName(...)` checks `isinstance`, then attributes. Keyword form `Point(x=0)` matches attribute `x`. Positional form `Point(a, b)` needs `__match_args__` — a tuple naming the attributes positions map to; `@dataclass` (a decorator that auto-writes common class code) generates it (`('x', 'y')`). Built-ins like `int`, `str`, `float` take one positional sub-pattern matching the whole subject.

```python
@dataclass
class Point:
    x: float
    y: float

match shape:
    case Point(x=0, y=0):                           # keyword attributes
        return "the origin"
    case Point(x, 0):                               # positional via __match_args__
        return f"on the x-axis at {x}"
    case Point():                                   # any Point
        return "some other point"
    case Circle(center=Point(x=0, y=0), radius=r):  # nested patterns
        return f"circle of radius {r} at the origin"
    case Circle(radius=r) if r <= 0:
        return "invalid circle"
    case int(n) | float(n):
        return f"a number {n}"
    case str() as s:
        return f"a string of length {len(s)}"
    case _:
        return "unknown shape"
```

```
Point(x=0, y=0) -> the origin
Point(x=5, y=0) -> on the x-axis at 5
Point(x=1, y=2) -> some other point
Circle(center=Point(x=0, y=0), radius=2) -> circle of radius 2 at the origin
Circle(center=Point(x=1, y=1), radius=-1) -> invalid circle
7 -> a number 7
2.5 -> a number 2.5
'hello' -> a string of length 5
[1] -> unknown shape
Point.__match_args__ = ('x', 'y')
```

The capture-pattern trap

A **bare name** is a *capture*, never a comparison. With `RED = "red"`, `case RED:` matches anything and rebinds `RED`:

```
matched?! blue
SyntaxError: name capture 'RED' makes remaining patterns unreachable
```

Python refuses to compile it if more cases follow. Compare against a constant with a **dotted name** such as an `Enum` member (a named constant in an enumeration class), a literal, or a guard (`case x if x == RED:`):

```python
class Color(Enum):
    RED = "red"
    GREEN = "green"

match c:
    case Color.RED:      # compared with ==
        return "stop"
    case Color.GREEN:
        return "go"
```

```
stop go
```

Trade-offs

	`match`	`if/elif`
Check structure and extract parts at once	✅	Manual `isinstance`, `len`, indexing
Arbitrary boolean conditions	Via guards only	✅
Python version	3.10+	All
Performance	Similar — cases tried in order, not a jump table	Similar

Don't use it for one or two boolean tests (`if`), constant key → value maps (a dict), or code that must run on ≤ 3.9.

Common mistakes

- `case RED:` with a plain variable (capture trap).
- Expecting fall-through or writing `break` in a case.
- Expecting `case [a, b]:` to match a string.
- No `case _:` → unexpected input silently ignored; consider raising in the default.
- OR patterns binding different names (`case [x] | [x, y]:`) → `SyntaxError`.

Real-world examples

- CLI and chat-bot command parsers (`command()` above).
- Dispatching JSON messages from WebSockets or message queues by `type` (see System Design Fundamentals).
- Compilers walking an AST: `case BinOp(left, Add(), right):` — the `ast` node classes define `__match_args__` for this.

In the interview

30-second answer: "`match` evaluates a subject once and tries case patterns top to bottom, with no fall-through. Patterns can be literals, captures, the `_` wildcard, dotted-name constants, OR-patterns, sequences with `*rest`, mappings that ignore extra keys, and class patterns that check

`isinstance` and attributes via `__match_args__`. Guards add an `if` condition. The main trap is that a bare name captures instead of comparing."

- *Match a constant?* → Dotted name, literal, or guard.
- *Mapping patterns need exact keys?* → No; extra keys ignored, `**rest` captures them.
- *What makes positional class patterns work?* → `__match_args__`, auto-generated by `dataclass` and `namedtuple`.

Self-check

1. What does `case [x, *_]:` match? → Any non-string sequence with ≥ 1 item; binds the first to `x`.
2. No case matches and no `_`? → Nothing; execution continues after `match`.
3. Capture which alternative of `"a" | "b"` matched? → `("a" | "b") as choice`.

TL;DR

`match` = one value against shape-aware patterns, first match wins, no fall-through; literals, `|`, `[... *rest]`, `{key: ...}`, `Class(attr=...)`, guards, `as`; dotted names for constants — bare names capture.

3.5 Loops — overview and the iteration protocol

What it is

Loop	Repeats…	Use when
`for`	once per item of an **iterable**	you have a collection or known count
`while`	while a **condition** stays truthy	the count isn't known in advance

Loop control statements: `break` (leave), `continue` (next iteration), `pass` (placeholder). Visiting every item is **traversal**.

How `for` really works — the iteration protocol

An **iterable** hands out items one at a time (lists, strings, dicts, sets, ranges, files, generators — objects producing values on demand); an **iterator** tracks the position.

1. `for x in obj:` calls `iter(obj)` to get an iterator.
2. Each round it calls `next(iterator)` and binds `x`.
3. When `next` raises `StopIteration`, the loop ends quietly.

```
it = iter(["a", "b"])
next(it)   # 'a'
next(it)   # 'b'
next(it)   # raises StopIteration
```

So one `for` works for every container, and lazy objects (`range`, file handles) are looped without loading everything into memory.

3.6 The `for` loop

What it is

```
for variable in iterable:
    ...   # body runs once per item; variable holds the current item
```

How it works — basic traversal

From `code/loops.py`:

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit, end=" ")
for ch in "hey":
    print(ch.upper(), end="")
```

```
apple banana cherry
HEY
```

No index bookkeeping, so no off-by-one errors. The loop variable **survives** the loop with its last value (unless the iterable was empty).

Looping through dictionaries

Iterating a dict yields **keys** (insertion order). Views (live read-only windows) give the rest:

You need	Write
keys	`for k in d:` (or `d.keys()`)
values	`for v in d.values():`
both	`for k, v in d.items():`

```
info = {"name": "John", "age": 30, "city": "New York"}
for key in info:
    print(key, "->", info[key])
for key, value in info.items():          # preferred: no second lookup
    print(f"{key}={value}", end=" ")
```

```
name -> John
age -> 30
city -> New York
name=John age=30 city=New York | values: ['John', 30, 'New York']
```

Nested loops

The inner loop runs fully for each outer item: `rows × cols` iterations.

```
matrix = [[1, 2, 3], [4, 5, 6]]
for row in matrix:
    for num in row:
        print(num, end=" ")
    print()                              # newline after each row
```

```
1 2 3
4 5 6
```

Two nested loops over n items are O(n²) (10× data → 100× work). If the inner loop only searches, a set/dict lookup often removes it.

`range()` — counting loops

Lazy integers, stop **exclusive**:

Call	Produces
`range(stop)`	0 … stop−1
`range(start, stop)`	start … stop−1
`range(start, stop, step)`	start, start+step, … while < stop (> stop for negative step)

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9] [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] [1, 3, 5, 7, 9] [10, 7, 4, 1]
```

`step` can't be 0 (`ValueError`). `range` supports `len()`, indexing, fast `in`, and is cheap at any size.

`enumerate()` — index and item together

Instead of `for i in range(len(names)): names[i]`:

```
for i, name in enumerate(names, start=1):
    print(f"{i}. {name}", end="  ")
```

```
1. ann  2. bob  3. cy
```

`zip()` — walk several iterables in parallel

`zip(a, b)` yields tuples `(a[0], b[0]), (a[1], b[1]), …` and **stops at the shortest**; `strict=True` (3.10+) raises if lengths differ — usually what you want for aligned data.

```
for name, score in zip(names, scores, strict=True):
    print(f"{name}:{score}", end="  ")
```

```
ann:91  bob:78  cy:85
zip strict: zip() argument 2 is shorter than argument 1
plain zip stops at the shortest: [(1, 'a'), (2, 'b')]
```

`dict(zip(keys, values))` builds a dict from two lists.

Common mistakes

- `for i in range(len(items)):` when you need items (iterate directly) or both (`enumerate`).
- `range(1, 10)` stops at 9.
- Expecting dict iteration to give pairs.
- Silent truncation with plain `zip`.

Real-world examples

- `for line in f:` streams a file; `csv.reader(f)` rows.
- `for i, batch in enumerate(dataloader):` — the standard PyTorch training loop.
- `zip(features, labels)` in ML pipelines.

In the interview

> **30-second answer:** "A `for` loop calls `iter()` on the iterable and `next()` until `StopIteration`, so it works on anything iterable. I iterate items directly, use `enumerate` for the index, `zip(strict=True)` for parallel sequences, `.items()` for dicts, and `range` for counts."

- *`zip` with unequal lengths?* → Stops at shortest; `strict=True` raises `ValueError`.
- *Loop variable after the loop?* → Keeps the last value.

Self-check

1. `list(range(2, 11, 3))`? → `[2, 5, 8]`.
2. Nested loops over 3 and 4 items: inner body runs? → 12 times.

TL;DR

`for item in iterable`; dict → keys (use `.items()`); `range` excludes stop; `enumerate` for indices; `zip(strict=True)` for parallel lists.

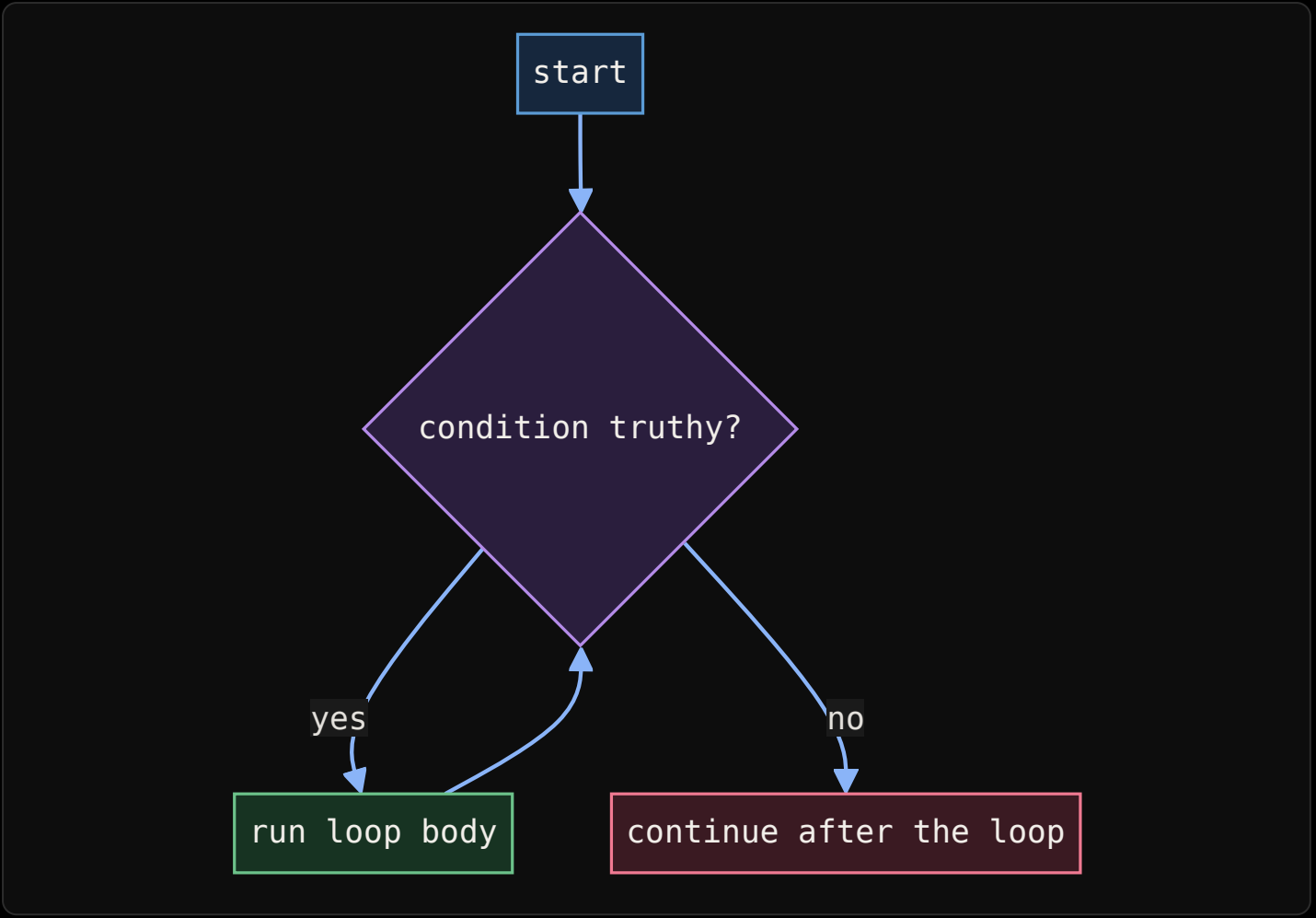
3.7 The `while` loop

What it is

```
while condition:
    ...   # repeats while condition is truthy; re-checked before every iteration
```

For unknown iteration counts: read until "quit", retry until success, iterate until a value converges.

How it works



The condition is checked first (the body may run **zero** times), and the body **must** change something the condition depends on.

```
count = 0
while count < 5:
    print(f"Count is {count}", end="; ")
    count += 1          # progress toward making the condition false
```

```
Count is 0; Count is 1; Count is 2; Count is 3; Count is 4;
```

A genuinely unknown count — halvings until below 1:

```
n, steps = 100.0, 0
while n >= 1:
    n /= 2
    steps += 1
```

```
halvings until < 1: 7
```

(100 → 50 → 25 → 12.5 → 6.25 → 3.125 → 1.5625 → 0.78125.)

Infinite loops on purpose

`while True:` + `break` is the idiomatic "loop and a half" (menus, server loops):

```
while True:
    answer = input("> ")
    if answer == "quit":
        break
    handle(answer)
```

With walrus: `while (answer := input("> ")) != "quit": handle(answer)`.

Trade-offs

	`for`	`while`
Termination	Automatic when exhausted	Your responsibility
Typical bug	Off-by-one in `range`	Infinite loop (forgot to update)
Use for	Collections, fixed counts	Unknown counts, event loops, retries, convergence

Common mistakes

- Forgetting the update → infinite loop (Ctrl-C raises `KeyboardInterrupt`).
- `while i < len(items): … i += 1` to traverse — use `for`.
- Float conditions like `while x != 1.0:` that never become exactly true — use `<` / `>` or a tolerance.

Real-world examples

- Game loops: `while running: handle_events(); update(); draw()`.
- Retry with back-off: `while attempts < 5 and not ok: …`.
- Chunked reading: `while (chunk := f.read(8192)): …`.

In the interview

> **30-second answer:** "`while` re-checks its condition before each iteration and is for unknown iteration counts. The body must make progress toward the condition becoming false; for intentional infinite loops I use `while True` with a clear `break`."

TL;DR

`while cond:` repeats until `cond` is falsy (possibly zero times); guarantee progress; `while True` + `break` for loop-and-a-half.

3.8 Loop control: `break`, `continue`, `pass`

What it is

Statement	Effect
`break`	Exit the **innermost** loop now; skips any loop `else`
`continue`	Skip to the next iteration (`while` re-checks its condition)
`pass`	Do nothing; fills a place where a statement is required

How it works

```python
for number in [1, 2, 3, 4, 5, 6]:
    if number == 4:
        print("Number 4 found, stopping loop.")
        break                                   # 5 and 6 are never visited

odds = []
for number in [1, 2, 3, 4, 5, 6]:
    if number % 2 == 0:
        continue                                # skip evens
    odds.append(number)

for number in [1, 2, 3]:
    if number % 2 == 0:
        pass                                    # placeholder - does nothing
    print(number, end=" ")                      # still runs for every number
```

```
Number 4 found, stopping loop.
continue kept odds: [1, 3, 5]
1 2 3
```

`pass` is **not** `continue`: the rest of the body still runs. Use it (or `...`) for stubs: `def todo(): pass`, `class MyError(Exception): pass`.

`break` exits only one level:

```python
for i in range(3):
    for j in range(3):
        if j == 1:
            break                   # leaves the inner loop only
        print(f"({i},{j})", end=" ")
```

```
(0,0) (1,0) (2,0)
```

To exit several levels, move the loops into a function and `return`, or use a flag variable.

Common mistakes

- Expecting `break` to leave all nested loops.
- Confusing `pass` and `continue`.
- `continue` in a `while` *before* the counter update → infinite loop.
- `continue`-based filtering where a comprehension is clearer.

In the interview

> **30-second answer:** "`break` leaves the innermost loop, `continue` jumps to the next iteration, `pass` is a no-op placeholder. To break out of nested loops I refactor into a function and `return`."

TL;DR

`break` = leave loop, `continue` = next iteration, `pass` = do nothing; all act on the innermost loop.

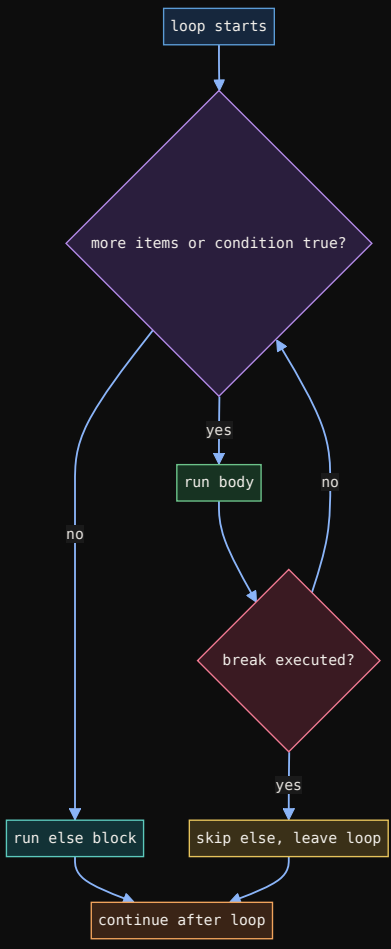
3.9 `for-else` and `while-else`

What it is

A loop's `else` runs **only if the loop finished without** `break` (iterable exhausted, or condition became false). Read it as **"no break"**. Classic use: search loops — `break` = found, `else` = not found, no flag needed.

```python
for item in iterable:
    ...
else:
    ...   # runs only if the loop didn't break
```

How it works



From `code/loops.py`:

```python
for number in [1, 2, 3, 4, 5]:
    if number == 3:
        print("Number 3 found, stopping search.")
        break
else:
    print("Number not found.")        # skipped: break happened

for number in [1, 2, 4, 5]:
    if number == 3:
        print("found 3")
        break
else:
    print("3 not in list (else ran: no break)")

count = 0
while count < 5:
    if count == 2:
        print("Breaking at count 2.")
        break
    count += 1
else:
    print("Loop was uninterrupted.")  # skipped: break happened
```

```
Number 3 found, stopping search.
3 not in list (else ran: no break)
Breaking at count 2.
```

With no `break` in the body the `else` always runs — legal but pointless.

Inside a function, `return` plays `break` and the final statement plays `else` — e.g. primality by trial division (try every divisor up to √n):

```python
def is_prime(n: int) -> bool:
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True
```

```
is 29 prime? True | is 21 prime? False
```

At module level, or when you need to continue afterwards, `for-else` is the flag-free equivalent.

Trade-offs / when not to use it

Many developers misread `else` as "if the loop didn't run". If that confuses your team, use a helper with `return`, or `any()` / `next()` (§3.10). Comment it `# no break`.

Common mistakes

- Thinking `else` runs only when the body never executed — it also runs after a normal full run.
- Thinking `else` runs after `break`.
- Misaligning `else` so it pairs with an `if` inside the loop (or vice versa).

In the interview

> **30-second answer:** "A loop's `else` runs only when the loop wasn't terminated by `break` — think 'no break'. It's handy for search loops: `break` on found, `else` for not-found, without a flag variable."

- *Runs on an empty iterable?* → Yes, no break happened.
- *`return` inside the loop?* → Skips `else` (leaves the function).

TL;DR

Loop `else` runs when the loop ends naturally, not via `break`; use it for search-with-not-found and comment it.

3.10 Common loop idioms

Aggregation with built-ins

Many loops are built-ins in disguise — shorter, clearer and run in C:

```python
nums = [4, -1, 7, 0, 3]
sum(nums), min(nums), max(nums)
any(x < 0 for x in nums)                # True if at least one; stops early
all(x < 10 for x in nums)               # True if every one; stops early
[x * x for x in nums if x > 0]          # list comprehension: map + filter
list(reversed(nums)), sorted(nums, reverse=True)
next((x for x in nums if x > 5), None)  # first match or a default
```

```
sum/min/max: 13 -1 7
any negative? True | all < 10? True
squares of positives: [16, 49, 9]
reversed: [3, 0, 7, -1, 4] | sorted desc: [7, 4, 3, 0, -1]
first > 5: 7
```

`next(generator, default)` replaces a whole `for … break … else` search. Comprehensions are covered in the lists chapter. Note `all([])` is `True`, `any([])` is `False`.

Counting / grouping with a dict

```
counts = {}
for word in "the cat and the hat".split():
    counts[word] = counts.get(word, 0) + 1
```

```
word counts: {'the': 2, 'cat': 1, 'and': 1, 'hat': 1}
```

(`collections.Counter` does this in one call — dictionaries chapter.)

Don't change a collection while looping over it

Removing during iteration shifts later items left, so the loop **skips** the next one:

```
buggy = [1, 2, 2, 3]
for x in buggy:
    if x == 2:
        buggy.remove(x)        # the second 2 slides into the visited slot and is skipped
```

```
remove-while-iterating left: [1, 2, 3] | filtered correctly: [1, 3]
RuntimeError: dictionary changed size during iteration
```

Fix: rebuild (`[x for x in data if x != 2]`) or iterate a copy (`for x in data[:]:`). Dicts refuse outright with `RuntimeError`; iterate `list(d)` if you must add/delete keys.

Idiom cheat-sheet

Instead of…	Write…
`for i in range(len(xs)): print(xs[i])`	`for x in xs: print(x)`
index + item via `range(len())`	`for i, x in enumerate(xs):`
parallel indexing `a[i], b[i]`	`for x, y in zip(a, b, strict=True):`
`for k in d: v = d[k]`	`for k, v in d.items():`
looping backwards with indices	`for x in reversed(xs):`
accumulator loop (keeps a running total)	`sum(xs)`
flag + loop to test a condition	`any(...)` / `all(...)`
loop with `append` to transform	list comprehension
`while` loop reading chunks	`while (chunk := f.read(n)):`

Real-world examples

- Log scanning: `for line in open(path): if "ERROR" in line: …`.
- ETL jobs (extract–transform–load) aggregating per-key totals in dicts.
- Validation: `if not all(field in payload for field in REQUIRED): return 400`.

In the interview

> **30-second answer:** "I replace hand-written loops with built-ins where possible — `sum`, `any`, `all`, `min` / `max`, `sorted`, comprehensions, `next` with a default — because they're clearer and run in C. I never mutate a list or dict while iterating it; I build a new one or iterate a copy."

- *Why does removing during iteration skip items?* → The list iterator advances by index; removal shifts later items down.
- *Do `any` / `all` evaluate everything?* → No, they short-circuit.

TL;DR

Prefer built-ins and comprehensions over manual loops; `enumerate` / `zip` / `.items()` over indexes; never mutate what you're iterating.

Chapter 3 TL;DR

- `if/elif/else`: first truthy branch wins; order specific → general; guard clauses keep code flat; `a if c else b` for values.
- `match/case` (3.10+): literals, `|`, captures, `_`, dotted constants, sequences with `*rest`, mappings (extra keys ignored), class patterns (`__match_args__`), guards, `as`; no fall-through; bare names capture.
- `for` uses the iteration protocol (`iter` / `next` / `StopIteration`); dicts yield keys; use `range`, `enumerate`, `zip(strict=True)`, `.items()`.
- `while` runs while a condition holds — guarantee progress; `while True` + `break` for loop-and-a-half.
- `break` leaves the innermost loop, `continue` skips ahead, `pass` does nothing.
- Loop `else` = "no break happened" — ideal for search loops.
- Reach for `sum` / `any` / `all` / `next` /comprehensions first; never modify a collection while iterating it.

Chapter 4 — Strings

Text is everywhere — user names, file paths, log lines, JSON, HTML, SQL, chat messages — and in Python it all lives in one type, `str`. This chapter covers what intermediate developers must know cold: immutability, slicing, the formatting mini-language, the useful methods and their costs, efficient joining, Unicode vs bytes (UTF-8), and a working knowledge of regular expressions.

What you'll learn

1. Creating, indexing and looping over strings.
2. Immutability and how to "change" a string anyway.
3. Operators (`+`, `*`, `in`, comparisons, `%`) and slicing.
4. Escape sequences and raw strings.
5. f-strings, the format-spec mini-language, `str.format()`, `%`.
6. The string methods you'll actually use, with costs.
7. Joining many pieces efficiently.
8. Unicode vs bytes, encodings, UTF-8.
9. Regular expressions: `search`, `match`, `findall`, `sub`, groups.

Runnable code is in `code/`; every output was produced with Python 3.14 (code also runs on 3.12+).

4.1 What a string is — creating, indexing, looping

What it is

A **string** (`str`) is an ordered, immutable sequence of Unicode characters (Unicode gives every character in every writing system a number — §4.9). *Ordered*: each character has a position. *Immutable*: characters can never be changed. *Sequence*: supports `len()`, indexing, slicing, `in`, `for` — like lists and tuples.

There is no separate character type: a character is a `str` of length 1 (`type("x")` is `str`, `len("x")` is `1`).

Why it matters

Almost every program reads, cleans or produces text; exact indexing rules prevent off-by-one bugs.

How it works

1. **Create** with any of three quote styles (all give `str`):
   - `'single'` — convenient when the text contains `"`.
   - `"double"` — convenient with apostrophes: `"It's raining"`.
   - `"""triple"""` / `'''triple'''` — spans lines, holds both quote kinds; used for docstrings.
2. **Index** with `s[i]` from `0`. **Negative indexes** count from the right (`-1` is last). Valid range for length *n*: `0 … n−1` and `−n … −1`; anything else raises `IndexError`.
3. **Loop** with `for ch in s:`; `enumerate(s)` adds positions.

```
index:   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14
char :   L   e   a   r   n   i   n   g   ␣   P   y   t   h   o   n
neg  : -15 -14 -13 -12 -11 -10  -9  -8  -7  -6  -5  -4  -3  -2  -1
```

```python
# code/01_string_basics.py (excerpt)
single = 'Hi there'
double = "It's raining"                      # apostrophe inside double quotes
triple = """Line one
Line two with "quotes" and 'apostrophes'"""
print(single, "|", double)
print(triple)
print(type("x"), len("x"))                   # a single character is just a str of length 1

word = "Learning Python"
print(word[0], word[9], word[-1], word[-6])  # L P n P
try:
    word[100]
except IndexError as e:
    print("IndexError:", e)

for i, ch in enumerate("abc"):
    print(f"{i}:{ch}", end=" ")
```

Output:

```
Hi there | It's raining
Line one
Line two with "quotes" and 'apostrophes'
<class 'str'> 1
L P n P
IndexError: string index out of range
0:a 1:b 2:c
```

Common mistakes

- Indexing starts at 0: the 8th character is `s[7]`.
- `for i in range(len(s)): s[i]` when you only need characters — write `for ch in s`.
- `'It's'` ends at the apostrophe → `SyntaxError`. Use `"It's"` or `'It\'s'`.

Self-check

1. `"Python programming is fun!"[-4]` ? → `'f'`.

TL;DR

`str` = immutable, ordered sequence of Unicode characters; three quote styles; zero-based and negative indexes; `for ch in s`.

4.2 Immutability — and how to "update" a string

What it is

Immutable: the value can't change after creation. `s[0] = "J"` is illegal; `upper()`, `replace()`, `+` all return a **new** string.

Why it matters

- **Safety:** functions can't alter strings you pass them — no defensive copies.
- **Hashability:** a fixed value means a fixed hash (an integer fingerprint used for fast lookup), so strings can be dict keys and set members (Chapter 6). CPython caches a string's hash after the first computation.
- **Sharing:** the interpreter may reuse one object in many places (identifiers are *interned* — stored once and shared), safe only because nobody can mutate it.

How it works

Item assignment → `TypeError`. To "update", slice what you keep, concatenate the new text, and rebind the name. The old object is untouched (garbage-collected if unreferenced).

```
s = "Hello, world!"
try:
    s[0] = "J"                          # not allowed
except TypeError as e:
    print("TypeError:", e)
before = id(s)
s = s[:7] + "Python!"                   # build a NEW string, rebind the name
print(s, "| same object?", id(s) == before)
```

Output:

```
TypeError: 'str' object does not support item assignment
Hello, Python! | same object? False
```

`id()` differs: a new object was created.

Trade-offs

	Immutable `str`	Mutable alternative (`list` of chars, `bytearray` , `io.StringIO`)
Safe to share / dict key	Yes	No
Change one character	O(n) — copy the whole string	O(1)
Build text piece by piece	`"".join(parts)` (4.8)	Natural

> **Key term — Big-O notation:** how cost grows with input size *n*. **O(1)**: constant; **O(n)**: grows in step with length; **O(n²)** ("quadratic"): doubling *n* quadruples the work. It describes growth, not timings.

Common mistakes

- Ignoring the result: `name.strip()` alone does nothing. Write `name = name.strip()`.
- Thinking `s += "x"` modifies in place — it creates a new string (4.8).

In the interview

> **30-second answer:** "Python strings are immutable: every 'modifying' operation returns a new string. That makes them safe to share and hashable, so they work as dict keys. The cost is that repeated modification copies data, so for building large strings you collect parts in a list and `join` them."

- *How does `s += 'x'` work?* → Builds `s + 'x'` and rebinds `s`. (CPython sometimes resizes in place when no other reference exists — don't rely on it.)
- *Why must dict keys be immutable?* → Strictly, *hashable*: a changing hash would make lookups search the wrong slot.

TL;DR

You never change a string; you make a new one and rebind. Immutability buys safety and hashability.

4.3 String operators

What it is

Operator	Meaning	Example	Result
`+`	Concatenation	`"Py" + "thon"`	`'Python'`
`*`	Repetition (int times)	`"Ha" * 3`	`'HaHaHa'`
`in`	Substring membership	`"fox" in "The quick brown fox"`	`True`
`not in`	Negated membership	`"cat" not in "The quick brown fox"`	`True`
`==`, `!=`	Equality by value	`"Python" == "Python"`	`True`
`<`, `>`, `<=`, `>=`	Lexicographic (dictionary-order) comparison	`"apple" < "banana"`	`True`
`%`	printf-style formatting (from C's `printf`; legacy, 4.6)	`"%s!" % "hi"`	`'hi!'`
`[i]`, `[a:b:c]`	Indexing and slicing (4.4)	`"abc"[1]`	`'b'`

How it works

1. `+` needs **two strings**: `"Age: " + 30` → `TypeError`; use `str(30)` or an f-string.
2. `*` with zero or negative gives `''`.
3. `in` tests **substrings**: `"row" in "brown"` is `True`.
4. Comparisons go character by character by **Unicode code point** (4.9); first difference decides; a prefix is smaller (`"app" < "apple"`).
   - `"Zebra" < "apple"`: `'Z'` (90) < `'a'` (97) → `True`. Uppercase sorts before lowercase.
   - `"10" < "9"` is `True` (`'1'` 49 < `'9'` 57) — numeric strings don't compare as numbers.

```
print("Ha" * 3, "+".join(["a", "b"]), "Py" + "thon")
print("fox" in "The quick brown fox", "cat" not in "The quick brown fox")
print("apple" < "banana", "Zebra" < "apple", "10" < "9")  # compares code points
```

Output:

```
HaHaHa a+b Python
True True
True True True
```

Common mistakes

- Sorting names with plain `<` (`"Zoe"` before `"adam"`). Use `key=str.casefold` (a *key function* decides each item's sort position).
- Comparing numeric strings — convert with `int()` / `float()`.
- `is` for strings checks identity; it may "work" for short interned strings then fail. Use `==`.

TL;DR

`+` joins, `*` repeats, `in` finds substrings, comparisons are by code point (case and digits matter), use `==` not `is`.

4.4 Slicing

What it is

`s[start:stop:step]` returns a new string:

- `start` — first position included (default: beginning).
- `stop` — first position **excluded** (default: end).
- `step` — jump size (default `1`; negative walks backwards).

The same syntax works on lists, tuples, bytes and arrays (Chapter 5): fixed-position fields, dropping prefixes, reversing, every *n*-th item.

How it works

1. Negatives get `len(s)` added (`len("Hello, Python!") == 14`, so `-7` → `7`).
2. Missing parts take defaults.
3. Walk from `start` toward `stop` by `step`, stopping **before** `stop`.
4. Out-of-range bounds are clipped — slicing never raises `IndexError`.

Half-open `[start, stop)` gives `len(s[a:b]) == b - a` and `s[:k] + s[k:] == s`.

```
text = "Hello, Python!"
        H  e  l  l  o  ,  ␣  P  y  t  h  o  n  !
index:  0  1  2  3  4  5  6  7  8  9 10 11 12 13
neg:  -14                   -7 -6 -5 -4 -3 -2 -1
```

```
text = "Hello, Python!"
print(repr(text[7:13]))    # 'Python'
print(repr(text[-7:-1]))   # 'Python'
print(repr(text[:5]), repr(text[7:]))
print(repr(text[::2]))     # every 2nd char: 'Hlo yhn'
print(repr(text[::-1]))    # reversed
print(repr(text[3:100]))   # slices never raise IndexError -> clipped
```

Output:

```
'Python'
'Python'
'Hello' 'Python!'
'Hlo yhn'
'!nohtyP ,olleH'
'lo, Python!'
```

`text[::2]` takes positions 0, 2, … 12 → `H l o ␣ y h n`; the comma (odd position 5) is skipped. `text[-5:-1]` is `'thon'`.

Common mistakes

- Expecting `stop` included: `text[7:12]` is `'Pytho'`.
- Negative step: `s[5:1:-1]` walks 5 **down to** 2; `s[1:5:-1]` is empty.
- `[::-1]` reverses code points, which can detach combining accents or split emoji with modifiers (skin tones) — 4.9.

In the interview

> **30-second answer:** "`s[start:stop:step]` returns a new sequence from `start` up to but not including `stop`. Omitted bounds default to the ends, negatives count from the end, bounds are clipped so it never raises, and `s[::-1]` reverses. It costs O(k) for k items copied."

- *Why is `stop` exclusive?* → `s[:k] + s[k:] == s`, `len(s[a:b]) == b - a`, no ±1 corrections.

- *Palindrome check* `s == s[::-1]`? → O(n) time and O(n) extra space; a two-pointer loop (indexes moving in from both ends) uses O(1) space.

Self-check

1. `"abcdef"[::3]`? → `'ad'`. 2. `"abc"[5:]`? → `''`.

TL;DR

`[start:stop:step]`, stop excluded, negatives from the end, never raises, `[::-1]` reverses, cost ∝ slice length.

4.5 Escape sequences and raw strings

What it is

An **escape sequence** is a backslash plus characters standing for one special character (`\n` = newline). A **raw string** (`r` prefix) disables escapes — every backslash is literal. Escapes put newlines, tabs, quotes and any Unicode into literals; raw strings suit backslash-heavy text: regular expressions (§4.10) and Windows paths.

How it works

Escape	Meaning	Example	Result
`\\`	Backslash	`"a\\b"`	`a\b`
`\'`	Single quote	`'It\'s'`	`It's`
`\"`	Double quote	`"He said \"hi\""`	`He said "hi"`
`\n`	Newline (line feed)	`"one\ntwo"`	two lines
`\t`	Horizontal tab	`"a\tb"`	`a` tab `b`
`\r`	Carriage return (cursor to line start)	`"Hello\rWorld"`	terminal shows `World`
`\b`	Backspace character (code 8)	`"ABC\b"`	terminal may erase `C` visually
`\f`	Form feed (printer page break)	`"Page\fNext"`	rarely visible
`\ooo`	Octal code (up to 3 digits)	`"\101"`	`A`
`\xhh`	Hex code (exactly 2 digits)	`"\x41"`	`A`
`\uXXXX`	Code point (4 hex digits)	`"\u00e9"`	`é`
`\UXXXXXXXX`	Code point (8 hex digits)	`"\U0001F600"`	😀
`\N{name}`	Character by Unicode name	`"\N{GREEK SMALL LETTER PI}"`	`π`

`\r` and `\b` are real characters; what you *see* depends on the terminal (`repr("ABC\b")` is `'ABC\x08'`).

```
print("Tab\tseparated\nnew line, quote: \" backslash: \\")
print("\x41\101\u00e9\N{GREEK SMALL LETTER PI}")   # A A é π
path = r"C:\new\table"                              # raw: backslashes kept literally
print(path, len(path), len("C:\new\table"))         # raw has 12 chars, cooked has 10
```

Output:

```
Tab	separated
new line, quote: " backslash: \
AAéπ
C:\new\table 12 10
```

Without `r`, `"C:\new\table"` silently contains a newline and a tab.

Common mistakes

- Windows paths without `r` — or better, use forward slashes or `pathlib.Path`.
- Unknown escapes like `"\d"`: kept, but `SyntaxWarning` since 3.12. Write regexes raw: `r"\d+"` (`"\b"` without `r` is a backspace, not a word boundary).
- A raw string can't end in one backslash: `r"C:\"` is a syntax error. Use `"C:\\"`.

TL;DR

Backslash escapes produce special characters; `r"..."` disables them — use it for regexes and Windows paths.

4.6 String formatting: f-strings, the format-spec mini-language, `str.format`, `%`

What it is

Formatting builds a string from a template plus values.

Style	Example	Introduced	Use today
f-string (formatted string literal)	`f"{name} is {age}"`	3.6	Default choice
`str.format()`	`"{} is {}".format(name, age)`	2.6 / 3.0	Templates stored as data
`%` (printf-style)	`"%s is %d" % (name, age)`	Original	Legacy code, `logging` calls
`string.Template`	`Template("$name is $age")`	2.4	Templates written by untrusted users

Python 3.14 added **t-strings** (`t"..."`), producing a `string.templatelib.Template` object (unrelated to `string.Template`) instead of a `str`, so libraries can escape values safely (e.g. for HTML or SQL). Still niche; f-strings remain the everyday tool.

Why it matters

Reports, log lines, money with two decimals, padded columns, hex or percent. The **format-spec mini-language** after `:` is shared by f-strings, `str.format()` and `format()` — learn it once.

How it works — f-strings

1. Prefix `f`; inside `{ }` put **any expression**.
2. Optional **conversion**: `!r` (`repr()`, the unambiguous developer form), `!s` (`str()`), `!a` (`ascii()`).
3. Optional `:format_spec`.
4. `{expr=}` (3.8+) prints the expression text *and* its value — a debugging shortcut.
5. `{{` / `}}` for literal braces.

```python
name, age = "Asha", 31
print(f"{name} is {age} years old.")
print(f"Next year: {age + 1}, upper: {name.upper()}")
print(f"{name!r} vs {name!s}")               # !r calls repr(), !s calls str()
width = 10
print(f"[{name:>{width}}]")                   # nested replacement field for the width
total = 42
print(f"{total = }")                          # self-documenting (3.8+)
print(f"{{literal braces}}")
```

Output:

```
Asha is 31 years old.
Next year: 32, upper: ASHA
'Asha' vs Asha
[      Asha]
total = 42
{literal braces}
```

Since 3.12 (PEP 701) f-strings are parsed by the real grammar: you can reuse the same quote type inside braces (`f"{d["key"]}"`), use backslashes, and split expressions over lines. On ≤ 3.11 use the other quote type inside.

How it works — the format-spec mini-language

```
[[fill]align][sign][#][0][width][grouping][.precision][type]
```

Part	Options	Meaning
fill	any character	Pad character (needs an align)
align	`<` `>` `^` `=`	Left, right, centre, pad after the sign
sign	`+` `-` space	Always sign / only minus (default) / space for positives
`#`		Alternate form: `0x`, `0o`, `0b` prefixes
`0`		Zero-pad numbers
width	integer	Minimum total width
grouping	`,` or `_`	Thousands separator
.precision	integer	Digits after the point (`f`, `%`), significant digits (`g`), max chars (strings)
type	`s d b o x X c e f g %`	String, decimal, binary, octal, hex, char, scientific (`1.2e+06`), fixed, general, percent

Defaults: strings left-aligned, numbers right-aligned.

Output of `code/02_formatting.py` (spec → result; `pi, n, ratio = 3.14159265, 1234567, 0.4567`):

```
{:.2f}                -> '3.14'
{:10.3f}              -> '     3.142'
{:<8}|                -> 'ab      |'
{:^8}|                -> '   ab   |'
{:*>8}                -> '******ab'
{:,}                  -> '1,234,567'
{:_}                  -> '1_234_567'
{:.1%}                -> '45.7%'
{:e}                  -> '1.234567e+06'
{:+d}                 -> '+42'
{:05d}                -> '00042'
{:b} {:o} {:x} {:X}   -> '11111111 377 ff FF'
{:#x}                 -> '0xff'
{:%Y-%m-%d}           -> '2024-03-09'
```

`f"{pi:10.3f}"`: fixed point, 3 decimals → `3.142` (5 chars), width 10, right-aligned → `' 3.142'`.

The last row: `date` passes everything after `:` to `strftime` (formatting with codes like `%Y`). Your classes can do the same by implementing `__format__`.

How it works — `str.format()` and `%`

```
print("{} + {} = {}".format(2, 3, 5))
print("{1} before {0}".format("A", "B"))
print("{name} has {n} cats".format(name="Li", n=2))
user = {"name": "Li", "n": 2}
print("{name} has {n} cats".format(**user))
template = "Dear {who}, your balance is {amt:,.2f}"   # reusable template stored as data
print(template.format(who="Sam", amt=12345.6))
print("%s is %d years, pi~%.2f" % ("Asha", 31, 3.14159))  # old printf style
```

Output:

```
2 + 3 = 5
B before A
Li has 2 cats
Li has 2 cats
Dear Sam, your balance is 12,345.60
Asha is 31 years, pi~3.14
```

- `{}` positional, `{1}` by index, `{name}` by keyword; `**user` unpacks a dict.
- `str.format()` shines when the **template is data** (from a file or config); f-strings must be in source.
- `%`: `%s`, `%d`, `%f`, `%x`, `%%`. A single tuple value misbehaves — one reason newer styles won.

Trade-offs

	f-string	str.format	%	string.Template
Readability	Best	Good	Fair	Good
Speed	Fastest (compiled into the function)	Slower (call + parse)	Similar to format	Slowest
Template can be data	No	Yes	Yes	Yes
Safe for user-written templates	n/a	**No** — `{obj.__class__...}` reaches attributes	Mostly	**Yes** — only `$name` substitution
Lazy formatting in logging	No	No	Yes (`logger.info("%s", x)`)	No

When not to use it

- **Logging:** `logger.info("user %s logged in", user)`, not an f-string. `logging` formats only if the level is enabled, and log aggregators (services collecting logs from many machines) group by the unformatted template.
- **SQL:** never format values into SQL. Use driver parameters (`cursor.execute("... WHERE id = ?", (user_id,))`) to prevent SQL injection (user input executed as database commands).
- **User-supplied templates:** `user_template.format(**objects)` can walk attributes (`{config.__class__.__init__.__globals__}`) and leak data. Use `string.Template` or a sandboxed template engine.

Common mistakes

- Forgetting `f` → braces print literally.

- `f"{price:.2}"` = **2 significant digits**, not two decimals; use `:.2f` (and `decimal.Decimal` for real money).
- `"%s" % some_tuple` → "not all arguments converted"; write `"%s" % (some_tuple,)`.

Real-world examples

- `logging` merges message arguments `%`-style; `style="{"` changes only the *Formatter's* layout string, not messages.
- `datetime`, `Decimal`, `ipaddress` implement `__format__`: `f"{now:%H:%M}"`.
- Config and email templates stored as data, filled with `str.format` or a template engine such as Jinja2.

In the interview

> **30-second answer:** "Use f-strings by default: readable and fastest. After the colon is the format-spec mini-language — fill, align, sign, width, grouping, precision, type — e.g. `{x:>10,.2f}`. Use `str.format` when the template is stored as data, `%` for logging's lazy formatting, and `string.Template` for untrusted templates."

- *`f"{x=}`? → `x=` followed by `repr(x)` (3.8).
- *Support `f"{obj:custom}`? → Implement `__format__(self, spec)`.
- *Why not f-strings in log calls?* → Built even when the level is disabled; prevents grouping by template.

Self-check

1. `f"{7:03d}"`? → `'007'`. 2. `f"{0.256:.0%}"`? → `'26%'`. 3. `f"{'hi':=^6}"`? → `'==hi=='`.

TL;DR

f-strings everywhere; learn `[[fill]align][sign][#][0][width][,][.prec][type]`; `str.format` for stored templates; `%` for logging; `Template` for untrusted input.

4.7 String methods

What it is

A **method** is a function attached to an object (`text.upper()`). `str` has about 47. None change the original: transformers **return a new string**, checkers return `True`/`False` or a number. They run in optimised C — far faster than hand-written loops for cleaning CSV fields, validating input, parsing logs.

How it works — the reference table

Case conversion

Method	Returns
`upper()` / `lower()`	All letters upper / lower
`capitalize()`	First character upper, rest lower
`title()`	First letter of each word upper, rest lower
`swapcase()`	Each letter's case flipped
`casefold()`	Aggressive lowercase for caseless comparison (`"ß"` → `"ss"`)

Alignment and padding

Method	Returns
`center(w, fill=" ")`	Centred in width `w`
`ljust(w, fill)` / `rjust(w, fill)`	Left / right justified
`zfill(w)`	Left-padded with zeros (keeps a leading sign)
`expandtabs(n)`	Tabs replaced by spaces

Stripping, splitting, joining

Method	Returns
`strip(chars)` / `lstrip` / `rstrip`	Remove whitespace (or any of `chars`) from both ends / left / right
`split(sep=None, maxsplit=-1)`	List of pieces; no `sep` → split on whitespace runs, drop empties
`rsplit(sep, maxsplit)`	Like `split`, counting `maxsplit` from the right
`splitlines()`	Lines (`\n`, `\r\n`, others) without breaks
`partition(sep)` / `rpartition(sep)`	`(before, sep, after)` around first / last `sep`
`sep.join(iterable)` (any loopable object)	Items (must be strings) glued with `sep`
`removeprefix(p)` / `removesuffix(s)`	Without that exact prefix/suffix, if present (3.9+)

Boolean checks (all `False` for `""`, except `isprintable()` and `isascii()`)

Method	`True` when…
`isalnum()`	Every character is a letter or number (Unicode sense)
`isalpha()`	Every character is a letter
`isdecimal()`	Every character is a decimal digit 0–9 (in any script, e.g. Arabic-Indic)
`isdigit()`	Decimal digits plus digits like superscript `²`
`isnumeric()`	Any numeric character, including `½`
`islower()` / `isupper()`	All cased characters lower / upper, at least one
`istitle()`	Upper only after uncased chars, lower only after cased ones
`isspace()`	Only whitespace, at least one
`isidentifier()`	Valid Python identifier (keywords included — check `keyword.iskeyword` too)
`isprintable()`	No non-printables (`\t`, `\n` count as non-printable); `True` for `""`
`isascii()`	All ASCII (the 128-character English set; 3.7+); `True` for `""`
`startswith(p)` / `endswith(s)`	Starts / ends with; accepts a **tuple** of options

Find, count, replace, translate

Method	Returns
`count(sub, start, end)`	Non-overlapping occurrences
`find(sub, start, end)` / `rfind`	Lowest / highest index, or `-1`
`index(sub, start, end)` / `rindex`	Same, but raises `ValueError`
`replace(old, new, count=-1)`	Copy with occurrences replaced (all, or first `count`)
`str.maketrans(...)` + `translate(table)`	Map/delete many single characters in one pass

Optional `start, end` (`beg`, `end` in older docs) search inside `s[start:end]` but report indexes in the full string.

Code — each group in action

```
s = "pYthon is FUN"
print(s.upper(), "|", s.lower(), "|", s.capitalize(), "|", s.title(), "|", s.swapcase())
print("they're bill's".title())           # title() treats ' as a word break
print("Straße".lower(), "Straße".casefold())  # casefold is the aggressive, caseless form
print("[" + "Menu".center(16, "-") + "]")
print("[" + "Menu".ljust(10) + "]", "[" + "Menu".rjust(10, ".") + "]", "42".zfill(5))
```

Output:

```
PYTHON IS FUN | python is fun | Python is fun | Python Is Fun | PyTHON IS fun
They're Bill'S
straße strasse
[------Menu------]
[Menu      ] [......Menu] 00042
```

With odd padding, which side of `center` gets the extra character depends on the lengths (`"ab".center(5)` is `' ab '`, `"abc".center(6)` is `' abc '`), whereas f-string `{:^w}` always puts it on the right. Pick one if columns must line up.

```
text = "  Hello, Python! Welcome to programming.  "
print(text.strip().split())               # any whitespace, runs collapsed
print(text.strip().rsplit(" ", 1))        # split once, from the right
print("a,,b".split(","), "a  b".split(" "), "a  b".split())
print("k=v=w".partition("="), "k=v=w".rpartition("="), "abc".partition("="))
print("line1\nline2\r\nline3".splitlines())
print("xxhixx".strip("x"), "--hi".lstrip("-"), "hi--".rstrip("-"))
print("report_2024.csv".removesuffix(".csv"), "Mr. Smith".removeprefix("Mr. "))
print("abcabc".strip("ab"))               # strip removes a SET of chars, not a prefix
print("-".join(["2024", "03", "09"]), ", ".join(str(n) for n in [1, 2, 3]))
```

Output:

```
['Hello,', 'Python!', 'Welcome', 'to', 'programming.']
['Hello, Python! Welcome to', 'programming.']
['a', '', 'b'] ['a', '', 'b'] ['a', 'b']
('k', '=', 'v=w') ('k=v', '=', 'w') ('abc', '', '')
['line1', 'line2', 'line3']
hi hi hi
report_2024 Smith
cabc
2024-03-09 1, 2, 3
```

Memorise:

- `split()` with **no argument** collapses whitespace runs and never yields empties; `split(" ")` yields `''` between adjacent separators.
- `strip("ab")` removes **any** of `a`/`b` from the ends — not the prefix `ab`; that's `removeprefix`.

```
samples = ["Hello123", "Hello", "123", "½", "²", "   ", "Hello World", "my_var", "3var"]
# prints one row per sample: isalnum isalpha isdecimal isdigit isnumeric isspace istitle isidentifier
```

Output (T = True, F = False):

```
value          alnum alpha decim digit numer space title ident
'Hello123'         T     F     F     F     F     F     T     T
'Hello'            T     T     F     F     F     F     T     T
'123'              T     F     T     T     T     F     F     F
'½'                T     F     F     F     T     F     F     F
'²'                T     F     F     T     T     F     F     F
'   '              F     F     F     F     F     T     F     F
'Hello World'      F     F     F     F     F     F     T     F
'my_var'           F     F     F     F     F     F     F     T
'3var'             T     F     F     F     F     F     F     F
True True
```

Nested sets: **decimal ⊂ digit ⊂ numeric**. To validate "can `int()` take this?", use `isdecimal()` (`int("²")` fails though `"²".isdigit()` is `True`). With signs, spaces or decimals, just `try: int(s)` / `float(s)`.

```python
t = "hello world, hello universe, hello multiverse"
print(t.count("hello"), t.find("hello"), t.rfind("hello"), t.find("bye"))
print(t.find("hello", 1))                  # search starting at index 1
try:
    t.index("bye")
except ValueError as e:
    print("ValueError:", e)
print(t.replace("hello", "hi", 2))         # only first 2 occurrences

table = str.maketrans({"a": "4", "e": "3", "o": "0", "!": None})  # None = delete
print("hello, leet coders!".translate(table))
```

Output:

```
3 0 29 -1
13
ValueError: substring not found
hi world, hi universe, hello multiverse
h3ll0, l33t c0d3rs
```

Cost of common operations (n = length of the string)

Operation	Time	Note
`len(s)`	O(1)	Length is stored
`s[i]`	O(1)	Fixed-width storage (4.9)
`s[a:b]`	O(b − a)	Copies the slice
`s + t`	O(len s + len t)	Always a new string
`sub in s`, `find`, `count`, `replace`	O(n) typical	Optimised C search
`split`, `strip`, `upper`, `translate`	O(n)	One pass
`sep.join(parts)`	O(total length)	Pre-computes size, one copy
`hash(s)`	O(n) first time, then O(1)	Cached on the object
`s == t`	O(n) worst case	Stops at first difference; O(1) if lengths differ

Common mistakes

- `if s.find("x"):` is **wrong** — `0` (match at start) is falsy, `-1` (absent) truthy. Use `if "x" in s:`.
- `lower()` for international caseless comparison — prefer `casefold()`.
- `title()` mangles apostrophes (`"They'Re"`); `string.capwords()` splits on whitespace only.

- `",".join([1, 2])` → `TypeError`; use `",".join(map(str, nums))` (`map` applies `str` to each item lazily, like a generator — Chapter 7).

Real-world examples

- `line.rstrip("\n").split(",")` in countless scripts (use `csv` when fields contain quoted commas).
- `url.removeprefix("https://")`, `filename.endswith((".jpg", ".png"))`.
- Search boxes normalise with `query.strip().casefold()`.

In the interview

> **30-second answer:** "String methods never mutate; they return new strings. Key groups: case (`lower`, `casefold`), cleaning (`strip`, `removeprefix`), splitting/joining (`split`, `partition`, `join`), checking (`isdecimal`, `startswith` with tuples) and searching (`in`, `find` returns −1, `index` raises). Most are O(n) and implemented in C."

- *`find` vs `index`?* → `-1` vs `ValueError` when absent.
- *`split()` vs `split(" ")`?* → No-arg collapses whitespace runs and drops empty fields.
- *Remove all punctuation fast?* → `s.translate(str.maketrans("", "", string.punctuation))`.

Self-check

1. `"a-b-c".rsplit("-", 1)`? → `['a-b', 'c']`. 2. `"Menu".center(20, "-")`? → `'--------Menu--------'`.

TL;DR

Methods return new strings. Memorise `strip/split/join/partition`, `startswith(tuple)`, `in` over `find`, `casefold` for comparisons, `isdecimal` for digits, `translate` for bulk mapping.

4.8 Joining strings efficiently

What it is

To build a big string from many pieces, collect them in a list and call `"".join(pieces)` once.

The problem it solves

Immutability means `s = s + piece` allocates a new string and copies **all** existing characters each time: 1 + 2 + … + n = n(n+1)/2 copies — **quadratic**, O(n²). `join` sums the lengths, allocates once, copies each piece once — **linear**, O(n).

How it works

1. Append pieces to a list (amortised O(1) — occasionally slow, constant on average; Chapter 5).
2. `sep.join(list)`: pass 1 adds up lengths; pass 2 copies into one pre-sized buffer.
3. Stream-style alternative: `io.StringIO` (in-memory text file) — `write()` pieces, then `getvalue()`.

CPython's trick: if the left string of `+=` has **no other references**, it may resize it in place, so the naive loop looks fast. Implementation detail — PyPy doesn't do it, and a second reference defeats it. The benchmark shows both:

```python
def plus_equal(n):          # naive, but CPython's in-place trick rescues it
    s = ""
    for _ in range(n):
        s += PIECE
    return s

def plus_equal_shared(n):   # same, but a second reference defeats the trick
    s = ""
    alias = []
    for _ in range(n):
        alias.append(s)
        s += PIECE
        alias.pop()
    return s

def join_list(n):
    parts = []
    for _ in range(n):
        parts.append(PIECE)
    return "".join(parts)
```

Output (`code/05_join_performance.py`, ms on the author's laptop — yours differ, the shape won't):

```
     n         plus_equal  plus_equal_shared          join_list          string_io
  2000            0.07 ms            0.23 ms            0.04 ms            0.04 ms
  5000            0.16 ms            0.89 ms            0.09 ms            0.10 ms
 10000            0.34 ms            3.06 ms            0.17 ms            0.20 ms
 20000            0.69 ms           18.84 ms            0.35 ms            0.41 ms
 40000            1.36 ms           81.27 ms            0.69 ms            0.82 ms
```

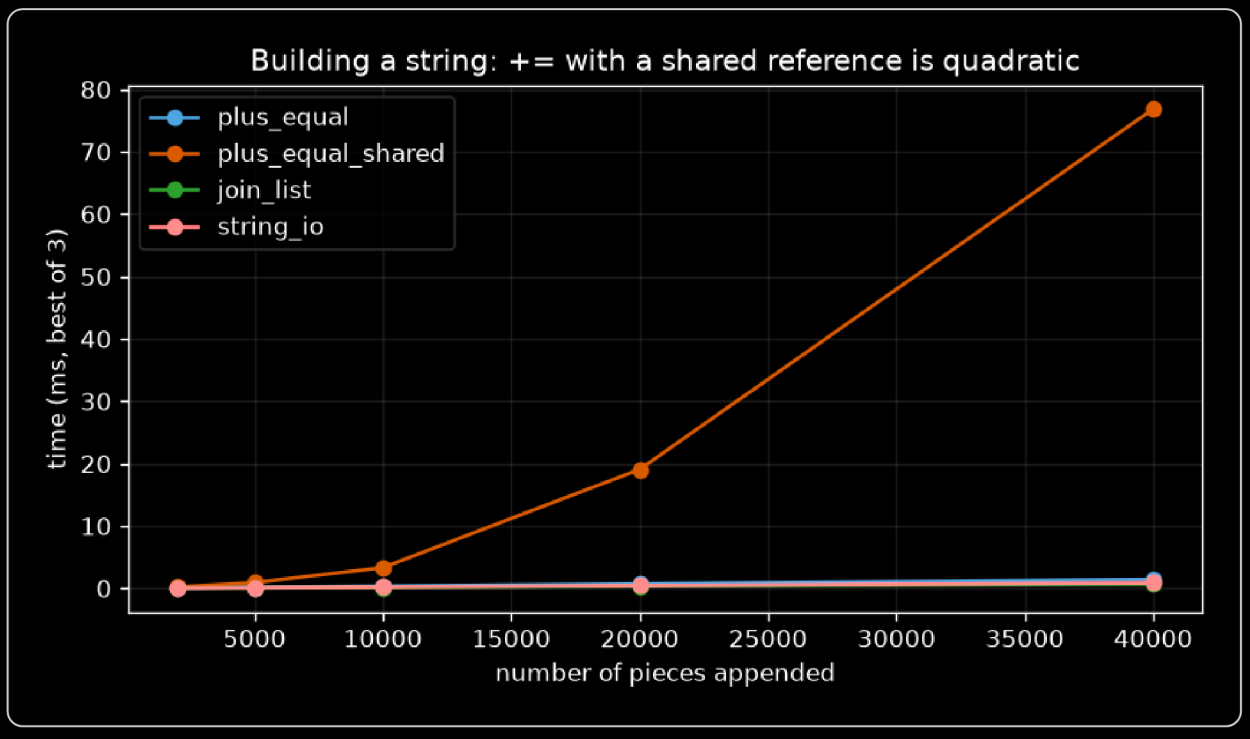


Figure — doubling n doubles `join` / `StringIO` time (linear) but roughly quadruples `+=` when the optimisation can't apply (quadratic).

Trade-offs

Approach	Complexity	Readability	Notes
`a + b + c` (few pieces)	Fine	Best	OK for a handful
f-string	Fine	Best	Fixed templates
`+=` in a loop	O(n²) worst case	Good	Fast only by CPython accident
`"".join(list_or_generator)`	O(n)	Good	The standard idiom
`io.StringIO`	O(n)	Good	When code "writes" output in many places

When not to use it

Two or three concatenations: `first + " " + last` or an f-string is clearer. The join rule is for loops and large inputs.

Common mistakes

- Joining numbers (`TypeError`) — `map(str, nums)`.
- `"".join([x for x in items])` when `"".join(items)` works.

In the interview

> **30-second answer:** "Repeated `+=` on an immutable string copies everything each time, so it's O(n²). Collect parts in a list and `''.join` once — O(n), because join sizes the result first and copies each part once."

- *My `+=` benchmark is fast?* → CPython resizes in place for single-reference strings; not guaranteed or portable.
- *Generator or list in `join`?* → `join` converts to a sequence anyway, so a list comprehension is slightly faster; both fine.

TL;DR

Many pieces → list + `"".join(...)` (or `StringIO`). A few → `+` or f-string.

4.9 Unicode vs bytes, encodings and UTF-8

What it is

- A **character** is an abstract symbol: "A", "é", "€", "😀".
- **Unicode** assigns each a **code point**, written `U+20AC` for "€". `str` is a sequence of code points; `ord(ch)` / `chr(n)` convert.
- **Bytes** are raw 8-bit values (0–255) — what files, sockets and disks hold. `bytes` is immutable, written `b"..."`.
- An **encoding** turns code points into bytes (`str.encode`) and back (`bytes.decode`).
- **UTF-8** dominates the web and Linux/macOS: variable-length, 1–4 bytes per code point, identical to ASCII for the first 128.

Read with the wrong encoding, text becomes gibberish ("mojibake").

Why it matters

Every boundary — reading a file, calling an HTTP API, writing to a database — converts between `str` and `bytes`. A wrong encoding crashes (`UnicodeDecodeError`) or, worse, silently corrupts text.

How it works

1. Work with `str` inside the program.
2. **Decode** incoming bytes early, **encode** outgoing text late (the "Unicode sandwich").
3. Name the encoding: `open(path, encoding="utf-8")`. Otherwise `open()` uses the locale encoding (OS language/region default), which varies by machine. Python 3.15 (due October 2026) makes UTF-8 the default (PEP 686) — still name it while supporting 3.12–3.14.
4. UTF-8 lengths:

Code points	Bytes	Example
U+0000 – U+007F (ASCII)	1	`A` → `41`
U+0080 – U+07FF	2	`é` → `C3 A9`
U+0800 – U+FFFF	3	`€` → `E2 82 AC`
U+10000 – U+10FFFF	4	`😀` → `F0 9F 98 80`

```
for ch in ["A", "é", "€", "😀"]:
    utf8 = ch.encode("utf-8")
    print(f"{ch!r}: ord={ord(ch):>6}  U+{ord(ch):04X}  utf-8={utf8!r:<22} {len(utf8)} byte(s)")

s = "café"
b = s.encode("utf-8")
print(type(b), b, "len(str)=", len(s), "len(bytes)=", len(b))
print(b.decode("utf-8"))
print(repr(b.decode("latin-1")))                    # wrong codec -> mojibake, no error!
try:
    b.decode("ascii")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
print(s.encode("ascii", errors="replace"), s.encode("ascii", errors="ignore"))
print(b"\xff\xfeok".decode("utf-8", errors="replace"))
print(b"abc"[0], list(b"hi"), bytes([104, 105]))        # indexing bytes gives ints
```

Output:

```
'A': ord=    65  U+0041  utf-8=b'A'                  1 byte(s)
'é': ord=   233  U+00E9  utf-8=b'\xc3\xa9'           2 byte(s)
'€': ord=  8364  U+20AC  utf-8=b'\xe2\x82\xac'       3 byte(s)
'😀': ord=128512  U+1F600  utf-8=b'\xf0\x9f\x98\x80'    4 byte(s)
<class 'bytes'> b'caf\xc3\xa9' len(str)= 4 len(bytes)= 5
café
'cafÃ©'
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 3: ordinal not in range(128)
b'caf?' b'caf'
��ok
97 [104, 105] b'hi'
```

- `"café"` is 4 characters but 5 UTF-8 bytes; database column limits and HTTP `Content-Length` count **bytes**.
- Decoding UTF-8 as Latin-1 (an old 256-character Western-European encoding) **doesn't fail** — it yields `cafÃ©`, classic mojibake.
- `errors=`: `"strict"` (default, raise), `"replace"` (`?` / `�`), `"ignore"` (drops data silently), `"backslashreplace"`, more.
- Indexing `bytes` gives an **int**.

Normalization. `é` can be one code point (U+00E9) or `e` + combining accent (U+0301): identical-looking, unequal. `unicodedata.normalize("NFC", s)` converts to the composed form:

```
a = "caf\u00e9"; b = "cafe\u0301"
print(a, b, a == b, len(a), len(b))          # café café False 4 5
nfc = unicodedata.normalize("NFC", b)
print("after NFC:", a == nfc, len(nfc))      # after NFC: True 4
```

How CPython stores `str`. Since 3.3 (PEP 393) each string uses the narrowest fixed width fitting its widest character: 1 byte/char for Latin-1, 2 up to U+FFFF, 4 otherwise. Fixed width makes `s[i]` O(1) — CPython does not store UTF-8 internally.

Trade-offs

	`str`	`bytes`	`bytearray`
Holds	Code points	Raw bytes 0–255	Raw bytes 0–255
Mutable	No	No	Yes
Literal	`"héllo"`	`b"hello"` (ASCII only in literal)	`bytearray(b"...")`
Use for	All text logic	Files, network, hashing, cryptography	Building/patching binary buffers

Encoding	Size per char	ASCII-compatible	Notes
UTF-8	1–4 bytes	Yes	Web, JSON (required by RFC 8259 for data exchanged between systems), Python source
UTF-16	2 or 4 bytes	No	Windows APIs, Java and JavaScript strings
Latin-1 (ISO-8859-1)	1 byte	Yes	256 characters; never fails to decode, so hides errors
ASCII	1 byte	—	128 characters

Common mistakes

- `"abc" + b"def"` → `TypeError`; Python 3 never guesses an encoding.
- Omitting `encoding=` in `open()` — works on your Mac, breaks on a Windows server.
- `len()` for user-visible characters: a flag or "👍🏽" is several code points. "Grapheme" (perceived character) counting needs `grapheme` or `regex` (`\X`).
- `errors="ignore"` to silence an error — silent data loss.

Real-world examples

- `requests`: `response.content` (bytes) vs `response.text` (decoded using the declared or guessed encoding).
- `json.dumps(..., ensure_ascii=False)` keeps non-ASCII readable instead of `\u00e9`.
- Python source is UTF-8 by default (PEP 3120), so `name = "Zoë"` needs no `# -*- coding: … -*-` comment.

In the interview

> **30-second answer:** "`str` is a sequence of Unicode code points; `bytes` is raw data. Encode text to bytes at the output boundary and decode bytes to text at the input boundary, always naming the encoding — normally UTF-8, which is variable-length (1–4 bytes) and ASCII-compatible."

- *`len('café'.encode())` is 5?* → `é` needs two bytes.
- *Mojibake?* → Text decoded with the wrong encoding (UTF-8 read as Latin-1 → `cafÃ©`).
- *Equal-looking strings unequal?* → Composed vs decomposed code points; normalise with NFC.

Self-check

1. Bytes for `€` in UTF-8? → 3. 2. Can `bytes.decode("latin-1")` raise? → No — every byte maps, which hides mistakes.

TL;DR

Text is `str` (code points); storage and network are `bytes`. Decode early, encode late, always say `utf-8`, normalise before comparing user text.

4.10 Regular expressions — a practical introduction

What it is

A **regular expression** (regex) is a pattern language for text *shapes*: "one or more digits", "a date like 2024-03-09". `re` compiles a pattern and searches strings. String methods handle fixed text; regex makes extracting emails, validating postcodes, parsing log fields or collapsing whitespace one-liners.

How it works — the core syntax

Pattern	Matches
`.`	Any character except newline
`\d` `\w` `\s`	Digit, word char (letter, digit, `_`), whitespace
`\D` `\W` `\S`	The opposites
`[abc]` `[a-z]` `[^0-9]`	One char from a set / range / not in the set
`^` `$`	Start / end of string (each line with `re.MULTILINE`); `$` also matches before a final `\n`
`\b`	Word boundary
`*` `+` `?`	0+, 1+, 0 or 1 (greedy — as much as possible)
`{3}` `{2,5}`	Exactly 3, 2 to 5
`*?` `+?`	Lazy — as little as possible
`a|b`	`a` or `b`
`( … )`	Capturing group (remembers its text)
`(?P<name> … )`	Named group
`(?: … )`	Non-capturing group

The main functions

Function	Does
`re.search(p, s)`	First match **anywhere** → `Match` or `None`
`re.match(p, s)`	Match only at the **start**
`re.fullmatch(p, s)`	**Whole string** must match
`re.findall(p, s)`	All matches (strings, or tuples with several groups)
`re.finditer(p, s)`	Iterator of `Match` objects (positions + groups)
`re.sub(p, repl, s)`	Replace; `repl` may use `\1` (a *backreference* to group 1's text) or be a function
`re.split(p, s)`	Split on a pattern
`re.compile(p)`	`Pattern` object with the same methods

A **`Match` object** has `.group()`, `.group(1)` / `.group("name")`, `.groups()`, `.groupdict()`, `.start()`, `.end()`, `.span()`.

Code

```
LOG = """2024-03-09 12:01:07 ERROR db timeout user=alice
2024-03-09 12:01:09 INFO  login ok user=bob
2024-03-10 08:15:00 ERROR disk full user=carol"""

s = "Order #123 shipped"
print(re.match(r"\d+", s))                  # None: match only tries at position 0
m = re.search(r"\d+", s)                    # scans the whole string
print(m, m.group(), m.start(), m.end(), m.span())
print(re.fullmatch(r"\d{3}", "123") is not None, re.fullmatch(r"\d{3}", "1234"))

print(re.findall(r"user=(\w+)", LOG))        # one group -> list of strings
print(re.findall(r"(\d{4})-(\d\d)-(\d\d)", "2024-03-09 and 2025-01-02"))  # tuples
for m in re.finditer(r"ERROR (.+?) user=(\w+)", LOG):
    print("  ", m.group(1), "->", m.group(2), "at", m.start())
```

Output:

```
None
<re.Match object; span=(7, 10), match='123'> 123 7 10 (7, 10)
True None
['alice', 'bob', 'carol']
[('2024', '03', '09'), ('2025', '01', '02')]
   db timeout -> alice at 20
   disk full -> carol at 112
```

```
pat = re.compile(r"(?P<date>\d{4}-\d\d-\d\d) (?P<time>[\d:]+) (?P<level>\w+)")
m = pat.search(LOG)
print(m.group("level"), m.group(1), m.groups())
print(m.groupdict())
print(re.search(r"(?:ab)+", "xxababab").group())   # (?:...) groups without capturing

print(re.sub(r"\s+", " ", "too    many   spaces"))
print(re.sub(r"(\d{4})-(\d\d)-(\d\d)", r"\3/\2/\1", "due 2024-03-09"))   # backrefs
print(re.sub(r"\d+", lambda m: str(int(m.group()) * 2), "3 apples, 10 pears"))
print(re.split(r"[,;]\s*", "a, b;c ,d"))

html = "<b>bold</b><i>it</i>"
print(re.findall(r"<.+>", html))     # greedy: grabs as much as possible
print(re.findall(r"<.+?>", html))    # lazy: as little as possible
print(re.escape("price: $5.00 (approx)"))
```

Output:

```
ERROR 2024-03-09 ('2024-03-09', '12:01:07', 'ERROR')
{'date': '2024-03-09', 'time': '12:01:07', 'level': 'ERROR'}
ababab
too many spaces
due 09/03/2024
6 apples, 20 pears
['a', 'b', 'c ', 'd']
['<b>bold</b><i>it</i>']
['<b>', '</b>', '<i>', '</i>']
price:\ \$5\.00\ \(approx\)
```

The date rewrite captures year, month, day as groups 1–3 and `r"\3/\2/\1"` writes them reversed (raw, so `\3` isn't an octal escape). `re.escape` escapes every special character — use it when inserting user text into a pattern.

Trade-offs

Task	Best tool
Fixed substring test	`in`, `startswith`, `endswith` — faster, clearer
Split on one fixed separator	`str.split`
Variable shapes, extraction, validation	`re`
HTML / XML	A real parser (`html.parser`, `lxml`, BeautifulSoup) — regex can't handle nesting
Strict email / URL validation	A dedicated library or `urllib.parse`

When not to use it

When a string method does the job, for nested formats (HTML, JSON, code), or with user-supplied patterns (see backtracking).

Common mistakes

- **Forgetting `r"..."`**: `"\b"` is a backspace.
- **`match` vs `search`**: `match` checks only the start; validate with `fullmatch`.
- **Greedy `.*`** swallowing too much — `.*?` or a negated class `[^<]*`.
- **Catastrophic backtracking:** Python's engine backtracks (retries alternatives when a path fails), so nested quantifiers (repeat counts like `*`, `+`, `{m,n}`) such as `(a+)+$` can take exponential time on near-matches, and simpler ones (`.*.*=.*`, `\s+$` on a long run of spaces) can go quadratic — the basis of ReDoS (regular-expression denial-of-service).
- `m.group()` without checking `m is not None` → `AttributeError`.

Real-world examples

- 2 July 2019: one new WAF (web application firewall) rule containing `.*(?:.*=.*)` backtracked so heavily it pinned CPUs at 100 % across Cloudflare's network — a global outage of about 27 minutes (Cloudflare post-mortem).
- 20 July 2016: Stack Overflow down 34 minutes — a trimming regex (`[\s\u200c]+$`) hit a post with ~20,000 consecutive whitespace characters and backtracked quadratically, O(n²), ~200 million steps for one string.
- Django routes URLs with regex via `re_path()` (before Django 2.0, `url()` regex routes were the only option); Logstash's grok filter is built on named regex patterns.

In the interview

> **30-second answer:** "`re.search` finds the first match anywhere, `match` only at the start, `fullmatch` the whole string. `findall` returns strings or tuples of groups, `finditer` gives Match objects, `sub` replaces with backrefs or a function. Use raw strings, lazy quantifiers when needed, named groups for readability, and beware catastrophic backtracking."

- *Why `re.compile`?* → Readability and reuse; `re` caches recent patterns, so speed gain is small.
- *`findall` with two groups?* → List of 2-tuples.
- *Non-capturing group?* → `(?:...)`.

Self-check

1. `re.match(r"\d", "a1")`? → `None`.
2. `re.sub(r"(\w+)@(\w+)", r"\2 at \1", "me@home")`? → `'home at me'`.

TL;DR

`search` / `match` / `fullmatch` for one match, `findall` / `finditer` for many, `sub` to replace, `()` to capture, `(?P<name>)` to name, `r""` always, keep patterns simple.

Chapter TL;DR

Topic	Remember
Basics	`str` is an immutable sequence of Unicode code points; zero-based and negative indexing.
Immutability	Methods and operators return new strings; rebind the name. Strings are hashable.
Operators	`+`, `*`, `in`; comparisons by code point; use `==`, not `is`.
Slicing	`[start:stop:step]`, stop excluded, never raises, `[::-1]` reverses.
Escapes	`\n \t \\ \' \" \xhh \uXXXX \N{...}`; `r"..."` for regex and paths.
Formatting	f-strings by default; `[[fill]align][sign][#][0][width][,][.prec][type]`; `format()` for stored templates, `%` for logging, `Template` for untrusted input.
Methods	`strip/split/join/partition/replace/startswith`; `in` not `find` for tests; `casefold` for comparisons; `isdecimal` for digits.
Building	Many pieces → list + `"".join`.
Unicode	Decode at input, encode at output, always `encoding="utf-8"`; normalise with NFC.
Regex	`search`, `match`, `fullmatch`, `findall`, `sub`, groups; raw strings; beware backtracking.

Chapter 5 — Lists, Tuples and Arrays

Python keeps items **in order** in several ways: the all-purpose `list`, the immutable `tuple`, the compact numeric `array.array`, and NumPy's `ndarray` (NumPy is the standard third-party library for fast numeric arrays). All support indexing, slicing, `len`, `in` and `for`, but they differ in mutability, memory, speed and safety. This chapter explains each, how it works underneath, and how to choose.

What you'll learn

1. The `array` module: type codes, insert/delete/search/update, methods.
2. Lists: creating, accessing, updating, joining, slicing, looping, operators.
3. Lists as dynamic arrays — the big-O cost of every operation.
4. Adding/removing items and the full list-method table.
5. Advanced slicing: steps, slice assignment, slice deletion.
6. List comprehensions.
7. Sorting: `sort()` vs `sorted()`, `key`, `reverse`, stability, multi-key sorts.
8. Copying: aliasing, shallow vs deep, the `[[0]*3]*3` trap.
9. Tuples: immutability, operators, unpacking (with `*`), joining, methods, records, `namedtuple`.
10. Choosing `list`, `tuple`, `array.array` or NumPy.

Runnable code is in `code/`. Outputs are real (Python 3.14; code runs on 3.12+). Timings are from one laptop — ratios hold.

5.1 The `array` module — typed, compact arrays

What it is

An **array** (standard-library `array` module) is a mutable sequence where **every element has the same C numeric type** (a fixed-size C number format) — all 32-bit ints, all 64-bit floats, etc. Values are stored as raw machine numbers side by side in one block (**contiguous** storage), not as separate Python objects. A list is a shelf of boxes holding anything; an array is an egg carton — uniform slots, packed densely.

Why it matters

For large volumes of plain numbers — sensor readings, audio samples, pixel values, binary file formats — an array uses a fraction of a list's memory and reads/writes files and sockets directly as bytes.

How it works

1. Create with a **type code**: `array.array("i", [10, 20, 30])`.
2. Each stored value is converted to that C type; wrong types → `TypeError`, out-of-range → `OverflowError`.
3. Indexing (incl. negative), slicing, `len`, `in`, `for` work as with lists.

Type codes (minimum sizes per the C standard; `itemsize` gives the real size)

Code	C type	Python type	Min bytes	Typical on 64-bit macOS/Linux
`'b'` / `'B'`	signed (allows negatives) / unsigned char	int	1	1 (−128…127 / 0…255)
`'h'` / `'H'`	signed / unsigned short	int	2	2
`'i'` / `'I'`	signed / unsigned int	int	2	4
`'l'` / `'L'`	signed / unsigned long	int	4	8 (4 on Windows)
`'q'` / `'Q'`	signed / unsigned long long	int	8	8
`'f'`	float	float	4	4
`'d'`	double	float	8	8
`'u'`	wchar_t (Unicode char)	str	2	4 — **deprecated** since 3.3, slated for removal
`'w'`	Py_UCS4 (Unicode char)	str	4	4 — added in 3.13 to replace `'u'`

```
a = array.array("i", [10, 20, 30, 40, 50])      # 'i' = C signed int
print(a, a.typecode, a.itemsize, "bytes per item")
print(a[0], a[1], a[-1], a[-2])
for x in a:
    print(x, end=" ")
try:
    a.append(3.5)                               # wrong type is rejected
except TypeError as e:
    print("TypeError:", e)
try:
    array.array("b", [200])                     # 'b' holds -128..127
except OverflowError as e:
    print("OverflowError:", e)
```

Output:

```
array('i', [10, 20, 30, 40, 50]) i 4 bytes per item
10 20 50 40
10 20 30 40 50
TypeError: 'float' object cannot be interpreted as an integer
OverflowError: signed char is greater than maximum
```

The four basic operations: insert, delete, search, update

> **Key term — Big-O notation:** how work grows with *n* elements. **O(1)**: constant. **O(n)**: linear. **O(k)**: linear in *k*, the size of the extra input. Growth, not seconds.

Insert (`insert`, shifts the rest right), **delete** (`pop(i)` / `remove(x)`, shift left), **search** (`index`) and **traverse** (`for x in a`) are O(n); **update** (`a[i] = x`) is O(1) — see the method table.

Array methods

Method	What it does	Cost
`append(x)`	Add at the end	O(1) amortised (constant on average)
`extend(iterable)`	Append every item (same type required)	O(k)
`insert(i, x)`	Insert before `i`	O(n)
`pop([i])`	Remove and return item `i` (default last)	O(1) last, O(n) otherwise
`remove(x)`	Delete first `x`	O(n)
`index(x)`	Position of first `x`	O(n)
`count(x)`	Occurrences of `x`	O(n)
`reverse()`	Reverse in place	O(n)
`clear()`	Remove all (**3.13+**; older: `del a[:]`)	O(1)
`tolist()` / `fromlist(l)`	To / append from a list	O(n)
`tobytes()` / `frombytes(b)`	Raw machine bytes out / in	O(n)
`tofile(f)` / `fromfile(f, n)`	Write / read raw values to a binary file	O(n)
`byteswap()`	Swap byte order (endianness) of every item	O(n)
`buffer_info()`	`(address, length)` of the underlying memory	O(1)

Arrays have **no** `.sort()` or `.copy()`. Sort with `sorted(a)` (returns a **list**) and rebuild with `array.array(code, sorted(a))`; copy with `a[:]`, `array.array(a.typecode, a)` or `copy.copy(a)`.

```
a = array.array("i", [1, 2, 3, 4, 5])
a.insert(1, 10);            print("insert(1, 10):", a.tolist())
removed = a.pop(1);         print("pop(1):", a.tolist(), "removed", removed)
print("index(3):", a.index(3))
a[2] = 20;                  print("a[2] = 20:", a.tolist())
a.append(4);                print("append(4):", a.tolist())
a.extend([5, 4, 6]);        print("extend:", a.tolist())
print("count(4):", a.count(4))
a.remove(4);                print("remove(4) (first 4 only):", a.tolist())
a.reverse();                print("reverse:", a.tolist())
print("sorted(a) -> list:", sorted(a))          # arrays have no .sort()
a = array.array("i", sorted(a));  print("back to array:", a)
b = copy.copy(a)                                # no .copy() method: use copy.copy or a[:]
b[0] = 99
print("copy independent:", a[0], b[0])
del a[:]                                        # empties it (array.clear() exists from 3.13)
print("emptied:", a)
```

Output:

```
insert(1, 10): [1, 10, 2, 3, 4, 5]
pop(1): [1, 2, 3, 4, 5] removed 10
index(3): 2
a[2] = 20: [1, 2, 20, 4, 5]
append(4): [1, 2, 20, 4, 5, 4]
extend: [1, 2, 20, 4, 5, 4, 5, 4, 6]
count(4): 3
remove(4) (first 4 only): [1, 2, 20, 5, 4, 5, 4, 6]
reverse: [6, 4, 5, 4, 5, 20, 2, 1]
sorted(a) -> list: [1, 2, 4, 4, 5, 5, 6, 20]
back to array: array('i', [1, 2, 4, 4, 5, 5, 6, 20])
copy independent: 1 99
emptied: array('i')
```

Memory. One million integers:

```
list of 1,000,000 ints :   36.0 MB (pointers + int objects)
array('i')             :    4.1 MB (raw 4-byte ints)
```

A list stores an 8-byte pointer (memory address) per slot **plus** a separate int object (28 bytes for small ints on 64-bit CPython) — ~9× more here.

Trade-offs, mistakes, and when not to use it

- **Not faster for arithmetic.** Every read creates an `int` / `float` object ("boxing"), so looping over an array is often *slower* than a list (5.14). For fast maths use NumPy.
- **Choose the type code carefully:** `'b'` overflows at 128; `'l'` is 4 bytes on Windows, 8 on Linux/macOS — use `'q'` for 64 bits everywhere.
- Arrays are for **compact storage and binary I/O**; lists for general work; NumPy for numerics.

TL;DR

`array.array(code, data)` = homogeneous, compact, mutable numbers; list-like indexing and most methods; no `sort` / `copy`; `clear()` from 3.13. A memory win, not a speed win.

5.2 Lists — the everyday sequence

What it is

A **list** is an ordered, **mutable** collection of objects of **any type**, written `[1, "Hello", 3.14, True]`, that grows and shrinks at run time. It's Python's default container: function results, file rows, API items, stacks (last-in-first-out) and queues (first-in-first-out) of work.

How it works — create, access, update

```
my_list = [1, "Hello", 3.14, True]          # mixed types are fine
print(my_list, len(my_list))
print(my_list[0], my_list[1], my_list[-1], my_list[-2])
my_list[1] = "World"
print("single update:", my_list)
my_list[1:3] = ["Changed", "Completely"]
print("slice update:", my_list)
print(list("abc"), list(range(4)), [0] * 3, [])
```

Output:

```
[1, 'Hello', 3.14, True] 4
1 Hello True 3.14
single update: [1, 'World', 3.14, True]
slice update: [1, 'Changed', 'Completely', True]
['a', 'b', 'c'] [0, 1, 2, 3] [0, 0, 0] []
```

Indexing works like strings; unlike strings you can **assign** to an index or slice (5.5). `list(iterable)` builds a list from anything iterable.

How it works — joining, slicing, looping

```
a, b = [1, 2, 3], [4, 5, 6]
c = a + b                                   # new list
a.extend(b)                                 # modifies a in place
print(c, a, c == a, c is a)
full = list(range(10))
print(full[2:7], full[-8:-3], full[::3], full[::-1])
for i, fruit in enumerate(["apple", "banana", "cherry"], start=1):
    print(i, fruit)
```

Output:

```
[1, 2, 3, 4, 5, 6] [1, 2, 3, 4, 5, 6] True False
[2, 3, 4, 5, 6] [2, 3, 4, 5, 6] [0, 3, 6, 9] [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
1 apple
2 banana
3 cherry
```

`+` builds a **new** list; `extend()` / `+=` **modify** the left list. `==` compares contents; `is` identity.

List operators

Operator	Meaning	Example	Result
`+`	Concatenate into a new list	`[1, 2] + [3]`	`[1, 2, 3]`
`*`	Repeat	`[1, 2] * 2`	`[1, 2, 1, 2]`
`in` / `not in`	Membership (linear scan)	`3 in [1, 2, 3]`	`True`
`==`, `<`, …	Element-by-element (lexicographic)	`[1, 2, 3] < [1, 3]`	`True`
`+=`	Extend in place	`a += [4]`	`a` changed
`del a[i]`	Delete by index or slice	`del a[0]`	

```
a, b = [1, 2, 3], [4, 5, 6]
print(a + b, a * 3, 3 in a, 7 not in a)
print([1, 2, 3] < [1, 3], [1, 2] == [1, 2], max(a), min(a), sum(a))
```

Output:

```
[1, 2, 3, 4, 5, 6] [1, 2, 3, 1, 2, 3, 1, 2, 3] True True
True True 3 1 6
```

`[1, 2, 3] < [1, 3]`: first elements tie, then 2 < 3 — like comparing words.

Common mistakes

- `a = a.extend(b)` sets `a` to `None` (`extend` returns `None`).
- Repeated `in` on a big list inside a loop — O(n) each; convert to a `set` (Chapter 6).
- Removing while iterating skips elements; iterate a copy (`lst[:]`) or rebuild with a comprehension.

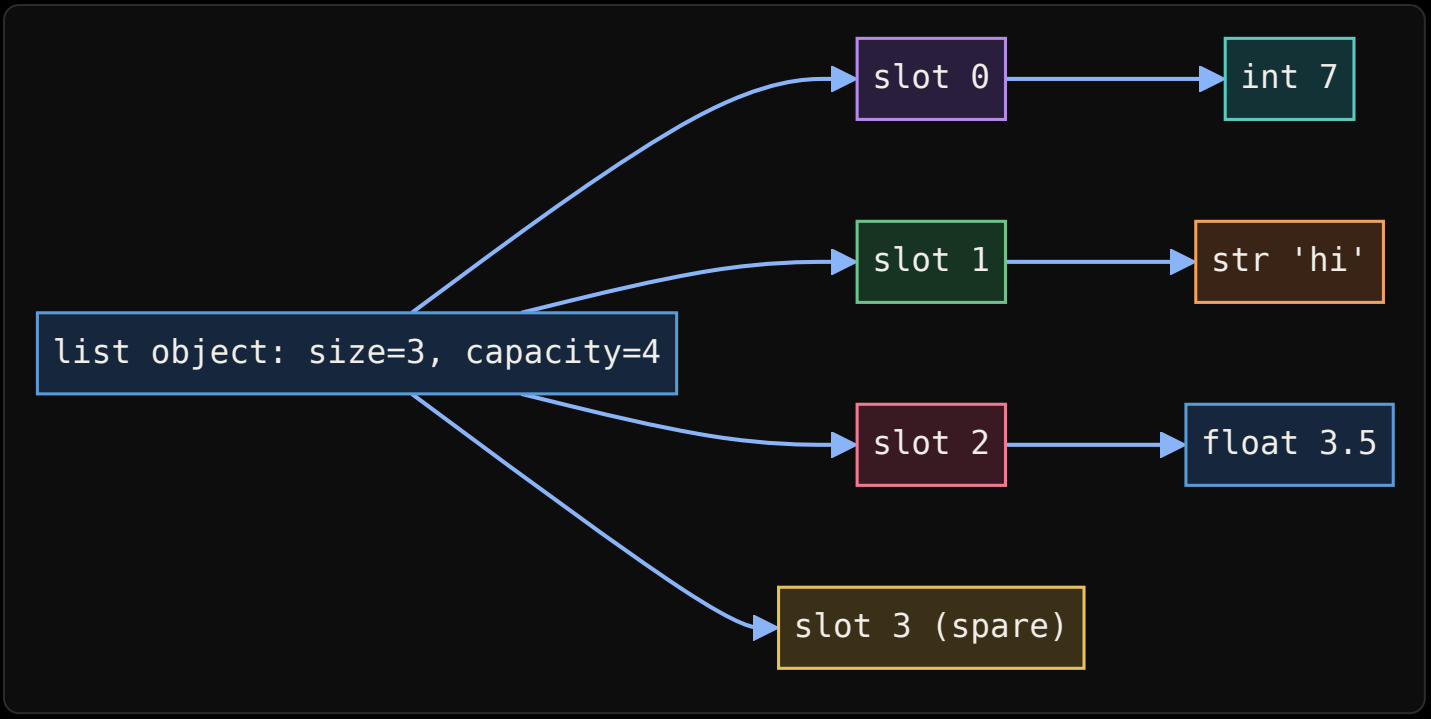
TL;DR

Ordered, mutable, heterogeneous (mixed types). Index, slice, update in place; `+` makes new lists, `extend` / `+=` modify.

5.3 List internals — a dynamic array — and the big-O table

What it is

In CPython a list is a **dynamic array**: one contiguous block of **pointers** to element objects, plus the used length (`len`) and the allocated slots (**capacity**). "Dynamic": it resizes automatically. The layout tells you which operations are cheap — the difference between O(n) and O(n²) programs.



How it works

1. **Index** `lst[i]`: address = start + i × 8 bytes → O(1).
2. **Append** with spare capacity: write the next slot, O(1).
3. **Append when full**: allocate a bigger block, copy all pointers, write. Growth is **~12.5% plus a small constant** (recent CPython: `new_size + new_size/8 + 6`, rounded down to a multiple of 4). Each resize leaves room for many appends, so the copying averages to a constant — **amortised O(1)**.
4. **Insert/delete at i**: every later pointer shifts → O(n − i); at the front, the whole list.
5. **Shrinking**: below half capacity, CPython may reallocate smaller.

Measured capacity while appending (`code/06_list_internals.py`, via `sys.getsizeof`):

```
len  capacity  (capacity = slots allocated)
  0         0
  1         4
  5         8
  9        16
 17        24
 25        32
 33        40
 41        52
 53        64
 65        76
```

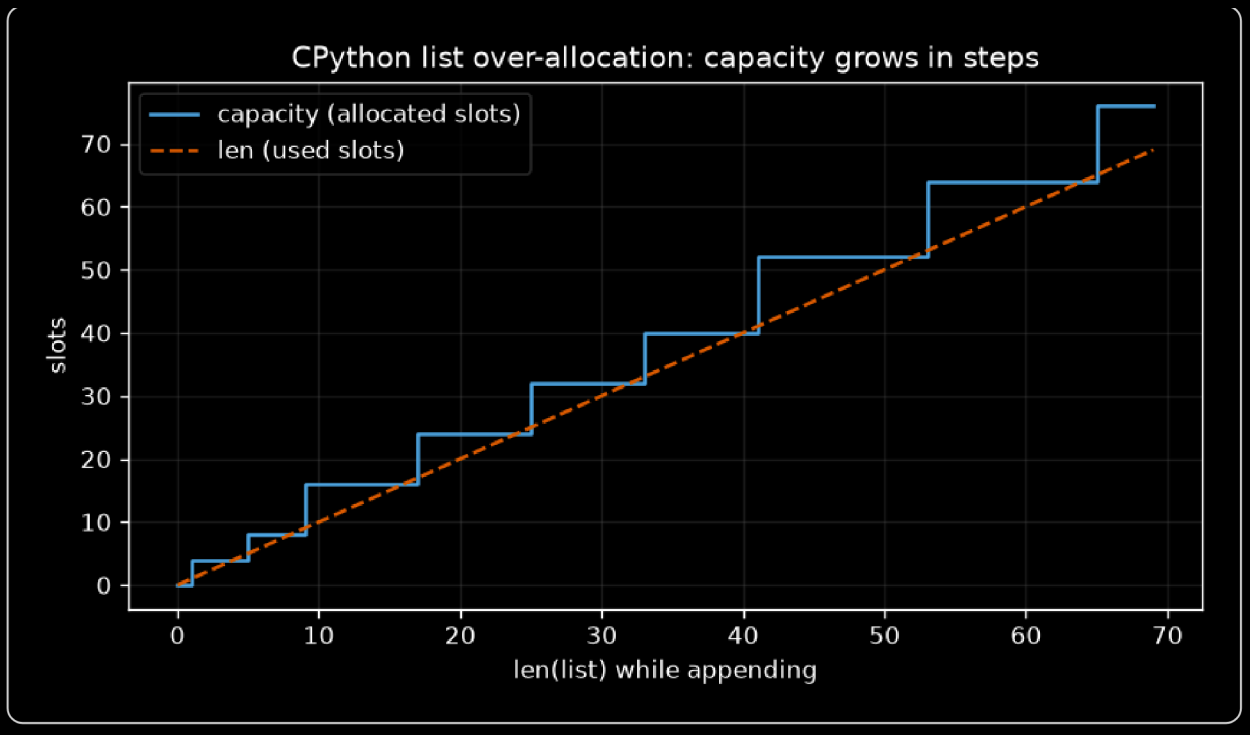


Figure — capacity (steps) stays at or above the used length (dashed); a copy happens only at each step.

Worked example: the 17th append finds 16 full, allocates 24 and copies 16 pointers; the next 7 appends are free — 2 copies each over 8 appends. Steps grow with the list, so the average stays bounded.

End vs front (µs — millionths of a second — per push-then-pop pair):

n	append()+pop()	insert(0)+pop(0)	(µs per pair)
10,000	0.017	3.588	
100,000	0.017	40.046	
1,000,000	0.018	414.395	

End operations stay flat as *n* grows ×100; front operations grow ×100 — O(1) vs O(n).

Big-O table for `list` (n = len, k = size of the other argument or slice)

Operation	Example	Average cost
Length	`len(a)`	O(1)
Get / set item	`a[i]`, `a[i] = x`	O(1)
Append	`a.append(x)`	O(1) amortised
Pop last	`a.pop()`	O(1)
Pop / delete at i	`a.pop(i)`, `del a[i]`	O(n − i) → O(n) at the front
Insert at i	`a.insert(i, x)`	O(n − i)
Extend	`a.extend(b)`, `a += b`	O(k)
Concatenate	`a + b`	O(n + k)
Repeat	`a * k`	O(n·k)
Get slice	`a[i:j]`	O(k)
Set slice	`a[i:j] = b`	O(k + n)
Delete slice	`del a[i:j]`	O(n)
Membership / search	`x in a`, `a.index(x)`, `a.count(x)`, `a.remove(x)`	O(n)
Min / max / sum	`min(a)`	O(n)
Copy	`a.copy()`, `a[:]`	O(n)
Reverse	`a.reverse()`	O(n)
Sort	`a.sort()`, `sorted(a)`	O(n log n) worst (slightly more than linear — the norm for good sorts), O(n) if already sorted
Iterate	`for x in a`	O(n)

Trade-offs

Need	Best structure	Why
Append/pop at the end (stack)	`list`	O(1)
Add/remove at both ends (queue)	`collections.deque` (double-ended queue; Chapter 6)	O(1) at both ends; `list.pop(0)` is O(n)
Frequent membership tests	`set` / `dict`	O(1) average vs O(n)
Keep sorted while inserting	`bisect` (binary-search helpers) on a list (Chapter 6) or a sorted-container library	O(log n) search (doubling n adds one step), O(n) insert

Common mistakes

- `list.pop(0)` as a queue — quadratic for large queues; use `deque.popleft()`.
- `insert(0, x)` in a loop to build a reversed list — append, then `reverse()` once.

In the interview

> **30-second answer:** "A Python list is a dynamic array of pointers. Indexing and append are O(1) (append amortised, thanks to proportional over-allocation of about 12.5%), but inserting or

> deleting anywhere except the end shifts elements and is O(n), and membership tests are O(n)."

- *Why amortised?* → Rare resizes copy everything, but total copying over n appends is O(n).
- *Linked list vs Python list?* → No built-in linked list (a chain of nodes each pointing to the next); `deque` is a doubly-linked list (nodes point both ways) of fixed-size blocks: O(1) at both ends, O(n) indexing in the middle.
- *Why is `x in lst` slow?* → Linear scan.
- *What's in the slots?* → Pointers, not the objects.

TL;DR

Dynamic array of pointers. O(1): index, append, pop-last. O(n): insert/delete at front or middle, `in`, `index`, `remove`. Queue → `deque`; lookups → `set`.

5.4 Adding and removing items

What it is

Goal	Use	Returns	Error when
Add one item at the end	`append(x)`	`None`	—
Add one item at position i	`insert(i, x)`	`None`	— (i beyond ends is clamped to the nearest end)
Add all items of an iterable	`extend(iterable)` / `+=`	`None`	Not iterable
Remove first item equal to x	`remove(x)`	`None`	`ValueError` if absent
Remove by position and get it	`pop(i=-1)`	The item	`IndexError` on empty/out of range
Remove by position or slice	`del a[i]`, `del a[i:j]`	—	`IndexError`
Remove everything	`clear()` (or `del a[:]`)	`None`	—

```
fruits = ["apple", "banana"]
fruits.append("cherry");              print("append:", fruits)
fruits.insert(1, "orange");           print("insert:", fruits)
fruits.extend(["grape", "mango"]);    print("extend:", fruits)
fruits.remove("banana");              print("remove:", fruits)
last = fruits.pop();                  print("pop():", fruits, "->", last)
first = fruits.pop(0);                print("pop(0):", fruits, "->", first)
del fruits[0];                        print("del [0]:", fruits)
fruits.clear();                       print("clear:", fruits)
x = [1, 2]
x.append([3, 4]);  print("append a list :", x)   # nests!
y = [1, 2]
y.extend([3, 4]);  print("extend a list :", y)
try:
    y.remove(99)
except ValueError as e:
    print("ValueError:", e)
```

Output:

```
append: ['apple', 'banana', 'cherry']
insert: ['apple', 'orange', 'banana', 'cherry']
extend: ['apple', 'orange', 'banana', 'cherry', 'grape', 'mango']
remove: ['apple', 'orange', 'cherry', 'grape', 'mango']
pop(): ['apple', 'orange', 'cherry', 'grape'] -> mango
pop(0): ['orange', 'cherry', 'grape'] -> apple
del [0]: ['cherry', 'grape']
clear: []
append a list : [1, 2, [3, 4]]
extend a list : [1, 2, 3, 4]
ValueError: list.remove(x): x not in list
```

Common mistakes

- `append([3, 4])` adds **one** (nested) element; `extend([3, 4])` adds **two**; `extend("hi")` adds `'h'`, `'i'`.
- `while x in lst: lst.remove(x)` — O(n²). Use `lst = [v for v in lst if v != x]`.
- `lst.clear()` empties the **same** object (all references see it); `lst = []` rebinds only this name.

TL;DR

`append` one, `extend` many, `insert` at a position; `remove` by value (first only), `pop` by index (returns it), `del` by index/slice, `clear` everything.

5.5 List methods (complete table) and slicing in depth

The full method table

Method	Does	Cost
`append(x)`	Add at the end	O(1) amortised
`extend(it)`	Add every item of `it`	O(k)
`insert(i, x)`	Insert before `i`	O(n)
`remove(x)`	Delete first equal item	O(n)
`pop([i])`	Remove and return item `i` (default last)	O(1) last / O(n)
`clear()`	Remove all	O(n) (drops each reference)
`index(x[, start[, end]])`	First index of `x`	O(n)
`count(x)`	Items equal to `x`	O(n)
`sort(*, key=None, reverse=False)`	Sort in place (stable — equal items keep their order, §5.7)	O(n log n)
`reverse()`	Reverse in place	O(n)
`copy()`	Shallow copy	O(n)

Only 11 methods; everything else is a built-in (`len`, `sorted`, `reversed`, `min`, `max`, `sum`, `enumerate`, `zip`) or an operator.

```
colors = ["red", "blue", "green", "blue"]
print(colors.count("blue"), colors.index("blue"), colors.index("blue", 2))
colors.remove("blue");   print(colors)          # only the first 'blue'
colors.append("amber")
colors.sort();           print("sort:", colors)
colors.reverse();        print("reverse:", colors)
nums = [3, 1, 2]
snapshot = nums.copy()
nums.sort(reverse=True)
print(nums, snapshot)
```

Output:

```
2 1 3
['red', 'green', 'blue']
sort: ['amber', 'blue', 'green', 'red']
reverse: ['red', 'green', 'blue', 'amber']
[3, 2, 1] [3, 1, 2]
```

`index("blue", 2)` starts at position 2, finding the second `"blue"` at 3.

Slicing in depth — steps and slice assignment

String slicing rules (Chapter 4) apply. Because lists are mutable you can also **assign** and **delete** slices.

Form	Rule
`a[i:j] = iterable`	Replace that range; new data may be **longer or shorter**
`a[i:i] = iterable`	Empty range → pure **insertion** at `i`
`a[i:j:k] = iterable`	Extended (stepped) slice → lengths **must match**
`del a[i:j:k]`	Delete the selected positions
`a[:] = []`	Empty in place (same as `clear()`)

```
x = list(range(10))
print(x[1:8:2], x[::-2], x[-3:])
y = x.copy(); y[2:5] = ["a"];            print("shrink :", y)      # lengths may differ
y = x.copy(); y[2:2] = ["new", "items"]; print("insert :", y)      # empty slice = insert
y = x.copy(); y[::2] = [0] * 5;          print("stepped:", y)      # lengths must match
y = x.copy(); del y[::2];                print("del ::2:", y)
y = x.copy(); y[:] = [];                 print("clear  :", y)
try:
    y = x.copy(); y[::2] = [1, 2]
except ValueError as e:
    print("ValueError:", e)
```

Output:

```
[1, 3, 5, 7] [9, 7, 5, 3, 1] [7, 8, 9]
shrink : [0, 1, 'a', 5, 6, 7, 8, 9]
insert : [0, 1, 'new', 'items', 2, 3, 4, 5, 6, 7, 8, 9]
stepped: [0, 1, 0, 3, 0, 5, 0, 7, 0, 9]
del ::2: [1, 3, 5, 7, 9]
clear  : []
ValueError: attempt to assign sequence of size 2 to extended slice of size 5
```

Common mistakes

- `a[1:3] = 5` — the right side must be iterable: `a[1:3] = [5]`.
- A slice **read** is a shallow copy; a slice **assignment** changes the original.

TL;DR

11 methods; slicing reads copies, slice assignment edits in place (length may change unless there's a step).

5.6 List comprehensions

What it is

A **list comprehension** builds a new list in one expression:

```
[expression for item in iterable if condition]
```

- **expression** — what goes in (e.g. `n ** 2`).
- **for item in iterable** — the source loop.
- **if condition** — optional filter.

The problem it solves

Empty-list + loop + `append` takes three or four lines to say one thing, and the repeated `.append` attribute lookup and call cost time. A comprehension is one readable line compiled to specialised bytecode.

How it works

1. Create an empty list; for each item, test `condition` (if any).
2. If true, evaluate `expression` and append.
3. Multiple `for` clauses nest **left to right**, as nested loops would.

```
numbers = [1, 2, 3, 4, 5]
squared = []
for n in numbers:                          # the traditional way
    squared.append(n ** 2)
print("loop:", squared)
print("comp:", [n ** 2 for n in numbers])
print("even squares:", [n ** 2 for n in numbers if n % 2 == 0])
print("if/else in expr:", ["even" if n % 2 == 0 else "odd" for n in numbers])
print([f"Number {n} squared is {n ** 2}." for n in numbers[:2]])

matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]          # same order as nested for-loops
transposed = [[row[i] for row in matrix] for i in range(3)]
pairs = [(c, n) for c in "ab" for n in (1, 2)]
print(flat, transposed, pairs)

total = sum(n * n for n in range(1_000_000))       # generator expression: no list built
```

Output:

```
loop: [1, 4, 9, 16, 25]
comp: [1, 4, 9, 16, 25]
even squares: [4, 16]
if/else in expr: ['odd', 'even', 'odd', 'even', 'odd']
['Number 1 squared is 1.', 'Number 2 squared is 4.']
[1, 2, 3, 4, 5, 6] [[1, 4], [2, 5], [3, 6]] [('a', 1), ('a', 2), ('b', 1), ('b', 2)]
sum of squares: 333332833333500000
loop+append: 39.0 ms   comprehension: 31.6 ms (ratio 1.2x; machine-dependent)
```

Two `if` positions:

- `[... for n in xs if cond]` — **filter** (fewer items).
- `[a if cond else b for n in xs]` — **choose a value** (same count).

The speed gain is modest (1.2×); choose comprehensions for **clarity**.

Trade-offs

	Comprehension	`for` + `append`	Generator expression `( … )` (lazy, one item at a time)	`map` / `filter` (apply a function / keep matches)
Readability for simple transforms	Best	OK	Best	OK with named functions
Memory	Whole list	Whole list	One item at a time	Lazy iterator
Complex logic, try/except, prints	Poor	Best	Poor	Poor

When not to use it

- Several statements, error handling, side effects (printing, writing files) — write a loop.
- One-pass consumers (`sum`, `any`, `max`, `"".join`) — a **generator expression** avoids materialising (building in memory) the list: `sum(x*x for x in xs)`.
- More than two `for` clauses or nesting deeper than one level (*Effective Python*: at most two control subexpressions).
- Side effects only (`[print(x) for x in xs]`) builds a useless list of `None`s.

Common mistakes

- Wrong nesting order: `[x for x in row for row in matrix]` fails (`row` undefined).
- Expecting the loop variable to leak: in Python 3 it's local to the comprehension.

Real-world examples

Data cleaning (`[c.strip().lower() for c in columns]`), SQL parameter lists, flattening paged API results (`[item for page in pages for item in page["results"]]`) in ETL (extract–transform–load) scripts.

In the interview

> **30-second answer:** "`[expr for x in it if cond]` builds a list in one expression; it's clearer and a bit faster than loop-and-append. Use a generator expression when you only need to iterate once, and a normal loop when the logic gets complicated."

- *Order of multiple fors?* → Left to right, like nested loops.
- *Dict and set versions?* → `{k: v for ...}`, `{x for ...}` (Chapter 6).

Self-check

1. `[c.upper() for c in "abc" if c != "b"]`? → `['A', 'C']`.
2. Flatten `[[1],[2,3]]`? → `[x for sub in lst for x in sub]`.

TL;DR

`[expr for x in it if cond]`; `if/else` goes before `for`; nested `for`s read left to right; `( … )` for one-pass use; loops for complex logic.

5.7 Sorting

What it is

- `lst.sort()` — sorts **in place**, returns `None`.
- `sorted(iterable)` — returns a **new** list from any iterable (list, tuple, string, set, dict keys, generator).

Both take `key=` (function computing the sort value) and `reverse=True`. Sorting underlies reports, rankings, deduplication, binary search (repeatedly halving a sorted list) and top-k queries; `key` plus

stability express almost any ordering in one line.

How it works

1. `key` is called **once per element**; Python sorts by the results (decorate–sort–undecorate for you).
2. Items (or keys) are compared with `<`; incomparable mixed types raise `TypeError`.
3. Algorithm: **Timsort** (since 2.3; 3.11 adopted the "powersort" merge policy). It finds runs (stretches already in order) and merges them: **O(n log n)** worst, **O(n)** on sorted data.
4. **Stable**: equal keys keep their original order — so multi-key sorts can be done in passes, secondary key first.

```
nums = [5, 2, 9, 1]
new = sorted(nums)                  # returns a NEW list, works on any iterable
print(new, nums)
result = nums.sort()                # sorts IN PLACE, returns None
print(nums, result)
print(sorted("banana"), sorted({3, 1, 2}), sorted({"b": 1, "a": 2}))

words = ["banana", "Apple", "cherry", "Date"]
print(sorted(words))                                # uppercase first (code points)
print(sorted(words, key=str.casefold))              # case-insensitive
print(sorted(words, key=len))                       # by length
print(sorted(words, key=len, reverse=True))
people = [("Ann", 31), ("Bob", 25), ("Cid", 31)]
print(sorted(people, key=itemgetter(1)))            # same as key=lambda p: p[1]
```

Output:

```
[1, 2, 5, 9] [5, 2, 9, 1]
[1, 2, 5, 9] None
['a', 'a', 'a', 'b', 'n', 'n'] [1, 2, 3] ['a', 'b']
['Apple', 'Date', 'banana', 'cherry']
['Apple', 'banana', 'cherry', 'Date']
['Date', 'Apple', 'banana', 'cherry']
['banana', 'cherry', 'Apple', 'Date']
[('Bob', 25), ('Ann', 31), ('Cid', 31)]
```

Stability: with `key=len`, `"banana"` and `"cherry"` (both 6) keep their order — even with `reverse=True`, which flips comparisons, not tie order.

Multi-key sorts — department ascending, then age descending:

```
people = [("Ann", "HR", 31), ("Bob", "IT", 25), ("Cid", "HR", 25), ("Dee", "IT", 31)]
by_age = sorted(people, key=itemgetter(2))
print("by age (ties keep original order):", [p[0] for p in by_age])
print("tuple key :", [p[0] for p in sorted(people, key=lambda p: (p[1], -p[2]))])
tmp = sorted(people, key=itemgetter(2), reverse=True)        # secondary key first
print("two passes:", [p[0] for p in sorted(tmp, key=itemgetter(1))])
```

Output:

```
by age (ties keep original order): ['Bob', 'Cid', 'Ann', 'Dee']
tuple key : ['Ann', 'Cid', 'Dee', 'Bob']
two passes: ['Ann', 'Cid', 'Dee', 'Bob']
```

A **tuple key** compares field by field; negating a number flips its direction. If a field can't be negated (a string to sort descending), use two stable passes.

Trade-offs

	`list.sort()`	`sorted()`
Input	Lists only	Any iterable
Result	Same list, changed	New list
Memory	No extra list	Extra O(n)
Keep original?	No	Yes

Common mistakes

- `lst = lst.sort()` → `None`.
- Mixed types (`[3, "a"]`) → `TypeError: '<' not supported between instances of 'str' and 'int'`.
- Numeric strings sorted as strings (`"10" < "9"`) — `key=int`.
- A `lambda` (one-line anonymous function) where `operator.itemgetter`/`attrgetter` (ready-made key functions) or `str.casefold` exist — clearer and slightly faster.

Real-world examples

Timsort (Tim Peters, for CPython) was adopted for object sorting in Java 7 (`Arrays.sort`), Android, and V8 (Chrome/Node.js JavaScript engine) since 2018. Leaderboards, "price then rating" filters, SQL `ORDER BY a, b DESC` and pandas `df.sort_values(["a", "b"], ascending=[True, False])` are key-based stable sorts.

In the interview

> **30-second answer:** "`sort` sorts a list in place and returns None; `sorted` returns a new list from any iterable. Both take `key` and `reverse`. Python uses Timsort — stable, O(n log n), O(n) on nearly sorted data — and stability lets you do multi-key sorts with a tuple key or successive passes."

- *Why is stability useful?* → Equal elements keep prior order, enabling multi-pass sorts.
- *How often is `key` called?* → Once per element.
- *Top 3 of a million?* → `heapq.nlargest(3, items)` (a *heap* keeps the smallest on top) — O(n log k) (Chapter 6).

TL;DR

`sort()` in place → `None`; `sorted()` new list; `key=` once per item; Timsort is stable and adaptive.

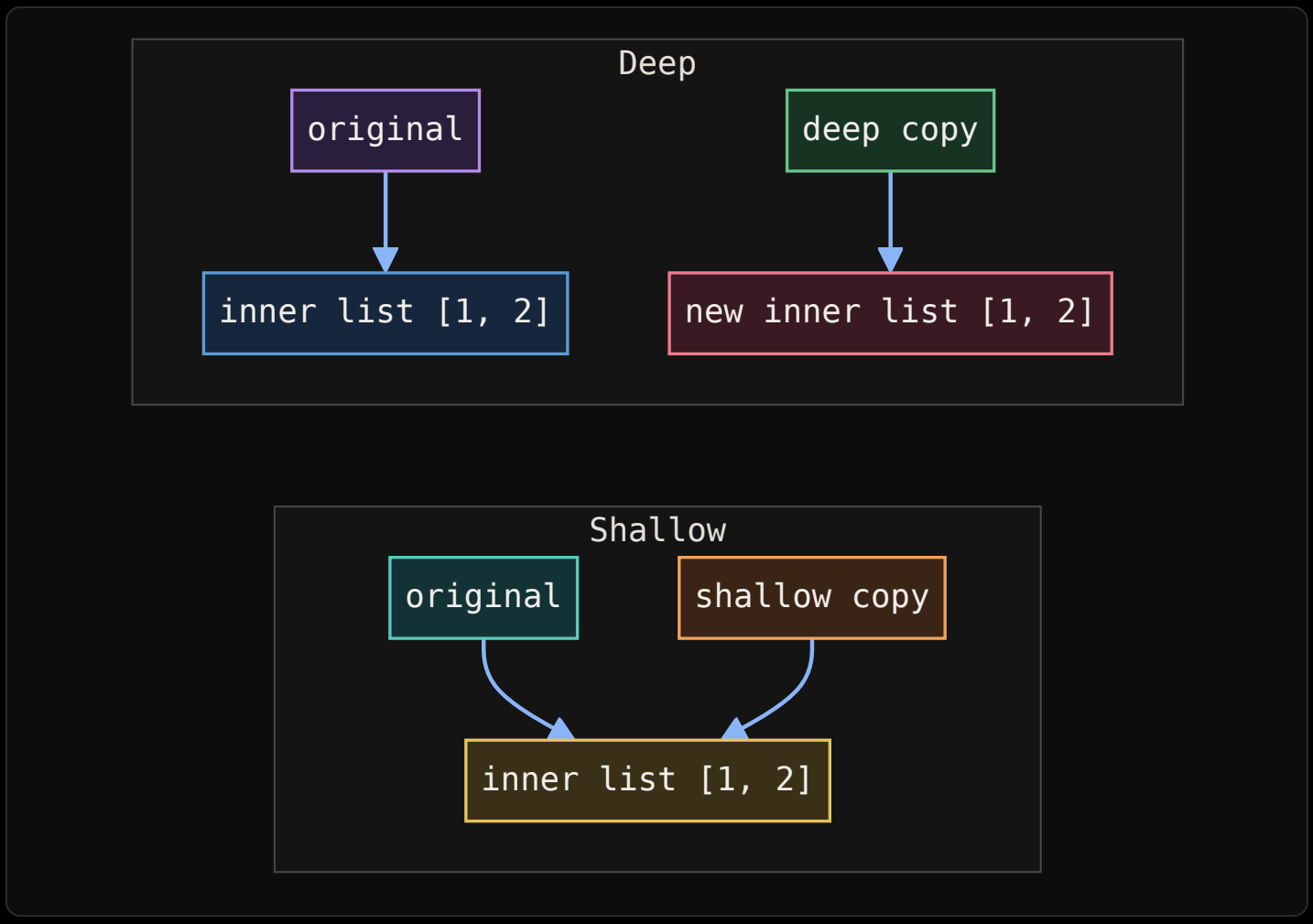
5.8 Copying — aliasing, shallow vs deep, and the nested-list trap

What it is

- **Aliasing**: two names, **same** object. `b = a` never copies.
- **Shallow copy**: new outer container, slots pointing to the **same** inner objects — `a.copy()`, `list(a)`, `a[:]`, `copy.copy(a)`.
- **Deep copy**: new outer container **and** recursive copies of everything inside — `copy.deepcopy(a)`.

Most "my data changed by itself" bugs come from aliasing or shallow copies of nested data.

How it works




```
a = [1, 2, 3]
b = a                        # NOT a copy: two names, one list
b.append(4)
print(a, b, a is b)

original = [[1, 2], [3, 4]]
shallow = original.copy()
deep = copy.deepcopy(original)
original[0].append(99)                  # mutate an INNER list
original.append([5])                    # mutate the OUTER list
print("original:", original)
print("shallow :", shallow, "<- shares inner lists")
print("deep    :", deep, "<- fully independent")
print(shallow[0] is original[0], deep[0] is original[0])
```

Output:

```
[1, 2, 3, 4] [1, 2, 3, 4] True
original: [[1, 2, 99], [3, 4], [5]]
shallow : [[1, 2, 99], [3, 4]] <- shares inner lists
deep    : [[1, 2], [3, 4]] <- fully independent
True False
```

The shallow copy didn't get `[5]` (separate outer list) but saw `99` (shared inner list). Passing a list to a function passes a reference too — `add_item(a)` appending `'x'` gives `[1, 2, 3, 4, 'x']` in the caller.

The `[[0] * 3] * 3` trap

`*` repeats **references**: one inner row, three pointers.

```
bad = [[0] * 3] * 3                    # 3 references to ONE row
bad[0][0] = 1
good = [[0] * 3 for _ in range(3)]     # 3 separate rows
good[0][0] = 1
```

Output:

```
bad : [[1, 0, 0], [1, 0, 0], [1, 0, 0]] ids: 1 distinct row(s)
good: [[1, 0, 0], [0, 0, 0], [0, 0, 0]] ids: 3 distinct row(s)
```

(`[0] * 3` is fine: ints are immutable.)

The mutable default argument trap

Defaults are evaluated **once**, when `def` runs, so a list default is shared by every call:

```
def bad_append(item, bucket=[]):
    bucket.append(item); return bucket
def good_append(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item); return bucket
print(bad_append(1), bad_append(2))    # [1, 2] [1, 2]
print(good_append(1), good_append(2))  # [1] [2]
```

Trade-offs

	Alias `b = a`	Shallow `a.copy()`	Deep `copy.deepcopy(a)`
Cost	O(1)	O(n)	O(total size of everything)
Independent outer list	No	Yes	Yes
Independent nested objects	No	No	Yes
Handles cycles (objects that contain each other)	—	—	Yes (memo of already-copied objects)

When not to use it

Don't `deepcopy` by reflex — it's slow and copies things you may want shared (caches, database connections). For flat or immutable-only data a shallow copy is effectively full. Immutable structures (tuples, frozen dataclasses — Chapter 10) avoid copying altogether.

Common mistakes

- `grid = [[None] * cols] * rows` — use a comprehension.
- Thinking `a[:]` copies nested lists.
- `def f(x=[])` / `def f(d={})` — use `None` and create inside.

In the interview

30-second answer: "Assignment never copies — it creates another reference. A shallow copy (`copy()`, `list()`, `[:]`) duplicates only the outer container; nested objects are shared. `copy.deepcopy` copies recursively. Classic traps: `[[0]*3]*3` shares one row, and mutable default arguments are shared between calls."

- *Does `a[:]` copy?* → Shallowly.
- *Does deepcopy handle cycles?* → Yes, via a memo dict.

Self-check

1. `a = [[1]]; b = a[:]; b[0].append(2)` — `a`? → `[[1, 2]]`.

TL;DR

`=` aliases, `copy()` shares the insides, `deepcopy` shares nothing. Never multiply a list of lists.

5.9 Tuples — immutable sequences and records

What it is

A **tuple** is an ordered, **immutable** sequence. **Commas** make it; parentheses only group: `(1, 2, 3)` or `1, 2, 3` (**tuple packing**). One element needs a trailing comma: `(42,)`.

Why it matters

- **Integrity:** coordinates, RGB colours, database rows, return bundles can't change by accident.
- **Hashable** (if all items are) → dict key or set element: `{(59.9, 10.7): "Oslo"}`.
- **Lighter:** fixed size, no spare capacity — a 3-item tuple is 72 bytes vs 88 for a list on this build; CPython can build constant tuples at compile time.
- **Thread safety by construction:** an unmutable object can be shared between threads without locks (the objects inside may still be mutable).

How it works — define, access, "update", loop, operators, slice

```
t = (1, 2, 3, 4, 5)
packed = "apple", "banana", "cherry"          # the commas make the tuple
single, not_a_tuple, empty = (42,), (42), ()
print(t, packed, type(single).__name__, type(not_a_tuple).__name__, empty)
print(t[0], t[2], t[-1], t[1:4], t[::-1])
try:
    t[1] = 20
except TypeError as e:
    print("TypeError:", e)
as_list = list(t); as_list[1] = 20; t2 = tuple(as_list)   # "update" = build a new tuple
print("updated copy:", t2)
for item in t:
    print(item, end=" ")
a, b = (1, 2, 3), (4, 5, 6)
print(a + b, a * 2, 3 in a, 7 not in a, (1, 2) < (1, 3))
```

Output:

```
(1, 2, 3, 4, 5) ('apple', 'banana', 'cherry') tuple int ()
1 3 5 (2, 3, 4) (5, 4, 3, 2, 1)
TypeError: 'tuple' object does not support item assignment
updated copy: (1, 20, 3, 4, 5)
1 2 3 4 5
(1, 2, 3, 4, 5, 6) (1, 2, 3, 1, 2, 3) True True True
```

Operator	Meaning	Example
`+`	Concatenate into a new tuple	`(1, 2) + (3,)` → `(1, 2, 3)`
`*`	Repeat	`(1, 2) * 2` → `(1, 2, 1, 2)`
`in` / `not in`	Membership	`3 in (1, 2, 3)` → `True`
`<`, `==`, …	Lexicographic comparison	`(1, 2) < (1, 3)` → `True`
`[i]`, `[i:j:k]`	Index and slice (slices are new tuples)	`(0,1,2,3)[1:3]` → `(1, 2)`

"Update" = list round-trip, or slicing: `t[:1] + (20,) + t[2:]`.

Immutability is shallow

Slots can't change, but a mutable object **inside** can:

```
t = ([1, 2], "x")
t[0].append(3)
print(t)                  # ([1, 2, 3], 'x')
hash(t)                   # TypeError: unhashable type: 'list'
```

A tuple is hashable only if everything inside is.

Tuples as records

A tuple is a **record** where *position has meaning*: `("Ann", "HR", 31)` = name, department, age. Lists hold **many things of one kind**; tuples hold **one thing with several fields** (*Fluent Python*'s guidance) — hence functions return tuples: `divmod(17, 5)` → `(3, 2)`.

Common mistakes

- `(42)` is the int 42; `type(("a"))` is `str` — write `(42,)`.
- `x = 1,` makes a tuple.
- A tuple of lists isn't hashable, so can't be a dict key.

TL;DR

Commas make tuples; shallowly immutable, hashable if contents are, lighter than lists, ideal fixed-shape records.

5.10 Unpacking tuples (and any iterable)

What it is

Unpacking assigns a sequence's items to several names: `a, b, c = (1, "Hello", 3.14)` — for any iterable. It removes index noise (`row[0]`), and powers multiple return values, swaps and pair loops.

How it works

1. The right side is fully evaluated first.
2. Items bind to names in order; counts must match or `ValueError` ("not enough" / "too many values").
3. **Star-unpacking** (`*name`) collects the rest into a **list** (possibly empty); one star per assignment.
4. Unpacking nests: `for name, (lat, lon) in ...`.

```
a, b, c = (1, "Hello", 3.14)
print(a, b, c)
try:
    w, x, y, z = (1, "Hello", 3.14)
except ValueError as e:
    print("ValueError:", e)
first, *middle, last = (1, 2, 3, 4, 5, 6)
print(first, middle, last)
head, *tail = [10]
print(head, tail)
x, y = 1, 2
x, y = y, x                               # swap
print(x, y)
for name, (lat, lon) in [("Oslo", (59.9, 10.7)), ("Lima", (-12.0, -77.0))]:
    print(f"{name}: {lat}, {lon}")
_, _, ext = "archive.tar.gz".partition(".")   # _ = "don't care"
print(ext)
```

Output:

```
1 Hello 3.14
ValueError: not enough values to unpack (expected 4, got 3)
1 [2, 3, 4, 5] 6
10 []
2 1
Oslo: 59.9, 10.7
Lima: -12.0, -77.0
tar.gz
```

`try/except ValueError` lets a program report bad input (a malformed CSV row) instead of crashing. The swap works because `(2, 1)` is built **before** assignment.

Star in calls and literals (the reverse):

Syntax	Meaning	Example
`f(*seq)`	Spread items as positional arguments	`print(*[1, 2, 3])` → `1 2 3`
`[*a, *b]`	Build a list from several iterables	`[*(1, 2), *[3]]` → `[1, 2, 3]`
`(*a, x)`	Build a tuple	`(*t, 99)`
`def f(*args)`	Collect extra positional arguments into a tuple	Chapter on functions

Common mistakes

- Two stars (`a, *b, *c = ...`) → `SyntaxError`.
- `*middle` is always a **list**, not a tuple.
- Unpacking a dict gives **keys**; use `.items()`.

In the interview

> **30-second answer:** "Unpacking binds items of any iterable to names; the count must match or you get ValueError. `first, *rest = seq` gathers the leftovers into a list. It powers multiple return values, swaps without a temp, and clean loops like `for k, v in d.items()`."

- *Is `a, b = b, a` atomic?* → Build tuple, then assign — no temp, but not thread-atomic (other threads can interleave).
- `*_`*?* → Discards any number of items: `first, *_ = row`.

Self-check

1. `a, *b = "hey"`? → `a='h'`, `b=['e', 'y']`.
2. `x, y = [1, 2, 3]`? → `ValueError: too many values to unpack (expected 2)`.

TL;DR

`a, b, c = seq`; counts must match; `*rest` gathers a list; `_` for ignored slots; works in loops and nests.

5.11 Joining tuples

What it is

Combining tuples always creates a **new** tuple.

How it works — five ways

```
parts = [(1, 2), (3, 4), (5, 6)]
out = ()
for p in parts:
    out += p                                # new tuple every time: O(n^2) overall
print("loop     :", out)
print("+        :", (1, 2, 3) + (4, 5, 6))
print("sum      :", sum(parts, ()))          # also O(n^2): repeated +
print("genexpr  :", tuple(x for p in parts for x in p))
print("chain    :", tuple(itertools.chain.from_iterable(parts)))
```

Output (last line: joining 3,000 two-item tuples):

```
loop     : (1, 2, 3, 4, 5, 6)
+        : (1, 2, 3, 4, 5, 6)
sum      : (1, 2, 3, 4, 5, 6)
genexpr  : (1, 2, 3, 4, 5, 6)
chain    : (1, 2, 3, 4, 5, 6)
3000 tuples: sum 14.55 ms vs chain 0.052 ms
```

`+` is fine for two or three. `+=` loops and `sum(parts, ())` copy everything so far each time — O(total²); the docs recommend `itertools.chain` (and `sum` refuses strings). The flattening generator is O(total); `tuple(itertools.chain.from_iterable(parts))` is the idiomatic linear C solution, ~280× faster here.

Common mistakes

- `(1,) + [2]` → `TypeError`; convert first.
- `t + (4)` fails; `t + (4,)` works.

TL;DR

Few → `+`. Many → `tuple(chain.from_iterable(...))` or a flattening generator. Avoid `+=` loops and `sum`.

5.12 Tuple methods

Only two — anything that would modify is missing by design.

Method	Returns	Cost
`count(x)`	Items equal to `x`	O(n)
`index(x[, start[, end]])`	First position of `x`; `ValueError` if absent	O(n)

```
fruit = ("apple", "banana", "cherry", "apple", "cherry")
print(fruit.count("apple"), fruit.count("cherry"), fruit.index("banana"), fruit.index("cherry"))
```

Output:

```
2 2 1 2
```

`index("cherry")` returns the **first** (2). Built-ins `len`, `min`, `max`, `sum`, `sorted` (returns a list), `reversed`, `any`, `all` work on tuples.

TL;DR

`count` and `index`, both O(n); the rest comes from built-ins.

5.13 `namedtuple` and `typing.NamedTuple` — tuples with field names

What it is

`collections.namedtuple` creates a **tuple subclass** whose positions also have names: `p.x` instead of `p[0]`. `typing.NamedTuple` is the class syntax with type hints and defaults.

The problem it solves

`emp[2]` says nothing; `emp.salary` says everything. Named tuples keep every tuple benefit (immutable, hashable, unpackable, tuple-sized — no per-instance `__dict__`, the attribute dictionary ordinary objects carry) while being self-documenting.

How it works

```
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
print(p, p.x, p[1], p._asdict(), p._replace(x=10))
x, y = p                                    # still a tuple

class Employee(NamedTuple):                 # typed, class-style syntax
    name: str
    dept: str
    salary: int = 0
e = Employee("Ann", "HR")
print(e, e.salary)
```

Output:

```
Point(x=3, y=4) 3 4 {'x': 3, 'y': 4} Point(x=10, y=4)
Employee(name='Ann', dept='HR', salary=0) 0
```

Extras: `_asdict()` (to dict, e.g. JSON), `_replace(**changes)` (modified **copy** — the way to "change" a field), `_fields`, `_make(iterable)`.

Trade-offs

	tuple	namedtuple / NamedTuple	`@dataclass` (Chapter 10)	dict
Access by name	No	Yes	Yes	Yes (`d["k"]`)
Immutable	Yes	Yes	Optional (`frozen=True`)	No
Hashable	If items are	If items are	If frozen	No
Memory	Smallest	Same as tuple	Bigger (`slots=True` shrinks)	Biggest
Methods / validation	No	Limited	Full class	No
Also behaves as a sequence	Yes	Yes (surprise: `p == (3, 4)` is `True`)	No	No

When not to use it

Records needing mutation, validation or methods — or where equality with plain tuples would be a bug — use `@dataclass`.

Real-world examples

`urllib.parse.urlparse()` → `ParseResult(scheme=..., netloc=...)` (a namedtuple subclass). `sys.version_info`, `time.struct_time`, `os.stat_result` are *struct sequences* — the C-level cousin. psycopg 3 (a PostgreSQL driver) offers a `namedtuple_row` row factory.

In the interview

> **30-second answer:** "A namedtuple is a tuple subclass with named fields — immutable, lightweight, unpackable, with `_asdict` and `_replace`. I use it (or `typing.NamedTuple` for type hints) for simple immutable records and switch to dataclasses when I need mutability, validation or methods."

TL;DR

Named fields on a tuple for immutable records; dataclasses for richer objects.

5.14 Choosing: `list` vs `tuple` vs `array.array` vs NumPy

What it is

NumPy's `ndarray` (third-party, `pip install numpy`) is a typed, contiguous, possibly multi-dimensional array whose operations run as compiled loops over the whole array (**vectorisation**).

How they compare — measured

One million integers, "multiply every element by 2":

```
container         memory    x*2 time
list              36.0MB      20.8ms
array.array        8.2MB      45.7ms
numpy              8.0MB       0.6ms
```

`list` holds pointers + int objects (most memory, interpreter loop); `array.array('q')` holds raw 8-byte ints but every read creates a Python int, so a Python loop is **slower** than the list; NumPy holds raw ints **and** does the arithmetic in C → ~35× faster than the list.

Trade-offs

	`list`	`tuple`	`array.array`	`numpy.ndarray`
Mutable	Yes	No	Yes	Yes (fixed size)
Element types	Any, mixed	Any, mixed	One C numeric type	One dtype (element data type: numeric, bool, fixed strings, objects)
Memory for numbers	High	High	Low	Low
Hashable	No	Yes (if items are)	No	No
Fast element-wise maths	No	No	No	Yes (vectorised)
Multi-dimensional	Via nesting	Via nesting	No	Yes
Standard library	Yes	Yes	Yes	No (third-party)
`+` means	Concatenate	Concatenate	Concatenate	**Element-wise add**

When to use which

`list` for changing collections; **`tuple`** for fixed records, dict keys, return values; **`array.array`** for compact numbers with no extra dependency and binary formats; **NumPy** for numerical computing (statistics, images, ML features, linear algebra) — the foundation under pandas, scikit-learn, SciPy and matplotlib.

Common mistakes

- Python-level loops over NumPy arrays lose the speed — use `arr * 2`, `arr[arr > 0]`.

- `np_a + np_b` adds element-wise; concatenate with `np.concatenate`.
- Picking `array.array` for speed — it's for memory.

In the interview

> **30-second answer:** "List for general mutable data, tuple for immutable records and dict keys, `array.array` for compact homogeneous numbers without dependencies, NumPy for fast numerical work because it stores raw values contiguously and runs vectorised C loops."

TL;DR

General → list. Fixed → tuple. Compact numbers → array. Fast numbers → NumPy.

Chapter TL;DR

Topic	Remember
`array` module	Typed, compact, mutable; type codes set size/range; no `sort`/`copy` methods; `clear()` from 3.13; memory win, not speed win.
Lists	Ordered, mutable, mixed types; index/slice/update; `+` new, `extend` in place.
Internals	Dynamic array of pointers; ~12.5% over-allocation → amortised O(1) append; front/middle insert/delete O(n); `in` O(n).
Add/remove	`append`, `extend`, `insert`; `remove` (value), `pop` (index, returns), `del`, `clear`.
Slicing	`[start:stop:step]`; slice assignment can resize; stepped assignment needs equal length.
Comprehensions	`[expr for x in it if cond]`; generator expression for one-pass use; loops for complex logic.
Sorting	`sort()` in place → None; `sorted()` new list; `key`, `reverse`; Timsort is stable, O(n log n), O(n) on sorted input.
Copying	`=` aliases; `copy()`/`[:]` shallow; `deepcopy` recursive; never `[[0]*n]*m`; no mutable defaults.
Tuples	Commas make them; `(x,)` for one; shallowly immutable; hashable if contents are; records.
Unpacking	Counts must match; `*rest` → list; swap with `a, b = b, a`.
Joining tuples	`+` for few; `chain.from_iterable` for many; `sum`/`+=` loops are quadratic.
Tuple methods	`count`, `index`.
namedtuple	Named, immutable records; `_asdict`, `_replace`; dataclass for more.
Choosing	list / tuple / array / NumPy by mutability, memory, and speed needs.

Chapter 6 — Dictionaries, Sets and the `collections` Toolbox

Lists and tuples find things by **position**; dictionaries and sets find things by **value**, in constant time on average whether they hold ten items or ten million. That makes them the workhorses of real code: caches (stored results reused to save time — see System Design Fundamentals), indexes, counters, configuration, JSON, de-duplication, graph algorithms (route finding over networks of connected points). This chapter covers how they work (hashing — turning a value into an integer that says where to store it), every method, and which specialised container from `collections`, `heapq` and `bisect` to use when a plain dict, set or list isn't quite right.

> **Key term — Big-O notation:** how work grows with *n* items. **O(1)** (constant): independent of size; **O(n)** (linear): proportional; **O(log n)**: grows very slowly (doubling *n* adds one step). The trend, not timings.

What you'll learn

1. Dictionaries: define, access, update, remove, copy, loop, operators (`in`, `|`), insertion order.
2. Hashing: "hashable", how a dict/set finds a key, the big-O table.
3. Nested dictionaries and JSON.
4. Every dict method; `get`, `setdefault`, `update`, `|` compared; view objects.
5. Dict and set comprehensions.
6. Sets: define, add/update, remove/discard, copy, loop.
7. Set operations and every set method.
8. `frozenset`.
9. `collections`: `Counter`, `defaultdict`, `OrderedDict`, `deque`, `ChainMap`, `namedtuple`.
10. `heapq` (priority queues — the most urgent item always comes out first), `bisect` (binary search — finding a position by repeatedly halving a sorted list), and a when-to-use table.

Runnable code is in `code/`. Outputs are real (Python 3.14; runs on 3.12+). Printed sets of **strings** can change order between runs — string hashes are randomised per process (one running instance of a program; 6.2).

6.1 Dictionaries — key → value

What it is

A **dictionary** (`dict`) stores **key–value pairs**, looked up by key (`ages["Alice"]`), not position. Keys are unique; values can be anything.

- Literal: `{"Alice": 22, "Bob": 25}`.
- **Mutable:** add, change, delete pairs any time.
- **Keys must be hashable** (roughly immutable: strings, numbers, tuples of hashables, `None`, `frozenset`) — not lists, dicts, sets (6.2).
- **Insertion-ordered:** iteration follows first-insertion order — a language guarantee since **Python 3.7** (in CPython 3.6 it was an implementation detail, not promised). Tutorials calling dicts "unordered" are out of date — but dicts aren't *indexable* by position (`d[0]` looks up the key `0`).

Why it matters

Anywhere you'd search a list for "the item whose name is X", a dict does it in one step. Python itself runs on dicts: module globals, object attributes (`obj.__dict__`), keyword arguments and class namespaces.

How it works — define and access

```
ages = {"Alice": 22, "Bob": 25, "Eve": 19}
print(ages, len(ages))
print(ages["Alice"], ages.get("Bob"), ages.get("Charlie"), ages.get("Charlie", "Not Found"))
try:
    ages["Charlie"]
except KeyError as e:
    print("KeyError:", e)
other_ways = dict(a=1, b=2), dict([("x", 1), ("y", 2)]), dict(zip("pq", [7, 8]))
print(*other_ways)
print({(59.9, 10.7): "Oslo", 42: "int key", None: "None key"})
try:
    {["a", "list"]: 1}
except TypeError as e:
    print("TypeError:", e)
```

Output:

```
{'Alice': 22, 'Bob': 25, 'Eve': 19} 3
22 25 None Not Found
KeyError: 'Charlie'
{'a': 1, 'b': 2} {'x': 1, 'y': 2} {'p': 7, 'q': 8}
{(59.9, 10.7): 'Oslo', 42: 'int key', None: 'None key'}
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

- `d[key]` raises `KeyError` if missing — use when absence is a bug.
- `d.get(key)` returns `None` (or your default) — use when absence is normal.
- Four ways to build: literal, `dict(k=v)` (string keys only), `dict(list_of_pairs)`, `dict(zip(keys, values))`.

How it works — update, remove, copy, loop

```
ages = {"Alice": 22, "Bob": 25, "Eve": 19}
ages["Alice"] = 23                          # overwrite
ages["Dave"] = 20                           # insert (goes to the end)
print(ages)
removed = ages.pop("Eve");  print("pop:", removed, ages)
del ages["Bob"];            print("del:", ages)
print(ages.pop("Zed", "no such key"))       # default avoids KeyError
```

Output:

```
{'Alice': 23, 'Bob': 25, 'Eve': 19, 'Dave': 20}
pop: 19 {'Alice': 23, 'Bob': 25, 'Dave': 20}
del: {'Alice': 23, 'Dave': 20}
no such key
```

`d[k] = v` **updates** (existing key keeps its position) or **inserts** (new key goes to the end). `pop` returns the value; `del` just deletes.

```
original = {"Alice": 23, "Charlie": 21, "tags": ["x"]}
copied = original.copy()                    # shallow
copied["Alice"] = 24
copied["tags"].append("y")                  # shared inner list!
print(original, copied, sep="\n")
for key in original:                        # iterates keys
    print(key, end=" ")
for key, value in {"Alice": 23, "Charlie": 21}.items():
    print(f"{key} is {value} years old")
```

Output:

```
{'Alice': 23, 'Charlie': 21, 'tags': ['x', 'y']}
{'Alice': 24, 'Charlie': 21, 'tags': ['x', 'y']}
Alice Charlie tags
Alice is 23 years old
Charlie is 21 years old
```

`copy()` (or `dict(d)`) is **shallow**: both share `tags`. Use `copy.deepcopy` for nested data (Chapter 5, 5.8). Looping yields **keys**; `.values()` values; `.items()` `(key, value)` pairs.

Dictionary operators

Operator	Meaning	Example
`k in d` / `k not in d`	Is `k` a **key**? (O(1))	`"Alice" in ages`
`d[k]`, `d[k] = v`, `del d[k]`	Get / set / delete	
`d1 | d2`	New merged dict; right wins on conflicts (3.9+)	`{"a":1} | {"a":2}` → `{'a': 2}`
`d1 |= d2`	Merge into `d1` in place (like `update`)	
`{**d1, **d2}`	Merge via unpacking (3.5+)	
`==`	Same pairs (order ignored)	

```
d = {"Alice": 23, "Charlie": 21}
print("Alice" in d, "Bob" not in d, 23 in d, 23 in d.values())
a, b = {"x": 1, "y": 2}, {"y": 20, "z": 3}
print("a | b :", a | b)
a |= {"w": 0};  print("a |= :", a)
print("{**a, **b}:", {**a, **b})
print({"a": 1, "b": 2} == {"b": 2, "a": 1})
views = d.keys()
d["New"] = 0
print(views)                                # views are live
```

Output:

```
True True False True
a | b : {'x': 1, 'y': 20, 'z': 3}
a |= : {'x': 1, 'y': 2, 'w': 0}
{**a, **b}: {'x': 1, 'y': 20, 'w': 0, 'z': 3}
True
dict_keys(['Alice', 'Charlie', 'New'])
```

`23 in d` is `False`: `in` checks **keys**; `in d.values()` is an O(n) scan.

Common mistakes

- **Changing size while looping** → `RuntimeError: dictionary changed size during iteration`. Loop over a snapshot `list(d)` or build a new dict.
- `d[k]` for optional keys → `KeyError`.
- Expecting `d.copy()` to copy nested containers.
- `d.keys()[0]` — views (6.4) aren't indexable; use `next(iter(d))`.

Real-world examples

- JSON objects are dicts (`json.loads`) — every parsed API response.
- `functools.lru_cache` and hand-written memo tables are dicts from arguments to results.
- Django/Flask pass template context and request data as (dict-like) mappings.

In the interview

> **30-second answer:** "A dict maps hashable keys to values with average O(1) get, set, delete and membership, using a hash table. Since 3.7 it preserves insertion order. `d[k]` raises KeyError, `get` returns a default, `|` merges (3.9+), and `copy()` is shallow."

- *Ordered?* → Yes, insertion order since 3.7; equality ignores order.
- *`d[k] = v` on an existing key?* → Value replaced, position unchanged.
- *`in` checks?* → Keys.

Self-check

1. `{"a": 1} | {"a": 2, "b": 3}`? → `{'a': 2, 'b': 3}`.

TL;DR

Key → value, O(1) average, insertion-ordered (3.7+), hashable keys, `[]` raises, `get` defaults, `|` merges, `copy` is shallow.

6.2 Hashing — how dicts and sets find things in O(1)

What it is

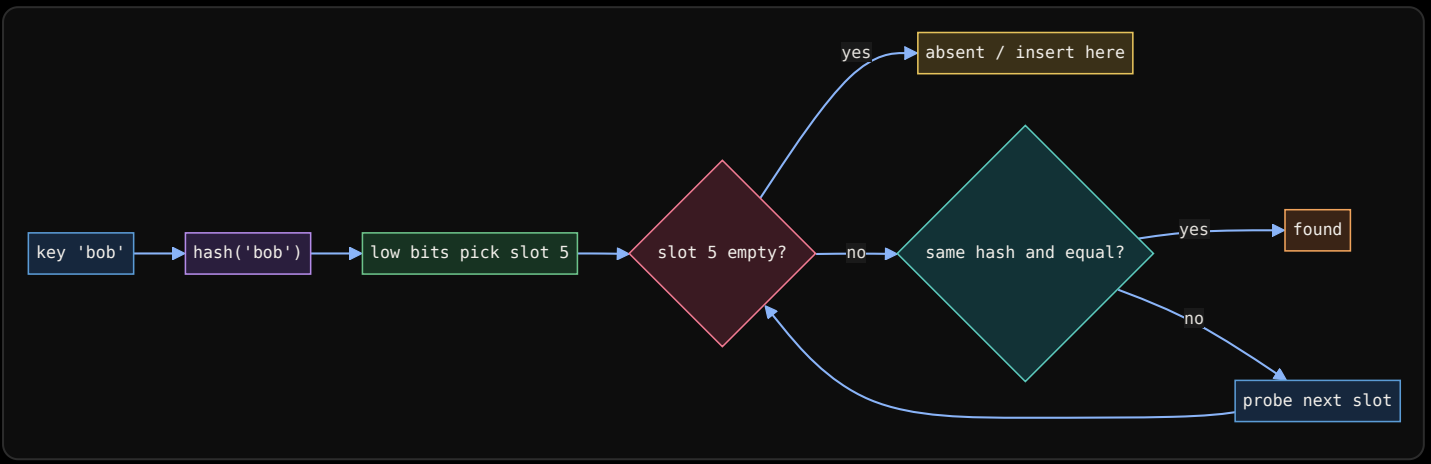
A **hash** is an integer computed by `hash(x)`. A **hash table** is an array of slots; the hash picks a key's slot. Dicts and sets are hash tables. An object is **hashable** if its hash never changes during its lifetime and it supports `==`.

The data-model contract: **if** `a == b` **then** `hash(a) == hash(b)`. Not the reverse — different values may share a hash (a **collision**).

Hashing explains why lookups are O(1), why keys must be immutable, why `1` and `1.0` are one key, and why sets have no order.

How it works — a lookup, step by step

1. `h = hash(key)`.
2. Low bits of `h` choose a slot (table size is a power of two).
3. Empty slot → absent (lookup) or insert here.
4. Occupied → same hash **and** (`is` or `==`) → found.
5. Otherwise a collision: CPython uses **open addressing** — it computes the next slot with a pseudo-random probe sequence (scrambled but repeatable, mixing in more hash bits) and repeats from 3.
6. The table stays at most about **2/3 full**; beyond that CPython allocates a bigger table and re-inserts everything (**resize**) — rare enough that insertion is **amortised O(1)**.



Since 3.6 CPython's dict is a **compact dict**: a sparse *index* table of slots pointing into a dense *entries* array of `(hash, key, value)` in insertion order. That's where ordering comes from, and it made dicts 20–25% smaller. Sets use a simpler non-compact table — no order.

Growth via `sys.getsizeof` (8 slots hold 5 entries, 16 hold 10, 32 hold 21, 64 hold 42 — the 2/3 rule):

```
len  bytes
  1  224
  6  352
 11  632
 22  1168
 43  2264
```

What is hashable?

```
42              hashable    hash=42
3.0             hashable    hash=3
'hi'            hashable    hash=<salted: differs every run>
(1, 2)          hashable    hash=-3550055125485641917
frozenset({1})  hashable    hash=-558064481276695278
None            hashable    hash=4238894112
True            hashable    hash=1
[1]             UNHASHABLE  (unhashable type: 'list')
{1: 2}          UNHASHABLE  (unhashable type: 'dict')
{1}             UNHASHABLE  (unhashable type: 'set')
(1, [2])        UNHASHABLE  (unhashable type: 'list')
```

Hashable	Not hashable
`int`, `float`, `complex`, `bool`, `str`, `bytes`, `None`, `frozenset`, `range`	`list`, `dict`, `set`, `bytearray`
`tuple` / `namedtuple` **if every item is hashable**	A tuple containing a list
Instances of your classes (hashed by identity by default)	Classes defining `__eq__` (behind `==`) without `__hash__` (behind `hash()`; Python sets `__hash__ = None`)
Frozen dataclasses (`@dataclass(frozen=True)`)	Regular dataclasses with `eq=True` (the default)

Equal values are the same key. `1 == 1.0 == True` and they hash equal:

```
d = {1: "int one"}
d[1.0] = "float one"
d[True] = "bool true"
print(d)          # {1: 'bool true'}  — one key; the first key object is kept, value replaced
```

String hashes are randomised. To stop "hash-flooding" denial of service (attackers crafting many colliding keys), `str` and `bytes` hashes are salted with a random per-process seed (default since 3.3; controlled by the `PYTHONHASHSEED` environment variable). Hence set-of-strings order varies between runs. Small ints hash to themselves (`hash(12345) == 12345`; `hash(-1)` is `-2` because `-1` signals an error in C).

Mutating a key breaks the table. If a key's hash depends on mutable state that changes while it's in a dict, the entry sits in the *old* hash's slot:

```
lookup with same object: LOST
lookup with old value  : LOST
```

The same object is looked for in the wrong slot; a fresh object with the old value finds the slot but isn't `==`. That's why built-in mutable types refuse to be hashable.

Big-O table (average case; worst case is O(n) when many keys collide)

Operation	`dict`	`set`
Lookup `d[k]` / `k in d` / `x in s`	O(1)	O(1)
Insert / update `d[k] = v`, `s.add(x)`	O(1) amortised	O(1) amortised
Delete `del d[k]`, `pop`, `s.remove(x)`, `discard`	O(1)	O(1)
`len()`	O(1)	O(1)
Iterate / `copy()` / `keys()`→list	O(n)	O(n)
`get`, `setdefault`, `popitem`	O(1)	`pop()` O(1)
`d1 | d2`, `update(other)`	O(len(other)) + copy	—
Union `s | t`	—	O(len(s) + len(t))
Intersection `s & t`	—	O(min(len(s), len(t)))
Difference `s - t`	—	O(len(s))
Symmetric difference `s ^ t`	—	O(len(s) + len(t))
`s <= t` (subset)	—	O(len(s))
Search a **value** `v in d.values()`	O(n)	—

Measured (`code/08_lookup_benchmark.py`, looking for the last element):

```
        n    list (µs)   set (µs)  dict (µs)
    1,000         5.26      0.030      0.031
   10,000        52.49      0.030      0.031
  100,000       536.93      0.030      0.031
1,000,000      5533.07      0.030      0.032
```

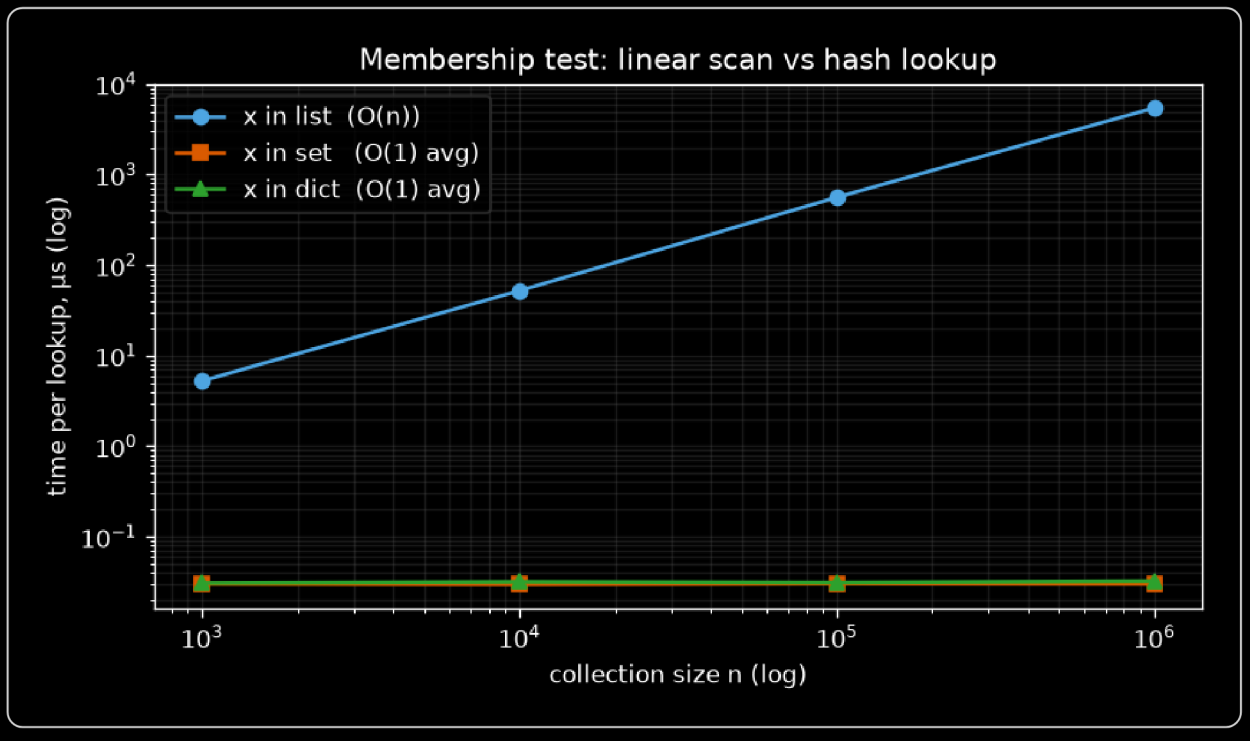


Figure — log–log: the list rises with slope 1 (linear); set and dict stay flat at ~30 ns. At a million items the set is ~180,000× faster.

Trade-offs

- **Memory:** spare slots (up to 1/3 empty) plus stored hashes — much more than a list.

- **No positional access**; sets have no order.
- **Worst case** O(n) under many collisions — rare, but possible with a bad `__hash__` (e.g. `return 1`).

Common mistakes

- Defining `__eq__` and losing hashability — define `__hash__` from the same immutable fields (or `@dataclass(frozen=True)`).
- A `__hash__` using mutable fields.
- Float keys after arithmetic: `0.1 + 0.2` is a different key from `0.3`.

In the interview

> **30-second answer:** "Dicts and sets are hash tables using open addressing. `hash(key)` picks a slot; collisions probe other slots; equality confirms the match. The table stays ≤ 2/3 full and resizes, so operations are O(1) average, O(n) worst. Keys must be hashable — their hash can't change — and equal objects must have equal hashes, which is why `1`, `1.0` and `True` are the same key."

- *Why can't lists be keys?* → Mutable; a changed hash loses the entry.
- *Why does set order change between runs?* → Per-process string-hash salting.
- *How are hash-table dicts ordered?* → Compact dict: index into an insertion-ordered entries array.

Self-check

1. `len({1, 1.0, True})`? → 1.

TL;DR

hash → slot → compare; ≤ 2/3 full; O(1) average. Hashable = hash never changes; equal ⇒ same hash; string hashes salted per run.

6.3 Nested dictionaries

What it is

A **nested dictionary** has dicts (or lists of dicts) as values, modelling **hierarchical** data: family trees, config files, JSON responses.

How it works

Read and write by chaining keys (`family["children"]["Alice"]["age"]`), add a branch by assigning a new dict at any level, loop one level with `.items()`.

```
family = {
    "name": "John Doe",
    "children": {
        "Alice": {"age": 10, "gender": "Female"},
        "Bob": {"age": 8, "gender": "Male"},
    },
}
print("Parent:", family["name"])
print("Alice's age:", family["children"]["Alice"]["age"])
family["children"]["Alice"]["age"] = 11                 # update deep value
family["children"]["Cara"] = {"age": 3, "gender": "Female"}  # add a new branch
for child, info in family["children"].items():
    print(f"  {child}: {info['age']} ({info['gender']})")
```

Output:

```
Parent: John Doe
Alice's age: 10
  Alice: 11 (Female)
  Bob: 8 (Male)
  Cara: 3 (Female)
```

Safe access when a level may be missing:

```
print(family.get("children", {}).get("Zed", {}).get("age", "unknown"))   # unknown

def deep_get(d, *keys, default=None):
    for k in keys:
        if not isinstance(d, dict) or k not in d:
            return default
        d = d[k]
    return d
print(deep_get(family, "children", "Bob", "age"), deep_get(family, "x", "y", default=-1))  # 8 -1
```

JSON round trip — nested dicts are what `json` reads and writes:

```
text = json.dumps(family, indent=2)
back = json.loads(text)
print(back == family)                                          # True
print(json.loads('{"1": [1, 2]}'), json.dumps({1: (1, 2)}))       # {'1': [1, 2]} {"1": [1, 2]}
```

JSON keys are always strings and JSON has no tuples: int keys return as strings, tuples as lists.

Trade-offs

Nested dicts	Dataclasses / Pydantic models (validating data classes)
Zero setup, matches JSON exactly	Named, typed fields, validation, IDE autocomplete
Key typos fail only at run time	Typos caught by the type checker
Ad-hoc/dynamic data	Stable, well-known schemas (agreed data shapes)

When not to use it

For deep, well-defined data used across a codebase, convert to dataclasses/Pydantic at the boundary (where data enters or leaves) and pass typed objects inside.

Common mistakes

- `KeyError` halfway down a chain — `get(..., {})` chains, `deep_get`, or `try/except KeyError`.
- Shallow-copying a nested config then mutating an inner dict — changes the original.
- Hand-written `if key not in d: d[key] = {}` — use `setdefault` or `defaultdict` (6.9).

Real-world examples

GitHub/Stripe API responses and JSON, YAML, TOML config files (TOML via `tomllib`) load as nested dicts; pandas' `json_normalize` flattens them into tables.

TL;DR

Chain keys to read and write; guard missing levels with `get(..., {})`; nested dicts ≈ JSON.

6.4 Dictionary methods and view objects

The full method table

Method	What it does	Cost
`d.get(k, default=None)`	Value for `k`, or `default` — never raises	O(1)
`d.setdefault(k, default=None)`	Return `d[k]`; if missing, first insert `k: default`	O(1)
`d.update(other, **kw)`	Merge pairs from a dict / iterable of pairs / keywords, overwriting	O(len(other))
`d.pop(k[, default])`	Remove `k`, return its value; `KeyError` if absent and no default	O(1)
`d.popitem()`	Remove and return the **last inserted** `(k, v)` (LIFO — last in, first out); `KeyError` if empty	O(1)
`d.clear()`	Remove everything	O(n)
`d.copy()`	Shallow copy	O(n)
`dict.fromkeys(keys, value=None)`	**Class method**: every key mapped to the **same** `value`	O(n)
`d.keys()` / `d.values()` / `d.items()`	Live **view** objects	O(1) to create

Plus `len(d)`, `k in d`, `del d[k]`, `reversed(d)` (3.8+), `d | other`, `d |= other` (3.9+).

```
info = {"name": "John", "age": 28, "city": "New York"}
print(info.get("name"), info.get("phone"))
info.update({"age": 29, "phone": "123-456-7890"})       # overwrite + add
info.update(zip_code="10001")                            # keyword form
print(info)
info.clear()
print(info)

new = dict.fromkeys(["a", "b", "c"], 1)
print(new)
print(new.pop("b"), new, new.items())
shared = dict.fromkeys(["x", "y"], [])                   # ONE list shared by all keys!
shared["x"].append(1)
print("fromkeys trap:", shared)
d = {"first": 1, "second": 2, "third": 3}
print(d.popitem(), d)                                    # LIFO: last inserted
```

Output:

```
John None
{'name': 'John', 'age': 29, 'city': 'New York', 'phone': '123-456-7890', 'zip_code': '10001'}
{}
{'a': 1, 'b': 1, 'c': 1}
1 {'a': 1, 'c': 1} dict_items([('a', 1), ('c', 1)])
fromkeys trap: {'x': [1], 'y': [1]}
('third', 3) {'first': 1, 'second': 2}
```

The `fromkeys` trap is the dict cousin of `[[0]*3]*3`: every key points at **one** list. Use `{k: [] for k in keys}`.

`get` vs `setdefault` vs `update` vs `|` — which one?

Want to…	Use	Changes `d`?
Read with a fallback	`d.get(k, default)`	No
Read, inserting a default if missing (grouping)	`d.setdefault(k, [])`	Only if missing
Merge another mapping into `d`	`d.update(other)` or `d |= other`	Yes
Get a **new** merged dict	`d | other` or `{**d, **other}`	No
Count / group many items	`Counter` / `defaultdict` (6.9)	—

```
words = ["apple", "avocado", "banana", "blueberry", "cherry"]
by_letter = {}
for w in words:
    by_letter.setdefault(w[0], []).append(w)            # insert [] only if missing
print(by_letter)
counts = {}
for ch in "hello":
    counts[ch] = counts.get(ch, 0) + 1                  # get with default for counting
print(counts)
```

Output:

```
{'a': ['apple', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
{'h': 1, 'e': 1, 'l': 2, 'o': 1}
```

`setdefault`'s default is **evaluated every call** even if the key exists — harmless for `[]`, costly for expensive defaults; `defaultdict` avoids it.

View objects

`keys()`, `values()`, `items()` return **views**: lightweight live windows onto the dict. Key views (and item views, if values are hashable) support set operations.

```
stock = {"apple": 3, "pear": 0}
k, v, it = stock.keys(), stock.values(), stock.items()
stock["fig"] = 7
print(k, v, it, sep="\n")
print(k & {"apple", "kiwi"}, sorted(k - {"pear"}))      # keys views support set ops
print(sorted(stock, key=stock.get, reverse=True))       # keys sorted by value
print(max(stock.items(), key=lambda kv: kv[1]))
```

Output:

```
dict_keys(['apple', 'pear', 'fig'])
dict_values([3, 0, 7])
dict_items([('apple', 3), ('pear', 0), ('fig', 7)])
{'apple'} ['apple', 'fig']
['fig', 'apple', 'pear']
('fig', 7)
```

Common mistakes

- `dict.fromkeys(keys, [])` shares one mutable value.
- `d.update(k=v)` with `k` a variable uses the literal name `"k"`; write `d[k] = v`.
- Assuming `popitem()` is random — since 3.7 it's always the last inserted.

In the interview

30-second answer: "`get` reads with a default, `setdefault` reads-or-inserts, `update` / `|=` merge in place, `|` returns a new merged dict with the right side winning. `pop` removes by key, `popitem`

> removes the last inserted. `keys/values/items` are live views, and key views support set operations."

- *Keys common to two dicts?* → `d1.keys() & d2.keys()`.

TL;DR

`get` (read), `setdefault` (read-or-insert), `update` / `|=` (merge in place), `|` (merge new), `pop` / `popitem`; views are live and set-like; avoid `fromkeys` with mutable values.

6.5 Dict and set comprehensions

What it is

Chapter 5's comprehension syntax also builds dicts and sets:

```
{key_expr: value_expr for item in iterable if condition}   # dict
{expr for item in iterable if condition}                   # set
```

How it works

```
words = ["apple", "banana", "cherry"]
print({w: len(w) for w in words})
prices = {"apple": 1.2, "banana": 0.5, "cherry": 3.0}
print({k: v for k, v in prices.items() if v >= 1})
print({v: k for k, v in {"a": 1, "b": 2}.items()})    # invert
print(sorted({c for c in "mississippi"}))              # set comprehension
```

Output:

```
{'apple': 5, 'banana': 6, 'cherry': 6}
{'apple': 1.2, 'cherry': 3.0}
{1: 'a', 2: 'b'}
['i', 'm', 'p', 's']
```

- Filter a dict by iterating `.items()`.
- **Inverting** swaps keys and values; shared values → the later key wins (values must be hashable).
- `{}` is an **empty dict**; empty set is `set()`.

Common mistakes

- Duplicate keys silently overwrite earlier ones.
- A comprehension where `dict(zip(keys, values))` is simpler.

TL;DR

`{k: v for ...}` for dicts, `{x for ...}` for sets; `{}` alone is a dict.

6.6 Sets — unique, unordered collections

What it is

A **set** is an unordered, mutable collection of **unique, hashable** items: `{"apple", "banana"}` or `set(iterable)`. Duplicates are dropped; no indexing (`s[0]` → `TypeError`).

Why it matters

- **De-duplication** in one step: `set(emails)`.
- **Fast membership** (O(1) average): "seen this before?" in crawlers (programs following links between web pages), graph searches, validation.

- **Set algebra** answers "which users are in both groups?" with one operator (6.7).

How it works — define, add, update, remove, copy, loop

```
fruits = {"apple", "banana", "cherry", "apple"}
colors = set(["red", "blue", "red", "green"])
print(sorted(fruits), sorted(colors), len(fruits))   # sorted() only for stable printing
empty_set, empty_dict = set(), {}
print(type(empty_set).__name__, type(empty_dict).__name__)
print(set("hello"), set(range(3)))                    # str order varies run to run

nums = {1, 2, 3}
nums.add(4); nums.add(2)                  # adding an existing item: no effect
nums.update([5, 6], (7,))                 # any iterables
print(nums)
nums.remove(1)
nums.discard(60)                          # absent: silently ignored
try:
    nums.remove(60)
except KeyError as e:
    print("KeyError:", e)
backup = nums.copy()
nums.clear()
print(nums, backup)
print("pop() ->", backup.pop(), backup)    # removes an ARBITRARY element
```

Output:

```
['apple', 'banana', 'cherry'] ['blue', 'green', 'red'] 3
set dict
{'l', 'h', 'e', 'o'} {0, 1, 2}          <- letter order varies from run to run
{1, 2, 3, 4, 5, 6, 7}
KeyError: 60
set() {2, 3, 4, 5, 6, 7}
pop() -> 2 {3, 4, 5, 6, 7}
```

- `add(x)` one item; `update(*iterables)` many.
- `remove(x)` raises `KeyError` if absent; `discard(x)` doesn't.
- `pop()` removes an **arbitrary** element.
- `copy()` is independent (items are immutable, so shallow = full).
- Small ints often *look* sorted because they hash to themselves — coincidence, not guarantee.

De-duplicate keeping order — sets lose order, dicts don't:

```
items = ["b", "a", "b", "c", "a"]
print(list(dict.fromkeys(items)))       # ['b', 'a', 'c']
```

Common mistakes

- `{}` for an empty set.
- `set("hello")` splits characters; `{"hello"}` is one item.
- Lists or dicts in a set → `TypeError: unhashable type`.
- Relying on set order in output — sort it.

TL;DR

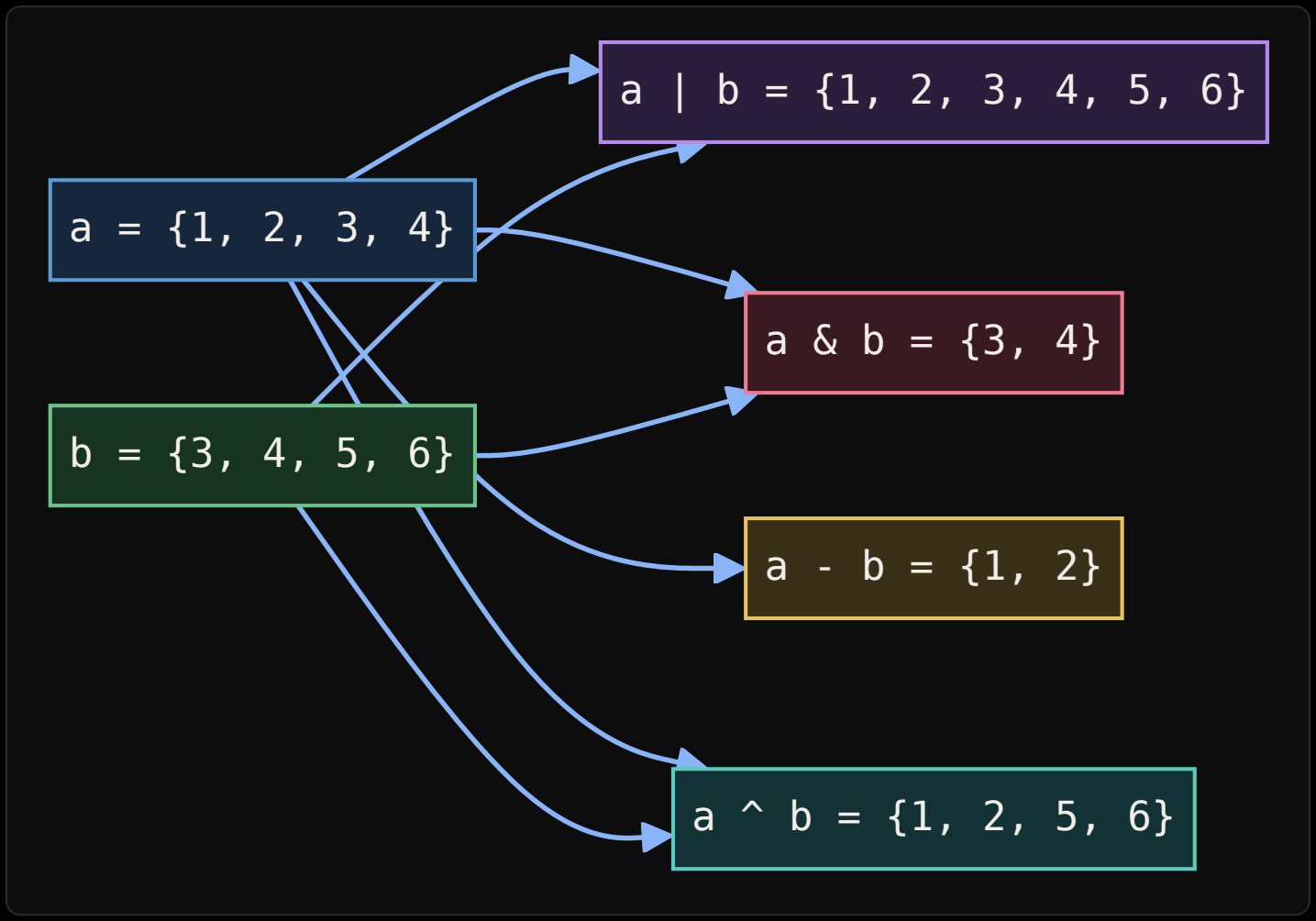
Unique, unordered, hashable items; `add` / `update`; `remove` (raises) / `discard` (silent); `set()` for empty; O(1) membership.

6.7 Set operations

What it is

Set algebra as **operators** (both sides sets/frozensets) or **methods** (any iterable, several arguments).

Operation	Operator	Method	Result
Union — in either	`a | b`	`a.union(b, ...)`	Everything from both
Intersection — in both	`a & b`	`a.intersection(b, ...)`	Common items
Difference — in `a` not `b`	`a - b`	`a.difference(b, ...)`	`a` minus `b`
Symmetric difference — in exactly one	`a ^ b`	`a.symmetric_difference(b)`	Union minus intersection
Subset	`a <= b`	`a.issubset(b)`	Every item of `a` is in `b`
Proper subset	`a < b`	—	Subset and not equal
Superset	`a >= b`	`a.issuperset(b)`	Every item of `b` is in `a`
Disjoint	—	`a.isdisjoint(b)`	No common items



```python
a, b = {1, 2, 3, 4}, {3, 4, 5, 6}
print("union        a | b :", a | b, a.union(b))
print("intersection a & b :", a & b, a.intersection(b))
print("difference   a - b :", a - b, a.difference(b), "  b - a:", b - a)
print("sym. diff    a ^ b :", a ^ b, a.symmetric_difference(b))
print("method accepts any iterable:", a.union([9, 10]), " operator needs sets")
try:
    a | [9]
except TypeError as e:
    print("TypeError:", e)
print("multi-arg:", a.intersection(b, {4, 99}))

small, big, other = {1, 2}, {1, 2, 3}, {7}
print(small <= big, small.issubset(big), small < big, big >= small, big.issuperset(small))
print(small.isdisjoint(other), small < small, small <= small)
```

Output:

```
union        a | b : {1, 2, 3, 4, 5, 6} {1, 2, 3, 4, 5, 6}
intersection a & b : {3, 4} {3, 4}
difference   a - b : {1, 2} {1, 2}   b - a: {5, 6}
sym. diff    a ^ b : {1, 2, 5, 6} {1, 2, 5, 6}
method accepts any iterable: {1, 2, 3, 4, 9, 10}  operator needs sets
TypeError: unsupported operand type(s) for |: 'set' and 'list'
multi-arg: {4}
True True True True True
True False True
```

Difference is **one-directional**; a set is a subset (`<=`) but not a proper subset (`<`) of itself.

Trade-offs

Operators are shorter and force explicit conversion (catching type mistakes); methods are flexible.

Real-world examples

- Permissions: `required <= user_permissions`.
- Sync diffs: `new_ids - old_ids` (added), `old_ids - new_ids` (removed).
- Recommendations via intersections; Jaccard similarity (shared ÷ all distinct) = `len(a & b) / len(a | b)` for near-duplicate detection.

In the interview

30-second answer: "`|` union, `&` intersection, `-` difference, `^` symmetric difference, `<=` / `>=` subset/superset, `isdisjoint`. Operators need sets; methods accept any iterable. Intersection costs O(min(len a, len b)) because it iterates the smaller set and probes the larger."

Self-check

1. `{1, 2, 3} ^ {2, 3, 4}`? → `{1, 4}`.

TL;DR

`| & - ^` plus subset/superset/disjoint; methods take any iterable; difference is one-directional.

6.8 Set methods (complete table) and `frozenset`

All set methods

Method	What it does	Operator form	Returns
`add(x)`	Add one element (no effect if present)	—	`None`
`update(*its)`	Add all elements from iterables	`|=`	`None`
`remove(x)`	Remove `x`; `KeyError` if absent	—	`None`
`discard(x)`	Remove `x` if present	—	`None`
`pop()`	Remove and return an arbitrary element; `KeyError` if empty	—	element
`clear()`	Remove everything	—	`None`
`copy()`	Shallow copy	—	new set
`union(*its)`	All elements	`|`	new set
`intersection(*its)`	Common elements	`&`	new set
`intersection_update(*its)`	Keep only common elements	`&=`	`None`
`difference(*its)`	Elements not in the others	`-`	new set
`difference_update(*its)`	Remove elements found in the others	`-=`	`None`
`symmetric_difference(it)`	In exactly one	`^`	new set
`symmetric_difference_update(it)`	Update to "in exactly one"	`^=`	`None`
`issubset(it)`	All my elements are in `it`	`<=`	bool
`issuperset(it)`	All of `it`'s elements are in me	`>=`	bool
`isdisjoint(it)`	No common elements	—	bool

`_update` methods mutate **in place** and return `None`; plain ones return a **new** set. Each `_update` has an augmented-operator twin (`&=`, `-=`, `^=`, `|=`) that requires sets on both sides.

```
s = {1, 2, 3, 4}
s.intersection_update({2, 3, 9});         print("&= :", s)
s.difference_update({3});                 print("-= :", s)
s.symmetric_difference_update({2, 8});    print("^= :", s)
s |= {100};                               print("|= :", s)
```

Output:

```
&= : {2, 3}
-= : {2}
^= : {8}
|= : {8, 100}
```

`frozenset` — the immutable set

What it is

A **frozenset** can't change after creation: all non-mutating methods (`union`, `issubset`, …), none of the mutating ones (`add`, `remove`, `update`, …).

Why it matters

Immutable ⇒ **hashable**: usable as a dict key or set element — which a normal set never can be.

```
fs = frozenset([1, 2, 3])
try:
    fs.add(4)
except AttributeError as e:
    print("AttributeError:", e)
groups = {frozenset({"alice", "bob"}): "team A"}   # usable as a dict key
print(groups[frozenset({"bob", "alice"})])
print(fs | {9}, type(fs | {9}).__name__)            # result type follows the left operand
```

Output:

```
AttributeError: 'frozenset' object has no attribute 'add'
team A
frozenset({1, 2, 3, 9}) frozenset
```

Lookup works regardless of member order — sets compare and hash by contents — ideal for "unordered pair" keys, e.g. an undirected edge (connection with no direction) `frozenset((u, v))`.

Common mistakes

- Expecting `s.union(t)` to modify `s`, or `s.update(t)` to return the result (`None`).
- Treating `pop()` as "smallest".

TL;DR

Plain methods return new sets, `_update` methods mutate; `frozenset` = immutable, hashable set for keys and sets of sets.

6.9 The `collections` module

Specialised containers for the common "plain dict/list isn't quite right" cases.

`Counter` — counting hashable things

What it is: a dict subclass mapping item → count; missing items count `0` (no `KeyError`).

```
words = "the cat and the hat and the bat".split()
c = Counter(words)
print(c)
print(c.most_common(2), c["the"], c["dog"])          # missing key -> 0, no KeyError
c.update(["cat", "cat"]); c.subtract(["the"])
print(c.most_common(3))
a, b = Counter("aabbb"), Counter("abccc")
print(a + b, a - b, a & b, a | b, sep=" | ")
print(Counter("mississippi").total())                # 3.10+
```

Output:

```
Counter({'the': 3, 'and': 2, 'cat': 1, 'hat': 1, 'bat': 1})
[('the', 3), ('and', 2)] 3 0
[('cat', 3), ('the', 2), ('and', 2)]
Counter({'b': 4, 'a': 3, 'c': 3}) | Counter({'b': 2, 'a': 1}) | Counter({'a': 1, 'b': 1}) | Counter({'b': 3, 'c': 3, 'a': 2})
11
```

- `most_common(n)` — top-n (ties keep first-seen order).
- `update` **adds** counts (unlike `dict.update`, which replaces); `subtract` may leave zero/negative counts.
- `+` / `-` **drop** results ≤ 0; `&` = per-key minimum, `|` = maximum.
- `total()` sums counts (3.10+).

Real-world: word frequencies, log-level tallies, top-k errors, "bag of words" (text as word counts) for text classification.

`defaultdict` — dict with automatic defaults

What it is: a dict subclass calling a **factory** (zero-argument callable — `list`, `int`, `set`, …) to create a value the first time a missing key is **accessed with `[]`**.

```
groups = defaultdict(list)
for kind, name in [("fruit", "apple"), ("veg", "kale"), ("fruit", "fig")]:
    groups[kind].append(name)                       # missing key -> list() created
print(dict(groups))
counts = defaultdict(int)
for ch in "banana":
    counts[ch] += 1
print(dict(counts))
print("veg" in groups, "meat" in groups, groups.get("meat"))   # get/in don't create keys
_ = groups["meat"]                                              # [] access DOES
print("after groups['meat']:", dict(groups))
tree = lambda: defaultdict(tree)                                # autovivifying tree
t = tree(); t["a"]["b"]["c"] = 1
```

Output:

```
{'fruit': ['apple', 'fig'], 'veg': ['kale']}
{'b': 1, 'a': 3, 'n': 2}
True False None
after groups['meat']: {'fruit': ['apple', 'fig'], 'veg': ['kale'], 'meat': []}
```

Just reading `d[missing]` inserts an empty value — mysterious extra keys. Use `in` / `.get()` for read-only checks.

Real-world: graph adjacency lists (each node's neighbours, `defaultdict(list)`), inverted indexes (word → documents containing it, `defaultdict(set)`), grouping rows by a column.

`OrderedDict` — a dict with reordering tools

What it is: the pre-3.7 ordered dict; now useful only for its extras:

- `move_to_end(key, last=True)` — O(1) move to end (front with `last=False`).
- `popitem(last=False)` — pop from the **front** (FIFO — first in, first out); `dict.popitem()` only pops the end.
- **Order-sensitive equality**.

```
od = OrderedDict.fromkeys("abcde")
od.move_to_end("b")                 # to the end
od.move_to_end("e", last=False)     # to the front
print(list(od))                     # ['e', 'a', 'c', 'd', 'b']
print(od.popitem(last=False))       # ('e', None)
print(OrderedDict(a=1, b=2) == OrderedDict(b=2, a=1), dict(a=1, b=2) == dict(b=2, a=1))  # False True
```

The classic use — an LRU (least-recently-used) cache:

```python
class LRUCache:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return None
        self.data.move_to_end(key)            # mark as most recently used
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)     # evict least recently used

cache = LRUCache(2)
cache.put("a", 1); cache.put("b", 2); cache.get("a"); cache.put("c", 3)
print("LRU keeps:", list(cache.data))         # LRU keeps: ['a', 'c']
```

Reading `"a"` made it recent, so adding `"c"` evicted `"b"`. For function results prefer `@functools.lru_cache`.

`deque` — double-ended queue

What it is: **O(1)** `append`/`pop` at **both** ends (a list is O(n) at the front). A doubly-linked list (each block links to next and previous) of fixed-size blocks, so middle indexing is O(n).

```python
q = deque([1, 2, 3])
q.append(4); q.appendleft(0)
print("after append/appendleft:", q)
left, right = q.popleft(), q.pop()
print("popleft ->", left, "| pop ->", right, "| now", q)
q.rotate(1); print("rotate(1):", q)
last3 = deque(maxlen=3)
for x in range(6):
    last3.append(x)                   # old items fall off the other end
print("maxlen=3:", last3)
```

Output:

```
after append/appendleft: deque([0, 1, 2, 3, 4])
popleft -> 0 | pop -> 4 | now deque([1, 2, 3])
rotate(1): deque([3, 1, 2])
maxlen=3: deque([3, 4, 5], maxlen=3)
```

- `maxlen` gives a bounded buffer — "last N lines/events" (like Unix `tail -n`).
- `append`/`popleft` are thread-safe in CPython, so deques back simple producer/consumer queues (one part adds work, another takes it); `queue.Queue` adds blocking (waiting for an item) and locking.

Real-world: BFS (breadth-first search, exploring a graph level by level) queues, sliding windows over recent items, capped undo history, rate limiters (capping requests per time period) keeping recent timestamps.

`ChainMap` — layered lookups

What it is: a view over several mappings searched **in order**; first hit wins; nothing copied.

```python
defaults = {"color": "red", "user": "guest", "debug": False}
env = {"user": "admin"}
cli = {"debug": True}
cfg = ChainMap(cli, env, defaults)    # first mapping wins
print(cfg["color"], cfg["user"], cfg["debug"])
cfg["color"] = "blue"                 # writes go to the FIRST mapping only
print(cli, defaults["color"])
child = cfg.new_child({"user": "tmp"})
print(child["user"], cfg["user"])
```

Output:

```
red admin True
{'debug': True, 'color': 'blue'} red
tmp admin
```

Real-world: configuration precedence (command line > environment variables > defaults); nested scopes in interpreters/template engines (`new_child()` pushes a scope). `string.Template.substitute` uses a `ChainMap` internally to combine a mapping with keyword arguments.

`namedtuple` — tuple with field names

Covered in Chapter 5 (5.13): `Card = namedtuple("Card", "rank suit")` → `Card("A", "spades").rank == "A"`, with `_fields`, `_asdict()`, `_replace()`.

```
A Card(rank='10', suit='hearts') ('rank', 'suit')
```

Common mistakes (collections)

- `Counter.update` expecting replacement.
- Creating `defaultdict` keys by reading.
- `list.pop(0)` instead of `deque.popleft()` for queues.
- `OrderedDict` just for ordering on 3.7+.

In the interview

30-second answer: "`Counter` counts and gives `most_common`; `defaultdict` auto-creates values for grouping; `deque` has O(1) appends and pops at both ends — use it for queues and bounded buffers; `OrderedDict` adds `move_to_end` and FIFO `popitem`, which makes an LRU cache easy; `ChainMap` layers mappings for config precedence; `namedtuple` gives tuples field names."

- *Implement an LRU cache?* → `OrderedDict`: `move_to_end` on access, `popitem(last=False)` on overflow — both O(1).
- *Why deque for BFS?* → `popleft()` O(1) vs `list.pop(0)` O(n).
- *`Counter` `+` vs `update`?* → `+` returns a new Counter dropping non-positive counts; `update` adds in place and keeps them.

Self-check

1. Keep only the last 100 log lines? → `deque(maxlen=100)`.
2. Which mapping receives `ChainMap` writes? → The first.

TL;DR

Counter = counts; defaultdict = auto-values; OrderedDict = reordering + FIFO pop (LRU); deque = fast both ends + maxlen; ChainMap = layered lookup; namedtuple = named fields.

6.10 `heapq` — priority queues on a plain list

What it is

A **binary min-heap** is a list where `heap[k] <= heap[2k+1]` and `heap[k] <= heap[2k+2]` for every `k`, so the smallest item is at `heap[0]`. `heapq` keeps a normal list in heap order. Whenever you repeatedly need "the smallest/most urgent next" — schedulers, Dijkstra's shortest path (cheapest route through a weighted network), merging sorted streams, top-k — a heap costs O(log n) per operation instead of re-sorting.

How it works

Function	Does	Cost
`heapify(list)`	Rearrange a list into a heap in place	O(n)
`heappush(h, x)`	Add `x` (sift up until heap order holds)	O(log n)
`heappop(h)`	Remove and return the smallest (sift down)	O(log n)
`h[0]`	Peek at the smallest	O(1)
`heappushpop(h, x)` / `heapreplace(h, x)`	Push then pop / pop then push, efficiently	O(log n)
`nsmallest(k, it)` / `nlargest(k, it)`	Top-k without full sort	O(n log k)
`merge(*sorted_its)`	Lazily merge sorted inputs	O(n log m)
`heapify_max`, `heappush_max`, `heappop_max`, …	Max-heap versions (**new in Python 3.14**)	as above

```python
def heapify_neg(xs):                        # helper: store negatives -> behaves as a max-heap
    h = [-x for x in xs]
    heapq.heapify(h)
    return h

h = [5, 1, 8, 3, 2]
heapq.heapify(h)                            # O(n), in place; h[0] is the smallest
print("heapified:", h, "min:", h[0])
heapq.heappush(h, 0)
print("pops:", [heapq.heappop(h) for _ in range(3)], "left:", h)
print(heapq.nsmallest(2, [7, 3, 9, 1]), heapq.nlargest(2, [7, 3, 9, 1]))
print("max-heap trick (negate):", -heapq.heappop(heapify_neg([3, 9, 4])))
```

Output:

```
heapified: [1, 2, 8, 3, 5] min: 1
pops: [0, 1, 2] left: [3, 5, 8]
[1, 3] [9, 7]
max-heap trick (negate): 9
```

`[1, 2, 8, 3, 5]` is **not sorted** — only the heap property holds. Before 3.14, max-heaps push negated numbers.

Priority queue with tie-breaking:

```python
tasks, counter = [], 0
for prio, name in [(2, "write docs"), (1, "fix prod bug"), (3, "refactor"), (1, "reply to CEO")]:
    heapq.heappush(tasks, (prio, counter, name))   # (priority, sequence, payload)
    counter += 1
while tasks:
    prio, _, name = heapq.heappop(tasks)
    print(f"  p{prio}: {name}")
```

Output:

```
  p1: fix prod bug
  p1: reply to CEO
  p2: write docs
  p3: refactor
```

Tuples compare element-wise: priority first; the increasing counter breaks ties in insertion order **and** stops Python ever comparing payloads (the carried data, which may not be comparable).

`heapq.merge([1, 4, 9], [2, 3, 10], [5])` → `[1, 2, 3, 4, 5, 9, 10]` — the core of external sorting (merging sorted files too big for memory).

Trade-offs

Need	Use
Smallest item repeatedly, with inserts	`heapq`
Just the top k once	`heapq.nlargest/nsmallest` (or `sorted(...)[:k]` if k ≈ n)
Only min or max once	`min()` / `max()` — O(n)
Thread-safe blocking priority queue	`queue.PriorityQueue` (heapq inside)
Sorted order at all times with lookups	`bisect` on a list, or a sorted-container library

Real-world examples

`asyncio`'s event loop keeps scheduled callbacks (`call_later`) in a heap; `sched` and `queue.PriorityQueue` use `heapq`; Python Dijkstra/A* (distance-estimate-guided shortest path) implementations almost always do.

Self-check

1. `heapify` vs n pushes? → O(n) vs O(n log n).

TL;DR

`heapify` O(n), push/pop O(log n), `h[0]` is the min; `(priority, count, item)` tuples; `nlargest` for top-k; max-heap = negate (or 3.14's `_max` functions).

6.11 `bisect` — binary search on sorted lists

What it is

`bisect` finds where a value belongs in an **already sorted** list with binary search — O(log n) comparisons.

How it works

- `bisect_left(a, x)` — first insertion index keeping order (before equal items).
- `bisect_right(a, x)` / `bisect(a, x)` — index **after** equal items.
- `insort(a, x)` (= `insort_right`) — insert in order: O(log n) search **+ O(n)** shift.
- Since 3.10, all accept `key=`.

On `[10, 20, 20, 30, 40]` searching 20: `bisect_left` lands at 1, `bisect_right` after the last 20 at 3.

```
scores = [10, 20, 20, 30, 40]
print(bisect.bisect_left(scores, 20), bisect.bisect_right(scores, 20), bisect.bisect(scores, 25))
bisect.insort(scores, 25)
print(scores)

def contains(sorted_list, x):
    i = bisect.bisect_left(sorted_list, x)
    return i < len(sorted_list) and sorted_list[i] == x
print(contains(scores, 30), contains(scores, 31))

def grade(score, cutoffs=(60, 70, 80, 90), letters="FDCBA"):
    return letters[bisect.bisect(cutoffs, score)]
print([grade(s) for s in (33, 60, 77, 89, 90, 100)])

people = [("Ann", 25), ("Bob", 31), ("Cid", 40)]           # sorted by age
i = bisect.bisect_left(people, 30, key=lambda p: p[1])      # key= is 3.10+
print("first person aged >= 30:", people[i])
```

Output:

```
1 3 3
[10, 20, 20, 25, 30, 40]
True False
['F', 'D', 'C', 'B', 'A', 'A']
first person aged >= 30: ('Bob', 31)
```

The grading function is the Python docs' classic: cutoffs become a lookup; exactly 60 lands **right** of the 60 cutoff → "D".

Trade-offs

- Search O(log n), but `insort` O(n). For many inserts plus lookups on big data, a set/dict (if order doesn't matter) or a balanced tree (self-adjusting, O(log n) inserts) / third-party `sortedcontainers.SortedList` is better.
- The list **must** be sorted by the same key — bisect doesn't check.

Real-world examples

`random.choices` bisects cumulative weights (running totals) for weighted sampling; time-series code finds the first event after a timestamp; tax brackets and grade cutoffs are one call.

TL;DR

Binary search on sorted lists: `bisect_left/right` O(log n), `insort` O(n), `key=` since 3.10.

6.12 When to use what — the decision table

You need…	Use	Why	Key costs
Ordered items, index access, append at end	`list`	Dynamic array	index O(1), append O(1)*, `in` O(n)
Fixed record / dict key / return several values	`tuple` / `namedtuple`	Immutable, hashable, light	index O(1)
Look up values by key	`dict`	Hash table	get/set/del O(1) avg
Fast membership / uniqueness	`set`	Hash table	`in` O(1) avg
A set as a key or inside a set	`frozenset`	Immutable, hashable	same as set
Count things / top-k frequencies	`Counter`	Counting dict with `most_common`	update O(k)
Group into lists/sets without key checks	`defaultdict(list/set)`	Auto-created values	O(1)
Queue / stack at both ends / last N items	`deque` (`maxlen=N`)	O(1) at both ends	append/pop O(1), middle index O(n)
LRU cache / reorder keys / FIFO pop	`OrderedDict` (or `functools.lru_cache`)	`move_to_end` , `popitem(last=False)`	O(1)
Layered config / scopes	`ChainMap`	Search mappings in order, no copying	lookup O(number of maps)
Repeatedly take the smallest/most urgent	`heapq`	Binary heap	push/pop O(log n), peek O(1)
Top-k from a large iterable	`heapq.nlargest/nsmallest`	Avoids a full sort	O(n log k)
Search/insert in a sorted list	`bisect`	Binary search	search O(log n), insert O(n)
Dedupe but keep order	`dict.fromkeys(items)`	Dicts keep insertion order	O(n)
Compact numeric storage / fast maths	`array.array` / NumPy	Chapter 5	—

(* amortised)

In the interview — putting it together

Prompt: "Find the 3 most frequent words in a log file and the first line where each appears."

Answer sketch: stream lines; `Counter` for counts; a dict with `setdefault(word, line_no)` for first occurrence; `most_common(3)` at the end. O(total words) plus O(u log 3) for the top-k — no nested loops, no `list.index` .

Chapter TL;DR

Topic	Remember
Dicts	Key → value, O(1) avg, insertion-ordered since 3.7; `[]` raises `KeyError`, `get` defaults; `|` merges (3.9+); `copy` is shallow; don't resize while iterating.
Hashing	hash → slot → compare; ≤ 2/3 full; equal ⇒ same hash; keys must be immutable/hashable; string hashes salted per run.
Nested dicts	Chain keys; guard with `get(..., {})`; map 1:1 to JSON (keys become strings).
Dict methods	`get`, `setdefault`, `update` / `|=`, `pop`, `popitem` (LIFO), `fromkeys` (shared value trap), live views with set ops.
Comprehensions	`{k: v for ...}`, `{x for ...}`; `{}` is a dict.
Sets	Unique, unordered, hashable items; `add` / `update`; `remove` raises, `discard` doesn't; `set()` for empty.
Set operations	`| & - ^`, `<= < >= >`, `isdisjoint`; methods accept iterables; `_update` versions mutate.
frozenset	Immutable, hashable set — keys and sets of sets.
collections	Counter, defaultdict, OrderedDict (LRU), deque (queues, maxlen), ChainMap (layers), namedtuple.
heapq	Min-heap on a list; push/pop O(log n); `(prio, count, item)`; `nlargest`; max-heap functions in 3.14.
bisect	Binary search on sorted lists; `insort` is O(n); `key=` since 3.10.

Chapter 7 — Functions

A **function** is a named, reusable block of code that takes inputs, does one job, and (usually) returns a result. Beyond ten-line scripts, everything is built from functions, and many "advanced" features — decorators (functions wrapping functions to add behaviour, 7.6), callbacks (functions handed to other code to call later), `sorted(key=...)`, caching (remembering results) — are just functions passed around as values.

This chapter goes from `def` to closures (functions remembering variables from where they were created), decorators and recursion (a function calling itself). Scripts are in `code/`, run with Python 3.14 (3.12-compatible); output is real.

What you'll learn

#	Topic	Script
7.1	Defining and calling, `return`, `None`, `pass`	`01_defining_and_arguments.py`
7.2	Every kind of argument: positional, keyword, default, `*args`, `**kwargs`, `/`, `*`, unpacking, the mutable-default trap, multiple return values	`01_defining_and_arguments.py`
7.3	Scope and namespaces: LEGB, `global`, `nonlocal`	`02_scope_and_closures.py`
7.4	Closures and the late-binding trap	`02_scope_and_closures.py`
7.5	First-class and higher-order functions, `lambda`, `map` / `filter`, `key=`	`03_first_class_and_functools.py`
7.6	Decorators and `functools`: `wraps`, `partial`, `reduce`, `lru_cache`, `cache`	`03_first_class_and_functools.py`
7.7	Recursion, the call stack, the recursion limit, memoization	`04_recursion.py`
7.8	Docstrings and type annotations	`05_docstrings_annotations.py`

7.1 Defining and calling functions

What it is

Define with `def`, a name, parentheses holding the **parameters**, and a colon; the indented lines are the **body**. Defining doesn't run it — it creates a function object bound to the name. **Call** it with its name plus parentheses holding **arguments**.

> **Parameter vs argument.** The *parameter* is the name in the definition (`def cube(x)` → `x`); the *argument* is the value at the call (`cube(3)` → `3`). Interviewers notice mix-ups.

Why it matters

1. **Modularity** — small pieces with one job, written and debugged alone.
2. **Reuse** — write once, call anywhere (or from other programs via modules, Chapter 8); fix a bug once.
3. **Scope** — names inside are private, so parts don't trample each other's variables (7.3).
4. **Readability and teamwork** — `send_invoice(order)` documents intent; people can own different functions.

How it works

1. `def` compiles the body to bytecode, creates a *function object*, evaluates default values (important in 7.2), binds the name.
2. A call creates a fresh **frame** (workspace of local variables), binds parameters to arguments, runs the body.
3. `return value` ends the call and hands `value` back; falling off the end or a bare `return` gives `None`.

4. The frame is discarded (unless a closure keeps part alive, 7.4).

`pass` does nothing — the placeholder where a block is required (empty function, class, `if`). `...` (Ellipsis) is used the same way in stubs.

```python
def say_hi():
    """A function with no parameters and no return statement."""
    print("hi from say_hi()")

def cube(x):
    """Return x cubed."""
    return x ** 3

def todo():
    pass  # placeholder body - keeps the syntax valid

say_hi()
print("say_hi() returned:", say_hi())
print("cube(3) =", cube(3))
print("todo() returned:", todo())
```

Output:

```
hi from say_hi()
hi from say_hi()
say_hi() returned: None
cube(3) = 27
todo() returned: None
```

`say_hi()` printed twice (plain call, then inside `print`), and returned `None` — a function that only *prints* still *returns* `None`.

Trade-offs: print vs return

	`print(...)` inside	`return ...`
Result usable by caller	No — goes to the screen	Yes — stored, tested, combined
Testable	Only by capturing stdout (the console stream)	Directly (`assert cube(3) == 27` — `assert` raises if false)
Good for	Debugging, CLI output at the edges	Almost all real logic

Common mistakes

- **Missing parentheses**: `result = cube` stores the function; `cube(3)` calls it.
- **Printing instead of returning** → `x = f()` is `None`.
- **Code after** `return` in the same block never runs.
- **Calling before defining** at module level — the `def` must have executed first (inside other functions it's fine; lookup happens at call time).

Real-world examples

Libraries are collections of functions (`len`, `open`, `json.dumps`, `requests.get`, `numpy.mean`); Flask and FastAPI map each URL to a plain function ("view"/"endpoint") returning the response.

In the interview

30-second answer: "A function is defined with `def`; defining creates a function object bound to a name, and calling it creates a new frame with its own local variables. `return` sends a value back; with no `return` the result is `None`. Functions give modularity, reuse and a private scope."

- *Are functions objects?* → Yes; they have attributes (`__name__`, `__doc__`, `__defaults__`) and can be passed around (7.5).
- `pass`*?* → No-op placeholder where syntax needs a statement.

Self-check

1. `x = print("a")` stores? → `None`.

TL;DR

`def name(params):` + indented body. Defining ≠ running. `return` gives a value back, otherwise `None`. `pass` fills an empty body.

7.2 Function arguments — every kind

What it is

At the **call site** (where the function is called) pass arguments **positionally** (by order) or **by keyword** (by name). In the **definition** add **defaults**, collect extras with `*args`/`**kwargs`, and force a style with `/` (positional-only) and `*` (keyword-only).

The full parameter list, in the only allowed order:

```
def f(pos_only, /, pos_or_kw, *args, kw_only, **kwargs): ...
#     ^^^^^^^^     ^^^^^^^^^^  ^^^^^  ^^^^^^^  ^^^^^^^^
#     before /     normal      extra  after *  extra keywords
#                              positionals     (dict)
#                              (tuple)
```

Kind	Declared as	Called as	Notes
Positional-or-keyword	`def f(a, b)`	`f(1, 2)` or `f(b=2, a=1)`	The default kind
Default	`def f(a, b=10)`	`f(1)` or `f(1, 5)`	Defaulted params come after non-defaulted (positional part)
Variable positional	`def f(*args)`	`f(1, 2, 3)`	`args` is a **tuple**
Variable keyword	`def f(**kwargs)`	`f(x=1, y=2)`	`kwargs` is a **dict** (insertion-ordered)
Positional-only	`def f(a, b, /)`	`f(1, 2)` only	Python 3.8+ (PEP 570)
Keyword-only	`def f(*, a, b)` or after `*args`	`f(a=1, b=2)` only	Can be required (no default)

Why it matters

Good argument design makes calls **self-documenting and hard to misuse**: `connect("db", 5432, True, False)` is a riddle; `connect("db", port=5432, ssl=True, retry=False)` explains itself. Defaults keep simple calls simple, `*args`/`**kwargs` enable flexible wrappers (functions calling another and adding something around it), and `/`/`*` let library authors rename parameters later.

How it works — binding arguments to parameters

1. Positional arguments fill parameters left to right; extras go to `*args` (or `TypeError`).
2. Keyword arguments match by name; unknown names go to `**kwargs` (or `TypeError`).
3. Filled twice (position *and* keyword) → `TypeError: got multiple values`.
4. Empty parameters take their **default**; none → `TypeError: missing ... required argument`.

At the call, positionals precede keywords: `f(1, b=2)` fine, `f(a=1, 2)` → `SyntaxError`.

Positional and keyword arguments

Positional arguments match purely by order — swap two and you silently get wrong results. **Keyword** arguments name the parameter, so order stops mattering and calls read clearly — best with many or optional parameters.

Default arguments

A **default** makes a parameter optional; a supplied argument overrides it.

```
def book_flight(origin, destination, seat="economy", *, meal="standard"):
    """origin/destination: positional-or-keyword; seat: default; meal: keyword-only."""
    return f"{origin}->{destination} [{seat}, meal={meal}]"

print(book_flight("SFO", "JFK"))                                  # defaults used
print(book_flight(destination="JFK", origin="SFO", seat="aisle"))  # any order by name
print(book_flight("SFO", "JFK", meal="vegan"))                    # keyword-only
book_flight("SFO")                          # missing arg
book_flight("SFO", "JFK", "aisle", "vegan")  # meal given positionally
book_flight("SFO", origin="LAX")             # same arg twice
```

Output (errors caught by a helper in the script):

```
SFO->JFK [economy, meal=standard]
SFO->JFK [aisle, meal=standard]
SFO->JFK [economy, meal=vegan]
  missing arg: TypeError: book_flight() missing 1 required positional argument: 'destination'
  meal given positionally: TypeError: book_flight() takes from 2 to 3 positional arguments but 4 were given
  same arg twice: TypeError: book_flight() got multiple values for argument 'origin'
```

Positional-only (`/`) and keyword-only (`*`)

- **Before** `/`: by position only. Names become internal, so they can be renamed later, and `value` can't collide with a `**kwargs` key. Built-ins do this: `len(obj, /)`, `abs(x, /)`, `divmod(x, y, /)` — `len(obj=[1])` is a `TypeError`.
- **After** `*` (or `*args`): by name only. Use for flags and non-obvious options: `sorted(iterable, /, *, key=None, reverse=False)` forbids `sorted(xs, None, True)`. Keyword-only parameters may be required.

```
def clamp(value, low, high, /):
    return max(low, min(value, high))

print("clamp(15, 0, 10) =", clamp(15, 0, 10))
clamp(value=15, low=0, high=10)
```

```
clamp(15, 0, 10) = 10
  clamp(value=15, ...): TypeError: clamp() got some positional-only arguments passed as keyword arguments: 'value, low, high'
```

Variable-length arguments: `*args` and `**kwargs`

`*args` gathers **extra positionals** into a tuple; `**kwargs` gathers **extra keywords** into a dict. The names are convention — the stars do the work.

```
def total(*numbers, start=0):
    print("  numbers is a", type(numbers).__name__, numbers)
    return sum(numbers, start)

def make_tag(tag, /, text, **attrs):
    attr_str = "".join(f' {k}="{v}"' for k, v in attrs.items())
    return f"<{tag}{attr_str}>{text}</{tag}>"

print("total(1, 2, 3) =", total(1, 2, 3))
print("total() =", total())
print(make_tag("a", "docs", href="https://docs.python.org", target="_blank"))
```

```
  numbers is a tuple (1, 2, 3)
total(1, 2, 3) = 6
  numbers is a tuple ()
total() = 0
<a href="https://docs.python.org" target="_blank">docs</a>
```

E.g. `configure(host="db1", port=5432, timeout="30s")` receives `{"host": "db1", "port": 5432, "timeout": "30s"}` and loops over `.items()`.

Unpacking at the call site

In reverse: `*iterable` spreads positionals, `**mapping` spreads keywords.

```
nums = [4, 5, 6]
opts = {"seat": "window", "meal": "kosher"}
print("total(*nums) =", total(*nums))
print(book_flight("SFO", "JFK", **opts))
```

```
  numbers is a tuple (4, 5, 6)
total(*nums) = 15
SFO->JFK [window, meal=kosher]
```

Returning several values

A function returns **one** object; `return a, b, c` builds a tuple that the caller unpacks.

```
def min_max_mean(values):
    return min(values), max(values), sum(values) / len(values)

result = min_max_mean([3, 9, 6])
print("raw result:", result, type(result).__name__)
lo, hi, avg = result
```

```
raw result: (3, 9, 6.0) tuple
lo=3 hi=9 avg=6.0
```

Beyond ~3 values, return a `dataclass` or `typing.NamedTuple` (records with named fields, Chapters 5 and 10) so callers write `stats.mean`.

The mutable-default pitfall

Defaults are evaluated once — when `def` runs — not per call. A mutable default (list, dict, set) is *one shared object*, so changes pile up.

```
def append_bad(item, bucket=[]):          # BUG: default evaluated once
    bucket.append(item)
    return bucket

def append_good(item, bucket=None):       # FIX: None sentinel
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket

print(append_bad("a"), append_bad("b"))
print("default is now:", append_bad.__defaults__)
print(append_good("a"), append_good("b"))
```

```
['a', 'b'] ['a', 'b']
default is now: (['a', 'b'],)
['a'] ['b']
```

Both `append_bad` calls returned the *same* list; `__defaults__` shows the mutated default. Fix with the `None` **sentinel** (placeholder meaning "nothing passed"): create the object inside. The same rule freezes

`def log(msg, when=datetime.now())` at definition time. `dataclasses` refuses bare mutable defaults and asks for `field(default_factory=list)`.

Trade-offs

Choice	Benefit	Cost
Positional	Short calls	Order-dependent, easy to swap by mistake
Keyword	Readable, order-free	Longer; names become public API others rely on
Keyword-only (`*`)	Forces clarity for flags/options	Slightly more typing
Positional-only (`/`)	Free to rename; no clash with `**kwargs`	Callers can't use names for clarity
`*args` / `**kwargs`	Flexible wrappers, forwarding	Hides the real signature (parameter list) from readers, IDEs and type checkers

When not to use it

- Don't use `**kwargs` when you know the options — explicit keywords error on typos (`timout=5`) and get IDE help; `**kwargs` swallows the typo.
- Don't stack boolean positional flags (`f(x, True, False)`) — make them keyword-only.

Common mistakes

- Mutable default (`=[]`, `={}`) — use `None`.
- `def f(a=1, b):` → `SyntaxError`; `f(a=1, 2)` → `SyntaxError`.
- `*args` is a **tuple** — `list(args)` to modify.
- `return a, b` is one tuple, not "two return values".

Real-world examples

- `print(*objects, sep=" ", end="\n", file=None, flush=False)` — `*objects` plus keyword-only options.
- `requests.get(url, params=None, **kwargs)` forwards `timeout=`, `headers=`.
- Matplotlib plotting calls pass `**kwargs` (`color=`, `linewidth=`) to underlying line/patch objects.

In the interview

> **30-second answer:** "Arguments bind positionally by order or by keyword by name. Defaults make parameters optional but are evaluated once at definition time, so never use a mutable default — use `None`. `*args` collects extra positionals into a tuple, `**kwargs` extra keywords into a dict. Parameters before `/` are positional-only, after `*` keyword-only."

- *Why is `def f(x=[])` dangerous?* → One list, created once, shared by every defaulted call.
- *Why positional-only in a library?* → Names stay private (renameable) and a `**kwargs` key may share the name.
- *Pass a list as separate arguments?* → `f(*my_list)`; dict as keywords → `f(**my_dict)`.

Self-check

1. `def f(a, /, b, *, c)` — which work: `f(1, 2, 3)`, `f(1, b=2, c=3)`, `f(a=1, b=2, c=3)`? → Only `f(1, b=2, c=3)` (and `f(1, 2, c=3)`).

TL;DR

Positional = order, keyword = name. Defaults evaluated **once** → `None` for mutables. `*args` → tuple, `**kwargs` → dict; `*`/`**` at the call unpack. `/` positional-only before, `*` keyword-only after. Multiple returns = one tuple.

7.3 Scope and namespaces — LEGB, `global`, `nonlocal`

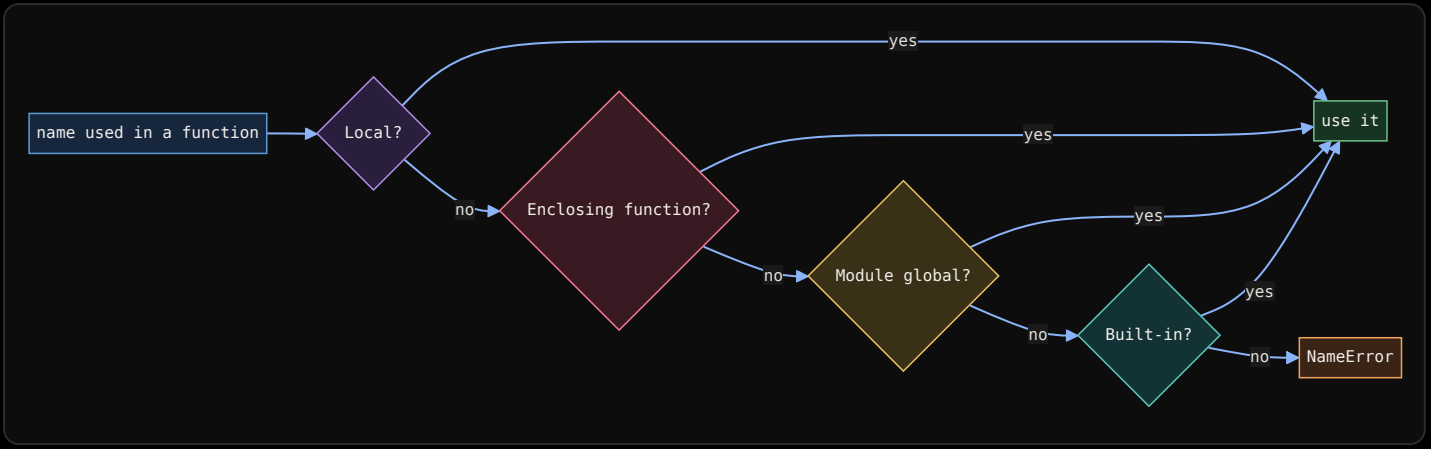
What it is

A **namespace** maps names to objects — usually literally a dict (`globals()` shows the module's). Names are unique within one namespace but can repeat across namespaces. A name's **scope** is the code region where it can be used directly.

Python resolves a bare name with the **LEGB rule**, stopping at the first hit:

Letter	Scope	Where it comes from	Lifetime
L	Local	Names assigned in the current function (incl. parameters)	The call
E	Enclosing	Locals of *outer* functions (nested functions)	While referenced (closures)
G	Global	Top-level names of the **module** (file)	Until program end / unload
B	Built-in	`len`, `print`, `ValueError` … from `builtins`	Whole program

Not found → `NameError`.



Why it matters

Scope makes functions safe to combine — one function's `total` can't clobber another's. Misunderstanding it causes the confusing `UnboundLocalError`.

How it works — the key rule

Locality is decided at compile time for the whole body: if a function *assigns* a name anywhere (`=`, `+=`, `for x in`, `import x`, `def x`), that name is local throughout — unless declared `global` or `nonlocal`. *Reading* alone doesn't make it local, so reading globals needs no declaration.

1. `global name` → assignments rebind the **module-level** variable.
2. `nonlocal name` → assignments rebind the **nearest enclosing function's** variable (must exist there; module globals don't count). Added in Python 3.
3. Only functions (and classes, comprehensions, lambdas) create scopes — **not** `if`, `for`, `while`, `with`; a variable assigned inside an `if` is visible after it.

```python
level = "global"                    # G

def outer():
    level = "enclosing"             # E for inner()
    def inner():
        level = "local"             # L
        print("  inner sees:", level)
    inner()
    print("  outer sees:", level)

def reads_global():
    print("  reads_global sees:", level)   # no assignment here -> global

counter = 0

def broken_increment():
    counter += 1                    # assignment makes counter LOCAL -> error

def global_increment():
    global counter
    counter += 1
```

```
== LEGB lookup ==
  inner sees: local
  outer sees: enclosing
  reads_global sees: global
  B (built-in) len is builtins.len: True

== assigning without global ==
  UnboundLocalError: cannot access local variable 'counter' where it is not associated with a value

== global ==
  counter = 2
```

`counter += 1` is `counter = counter + 1`: the assignment makes `counter` local, so reading it first raises `UnboundLocalError` (a `NameError` subclass). `nonlocal` does the same one level in (`make_counter`, 7.4).

Trade-offs

Shared state via	Good	Bad
Pass in / return values	Explicit, testable, thread-friendly	More parameters
`global`	Quick for scripts	Hidden coupling; any function can change it; hard to test
`nonlocal` / closure	State private to one function family	Less obvious than a class
A class with attributes	Clear home for state + behaviour	More code

When not to use it

Avoid `global` except for true module-level config or caches in small scripts. Mutating a global *object* (`CACHE[key] = v`) needs no `global` — only rebinding the *name* does — but it's still shared mutable state.

Common mistakes

- `UnboundLocalError` from `x += 1` on a global — `global x`, or better, return the new value.
- **Shadowing built-ins**: `list = [1, 2]` then `list('xy')` → `'list' object is not callable` (likewise `dict`, `id`, `input`, `sum`, `type`).
- A class body is not an enclosing scope for its methods — use `self.x` (`self` = the object the method was called on, Chapter 10) or `ClassName.x`.
- Comprehension variables don't leak (own scope); a plain `for` variable does.

Real-world examples

- `logger = logging.getLogger(__name__)` at module level, only *read* in functions.
- `MAX_RETRIES = 3` constants: global, UPPER_CASE (PEP 8), treated as read-only.

In the interview

> **30-second answer:** "Python looks names up via LEGB: local, enclosing, global, built-in. Any assignment in a function makes that name local for the whole function, which is why `x += 1` on a global raises `UnboundLocalError`. `global` rebinds module-level names, `nonlocal` rebinds the nearest enclosing function's name. Blocks like `if` and `for` don't create scopes."

- *`global` needed to append to a global list?* → No — `.append` mutates, not rebinds.
- *`global` vs `nonlocal`?* → Module scope vs nearest enclosing *function* scope (must exist there).
- *Why does `def f(): print(x); x = 1` fail with a global `x`?* → The later assignment makes `x` local for the whole body.
- *Can `nonlocal` target a module variable?* → No — `SyntaxError: no binding for nonlocal ... found`.

TL;DR

Lookup **L → E → G → B**. Assignment anywhere in a function ⇒ local throughout. `global` for module names, `nonlocal` for outer-function names; prefer passing and returning values.

7.4 Closures

What it is

A **closure** is an inner function that *remembers* variables from the enclosing scope after the outer function has returned. Those are **free variables**, stored in "cells" on the function (`fn.__closure__`).

Why it matters

Closures build **functions from functions** — configured functions (`make_multiplier(3)`), counters with private state, callbacks carrying context — without a class. Decorators (7.6) are built on them.

How it works

1. `make_multiplier(3)` runs; `factor = 3` is local.
2. Inner `multiply` reads `factor` → the compiler makes it a free variable in a cell.
3. `multiply` is returned; the frame ends but the cell survives.
4. Each `make_multiplier` call creates a **new** cell — `double` and `triple` are independent.
5. *Rebinding* (not just reading) a captured variable needs `nonlocal`.

```python
def make_counter():
    count = 0
    def step():
        nonlocal count              # rebind the enclosing variable
        count += 1
        return count
    return step

def make_multiplier(factor):
    def multiply(x):
        return x * factor           # 'factor' is captured (a free variable)
    return multiply

c1, c2 = make_counter(), make_counter()
print("  c1:", c1(), c1(), c1(), " c2:", c2())
double, triple = make_multiplier(2), make_multiplier(3)
print("  double(5) =", double(5), " triple(5) =", triple(5))
print("  free vars of double:", double.__code__.co_freevars,
      "cell value:", double.__closure__[0].cell_contents)
```

```
  c1: 1 2 3  c2: 1
  double(5) = 10  triple(5) = 15
  free vars of double: ('factor',) cell value: 2
```

The late-binding trap

Closures capture **variables, not values**; the value is read when the inner function *runs*.

```python
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])         # [2, 2, 2]  - all see the final i
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])         # [0, 1, 2]  - default captures the current value
```

`i=i` works *because* defaults are evaluated at definition (7.2); `functools.partial` is cleaner.

Trade-offs

	Closure	Class
Code size	Tiny	Larger
Number of operations	Best for one (a callable)	Many methods
Inspectability	State hidden in cells	Attributes visible, easy to debug

When not to use it

Several operations (increment, reset, read), or state others must inspect/serialise (convert to bytes/text) → write a class. Closures can't be pickled by `pickle` (Python's object-serialisation module).

Common mistakes

- Late binding in loops.
- Missing `nonlocal` → `UnboundLocalError` on `count += 1`.

Real-world examples

- Every decorator (`@functools.lru_cache`, Flask's `@app.route("/")`) returns a closure.
- GUI callbacks (Tkinter `command=`) use closures or `partial` to carry which button was pressed.

In the interview

> **30-second answer:** "A closure is a nested function that keeps references to variables of its enclosing scope after that scope has returned. It captures variables, not values, so lambdas in a

loop all see the last value unless you bind with a default argument or `partial`. `nonlocal` lets it rebind the captured variable."

- *Where are captured variables stored?* → Cells in `fn.__closure__`; names in `fn.__code__.co_freevars`.
- *Fix the loop-lambda bug?* → `lambda i=i: ...` or `functools.partial(f, i)`.

TL;DR

Inner function + remembered outer variables = closure. Captures variables (late binding); `nonlocal` to rebind. Foundation of decorators.

7.5 First-class and higher-order functions, `lambda`, `map`/`filter`, `key=`

What it is

Functions are **first-class objects**: assign, store in lists/dicts, pass, return. A **higher-order function** takes or returns a function (`map`, `sorted`, decorators, `make_multiplier`). A `lambda` is an anonymous single-expression function: `lambda x: x * 2` ≈ `def` returning `x * 2`, but no statements, no annotations, name `<lambda>`. Passing **behaviour** as data ("sort by this", "call this when done") removes families of near-duplicate functions.

How it works

```
yell = shout                                  # assign to a name
table = {"up": str.upper, "low": str.lower}  # store in a dict
print(apply_twice(shout, "hey"))              # pass as argument  -> HEY!!

nums = [5, -2, 8, -7, 3]
list(map(abs, nums))                  # [5, 2, 8, 7, 3]
list(filter(lambda n: n > 0, nums))   # [5, 8, 3]
[n * n for n in nums if n > 0]        # [25, 64, 9]
```

- `map(fn, it)` applies `fn` to each item; `filter(pred, it)` keeps items where the *predicate* (yes/no function) is truthy. Both return **lazy iterators** (values produced on request) — wrap in `list()`; consumable once.
- `sorted(it, key=fn, reverse=False)` returns a **new list**; `list.sort()` sorts in place, returns `None`. `min`, `max`, `heapq.nlargest`, `itertools.groupby` take `key=` too.
- `key` is called **once per element**; sorting is **stable** (equal keys keep order) — the basis of multi-level sorts (§5.7).
- `operator.itemgetter(1)`, `operator.attrgetter("age")`, `str.lower` are ready-made, faster alternatives to many lambdas.

```
people = [("Ana", 31), ("bob", 25), ("Cy", 31), ("dee", 19)]
sorted(people, key=lambda p: p[1])            # by age
sorted(people, key=lambda p: (-p[1], p[0]))   # age desc, then name
sorted(["bob", "Ana", "dee", "Cy"], key=str.lower)
max(people, key=operator.itemgetter(1))
```

```
by age:             [('dee', 19), ('bob', 25), ('Ana', 31), ('Cy', 31)]
by age desc, name:  [('Ana', 31), ('Cy', 31), ('bob', 25), ('dee', 19)]
case-insensitive:   ['Ana', 'bob', 'Cy', 'dee']
itemgetter(1):      ('dee', 19)
oldest:             ('Ana', 31)
```

A **tuple** key sorts by first element, then second…; negating a number flips that level.

Trade-offs

Tool	Best for	Watch out
Comprehension	Transform + filter in one line	Builds a whole list (generator `( … )` for laziness)
`map` / `filter` with a named function	`map(str.strip, lines)`	With a lambda, usually less readable than a comprehension
`lambda`	Tiny one-off `key=` functions	No statements; `<lambda>` in tracebacks; PEP 8: don't assign one to a name — use `def`

When not to use it

`f = lambda x: x + 1` → write `def` (better traceback name, docstring). Don't nest `map(lambda..., filter(lambda..., ...))` when a comprehension reads plainly.

Common mistakes

- `sorted_list = my_list.sort()` → `None`.
- Consuming a `map` / `filter` twice — second pass empty.
- `key=len()` (calling) instead of `key=len`.

Real-world examples

- pandas: `df["name"].map(str.title)`, `df.sort_values(key=...)`, `df.apply(fn)`.
- `sorted(files, key=os.path.getmtime)` (last-modified time).
- Callbacks: `threading.Thread(target=worker)`; `executor.map(fetch, urls)` in `concurrent.futures` (an *executor* is a worker pool running calls in parallel).

In the interview

> **30-second answer:** "Functions are objects, so they can be passed, stored and returned; a higher-order function takes or returns one. `lambda` makes a one-expression anonymous function, mostly for `key=`. `map` / `filter` return lazy iterators; comprehensions are usually preferred. `sorted` with `key` computes the key once per element and is stable, so tuple keys give multi-level sorts."

- *`sorted` vs `.sort()`?* → Any iterable → new list vs list method, in place, `None`.
- *Two fields, one descending?* → Tuple key with a negated number, or two stable sorts (secondary first).
- *`if` in a lambda?* → No statements; a conditional *expression* is fine.

TL;DR

Functions are values; higher-order functions take/return them. `lambda` for tiny keys. `map` / `filter` lazy; comprehensions clearer. `sorted(key=…)`: once per item, stable, tuple keys for multi-level.

7.6 Decorators and `functools`

What it is

A **decorator** takes a function and returns a replacement (usually a closure calling the original plus extra behaviour). `@` is shorthand:

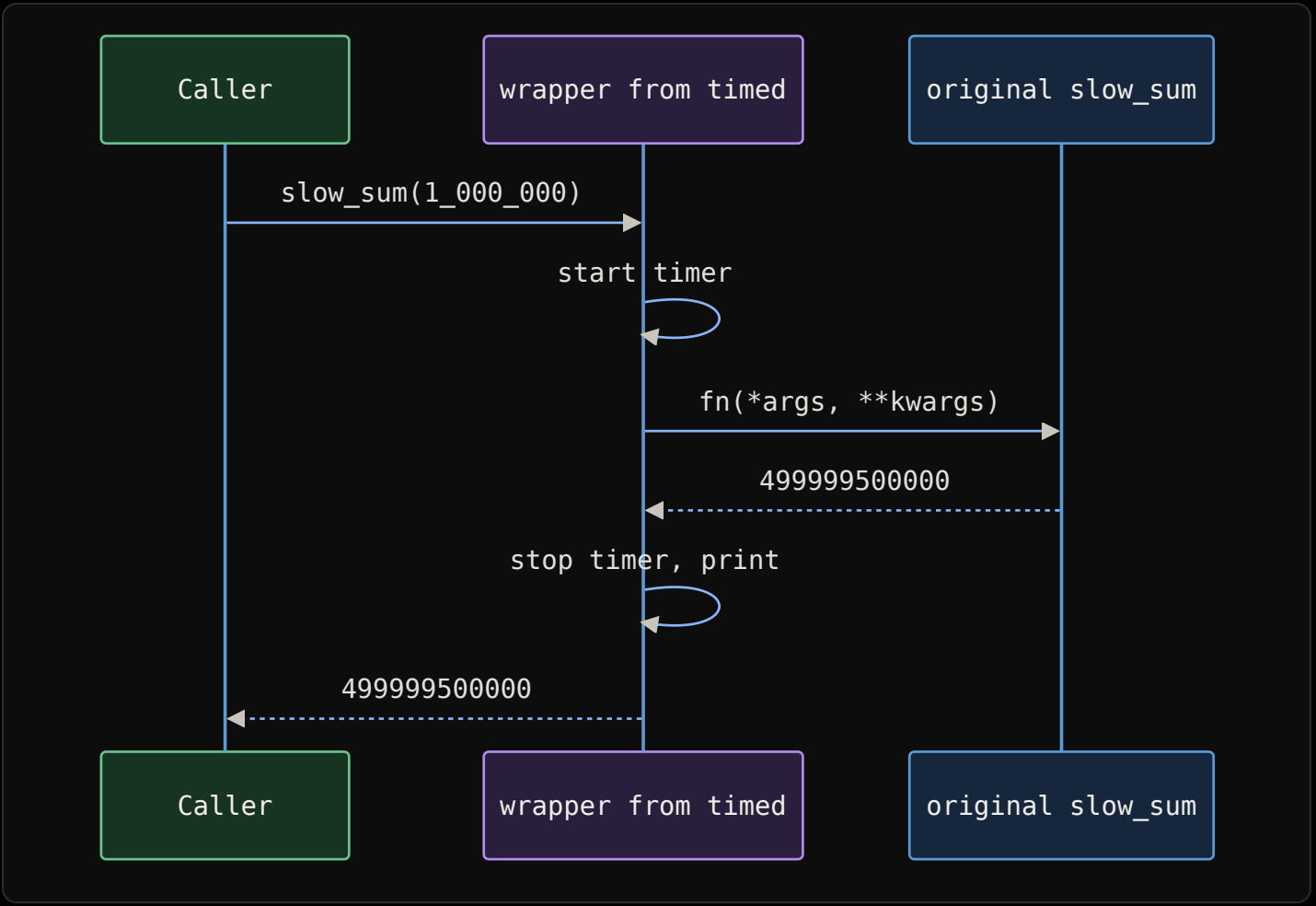
```
@timed
def slow_sum(n): ...
# is exactly
def slow_sum(n): ...
slow_sum = timed(slow_sum)
```

`functools` collects function tools: `wraps`, `partial`, `reduce`, `lru_cache`, `cache`, `cached_property`, `singledispatch`, `total_ordering`.

Why it matters

Decorators add cross-cutting behaviour (concerns spanning many functions) — timing, logging, caching, auth checks, retries, route registration — without editing each function; `functools` provides battle-tested versions.

How it works




```python
def timed(fn):
    @functools.wraps(fn)                # copy __name__, __doc__, etc.
    def wrapper(*args, **kwargs):       # accept any signature
        t0 = time.perf_counter()
        result = fn(*args, **kwargs)    # call the original
        ms = (time.perf_counter() - t0) * 1000
        print(f"  [{fn.__name__} ran in {'under' if ms < 500 else 'over'} 500 ms]")
        return result                   # don't forget to return!
    return wrapper
```

```
  [slow_sum ran in under 500 ms]
  slow_sum(1_000_000) = 499999500000
  with wraps:    name = slow_sum | doc = Add 0..n-1 the slow way.
  without wraps: name = wrapper | doc = None
```

`*args, **kwargs` let the wrapper accept and forward any signature.

`functools.wraps` copies `__name__`, `__doc__`, `__module__`, `__qualname__`, `__dict__` and sets `__wrapped__`; without it every decorated function appears as `wrapper` in logs, `help()` and tracebacks.

`functools.partial(fn, *args, **kwargs)` pre-fills arguments:

```
int_from_binary = functools.partial(int, base=2)
int_from_binary("1011")   # 11
```

`functools.reduce(fn, iterable[, initial])` folds pairwise left to right: `reduce(mul, [1,2,3,4,5])` = ((((1·2)·3)·4)·5) = 120. Without `initial`, empty input → `TypeError`. Prefer `sum`, `math.prod`, `max`, `"".join` where they fit.

`functools.lru_cache(maxsize=128)` memoizes: stores results keyed by arguments and returns them on repeat calls; beyond `maxsize` keys, the **least recently used** is evicted. **`functools.cache`** (3.9+) = `lru_cache(maxsize=None)` — unbounded, slightly faster.

```
@functools.lru_cache(maxsize=128)
def square_slowly(n):
    time.sleep(0.05)
    return n * n
for n in (4, 4, 5, 4):
    square_slowly(n)
print(square_slowly.cache_info())
```

```
  CacheInfo(hits=2, misses=2, maxsize=128, currsize=2)
  after cache_clear: CacheInfo(hits=0, misses=0, maxsize=128, currsize=0)
  unhashable argument -> unhashable type: 'list'
```

4, 4, 5, 4 → misses on first 4 and 5, hits on the repeats. Arguments must be **hashable**.

Trade-offs

Tool	Win	Cost / caveat
`lru_cache(maxsize=N)`	Bounded memory	Bookkeeping; stale results if data changes
`cache`	Fastest, simplest	Grows forever — a memory leak for unbounded inputs
`partial`	Picklable, introspectable (`.func`, `.args`)	Only pre-fills; can't reorder
`reduce`	General fold	Often less readable than a loop or built-in

When not to use it

- Don't cache functions with side effects (sending email) or results depending on time, randomness or mutable globals.
- `lru_cache` on an **instance method** keys on `self` and keeps every instance alive; prefer `functools.cached_property` or a module-level cached function.

Common mistakes

- No `return result` in the wrapper → decorated function returns `None`.
- No `@functools.wraps`.
- `@timed()` vs `@timed` confusion. A decorator *with* arguments (`@retry(3)`) is a function returning a decorator — three levels of nesting.

Real-world examples

- Flask/FastAPI `@app.get("/items")` registers route handlers.
- pytest: `@pytest.fixture`, `@pytest.mark.parametrize(...)`.
- Standard library: `@property`, `@staticmethod`, `@classmethod`, `@dataclass`, `@contextlib.contextmanager`.
- `re` caches compiled patterns so repeated `re.match(pattern, s)` doesn't recompile.

In the interview

> **30-second answer:** "A decorator is a callable that takes a function and returns a replacement; `@d` above `def f` means `f = d(f)`. Inside you define a `wrapper(*args, **kwargs)` closure and use `functools.wraps` to keep the original's name and docstring. `functools` also gives `partial` to pre-fill arguments, `reduce` to fold, and `lru_cache` / `cache` to memoize pure functions (output depends only on inputs, no side effects) with hashable arguments."

- *`wraps`?* → Copies metadata (`__name__`, `__doc__`, …), sets `__wrapped__`.
- *`@retry(3)`?* → Outer function takes `3`, returns the decorator, which returns the wrapper.
- *When is `lru_cache` wrong?* → Impure functions, unhashable args, unbounded keys with `maxsize=None`, instance methods.

TL;DR

Decorator = function in, function out; always `@wraps` and `return result`. `partial` pre-fills, `reduce` folds, `lru_cache` (default `maxsize` 128) / `cache` memoize pure functions with hashable arguments.

7.7 Recursion

What it is

A **recursive function** calls itself on a smaller version of the problem — *divide and conquer*. It needs:

1. A **base case** — small enough to answer directly.
2. A **recursive case** — shrink the input and recurse, guaranteeing progress toward the base case.

Why it matters

Natural for **self-similar data**: nested lists/JSON, folder trees, parse trees, tree/graph traversal, merge sort and quicksort, backtracking (try an option, undo if it fails: permutations, sudoku). Interviews use it constantly.

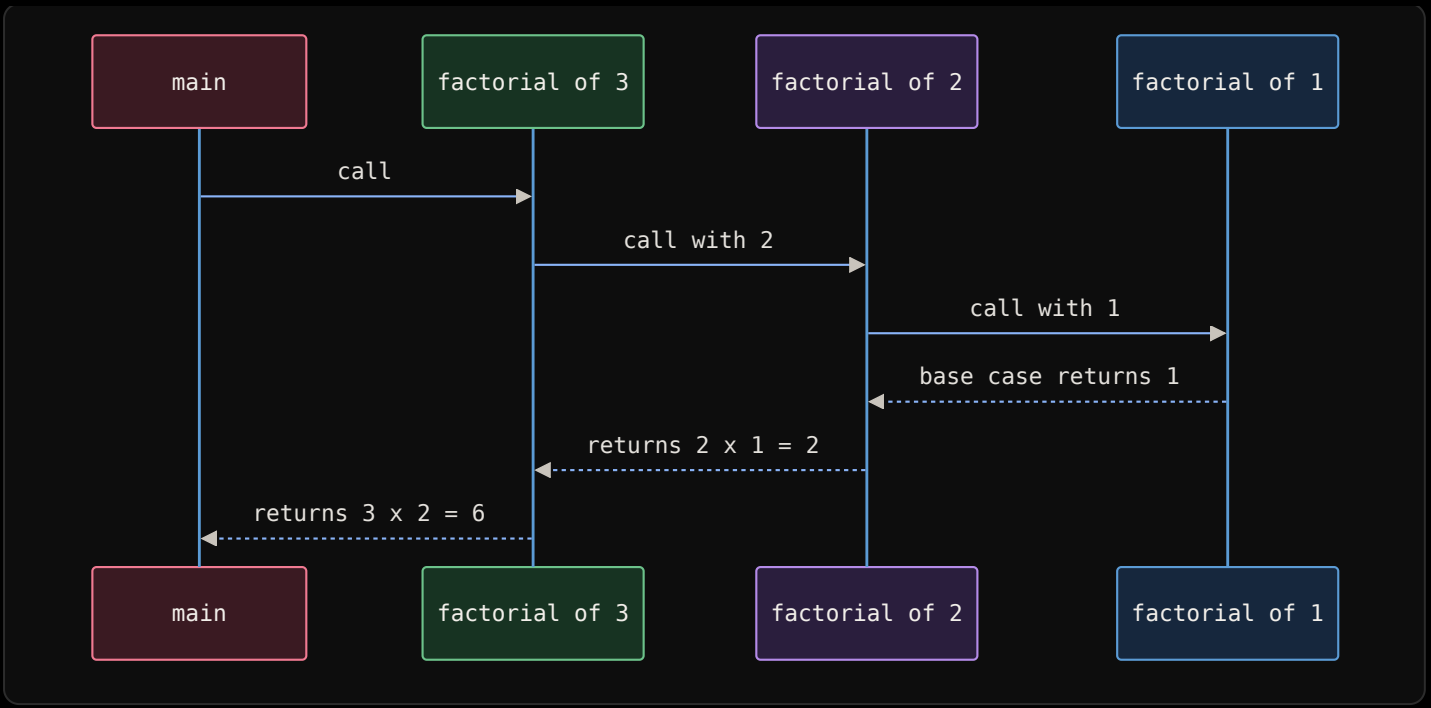
How it works — the call stack

Each call gets a frame on the **call stack** (last-in-first-out pile of active calls); frames pile up to the base case, then results flow back. Factorial: n! = n × (n−1)!, 0! = 1! = 1.

```python
def factorial(n):
    if n <= 1:                # base case (also handles 0! = 1)
        return 1
    return n * factorial(n - 1)
```

Trace of `factorial(3)`, and `factorial(5)` = 5 × 4 × 3 × 2 × 1 = 120:

```
factorial(5) = 120 | factorial(0) = 1
factorial(3)
  factorial(2)
    factorial(1)
    -> 1
  -> 2
-> 6
```



The recursion limit

CPython has **no tail-call optimisation** (TCO — reusing the frame when the last step is a self-call), by design to keep tracebacks complete, so every level costs a frame. Depth is capped: `sys.getrecursionlimit()` is **1000** by default; exceeding it raises `RecursionError` (a `RuntimeError` subclass).

```
sys.getrecursionlimit() = 1000
depth_of(500) = 500
RecursionError: maximum recursion depth exceeded
```

`sys.setrecursionlimit(n)` raises it, but deep recursion costs memory and can crash the process — for thousands of levels **rewrite as a loop** (explicit stack list for tree walks).

Memoization — fixing exponential recursion

Fibonacci: F(0)=0, F(1)=1, F(n)=F(n−1)+F(n−2); F(7) = 13. Naive recursion recomputes the same values (F(5) calls F(4) and F(3); F(4) calls F(3) *again*), so calls grow ~1.6^n. **Memoization** computes each F(k) once — linear work.

```python
@functools.cache
def fib_memo(n):
    if n < 2:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

def fib_iter(n):              # bottom-up: no recursion at all
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

```
fib(7) = 13
naive calls for n=10, 20, 25: 177 21891 242785
memoized: CacheInfo(hits=23, misses=26, maxsize=None, currsize=26)
fib_iter(90) = 2880067194370816120
```

n = 25: 242,785 naive calls vs 26 distinct values computed (misses) and 23 cache hits.

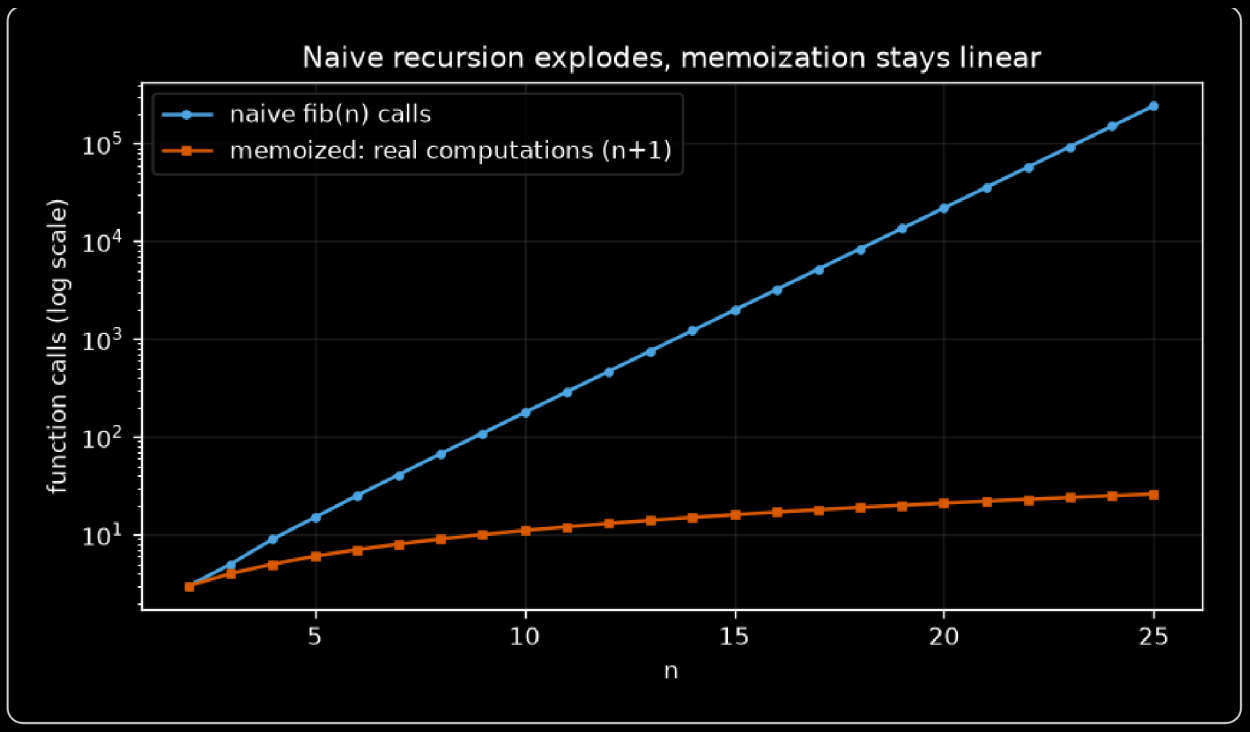


Figure — calls for fib(n): naive grows exponentially (a straight line on a log axis), memoized linearly.

Recursion over nested data

```python
def flatten(nested):
    out = []
    for item in nested:
        if isinstance(item, list):
            out.extend(flatten(item))   # recursive case: a smaller list
        else:
            out.append(item)            # base case: a plain value
    return out

flatten([1, [2, [3, [4]], 5], [], 6])   # [1, 2, 3, 4, 5, 6]
```

The same shape walks directories (`os.walk` / `Path.iterdir`), JSON and trees.

Trade-offs

	Recursion	Iteration
Fits	Trees, nested data, divide and conquer, backtracking	Linear sequences, counting, very deep processes
Clarity	Mirrors the definition	May need an explicit stack
Memory	One frame per level	Constant (or your own stack)
Depth limit	~1000 by default	None
Speed in CPython	Call overhead per level	Usually faster

When not to use it

Linear problems (sum a list, walk a 100k-node linked list) — loop. Depth driven by untrusted input size.

Common mistakes

- **Missing/unreachable base case**: base `n == 1` makes `factorial(0)` recurse 0 → −1 → … until `RecursionError`; `n <= 1` fixes it.

- Not shrinking the input (`f(n)` calling `f(n)`).
- Forgetting to `return` the recursive result → `None`.
- Naive recursion on overlapping sub-problems (the same smaller problems solved repeatedly: Fibonacci, grid paths) — memoize.
- Mutable default accumulators (`def walk(node, seen=set())`).

Real-world examples

- `os.walk`, `shutil.copytree`, `json` encoding (CPython's encoder is recursive and can raise `RecursionError` on extremely deep input), `copy.deepcopy`.
- Recursive-descent parsers (one function per grammar rule, calling each other).

In the interview

> **30-second answer:** "Recursion is a function calling itself on a smaller input until a base case. Each call uses a stack frame; CPython doesn't optimise tail calls and limits depth to about 1000, raising `RecursionError`. Overlapping sub-problems like Fibonacci are exponential unless memoized with `functools.cache`, which makes them linear. For very deep or linear problems, convert to a loop."

- *Naive vs memoized Fibonacci?* → O(φⁿ) (φ ≈ 1.618, golden ratio) time vs O(n) time and O(n) space.
- *Tail-call optimisation?* → No, by design.
- *Recursive DFS (depth-first search) to iterative?* → Explicit stack: pop a node, process, push children.

TL;DR

Base case + shrinking recursive case. Frames pile up; limit ≈ 1000; no TCO. Memoize overlapping sub-problems (`@cache`); loop when deep.

7.8 Docstrings and annotations

What it is

- A **docstring** is a string literal as the **first statement** of a function, class or module, stored in `__doc__` and shown by `help()`, IDEs and documentation generators (Sphinx). PEP 257: one-line summary, blank line, details.
- **Annotations** (type hints) declare expected types: `def f(x: int) -> str`. **Not enforced** at runtime — stored and ignored. **mypy**, **pyright** (static type checkers) and IDEs read them to catch bugs before running; **FastAPI** (web framework) and **Pydantic** (data validation) read them at runtime to validate. Full coverage: Chapter 13.

Why it matters

Docstrings explain *what and why*; annotations make the contract checkable, so a type checker catches `average("abc")` in CI (automated checks on every change).

How it works

```
from collections.abc import Callable, Iterable

def average(values: Iterable[float], *, default: float | None = None) -> float | None:
    """Return the arithmetic mean of *values*.

    Args:
        values: numbers to average.
        default: returned when *values* is empty.

    Returns:
        The mean, or *default* for empty input.

    >>> average([1, 2, 3])
    2.0
    """
    items = list(values)
    if not items:
        return default
    return sum(items) / len(items)

def apply(fn: Callable[[int], int], x: int) -> int:
    return fn(x)

type Pair = tuple[int, int]          # 3.12+ type alias statement
def swap(p: Pair) -> Pair:
    return p[1], p[0]
```

```
first docstring line: Return the arithmetic mean of *values*.
signature: (values: collections.abc.Iterable[float], *, default: float | None = None) -> float | None
return annotation: float | None
average([1, 2, 3]) = 2.0
average([], default=0.0) = 0.0
swap(('x', 'y')) = ('y', 'x') <- str, not int, and no error
apply(lambda n: n + 1, 41) = 42
swap((1, 2)) = (2, 1)
doctest: 1 test(s), 0 failure(s)
```

Syntax	Meaning	Since
`x: int`, `-> str`	Parameter / return annotation	3.0 (PEP 3107); typing semantics 3.5 (PEP 484)
`list[int]`, `dict[str, float]`	Built-in generics (containers with a type parameter)	3.9
`int | None`	Union ("either type"; replaces `Optional[int]`)	3.10
`type Alias = ...`, `def f[T](x: T) -> T`	Type alias / generic syntax	3.12 (PEP 695)
Lazy evaluation of annotations	Forward references (types defined later) work without quotes	3.14 (PEP 649)

`inspect.signature(fn)` reads parameters, defaults and annotations at runtime. The `>>>` lines form a **doctest**: `doctest.testmod()` runs them and compares output.

Trade-offs

Annotations add noise and need a checker to pay off, but on shared code they catch whole bug classes and power autocompletion. Docstrings drift — keep them short and about behaviour.

When not to use it

Throwaway scripts needn't be fully annotated; don't restate the obvious (`"""Returns x."""`).

Common mistakes

- Believing annotations validate input — `swap(("x", "y"))` ran fine.
- A `# does X` comment instead of a docstring → invisible to `help()`.
- Docstring not *first* → `__doc__` is `None`.

Real-world examples

- FastAPI builds request validation and OpenAPI docs (machine-readable API description) from annotations.
- NumPy/Google docstring styles rendered by Sphinx (numpy.org).
- Dropbox runs mypy in CI over millions of lines.

In the interview

> **30-second answer:** "A docstring is the first string in a function, stored in `__doc__` and shown by `help()`. Annotations declare expected types but aren't enforced at runtime; static checkers like mypy use them, and frameworks like FastAPI/Pydantic read them to validate."

- *Checked at run time?* → No, only by tools/libraries that read them.
- *Read annotations at runtime?* → `inspect.signature(f)`, `inspect.get_annotations(f)` / `f.__annotations__`.

TL;DR

Docstring = first-statement string shown by `help()`. Annotations = documentation checkable by mypy/pyright, not enforced at runtime.

Chapter TL;DR

- `def` creates a function object; calling runs it in a fresh frame; no `return` ⇒ `None`; `pass` for empty bodies.
- Arguments: positional (order) and keyword (name); defaults evaluated **once** ⇒ never a mutable default, use `None`; `*args` tuple, `**kwargs` dict; `/` positional-only, `*` keyword-only; `*`/`**` unpack at call sites; `return a, b` returns a tuple.
- Scope: **LEGB**. Assignment makes a name local for the whole function (`UnboundLocalError`). `global` → module, `nonlocal` → enclosing function. `if`/`for` don't create scopes.
- Closures remember enclosing variables (by reference ⇒ late binding). Decorators are closures: `@d` means `f = d(f)`; use `functools.wraps`.
- Functions are first-class: pass them, return them, `key=` them. `map`/`filter` are lazy; `sorted` is stable.
- `functools`: `partial`, `reduce`, `lru_cache`/`cache` (pure functions, hashable args), `wraps`.
- Recursion: base case + progress; limit 1000, no tail-call optimisation; memoize overlapping sub-problems; loop when deep.
- Docstrings for `help()`, annotations for tools — not runtime checks.

Chapter 8 — Modules, Packages and Environments

Real programs span many files, depend on other people's code (NumPy, requests, Django), and must run the same on your laptop, a colleague's machine and a server. Three layers make that possible:

1. **Modules and packages** — splitting code into files and folders, pulled together with `import`.
2. **The import system** — where Python looks for code, what it caches, why imports fail.
3. **Environments and packaging** — `pip` (the package installer), virtual environments (isolated per-project package folders, 8.7), `requirements.txt` vs `pyproject.toml`, `uv` and Poetry, project layout, publishing to PyPI (the Python Package Index, the public package store).

Scripts are in `code/`; they build demo packages in temporary folders and leave nothing behind. Output is real (Python 3.14; code is 3.12-compatible).

#	Topic	Script
8.1	Modules, the four `import` forms, aliases	`01_import_basics.py`
8.2	How `import` works: `sys.modules`, finders/loaders, `sys.path`	`01_import_basics.py`, `02_packages_and_sys_path.py`
8.3	Packages, `__init__.py`, namespace packages, `__all__`	`02_packages_and_sys_path.py`
8.4	Absolute vs relative imports	`02_packages_and_sys_path.py`
8.5	`if __name__ == "__main__"` and `python -m`	`02_packages_and_sys_path.py`
8.6	Common import errors, circular imports	`03_import_errors.py`
8.7	`pip` and virtual environments (`venv`)	`04_environments.py`
8.8	`requirements.txt` vs `pyproject.toml`; `uv` and Poetry	`04_environments.py`
8.9	Project layout (src layout) and publishing basics	—

8.1 Modules and `import`

What it is

A **module** is a `.py` file of functions, classes and variables that other code loads with `import`; its name is the file name without `.py` (`pricing.py` → `pricing`). Modules can also be compiled extensions (C/C++ compiled to machine code, `.so` / `.pyd`, e.g. parts of NumPy) or built into the interpreter (`sys`).

The **standard library** ("stdlib", "batteries included") ships a few hundred modules — `math`, `json`, `pathlib`, `datetime`, `random`, `csv`, `sqlite3`, `http`, `unittest` — for files, networking, data formats, testing and concurrency, with no installation.

Why it matters

- **Organisation** — one theme per file.
- **Reuse and sharing** — PyPI is just modules.
- **Namespaces** — each module has its own global names, so `json.load` and `pickle.load` (`pickle` saves objects as bytes) never collide.

How it works — the four import forms

Form	You then write	Brings into your namespace
`import math`	`math.sqrt(81)`	The name `math` (the module)
`from math import sqrt`	`sqrt(64)`	Only `sqrt`
`import numpy as np`	`np.array(...)`	The module under an **alias**
`from math import sqrt as square_root`	`square_root(2)`	One name under an alias

```
import math                                 # whole module
import math as m                            # module with an alias
from math import sqrt                       # one name
from math import sqrt as square_root        # one name with an alias
from os import path as os_path

print("math.sqrt(81)       =", math.sqrt(81))
print("sqrt(64)            =", sqrt(64))
print("square_root(m.pi)   =", square_root(m.pi))
print("m is math           =", m is math)
```

Output:

```
== four import forms ==
math.sqrt(81)       = 9.0
sqrt(64)            = 8.0
square_root(m.pi)   = 1.7724538509055159
m is math           = True
os_path.join('a','b') = a/b

== a module is an object with a namespace ==
sys   type=module  compiled into the interpreter
json  type=module  file ending __init__.py
names in math starting with 'is': ['isclose', 'isfinite', 'isinf', 'isnan', 'isqrt']
```

- **Dot notation** (`math.sqrt`) reaches attributes inside the module.
- An **alias** shortens names (`np`, `pd`, `plt` are conventions) or avoids clashes; `m is math` — same module object.
- `dir(module)` lists contents; `help(module)` shows docs. `json` reports `__init__.py` — it's a *package* (8.3).

`from module import *` copies every public name (no leading `_`, or those in `__all__`). PEP 8 discourages it outside interactive sessions: origins become invisible and your names can be silently overwritten.

Style (PEP 8): imports at the top, one per line, grouped *stdlib → third-party → your code* with blank lines between; `ruff` / `isort` sort them.

Trade-offs

	`import math`	`from math import sqrt`
Clarity at call site	`math.sqrt` shows origin	Shorter, origin hidden
Name clashes	None	Possible
Circular-import safety	Safer (attribute looked up at call time, 8.6)	Riskier
Monkey-patching/mocking (swapping an attribute at run time, e.g. a fake in tests)	Sees patched attributes	Holds the *old* object

Common mistakes

- Naming your file after a module you use (`random.py`, `json.py`) — 8.6.
- `import math.sqrt` → `ModuleNotFoundError: No module named 'math.sqrt'; 'math' is not a package`; use `from math import sqrt`.
- Wildcard imports in production code.

Real-world examples

Notebooks start `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt`; Django projects `from shop.models import Order`.

In the interview

> **30-second answer:** "A module is a `.py` file; `import m` loads it once and binds the name `m`, and `from m import x` binds just `x`. `as` gives an alias. Prefer explicit imports, avoid `import *`, and group imports stdlib/third-party/local."

- *`import a` vs `from a import b`?* → Both load all of `a`; only the bound name differs (after `from math import sqrt`, `math.pi` fails).
- *Importing twice runs twice?* → No — cached in `sys.modules` (8.2).

TL;DR

Module = file. `import m` / `from m import x` / `as` aliases. Explicit imports, grouped, at the top.

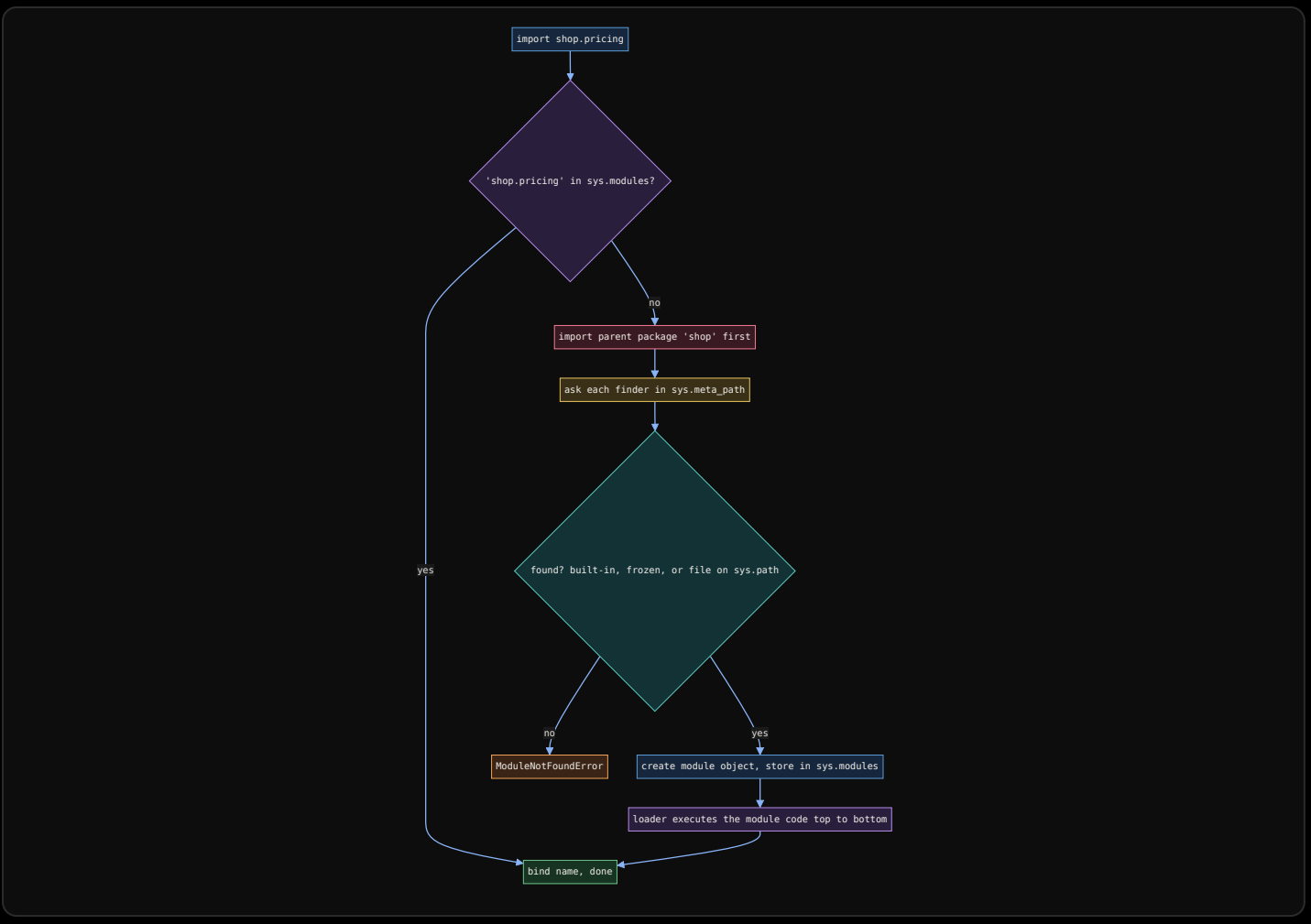
8.2 How `import` really works

What it is

The **import system** turns `import shop.pricing` into a module object:

- **`sys.modules`** — dict cache of every loaded module, keyed by full name.
- **Finders and loaders** — *finders* (in `sys.meta_path`) locate a module; a *loader* creates and executes it. The built-in chain handles built-in modules, frozen modules (baked into the interpreter binary) and the file system.
- **`sys.path`** — folders the file-system finder searches, in order.

How it works



1. **Check the cache.** In `sys.modules` → reuse; module code runs **once per process**. (`importlib.reload(m)` forces a re-run.)
2. **Import parents first:** `a`, then `a.b`, then `a.b.c`.
3. **Find.** The path finder walks **`sys.path`**:
    1. the running script's folder (or the current directory for `python -m` / `-c` / the REPL),
    2. folders in the `PYTHONPATH` environment variable,
    3. the standard library,
    4. `site-packages` — where `pip` installs (the active venv's, if any).
4. **Load.** The new module is put into `sys.modules` **before** its code runs (why circular imports see half-built modules, 8.6), then its top level executes.
5. **Bind** the requested names.

Bytecode is cached in `__pycache__/*.pyc` to skip re-parsing; safe to delete; add to `.gitignore`.

```
print("'math' in sys.modules:", "math" in sys.modules)
again = importlib.import_module("math")
print("second import returns the same object:", again is math)
json_mod = importlib.import_module("json")     # import by string name
```

```
'math' in sys.modules: True
second import returns the same object: True
json.dumps via import_module: {"a": 1}
```

`importlib.import_module("name")` imports by string — how plugin systems and frameworks (Django's `INSTALLED_APPS`) load modules named in configuration.

Common mistakes

- Scattering `sys.path.append(...)` hacks. Install the project (`pip install -e .`, 8.9) or run `python -m` from the project root.
- Expecting module edits to appear in a running REPL/notebook without `importlib.reload` (or Jupyter's `%autoreload`).

In the interview

> **30-second answer:** "`import` first checks `sys.modules`; if missing, finders search built-ins and then the folders in `sys.path` — script directory, PYTHONPATH, stdlib, site-packages. The loader creates the module, registers it in `sys.modules`, then runs its top-level code once. Later imports reuse the cached object."

- *`__pycache__`?* → Cached `.pyc` bytecode.
- *`sys.path` order?* → Script dir, PYTHONPATH, stdlib, site-packages; first match wins.

TL;DR

Cache (`sys.modules`) → find (`sys.meta_path`, `sys.path`) → load (register, execute once) → bind. First match on `sys.path` wins.

8.3 Packages, `__init__.py`, namespace packages, `__all__`

What it is

A **package** is a folder of modules imported as a unit, giving dotted names (`shop.pricing`, `sklearn.linear_model` — scikit-learn). Technically: a module with a `__path__` attribute (folders to search for submodules).

- **Regular package**: folder with `__init__.py`, which runs on the first import of the package or anything inside. Empty, or defines the public API (re-exports names, `__version__`).
- **Namespace package** (PEP 420, 3.3+): folder *without* `__init__.py`. Still importable; several `sys.path` folders can contribute to one namespace (splitting a namespace over separately installed distributions). No `__file__`.

	Module	Regular package	Namespace package
On disk	`x.py`	folder + `__init__.py`	folder, no `__init__.py`
Has `__path__`	No	Yes	Yes
Runs code on import	The file	`__init__.py`	Nothing
Use for	Single unit	Normal libraries and apps	One namespace over several installs

`__all__` — optional list of strings defining what `from package import *` exports; documents the public API.

How it works — the demo package

`02_packages_and_sys_path.py` writes this tree into a temp folder:

```
shop/
    __init__.py      # from .pricing import price_with_tax ; __all__ ; __version__
    __main__.py      # makes "python -m shop" work
    pricing.py       # uses a relative and an absolute import of utils
    utils.py
plugins/             # no __init__.py -> namespace package
    extra.py
```

```python
# shop/__init__.py
"""shop - a tiny demo package."""
from .pricing import price_with_tax      # re-export the public API
__all__ = ["price_with_tax"]
__version__ = "0.1.0"
print(f"  [running shop/__init__.py, __name__={__name__!r}]")
```

```python
sys.path.insert(0, str(root))
import shop                          # runs shop/__init__.py once
import shop                          # cached: __init__ does NOT run again
from shop import price_with_tax
```

```
== import it: put its parent folder on sys.path ==
  [running shop/__init__.py, __name__='shop']
  price_with_tax(100) = 108.0
  shop.__version__    = 0.1.0
  shop.__path__ is a list of folders: ['shop']
  pricing.__name__    = shop.pricing | __package__ = shop
  namespace package: extra plugin (namespace package, no __init__.py)
  plugins has __file__? None

== star import only takes names in __all__ ==
  names brought in: ['price_with_tax']
```

`__init__.py` ran once despite two imports. The re-export "facade" (a simple front hiding internal layout) is how `pd.DataFrame` works although `DataFrame` lives in `pandas/core/frame.py`.

Trade-offs

A heavy `__init__.py` importing every submodule slows `import pkg` and risks cycles; an empty one forces users to learn the layout. Middle ground: re-export the small public API; keep the rest `_private`.

When not to use it

Don't rely on namespace packages accidentally — tools (test runners, type checkers, packaging configs) handle regular packages more predictably.

Common mistakes

- Slow work (network calls, big files) at the top level of `__init__.py`.
- Assuming `import pkg` imports all submodules — only what `__init__.py` imports; otherwise `import pkg.sub`.

Real-world examples

- `from sklearn.linear_model import LogisticRegression` — subpackages.
- Stdlib packages: `json`, `email`, `http`, `urllib`, `concurrent`.
- Google's `google.cloud` namespace package spans `google-cloud-storage`, `google-cloud-bigquery`, etc.

In the interview

30-second answer: "A package is a folder of modules; with `__init__.py` it's a regular package whose init file runs on first import and usually re-exports the public API. Without it, it's a PEP 420 namespace package that can span multiple folders. `__all__` controls `import *` and documents the public surface."

- *`__init__.py` required in Python 3?* → No, but without it you get a namespace package, which behaves differently.

TL;DR

Package = folder of modules (+ `__init__.py` for regular). `__init__.py` runs once and defines the public API; `__all__` limits `import *`; packages have `__path__`.

8.4 Absolute vs relative imports

What it is

- **Absolute**: full path from a top-level package on `sys.path` — `from shop.utils import round_money`.
- **Relative**: dots relative to the *current package* — `from .utils import ...` (same package), `from ..core import x` (parent). Only `from … import …` can be relative.

How it works

Relative imports resolve via `__package__` (`'shop'` for `shop.pricing`). A file run directly is `__main__` with no parent package, so its relative imports fail:

```
python shop/pricing.py  -> ImportError: attempted relative import with no known parent package
python -m shop.pricing  ->
  [running shop/__init__.py, __name__='shop']
  pricing run directly: 10 -> 10.8
  stderr: RuntimeWarning: 'shop.pricing' found in sys.modules after import of package 'shop', but prior to
execution of 'shop.pricing'; this may result in unpredictable behaviour
```

`python -m` from the project root imports the package first, so it works. The `RuntimeWarning` is a real gotcha: `shop/__init__.py` already imported `shop.pricing`, so `-m` executes a *second* copy as `__main__`. Keep entry points (modules that start a program) in modules `__init__` doesn't import (`__main__.py`, `cli.py`).

Trade-offs

	Absolute	Relative
Readability	Full origin visible	Short in deep packages
Renaming the top package	Must edit imports	Keeps working
Works in a script run directly	Yes (if package is on `sys.path`)	No
PEP 8	**Recommended**	Acceptable intra-package

Python 2's *implicit* relative imports are gone: bare `import utils` is always absolute.

Common mistakes

- `python pkg/mod.py` → relative imports fail; use `python -m pkg.mod`.
- Dots beyond the top package → `ImportError: attempted relative import beyond top-level package`.

In the interview

> **30-second answer:** "Absolute imports use the full dotted path and are PEP 8's default; relative imports use dots relative to the current package and work only inside packages, not in files run as scripts — run those with `python -m`."

TL;DR

Absolute by default; dots for intra-package imports; run package modules with `python -m`.

8.5 `if __name__ == "__main__":` and `python -m`

What it is

`__name__` is the module name when **imported** (`'shop.pricing'`) and `'__main__'` when **run directly** (`python file.py` or `python -m pkg.mod`). The guard (intro in §1.5)

```python
if __name__ == "__main__":
    main()
```

runs code only on execution, not import.

Why it matters

One file is both a **reusable module** and a **runnable script**: importers (tests) don't trigger script behaviour (starting a server, parsing command-line arguments, printing). It's also required with `multiprocessing` (work in separate Python processes) where workers start by re-importing the main module — the `spawn` and `forkserver` start methods (Windows; macOS by default since 3.8; Linux defaults to `forkserver` since 3.14) — or each worker re-runs the script.

How it works

- `python -m package.module` runs a module found via `sys.path` as `__main__` — the right way to run code in packages, and how tools run: `python -m pip`, `-m venv`, `-m pytest`, `-m http.server`, `-m json.tool`.
- `python -m package` runs `package/__main__.py`.
- In a script `sys.path[0]` is the script's folder; with `-m`, the current directory.

```
python -m shop          ->
  [running shop/__init__.py, __name__='shop']
  python -m shop says: 19.99 -> 21.59
```

Common mistakes

- Top-level test calls or `print`s without the guard run on every import.
- Lots of logic *inside* the guard — keep it to `main()` so logic stays importable and testable.

In the interview

30-second answer: " `__name__` is `'__main__'` for the file being run and the module name when imported. The guard keeps script-only code from running on import. `python -m pkg.mod` runs a module inside a package with correct imports; `python -m pkg` runs `pkg/__main__.py`."

TL;DR

Guard script behaviour with `if __name__ == "__main__": main()`; use `python -m` for modules inside packages.

8.6 Common import errors and circular imports

What it is

`03_import_errors.py` reproduces each common failure in a throwaway folder:

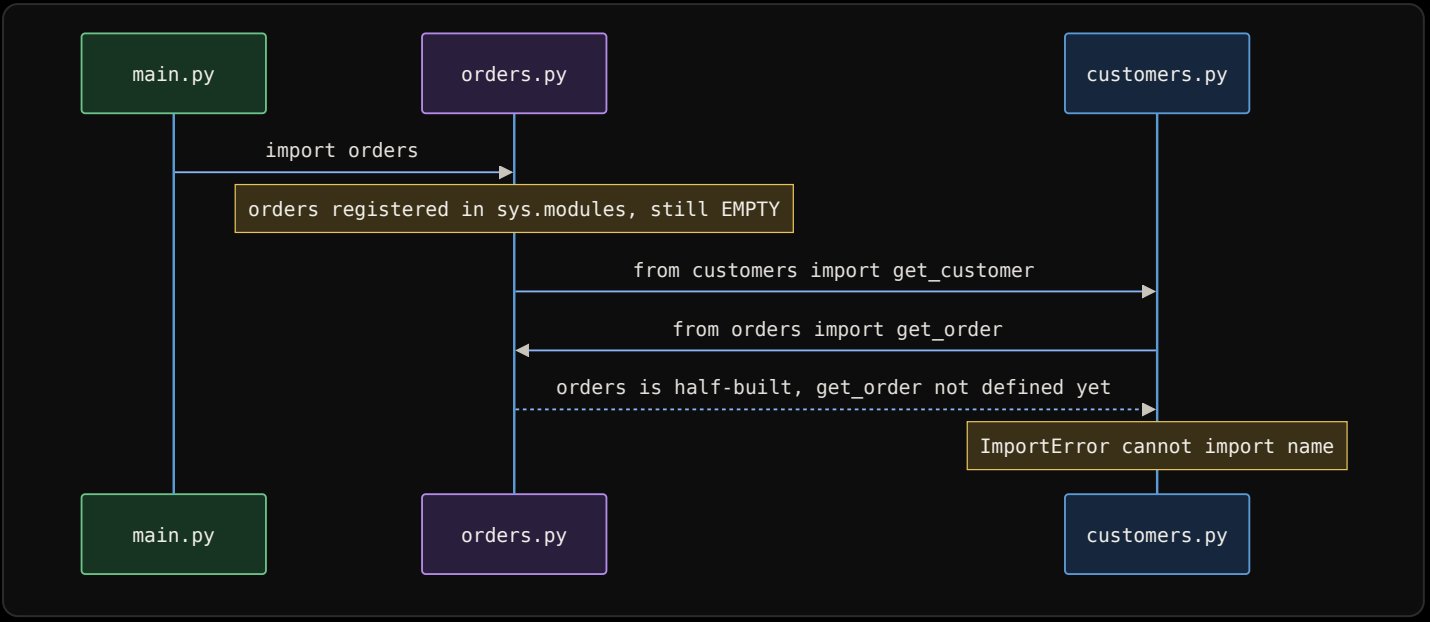
```
1. missing module                  -> ModuleNotFoundError: No module named 'reqests'
2. name not in module              -> ImportError: cannot import name 'square_root' from 'math' (.../math.cpython-
314-darwin.so)
3. shadowing the stdlib            -> AttributeError: module 'random' has no attribute 'randint' (consider
renaming '.../random.py' since it has the same name as the standard library module named 'random' and prevents
importing that standard library module)
4. circular import (from-import)   -> ImportError: cannot import name 'get_order' from 'orders' (consider renaming
'.../orders.py' if it has the same name as a library you intended to import)
4b. fixed: import module, use attr -> {'id': 7, 'customer': {'id': 1}}
4c. fixed: import inside function  -> {'id': 7, 'customer': {'id': 1}}
```

Error	Usual cause	Fix
`ModuleNotFoundError` (subclass of `ImportError`)	Typo; not installed **in the interpreter you're running**; wrong working directory	Check spelling; `python -m pip install x` with the same `python`; install the project or use `-m`
`ImportError: cannot import name`	Name doesn't exist (typo, removed in a new version) — or a circular import	Check the module's contents / version
`AttributeError: module 'x' has no attribute 'y'`	Your `x.py` shadows the real module; or submodule not imported	Rename your file (delete its `__pycache__`)
`ImportError: attempted relative import …`	Ran a package module as a script	`python -m pkg.mod`

Case 3: `sys.path[0]` (script folder) precedes the stdlib, so a local `random.py` wins. 3.13+ adds the "consider renaming" hint.

Circular imports

A **circular import**: A imports B while B (directly or indirectly) imports A.



Why: modules are registered in `sys.modules` *before* their body runs (8.2 step 4), so `customers` gets a half-initialised `orders` whose `def get_order` hasn't executed. Python 3.12 says it plainly ("cannot import name 'get_order' from partially initialized module 'orders' (most likely due to a circular import)"); the 3.14 run above appends a misleading generic "consider renaming" hint instead. If `cannot import name` names something that clearly exists, suspect a cycle.

Fixes, best first:

1. **Restructure** — move shared code to a third module (the right design fix).
2. **Import the module, not the name** (`import orders`; `orders.get_order(...)` inside functions) — looked up at *call* time, after both finished loading (4b).
3. **Deferred import** inside the function (4c) — cheap, local, but hides a dependency.
4. Type-hint-only imports: `if TYPE_CHECKING: from orders import Order` (`from typing import TYPE_CHECKING`; seen by checkers, not at runtime).

Common mistakes

- "`pip install` worked but import fails": different interpreter. Use `python -m pip` and check `which python` / `sys.executable`.
- Files named `test.py`, `code.py`, `email.py`, `random.py`, `numpy.py`.

Real-world examples

Django models referencing each other across apps use string references (`models.ForeignKey("customers.Customer", ...)` — a *foreign key* links one database record to another) — "look it up later" at framework level.

In the interview

> **30-second answer:** "A circular import happens because a module is put in `sys.modules` before its body finishes, so the other module sees a half-built object and `from x import y` fails. Fix by moving shared code to a third module, importing the module instead of names so lookup happens at call time, or deferring the import into a function."

- *`ModuleNotFoundError` though installed?* → Different interpreter/venv, or a shadowing local file.
- *Why does `import a` survive a cycle when `from a import f` doesn't?* → `import a` needs only the module object; `from a import f` needs `f` defined.

TL;DR

Check spelling, interpreter and shadowing first. Cycles break `from x import y`; restructure, `import module`, or defer.

8.7 `pip` and virtual environments

What it is

- **PyPI** (pypi.org) — the public repository of third-party packages.
- `pip` — the standard installer: downloads packages (preferably pre-built **wheels**, `.whl`) and their dependencies into an environment's `site-packages`.
- **Virtual environment (venv)** — isolated folder with its own `site-packages` and a `python` link to a base interpreter; installs don't affect other projects or the system Python. (Intro in §1.4.)

Why it matters

Project A needs Django 4.2, B needs 5.1 — one `site-packages` can't hold both. Venvs give per-project versions, reproducibility, and protect the OS's Python. Homebrew Python and Debian/Ubuntu now **refuse** global `pip install` (`externally-managed-environment`, PEP 668).

How it works

```
python3 -m venv .venv                 # create (folder name .venv is the convention)
source .venv/bin/activate             # macOS/Linux   (Windows: .venv\Scripts\activate)
python -m pip install requests        # install into THIS venv
python -m pip install "django>=5,<6"  # version specifiers
python -m pip list                    # what's installed
python -m pip show requests           # details, dependencies
python -m pip install -U requests     # upgrade
python -m pip uninstall requests
python -m pip freeze > requirements.txt   # exact versions of everything installed
python -m pip install -r requirements.txt # recreate elsewhere
deactivate                            # leave the venv
```

"Activating" only prepends the venv's `bin/` to `PATH` (folders the shell searches for commands) and changes the prompt — you can instead call `.venv/bin/python` directly. A venv is **not portable**: never copy or commit it; commit the dependency list and add `.venv/` to `.gitignore`.

Inside a venv `sys.prefix` (the venv) differs from `sys.base_prefix` (the base install):

```python
def in_venv() -> bool:
    return sys.prefix != sys.base_prefix

venv.create(env_dir, with_pip=False)   # same as: python -m venv .venv --without-pip
```

```
== which interpreter am I? ==
  Python 3.14.7
  running inside a venv: True
  sys.prefix folder name: .venv-books

== installed distributions (importlib.metadata) ==
  numpy         2.5.3
  pandas        3.0.6
  scikit-learn  1.9.1
  numpy requires python: >=3.12

== a fresh, empty venv is isolated ==
  created: ['bin', 'include', 'lib', 'pyvenv.cfg']
  new venv -> in venv: True
  new venv -> No module named 'numpy'
```

A brand-new venv can't see the book environment's NumPy — isolation. `importlib.metadata.version("numpy")` reads installed versions at runtime.

Trade-offs

Tool	What it isolates	Notes
`venv` (stdlib)	Python packages	Built in; uses an existing Python
`pipx`	Each CLI *tool* in its own venv	For tools like `black`, `httpie`, not project libraries
`conda`	Packages **and** non-Python libraries (CUDA, MKL — GPU/maths libraries — compilers), Python itself	Popular in data science; own channels
Docker (app + OS files in a *container*)	Whole OS userland (OS files outside the kernel)	Deployment-level isolation, heavier

When not to use it

Stdlib-only scripts don't need a venv; everything else gets one per project.

Common mistakes

- `pip` running a different Python → `python -m pip`.
- `sudo pip install` into the system Python.
- Committing or copying `.venv/`.
- Unpinned production installs (today's and next year's `pip install requests` differ).

Real-world examples

CI pipelines (e.g. GitHub Actions) create a fresh environment per run from a lock/requirements file; Heroku-style platforms and Docker images run `pip install -r requirements.txt` at build time.

In the interview

30-second answer: "pip installs packages from PyPI into an environment's site-packages. A venv is an isolated environment per project — `python -m venv .venv`, activate, `python -m pip install`. It prevents version conflicts between projects and protects the system Python. Don't commit the venv; commit the dependency list."

- *Detect a venv in code?* → `sys.prefix != sys.base_prefix`.

- *venv vs conda?* → venv: packages for an existing interpreter; conda also manages Python versions and native libraries.

TL;DR

One venv per project: `python -m venv .venv` → activate (changes `PATH`) → `python -m pip install …`. Never commit the venv.

8.8 `requirements.txt` vs `pyproject.toml`; `uv` and Poetry

What it is

- **`requirements.txt`** — pip-specific list of packages, often exact pins (`requests==2.32.3`); describes an *environment*, not a project.
- **`pyproject.toml`** — the standard project file (TOML syntax): **build requirements** (PEP 518, `[build-system]`), **project metadata** — name, version, Python range, dependencies, optional extras (named groups of extra dependencies), entry points (PEP 621, `[project]`) — and tool settings (`[tool.ruff]`, `[tool.pytest.ini_options]`).
- A **lock file** (`uv.lock`, `poetry.lock`, or fully pinned requirements) records the exact version of *every* package, including transitive dependencies (dependencies of dependencies), for reproducible installs.

How it works

A minimal `pyproject.toml`, parsed by the script with stdlib `tomllib` (3.11+):

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "shop"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["requests>=2.31,<3", "pydantic>=2"]

[project.optional-dependencies]
dev = ["pytest>=8", "ruff"]

[project.scripts]
shop = "shop.cli:main"
```

```
== reading pyproject.toml (tomllib, 3.11+) ==
  name/version : shop 0.1.0
  dependencies : ['requests>=2.31,<3', 'pydantic>=2']
  dev extras   : ['pytest>=8', 'ruff']
  console script 'shop' -> shop.cli:main
  build backend: hatchling.build
```

- `dependencies` holds **ranges** (what you're compatible with); the lock holds **exact pins** (what you tested).
- `pip install -e ".[dev]"` = **editable** install (points at your source folder, edits apply without reinstalling) plus `dev` extras.
- `[project.scripts]` makes pip create a `shop` command calling `shop.cli:main()`.
- The **build backend** (hatchling, setuptools, flit-core, poetry-core, uv_build…) builds distributable files (PEP 517).

	`requirements.txt`	`pyproject.toml`
Standard?	pip convention	PEP 518/517/621 standard
Describes	An environment to install	A project: metadata, deps, build, tool config
Typical content	Exact pins	Compatible ranges
Used by	`pip install -r`	pip, build, uv, Poetry, hatch, ruff, pytest…
Best for	Deploying an app; simple scripts	Any package or modern project

Version specifiers: `==2.1.0` exact, `>=2.1` minimum, `<3` upper bound, `~=2.1` "compatible release" (≥2.1, <3.0), `!=2.1.3` exclude.

Modern tools: `uv` and Poetry (brief)

- **uv** (Astral, written in Rust) — one fast binary replacing pip, venv, pip-tools and pipx: `uv init` (new project + `pyproject.toml`), `uv add requests` (add + update `uv.lock` + install), `uv sync` (make `.venv` match the lock), `uv run pytest`, `uv python install 3.12` (manage Python versions), `uv pip install ...` (pip-compatible), `uv build` / `uv publish`.
- **Poetry** — older, widely used all-in-one: `poetry new`, `add`, `install`, `poetry.lock`, `build`, `publish`; 2.x supports the standard `[project]` table.

Both manage `pyproject.toml`, resolve dependencies, write a lock file and create the venv.

When not to use it

A one-file script needs only a venv + `requirements.txt`. Don't mix tools in one project — pick one source of truth.

Common mistakes

- Exact pins in a **library's** `dependencies` — makes it uninstallable alongside others. Libraries use ranges; applications lock.
- `pip freeze` for a library's requirements (captures your whole environment, dev tools included).
- Not committing the lock file for applications.

Real-world examples

pandas, FastAPI, pytest and scikit-learn declare metadata in `pyproject.toml`; ruff, black, mypy and pytest read config from it.

In the interview

> **30-second answer:** "`requirements.txt` is a pip input listing packages, usually pinned — an environment snapshot. `pyproject.toml` is the standard project file: build system, metadata, dependency ranges, extras, entry points and tool config. Reproducibility comes from a lock file; tools like uv and Poetry manage the pyproject, lock and venv together."

- *Library vs application dependencies?* → Libraries: loose ranges; applications: exact pins in a lock.
- *Wheel?* → Pre-built distribution (`.whl`, a zip) installed without a build step — fast, no compiler.

TL;DR

`pyproject.toml` = the project (ranges, metadata, build, tool config). Lock file / pinned `requirements.txt` = exact environment. uv and Poetry manage both.

8.9 Project layout and publishing basics

What it is

The **src layout** puts the package under `src/`:

```
shop-project/
├── pyproject.toml
├── README.md
├── src/
│   └── shop/
│       ├── __init__.py
│       ├── __main__.py
│       ├── cli.py
│       └── pricing.py
└── tests/
    └── test_pricing.py
```

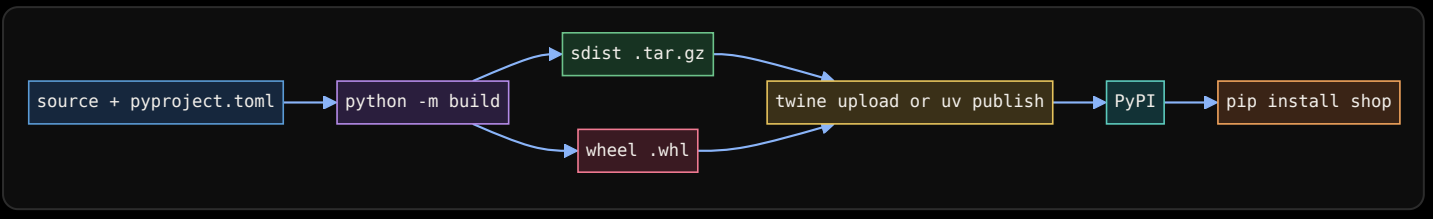
The **flat layout** puts `shop/` next to `pyproject.toml`.

Why it matters

With a flat layout, running from the root puts it on `sys.path`, so `import shop` uses your *source folder* — tests pass even when packaging is broken (a file missing from the wheel, a wrong `package-data` rule). With src, `shop` imports **only after installing** (`pip install -e .`), so tests exercise what users get — the advantage the Python Packaging User Guide describes.

How it works — publishing to PyPI

1. Metadata in `pyproject.toml` (unique PyPI name; bump `version` every release — PyPI never accepts a re-upload).
2. **Build**: `python -m build` (or `uv build`) → `dist/shop-0.1.0.tar.gz` (**sdist**, source, built on install) and `dist/shop-0.1.0-py3-none-any.whl` (**wheel**, installs by unpacking).
3. **Test** on **TestPyPI** (test.pypi.org), a sandbox index.
4. **Upload**: `python -m twine upload dist/*` (or `uv publish`) with an API token — or better, **Trusted Publishing** from CI (GitHub Actions proves identity to PyPI via OpenID Connect; no stored long-lived token).
5. Users `pip install shop`.



Trade-offs

	src layout	flat layout
Catches packaging mistakes	Yes (must install to import)	No
Setup effort	Needs `pip install -e .`	Works from root immediately
Used by	Many libraries (and uv's `--package` / `--lib` templates)	Many apps and simple projects

When not to use it

Never-packaged apps (a Django site, a notebook project) gain little.

Common mistakes

- Uploading secrets — inspect `dist/` first (a wheel is a zip: `unzip -l`).
- Re-using a version number.
- PyPI name ≠ import name (`scikit-learn` → `sklearn`, `Pillow` → `PIL`) — keep them aligned in new projects.

Real-world examples

`pip install scikit-learn` → `import sklearn` is the classic mismatch; many popular projects publish from GitHub Actions via Trusted Publishing.

In the interview

> **30-second answer:** "Use a `pyproject.toml`, ideally a src layout so tests run against the installed package. Build an sdist and a wheel with `python -m build` or `uv build`, try them on TestPyPI, then upload with twine or `uv publish`, preferably via Trusted Publishing from CI. Versions are immutable on PyPI."

- *sdist vs wheel?* → Source built at install vs pre-built, unpacked.
- *Why src layout?* → Prevents importing the working copy instead of the installed package.

TL;DR

src layout + `pyproject.toml` + editable install for dev. `build` → sdist + wheel → TestPyPI → PyPI (Trusted Publishing).

Chapter TL;DR

- **Module** = a `.py` file; **package** = folder of modules (`__init__.py` → regular, none → namespace). `import m`, `from m import x`, `as` aliases; avoid `import *`.
- `import` checks `sys.modules` → finds via `sys.path` (script dir, PYTHONPATH, stdlib, site-packages) → registers then runs the module **once** → binds names.
- Absolute imports by default; relative (`.`) only inside packages; run package modules with `python -m`.
- `if __name__ == "__main__": main()` separates "run" from "import".
- Import errors: typo / wrong interpreter / shadowing file / circular import (module registered before its body runs — restructure, `import module`, or deferred import).
- One **venv** per project; `python -m pip`; never commit `.venv`.
- `pyproject.toml` (standard: metadata, dependency ranges, build, tool config) vs `requirements.txt` (pip list, pins); lock files for reproducibility; **uv** or **Poetry** manage it all.
- **src layout**, build **sdist + wheel**, TestPyPI, then PyPI via twine/`uv publish`/Trusted Publishing.

Chapter 9 — Errors, Exceptions and Files

Programs meet bad input, missing files, full disks and broken networks. Python reports these with **exceptions** — objects that interrupt normal flow and travel up the call stack (the pile of calls in progress) until handled. Most failures involve I/O (input/output: files, networks, devices), so this chapter pairs error handling with files: text and bytes, paths, CSV, JSON, pickle (§9.11), temporary files and folder operations.

Scripts are in `code/`; all file operations happen in a temporary directory removed afterwards. Output is real (Python 3.14; code is 3.12-compatible).

#	Topic	Script
9.1	Exceptions, tracebacks, the built-in hierarchy	`01_exceptions.py`
9.2	`try` / `except` / `else` / `finally`	`01_exceptions.py`
9.3	Raising, re-raising, custom exceptions	`01_exceptions.py`
9.4	Chaining (`raise … from …`), `add_note`	`01_exceptions.py`
9.5	EAFP vs LBYL	`01_exceptions.py`
9.6	`ExceptionGroup` and `except*` (3.11+)	`01_exceptions.py`
9.7	`assert`	`01_exceptions.py`
9.8	Context managers and `with`	`02_context_managers.py`
9.9	File I/O: modes, text vs binary, encodings	`03_files_pathlib.py`
9.10	`pathlib`, `shutil`, `tempfile`	`03_files_pathlib.py`
9.11	`csv`, `json`, `pickle` (and why pickle is unsafe)	`04_csv_json_pickle.py`

9.1 Exceptions and the hierarchy

What it is

An **exception** is an object describing an error, **raised** at the point of failure. Raising stops the current code; Python unwinds the call stack, leaving each function in turn, looking for a matching `except`. If none matches, the program stops and prints a **traceback** — the chain of calls that led there. Read it **bottom-up**: last line = exception type and message, the line above = where it happened.

```
Traceback (most recent call last):
  File "app.py", line 8, in <module>
    total = parse("12a")
  File "app.py", line 3, in parse
    return int(text)
ValueError: invalid literal for int() with base 10: '12a'
```

- **Syntax errors** (`SyntaxError`, `IndentationError`) are found when Python *compiles* the file — nothing runs, so they can't be caught in that file.
- **Exceptions** happen while code *runs* (`ZeroDivisionError`, `KeyError`, …) and can be handled.

How it works — the class hierarchy

Every exception is an instance of a class deriving from `BaseException`. Handlers match a class *and all its subclasses*, so the tree decides what an `except` catches.

```
== part of the built-in hierarchy ==
  BaseException
      BaseExceptionGroup
      Exception
      GeneratorExit
      KeyboardInterrupt
      SystemExit
  ...under Exception (selection):
  LookupError
      IndexError
      KeyError
  ArithmeticError
      FloatingPointError
      OverflowError
      ZeroDivisionError
  OSError
      FileNotFoundError
      IsADirectoryError
      PermissionError
      TimeoutError
  issubclass(ModuleNotFoundError, ImportError): True
```

(The script walks `__subclasses__()` of the real classes; a selection is printed.)

Class	Raised when	Example
`SystemExit`, `KeyboardInterrupt`, `GeneratorExit`	Program exit, Ctrl-C, generator (function yielding values one at a time) closing	Deliberately **not** under `Exception` so `except Exception` doesn't swallow (catch and hide) them
`ValueError`	Right type, bad value	`int("abc")`
`TypeError`	Wrong type / wrong call	`"a" + 1`, too many arguments
`KeyError` / `IndexError` (both `LookupError`)	Missing key / index out of range	`{}["x"]`, `[][0]`
`AttributeError`	Missing attribute	`None.upper()`
`NameError` (→ `UnboundLocalError`)	Unknown name	typo in a variable
`ZeroDivisionError` (`ArithmeticError`)	Division by zero	`1 / 0`
`OSError` family	OS-level failures	`FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `TimeoutError`, `ConnectionError`
`ImportError` (→ `ModuleNotFoundError`)	Import failures	Chapter 8
`StopIteration`	Iterator exhausted	used by `for` internally
`RecursionError` (`RuntimeError`)	Too-deep recursion	Chapter 7
`NotImplementedError` (`RuntimeError`)	Abstract method (declared by a base class for subclasses to fill in) not overridden	
`AssertionError`	`assert` failed	9.7
`UnicodeDecodeError` (→ `ValueError`)	Bytes invalid in the codec (a text encoding's encoder/decoder)	9.9

In the interview

> **30-second answer:** "An exception is an object raised on error; it unwinds the stack until an `except` clause whose class matches (including subclasses) handles it, otherwise the program ends with a traceback. All exceptions derive from `BaseException`; application errors derive from `Exception`, while `SystemExit`, `KeyboardInterrupt` and `GeneratorExit` sit directly under `BaseException` so generic handlers don't catch them."

- *Why isn't `KeyboardInterrupt` an `Exception`?* → So `except Exception:` doesn't break Ctrl-C.
- *Read a traceback?* → Bottom line = type and message; frames above = call path, most recent last.

TL;DR

Exceptions are objects that unwind the stack until matched by class. Know the tree: `BaseException` → `Exception` → your errors.

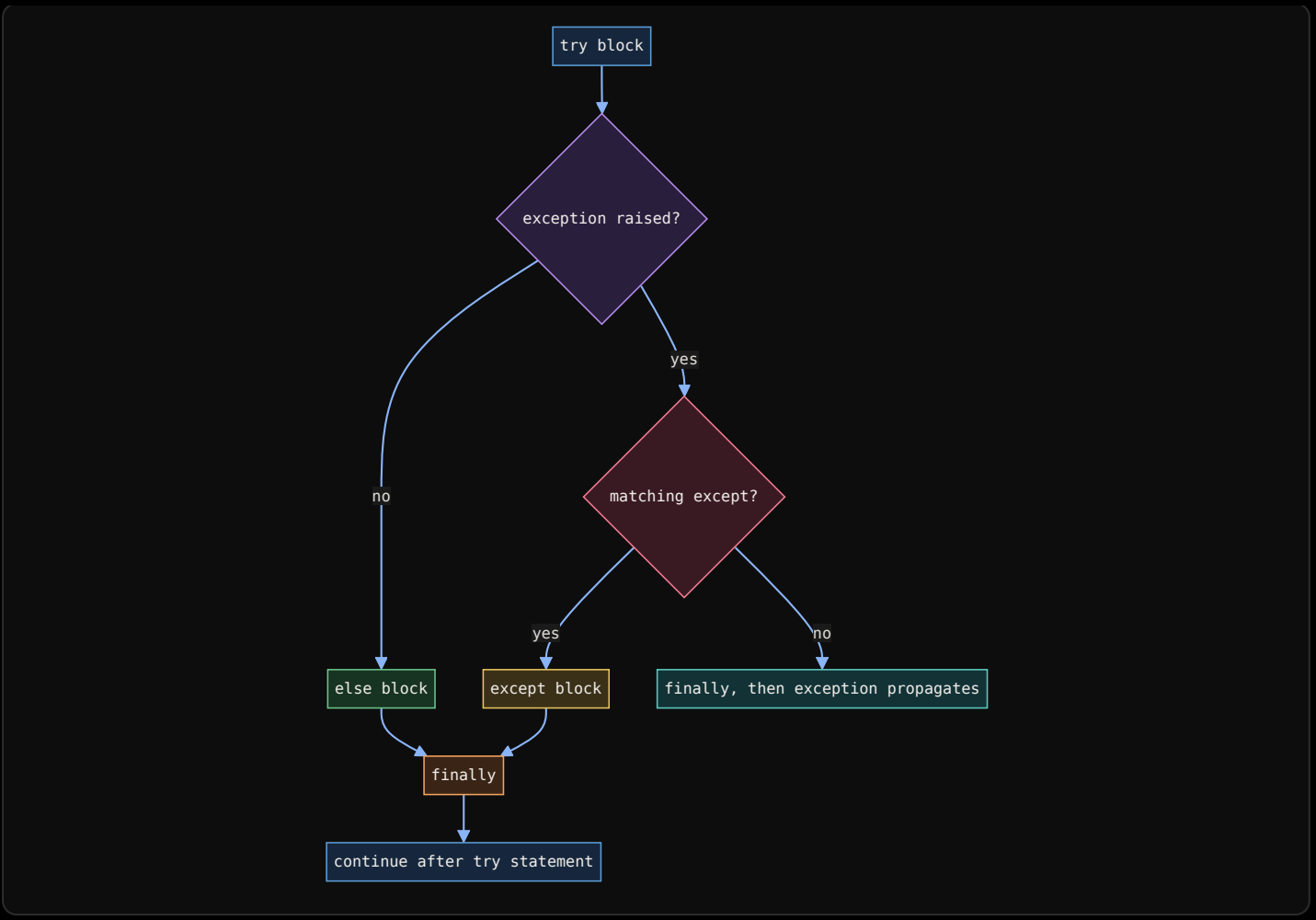
9.2 `try` / `except` / `else` / `finally`

What it is

```
try:
    risky()                 # code that might fail
except SomeError as exc:    # runs if SomeError (or subclass) was raised in try
    handle(exc)
else:                       # runs only if try raised NOTHING
    use_result()
finally:                    # runs ALWAYS: success, handled, unhandled, return, break
    clean_up()
```

How it works

1. Run `try`.
2. On an exception, skip the rest of `try`; check `except` clauses **top to bottom**; the **first** match runs. `except (A, B) as e:` catches either.
3. No exception → run `else`.
4. Always run `finally` — even while an exception propagates (travels up the stack) or the block `return`s.
5. No match → the exception continues upward (after `finally`).




```python
def parse_age(text):
    try:
        age = int(text)
    except ValueError:
        print(f"  except: {text!r} is not a number")
        return None
    else:
        print(f"  else:   parsed {age}")
        return age
    finally:
        print("  finally: always runs")
```

```
== try / except / else / finally ==
  else:   parsed 42
  finally: always runs
  except: 'forty' is not a number
  finally: always runs
```

`finally` ran though both branches `return`ed.

Why `else`? It keeps `try` tiny — only the line that can raise the handled error — so a bug in success-path code isn't silently caught by the wrong handler.

Trade-offs / rules of thumb

Do	Don't
Catch the **most specific** exception you can handle	Bare `except:` — also catches `KeyboardInterrupt` and `SystemExit`
Order handlers specific → general	`except Exception` first (shadows the rest — later handlers become dead code)
Keep `try` blocks small	Wrap 50 lines in one `try`
Handle where you can *do something* (retry, default, message)	Catch just to `pass` (hides bugs)
Use `finally` / `with` for cleanup	`return` inside `finally` — swallows exceptions (3.14 emits `SyntaxWarning`, PEP 765)

When not to use it

Not for ordinary control flow when a check or API exists (`dict.get`, `str.isdigit` for trusted formats). Don't catch errors you can't handle — let a top-level handler log them.

Common mistakes

- `except Exception as e: pass` — the bug resurfaces far away.
- Using `e` after the handler — Python **deletes** the `as` name at the end (to break reference cycles, objects referring to each other that can't be freed promptly) → `NameError`. Copy it if needed.
- Catching `Exception` around your own bugs (a typo's `TypeError`) and reporting "invalid input".

Real-world examples

- **Log, don't hide**: `logging.exception("msg")` inside an `except` logs the message *plus* traceback; `traceback.print_exc()` / `format_exc()` give the same text on stderr or as a string (Chapter 14).
- **Warnings aren't exceptions**: `warnings.warn("old_api() is deprecated", DeprecationWarning)` prints and continues; `python -W error` (or pytest's `-W error`) turns warnings into exceptions so CI catches deprecations.
- Django, Flask, FastAPI wrap each request in a top-level handler turning uncaught exceptions into a 500 response ("internal server error"; HTTP basics in System Design Fundamentals) plus a log entry.
- `requests`: `resp.raise_for_status()` raises `requests.HTTPError` for 4xx (client's fault) / 5xx (server's fault); all errors derive from `requests.RequestException`.

In the interview

> **30-second answer:** "`try` runs risky code; the first matching `except` handles; `else` runs only if nothing was raised, which keeps the `try` minimal; `finally` always runs for cleanup. Catch specific exceptions, order specific to general, never a bare `except`."

- *Does `finally` run if `try` returns?* → Yes, just before the return completes.
- *Bare `except:` vs `except Exception:`?* → Bare also catches `KeyboardInterrupt` / `SystemExit`.

TL;DR

`try` small, `except` specific (first match wins), `else` on success, `finally` always; the `as` name is deleted after the handler.

9.3 Raising, re-raising and custom exceptions

What it is

- `raise SomeError("message")` — signal an error yourself, typically when inputs violate your function's contract (its promises about valid inputs/outputs).
- **Re-raising**: a bare `raise` inside `except` re-throws the current exception with its original traceback, after logging or cleanup. Outside a handler: `RuntimeError: No active exception to reraise`.
- **Custom exceptions** — your own `Exception` subclasses so callers can catch *your* errors precisely.

How it works

1. Use a built-in when it fits: bad value → `ValueError`, wrong type → `TypeError`, missing key-like thing → `KeyError` / `LookupError`, unsupported operation → `NotImplementedError`.
2. For domain errors (errors meaningful in your problem area, e.g. payments), define a module **base class** plus specific subclasses — callers catch all (`except PaymentError`) or one (`except CardDeclined`).
3. Suffix names with `Error` (PEP 8); add attributes the handler needs, set in `__init__`.

```
class PaymentError(Exception):
    """Base class for this module's errors."""

class CardDeclined(PaymentError):
    def __init__(self, card_last4, reason):
        super().__init__(f"card ending {card_last4} declined: {reason}")
        self.card_last4 = card_last4
        self.reason = reason

class InsufficientFunds(PaymentError):
    pass

def charge(balance, amount):
    if amount <= 0:
        raise ValueError(f"amount must be positive, got {amount}")
    if amount > balance:
        raise InsufficientFunds(f"need {amount}, have {balance}")
    return balance - amount

def reraise_after_logging(x):
    try:
        return 1 / x
    except ZeroDivisionError:
        print("  logging the error, then re-raising")
        raise                           # bare raise keeps the original traceback
```

```
== raising and catching custom exceptions ==
  charge(100, 30) -> 70
  InsufficientFunds: need 500, have 100
  ValueError: amount must be positive, got -5
  caught CardDeclined, attrs: last4=4242 reason=expired

== re-raising ==
  logging the error, then re-raising
  caller still sees: ZeroDivisionError('division by zero')
```

Pass the message to `super().__init__` so `str(exc)` and tracebacks show it.

Trade-offs

Too few custom exceptions → callers parse message strings (fragile); too many → a zoo. A base class plus a few meaningful subclasses is the sweet spot. Many libraries also inherit a matching built-in (`class InvalidAmount(PaymentError, ValueError)`) so generic `except ValueError` keeps working.

Common mistakes

- Subclassing `BaseException` — escapes `except Exception`.
- `raise exc` instead of bare `raise` in the handler — works but adds the current line to the traceback.
- `raise "error"` — `TypeError` in Python 3; only exception instances/classes.
- Returning error codes (`return -1`) — callers forget to check.

Real-world examples

- `requests.RequestException` → `ConnectionError`, `Timeout`, `HTTPError`.
- Django: `ObjectDoesNotExist` base with per-model `Model.DoesNotExist`; `ValidationError`.
- `json.JSONDecodeError` subclasses `ValueError`, so old `except ValueError` code works.

In the interview

> **30-second answer:** "Raise built-ins like `ValueError` / `TypeError` for contract violations; for domain errors create a base exception subclassing `Exception` plus specific subclasses with useful attributes. Inside a handler, bare `raise` re-throws with the original traceback."

- *Why a base class per library?* → One clause catches all its errors.
- *`raise` vs `raise e`?* → Bare re-raises the active exception unchanged; `raise e` re-raises the object from the current line.

TL;DR

`raise` built-ins for contract errors; custom hierarchy (base + `...Error` subclasses) for domain errors; bare `raise` to re-throw.

9.4 Exception chaining and notes

What it is

Raising a higher-level error while handling a low-level one records the link:

Syntax	Stored in	Traceback says
`raise New(...)` inside an `except` (implicit)	`new.__context__`	"During handling of the above exception, another exception occurred"
`raise New(...) from orig` (explicit)	`new.__cause__`	"The above exception was the direct cause of the following exception"
`raise New(...) from None`	suppresses the context	Only the new exception is shown

`exc.add_note("…")` (3.11+) attaches context lines shown under the message — "which row/file/request?".

Why it matters

Translating errors keeps your API clean (callers catch `ConfigError`, not your internals' `KeyError`/`ValueError`) *without* losing the root cause needed for debugging.

How it works

```
class ConfigError(Exception):
    pass

def load_port(cfg):
    try:
        return int(cfg["port"])
    except (KeyError, ValueError) as exc:
        raise ConfigError("invalid 'port' setting") from exc
```

```
== exception chaining: raise ... from ... ==
  port = 8080
  ConfigError: invalid 'port' setting; __cause__ = KeyError('port')
  ConfigError: invalid 'port' setting; __cause__ = ValueError("invalid literal for int() with base 10: 'eighty'")
  implicit: __context__ = KeyError('missing')
  formatted traceback of an explicit chain (last lines):
    | Traceback (most recent call last):
    | KeyError: 'port'
    |
    | The above exception was the direct cause of the following exception:
    |
    | Traceback (most recent call last):
    | ConfigError: invalid 'port' setting

== add_note (3.11+) ==
   invalid literal for int() with base 10: 'x1' | notes: ['while reading row 7 of prices.csv']
```

(Only unindented traceback lines are printed.)

Common mistakes

- Wrapping without `from` and without a message — hides what happened.
- `from None` everywhere — discards debugging information; use only when the original is noise.

In the interview

30-second answer: "Raising inside an `except` chains implicitly via `__context__`; `raise X from e` sets `__cause__` and says 'direct cause' in the traceback; `from None` hides the chain. Use explicit chaining when translating low-level errors into your own exception types. `add_note` adds context in 3.11+."

TL;DR

Translate with `raise MyError(...) from exc`; add context with `add_note`.

9.5 EAFP vs LBYL

What it is

- **LBYL — Look Before You Leap**: check, then act. `if key in d: v = d[key]`.
- **EAFP — Easier to Ask Forgiveness than Permission**: try, handle the exception. `try: v = d[key] except KeyError: ...`.

The official glossary calls EAFP the common Python style.

Why it matters

EAFP avoids **race conditions** — the time-of-check-to-time-of-use (TOCTOU) gap: `if os.path.exists(p): open(p)` fails if another process deletes the file in between; `try: open(p) except FileNotFoundError:` has no gap. It also avoids double work (check *and* lookup) and suits duck typing (call the method instead of checking the type).

```python
stock = {"apple": 3}
qty = stock["pear"] if "pear" in stock else 0      # LBYL
try:                                               # EAFP
    qty = stock["pear"]
except KeyError:
    qty = 0
stock.get("pear", 0)                               # the idiomatic one-liner
```

```
LBYL: 0
EAFP: 0 | idiomatic: stock.get('pear', 0) = 0
```

Trade-offs

	EAFP	LBYL
Races (files, shared state)	Safe	Check can go stale
Speed when failure is **rare**	Fast (`try` is almost free when nothing raises; 3.11+ "zero-cost")	Pays for the check every time
Speed when failure is **common**	Raising/catching is relatively expensive	Cheaper
Readability	Can hide which line failed if `try` is big	Explicit

When not to use it

Prefer LBYL when failure is common, when partial side effects can't be undone, or a dedicated API exists (`dict.get` , `getattr(obj, "x", default)` , `str.isdigit()`).

In the interview

> **30-second answer:** "LBYL checks before acting; EAFP attempts and catches. Python favours EAFP: it avoids race conditions like checking a file exists then opening it, and it's cheap when errors are rare. Use LBYL when failure is frequent or partial work is hard to undo."

TL;DR

Python prefers EAFP — race-free and fast when errors are rare; LBYL when failure is common.

9.6 `ExceptionGroup` and `except*` (Python 3.11+)

What it is

When **several** independent errors happen at once — concurrent tasks, validating many fields, many cleanup callbacks — an `ExceptionGroup` (PEP 654) wraps them in one raisable object. `except*` handles the matching *part*: each clause receives a sub-group; anything unhandled is re-raised in a new group.

How it works

```
def run_all():
    errors = []
    for task, fn in [("a", lambda: 1 / 0), ("b", lambda: int("z")),
                     ("c", lambda: 42), ("d", lambda: [][1])]:
        try:
            fn()
        except Exception as exc:
            exc.add_note(f"task {task}")
            errors.append(exc)
    if errors:
        raise ExceptionGroup("some tasks failed", errors)

try:
    run_all()
except* ZeroDivisionError as eg:
    print("  math errors: ", [(repr(e), e.__notes__) for e in eg.exceptions])
except* (ValueError, IndexError) as eg:
    print("  input errors:", [repr(e) for e in eg.exceptions])
```

```
== ExceptionGroup and except* (3.11+) ==
  math errors:  [("ZeroDivisionError('division by zero')", ['task a'])]
  input errors: ['ValueError("invalid literal for int() with base 10: \'z\'")', "IndexError('list index out of
range')"]
```

Unlike plain `except`, **more than one** `except*` clause can run. Rules: no mixing `except` and `except*` in one `try`; `eg` is always a group (`eg.exceptions`); `ExceptionGroup` holds `Exception` subclasses, `BaseExceptionGroup` anything.

Real-world examples

- `asyncio.TaskGroup` (3.11+; runs several asyncio tasks together) raises an `ExceptionGroup` when children fail.
- The Trio async library pioneered the model, motivating PEP 654.

When not to use it

Single errors — plain `except`.

In the interview

30-second answer: "An `ExceptionGroup` bundles several exceptions; `except*` lets each clause handle the matching subset, and unhandled ones are re-raised. It's used by `asyncio.TaskGroup` when multiple tasks fail."

TL;DR

Many errors at once → `ExceptionGroup`; split-handle with `except*`.

9.7 `assert`

What it is

`assert condition, "message"` raises `AssertionError("message")` if `condition` is false — documenting and checking **internal assumptions** (invariants: conditions that should always hold if your code is correct).

```
def mean(xs):
    assert len(xs) > 0, "mean() of empty list"
    return sum(xs) / len(xs)
```

```
mean([2, 4]) = 3.0
AssertionError: mean() of empty list (disabled entirely by python -O)
```

The crucial caveat

`-O` (optimise) sets `__debug__` to `False` and **removes all asserts**. Never use `assert` for input validation, security checks or anything production needs — raise a real exception (`ValueError`).

Common mistakes

- `assert (x > 0, "msg")` — a non-empty tuple is always truthy, never fails (CPython warns "assertion is always true").
- Validating user input or permissions with `assert`.

Real-world examples

`pytest` uses plain `assert`, rewriting it to show both sides on failure (`assert result == 5`).

In the interview

> **30-second answer:** "`assert` checks internal invariants and raises `AssertionError`; it's stripped under `python -O`, so use it for debugging and tests, never for validating input."

TL;DR

`assert` = developer sanity check and test tool; not validation.

9.8 Context managers and `with`

What it is

A **context manager** sets something up on entering a `with` block and **guarantees** teardown on leaving — normally, via `return`/`break`, or by exception. Typical: files, locks (one thread at a time), database transactions (changes that succeed or fail together), temp directories, timers.

```
with open("data.txt", encoding="utf-8") as f:   # file opened
    data = f.read()
# file closed here, even if read() raised
```

How it works — the protocol

The *protocol* (required special methods) for `with EXPR as NAME: BODY`:

1. Evaluate `EXPR` → manager `m`.
2. Call `m.__enter__()`; bind its return value to `NAME`.
3. Run `BODY`.
4. Call `m.__exit__(exc_type, exc, tb)` — `None, None, None` on success, else the exception details.
5. If `__exit__` returns **truthy**, the exception is **suppressed**; otherwise it propagates.

It's `try/finally` as a reusable object.

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        print("  __enter__: timer started")
        return self                     # bound to the name after 'as'

    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"  __exit__: exc_type={exc_type.__name__ if exc_type else None}")
        return False                    # let exceptions propagate
```

```
== class-based: normal exit ==
  __enter__: timer started
  __exit__: exc_type=None
  elapsed measured: True

== class-based: exception inside the block ==
  __enter__: timer started
  __exit__: exc_type=KeyError
  exception still reached caller: KeyError('boom')
```

`@contextlib.contextmanager` builds one from a generator (function pausing at `yield`): before `yield` = enter, yielded value = `as` target, after = exit. Wrap `yield` in `try` to react to errors:

```python
@contextlib.contextmanager
def transaction(log):
    log.append("BEGIN")
    try:
        yield log
    except Exception:
        log.append("ROLLBACK")
        raise
    else:
        log.append("COMMIT")
```

```
  log: ['BEGIN', 'UPDATE accounts ...', 'COMMIT', 'BEGIN', 'UPDATE bad ...', 'ROLLBACK']
```

`contextlib` helper	Does	Output
`suppress(FileNotFoundError)`	Ignore listed exceptions in the block	`missing file ignored`
`closing(obj)`	Call `obj.close()` on exit	
`ExitStack()`	Manage a *dynamic number* of managers; exits in **reverse** order (LIFO)	`open db, cache, queue` → `close queue, cache, db`
Parenthesised `with (A() as a, B() as b):`	Several managers in one statement (3.10+)	

Trade-offs

Class-based when the object has other methods (a connection); `@contextmanager` for one-off setup/teardown; `ExitStack` when the resource count is known only at runtime.

Common mistakes

- Files without `with` → leaked file handles (OS resources behind open files), and buffered writes may never reach disk.
- `@contextmanager` without `try/finally` around `yield` → no cleanup if the body raises.
- Accidentally returning `True` from `__exit__` → swallows every exception.

Real-world examples

- `with lock:` (`threading.Lock()`) acquires and always releases.
- `with conn:` on `sqlite3` commits on success, rolls back on exception — and does **not** close the connection.
- `torch.no_grad()`, `unittest.mock.patch(...)`, `decimal.localcontext()`, `tempfile.TemporaryDirectory()`.

Deeper treatment (async managers, reentrancy — re-entering an active manager) in Chapter 12.

In the interview

> **30-second answer:** "A context manager implements `__enter__` and `__exit__`; `with` calls enter, runs the block, and always calls exit with any exception info — exit returning true suppresses it. It's

> a reusable try/finally for resources like files and locks. `contextlib.contextmanager` builds one from a generator; `ExitStack` handles a dynamic set."

- *`__exit__` receives?* → `exc_type, exc_value, traceback`, all `None` on success.
- *Suppress an exception?* → Return `True` from `__exit__`, or `contextlib.suppress`.

TL;DR

`with` = guaranteed cleanup. `__enter__`/`__exit__`, or `@contextmanager` with `try` around `yield`; `suppress`, `closing`, `ExitStack`.

9.9 File I/O — modes, text vs binary, encodings

What it is

`open(path, mode="r", encoding=None, newline=None)` returns a **file object**. **Text mode** reads/writes `str`, converting via an **encoding** (characters ↔ bytes, e.g. UTF-8 — see §4.9); **binary mode** (`"b"`) reads/writes raw `bytes`.

Mode	Meaning	If file exists	If missing
`"r"` (default)	read	read from start	`FileNotFoundError`
`"w"`	write	**truncates** to empty	creates
`"a"`	append	writes go to the end	creates
`"x"`	exclusive create	`FileExistsError`	creates
`"+"` added (`"r+"`, `"w+"`)	read and write		
`"b"` added (`"rb"`, `"wb"`)	binary (bytes)		
`"t"` (default)	text (str)		

How it works

```
notes = base / "notes.txt"
with open(notes, "w", encoding="utf-8") as f:    # 'w' creates/truncates
    f.write("first line\n")
    f.writelines(["second line\n", "café ☕\n"])
with open(notes, "a", encoding="utf-8") as f:    # 'a' appends
    print("appended line", file=f)
with open(notes, encoding="utf-8") as f:         # default mode 'r'
    for i, line in enumerate(f, 1):              # lazy, line by line
        print(f"  {i}: {line.rstrip()}")
```

```
== writing and reading text ==
  1: first line
  2: second line
  3: café ☕
  4: appended line
  read() whole file -> 44 characters

== text vs bytes, encodings ==
  bytes on disk: 47 (é is 2 bytes, ☕ is 3 bytes in UTF-8)
  first 12 bytes: b'first line\ns'
  ascii decode -> UnicodeDecodeError ordinal not in range(128) at byte 26
  latin-1 (wrong codec) mangles: 'cafÃ© â\x98\x95'
  errors='replace': caf�� ���

== binary mode, seek/tell ==
  tell after seek(4): 4 | read(3): [4, 5, 6]

== 'x' mode refuses to overwrite ==
  FileExistsError: File exists
```

Method	Returns	Memory
`for line in f:`	one line at a time (with `\n`)	Constant — best for big files
`f.read()` / `f.read(n)`	whole file / up to `n` chars (bytes in binary)	Whole file
`f.readline()`	next line	One line
`f.readlines()`	list of all lines	Whole file
`f.write(s)`	characters written	—
`f.seek(pos)`, `f.tell()`	move / report position	—

Key facts:

1. **Always pass** `encoding="utf-8"` in text mode. Otherwise Python uses the locale encoding (OS language/region default; historically cp1252 on many Windows systems), so a file written on macOS may not read on Windows. PEP 686 plans UTF-8 as default in 3.15; until then be explicit (or run with `-X utf8`).
2. A wrong codec fails (`UnicodeDecodeError`) or silently produces **mojibake** (`cafÃ©`). `errors="replace"` / `"ignore"` trade errors for data loss — last resorts.
3. Text mode translates newlines: reading turns `\r\n` and `\r` into `\n` ("universal newlines"); writing turns `\n` into the platform separator. Binary never changes bytes.
4. Writes are **buffered**; data is safe after `close()` (or `flush()` + `os.fsync()` for durability — surviving a crash or power cut).
5. Images, zips, pickles, anything non-text → binary mode.
6. **File-like objects**: anything with `read()`/`write()` substitutes for a file — `io.StringIO("text")` (in-memory text) and `io.BytesIO(b"...")` (in-memory binary), handy in tests and for APIs expecting files (`csv.reader(io.StringIO(data))`, `pd.read_csv(io.StringIO(...))`).

Common mistakes

- `"w"` when you meant `"a"` — file wiped instantly.
- Omitting `encoding=`.
- `f.read()` on a multi-GB log — iterate lines.
- Paths by string `+` — use `pathlib`.

Real-world examples

Log processors stream line by line; `pandas.read_csv(..., encoding="utf-8")` exposes the same choice; most "garbled characters" bugs in data pipelines are encoding mismatches.

In the interview

> **30-second answer:** "`open` returns a file object; use it in a `with` block. Text mode works with `str` and needs an encoding — always pass `utf-8` explicitly; binary mode works with `bytes`. `w` truncates, `a` appends, `x` fails if the file exists. Iterate over the file for large inputs."

- *Why can 'é' be 2 bytes?* → UTF-8 is variable-width: 1 byte for ASCII, 2–4 for others.

Self-check

1. Mode for appending bytes? → `"ab"`.

TL;DR

`with open(p, mode, encoding="utf-8")`. Text = `str` + encoding, binary = `bytes`. `w` truncates, `a` appends, `x` safe-creates. Stream big files.

9.10 `pathlib`, `shutil`, `tempfile`

What it is

- **`pathlib.Path`** — paths as objects with methods: join with `/`, inspect parts, read/write/list without string juggling or `os.path`. Works on Windows and POSIX (the Unix standard Linux and macOS follow).
- **`shutil`** ("shell utilities") — high-level file and tree operations: copy, move, delete trees, archives.
- **`tempfile`** — safely creates uniquely named temp files and directories that clean up automatically.

How it works — `pathlib`

```
p = base / "reports" / "2026" / "q3.final.csv"
p.name, p.stem, p.suffix, p.suffixes
p.parent.mkdir(parents=True, exist_ok=True)
p.write_text("id,total\n1,9.5\n", encoding="utf-8")
(base / "reports").glob("*/*.csv")
base.rglob("*.md")
```

```
== pathlib ==
  name: q3.final.csv | stem: q3.final | suffix: .csv | suffixes: ['.final', '.csv']
  parent relative to base: reports/2026
  with_suffix('.json'): q3.final.json
  exists: True | is_file: True | size: 15 bytes
  glob('*/*.csv'):    ['q2.csv', 'q3.final.csv']
  rglob('*.md'):      ['readme.md']
  iterdir(reports):   ['2026', 'readme.md']
```

Task	pathlib	older os / os.path
Join	`base / "a" / "b.txt"`	`os.path.join(base, "a", "b.txt")`
Current dir / home	`Path.cwd()`, `Path.home()`	`os.getcwd()`, `os.path.expanduser("~")`
Absolute, symlinks (shortcut files) resolved	`p.resolve()`	`os.path.realpath(p)`
Parts	`p.name`, `.stem`, `.suffix`, `.parent`	`os.path.basename`, `splitext`, `dirname`
Exists / type	`p.exists()`, `.is_file()`, `.is_dir()`	`os.path.exists` …
Read / write whole file	`p.read_text()`, `p.write_bytes(b)`	`open(...)`
Make dirs	`p.mkdir(parents=True, exist_ok=True)`	`os.makedirs(p, exist_ok=True)`
Delete file / empty dir	`p.unlink(missing_ok=True)`, `p.rmdir()`	`os.remove`, `os.rmdir`
Rename / replace	`p.rename(t)`, `p.replace(t)`	`os.rename`, `os.replace`
List / search (glob = wildcard name match like `*`)	`p.iterdir()`, `p.glob("*.py")`, `p.rglob("*.py")`, `p.walk()` (3.12+)	`os.listdir`, `glob.glob`, `os.walk`

`open()` and nearly every library accept `Path` objects. `Path("x.tar.gz").suffix` is `".gz"` (`.suffixes` → `['.tar', '.gz']`).

How it works — `shutil` and `tempfile`

```
== shutil: copy, move, archive, delete trees ==
  copytree -> ['2026', '2026/q2.csv', '2026/q3.final.csv', 'readme.md']
  moved readme -> True
  archive: reports_bak.zip True
  rmtree backup -> exists? False
  disk_usage works: True

== tempfile ==
  temp file .txt contains: scratch
  deleted after close: True

== atomic replace pattern ==
  settings now: {"v": 2}

temp dir removed: True
```

shutil function	Does
`copy(src, dst)` / `copy2`	Copy a file (+ permission bits / + timestamps and other metadata)
`copytree(src, dst)`	Copy a whole folder tree
`move(src, dst)`	Move/rename, across filesystems (disks/partitions) if needed
`rmtree(path)`	Delete a folder **and everything in it** — permanent, no recycle bin
`make_archive(base, "zip", root_dir)` / `unpack_archive`	Create/extract zip, tar, gztar…
`which("git")`, `disk_usage(path)`	Find an executable on `PATH`; total/used/free bytes

`tempfile` API	Gives
`TemporaryDirectory()`	Folder deleted (with contents) when the `with` block ends
`NamedTemporaryFile()`	File with a real name, deleted on close by default (3.12 adds `delete_on_close=False` for reopening by name, needed on Windows)
`TemporaryFile()`	Anonymous file — may have no visible name
`mkstemp()` / `mkdtemp()`	Low-level: create and return path; **you** delete it

Atomic replace pattern: write to a temp file *in the same directory*, then `os.replace(tmp, target)`. The rename is atomic (all at once or not at all) on one filesystem, so readers see old or new — never half-written. Common for config/state files.

Trade-offs

`pathlib` is the modern default; `os`/`os.path` remain for low-level calls (`os.replace`, `os.fsync`, file descriptors — the OS's integer handles for open files) and older code. `shutil` isn't transactional (all-or-nothing) — an interrupted `copytree` leaves a partial copy.

Common mistakes

- Hard-coded `"/tmp/myfile"` — predictable names are a security risk and unportable; use `tempfile`.
- `shutil.rmtree` on unchecked user-supplied paths.
- String concatenation or backslashes for paths.
- `Path("a").glob("**/*.py")` ≡ `rglob("*.py")`; `rglob` is shorter.

Real-world examples

pytest's `tmp_path` fixture gives each test a fresh `Path` temp dir; build tools use `copytree`/`make_archive`; `shutil.which` checks that `git` or `ffmpeg` is installed.

In the interview

> **30-second answer:** "`pathlib.Path` is the object-oriented way to build and inspect paths and to read, write, list and glob; `shutil` does high-level file and tree operations like copy, move, rmtree and archives; `tempfile` creates unique temp files and directories that clean themselves up. For safe file updates, write a temp file in the same folder and `os.replace` it."

TL;DR

`Path` for paths, `shutil` for copying/moving/deleting trees, `tempfile` for throwaway files; atomic writes via temp + `os.replace`.

9.11 `csv`, `json`, `pickle`

What it is

Format	Module	Looks like	Holds	Readable by other languages	Safe to load untrusted data
CSV — comma-separated values	`csv`	Text table	Rows of **strings**	Yes (Excel, databases)	Yes
JSON — JavaScript Object Notation	`json`	Text	dict, list, str, int, float, bool, None	Yes (the web's lingua franca)	Yes
pickle	`pickle`	Binary	Almost any Python object	**No** — Python only	**NO** — can run arbitrary (attacker-chosen) code

CSV

Use `csv`, not `line.split(",")` — real CSV has quoted fields containing commas, quotes, even newlines.

```
with open(path, "w", newline="", encoding="utf-8") as f:   # newline="" per csv docs
    w = csv.DictWriter(f, fieldnames=["id", "customer", "total", "placed"])
    w.writeheader()
    for o in orders:
        w.writerow(asdict(o))
with open(path, newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
```

```
raw file:
  id,customer,total,placed
  1,"Ana, Inc.",19.5,2026-09-01
  2,"Bob ""B"" Ltd",5.25,2026-09-02
first row: {'id': '1', 'customer': 'Ana, Inc.', 'total': '19.5', 'placed': '2026-09-01'}
total is a str -> convert yourself: 24.75
naive split(',') breaks on quoted commas: ['1', '"Ana', ' Inc."', '19.5', '2026-09-01']
tab-separated: [['a', 'b'], ['1', '2']]
```

- The writer quoted `"Ana, Inc."` and doubled inner quotes automatically.
- **Everything reads back as `str`** — convert yourself.
- Open with **`newline=""`** (required by the docs) so `csv` handles line endings — else blank rows on Windows or broken quoted newlines.
- `reader` / `writer` use lists; `DictReader` / `DictWriter` use dicts keyed by the header; `delimiter="\t"` for TSV; `csv.Sniffer` guesses the dialect (delimiter, quoting rules).
- For analysis-sized data, `pandas.read_csv` is usually better.

JSON

Python → JSON	JSON → Python
`dict` → object	object → `dict`
`list`, `tuple` → array	array → `list`
`str` → string	string → `str`
`int`, `float` → number	number → `int` or `float`
`True` / `False` → `true` / `false`	→ `True` / `False`
`None` → `null`	→ `None`

`dumps` / `loads` use strings; `dump` / `load` files.

```
text = json.dumps(payload, default=to_json_default, indent=2, ensure_ascii=False)
back = json.loads(text)
```

```
first lines of dumps():
  {
    "orders": [
      {
        "id": 1,
        "customer": "Ana, Inc.",
        "total": 19.5,
        "placed": "2026-09-01"
      },
loads() types: paid -> bool | discount -> None | placed -> str
load() from file, customers: ['Ana, Inc.', 'Bob "B" Ltd']
tuple becomes list: {'pt': [1, 2]}
int keys become str: {'1': 'a'}
without default=: Object of type date is not JSON serializable
JSONDecodeError: Expecting property name enclosed in double quotes at pos 1
```

- Unknown types (`date`, `Decimal`, `set`, your classes) need `default=` (e.g. returning `isoformat()`), a custom `JSONEncoder`, or pre-conversion (`dataclasses.asdict`); they return as plain strings/lists — the round trip is **lossy**.
- Tuples → lists; non-string keys → strings.
- JSON needs double quotes; `'single'` → `json.JSONDecodeError` (a `ValueError`).
- `indent=2` pretty-prints; `ensure_ascii=False` keeps "é" instead of the escape `\u00e9`; `sort_keys=True` stabilises output; `python -m json.tool file.json` pretty-prints from the shell.
- `json.dumps(float("nan"))` writes `NaN`, **invalid** for strict parsers; `allow_nan=False` raises instead.

pickle

`pickle` serialises almost any Python object graph (an object plus everything it references — custom classes, dates, shared references) to bytes and back.

```
blob = pickle.dumps(orders)
restored = pickle.loads(blob)
with open(base / "orders.pkl", "wb") as f:                # binary mode!
    pickle.dump(orders, f)
```

```
round trip equal: True | type: Order | placed: 2026-09-01
load() from file: 2 orders
```

Security caveat — never unpickle untrusted data. An object can specify via `__reduce__` a function to *call* on load. The demo uses a harmless `print`; an attacker would use `os.system` (runs any shell command):

```
class Sneaky:
    def __reduce__(self):
        return (print, ("  !!! code ran during pickle.loads() !!!",))

pickle.loads(pickle.dumps(Sneaky()))
```

```
loading a crafted pickle...
!!! code ran during pickle.loads() !!!
loads() returned: None
```

Code ran merely by *loading*; the official docs warn prominently. Other limits: Python-only; tied to class definitions (renaming a class breaks old pickles); lambdas, open files, sockets and locks can't be pickled. Current protocol: 5 (`pickle.HIGHEST_PROTOCOL`).

Trade-offs / when to use which

- **Config, APIs, data exchange** → JSON (TOML for human-edited config, read with `tomllib`).

- **Tables for spreadsheets/other tools** → CSV (or Parquet — compressed column-oriented files — via pandas for big data).
- **Short-lived caching of your own Python objects** (pipeline steps, `multiprocessing` internals) → pickle.
- **Never** pickle across a trust boundary (data from outside your control: uploads, network, downloads).

Common mistakes

- Pickle files in text mode → `TypeError`; use `"wb"`/`"rb"`.
- Forgetting `newline=""` with `csv`.
- Expecting `datetime`s back from `json.loads`.
- Loading downloaded model files with plain `pickle.load`.

Real-world examples

- **PyTorch** checkpoints (`torch.save`) use pickle; since PyTorch 2.6 `torch.load` defaults to `weights_only=True` to block code execution, and Hugging Face created **safetensors** as a pickle-free format for model weights.
- scikit-learn documents pickle/joblib persistence with the same warning and points to the safer `skops` format.
- REST APIs (web APIs built from plain HTTP requests to URLs; GitHub, Stripe) speak JSON; spreadsheet exports are CSV.

In the interview

> **30-second answer:** "CSV is text rows of strings — use the `csv` module with `newline=''` because of quoting. JSON maps to dict/list/str/number/bool/None, is language-neutral and safe; other types need a `default=` hook and come back as plain values. Pickle stores arbitrary Python objects in binary but is Python-only and unsafe: loading can execute code via `__reduce__`, so never unpickle untrusted data."

- *Why is `pickle.loads` dangerous?* → A pickle can make the loader call arbitrary callables.
- *JSON-encode a `datetime`?* → `default=` returning `isoformat()`, or a custom `JSONEncoder`.
- *`json.dump` vs `dumps`?* → File vs string.

TL;DR

CSV = strings in rows (`csv` module, `newline=""`). JSON = safe, portable, limited types. Pickle = any Python object, binary, **never from untrusted sources**.

Chapter TL;DR

- Exceptions are objects that unwind the stack until an `except` matches by class; `Exception` is the base for application errors, `BaseException` also covers exit and Ctrl-C.
- `try` small, `except` specific (first match wins), `else` on success, `finally` always; never a bare `except:`; the `as e` name is deleted after the handler.
- `raise` built-ins for contract errors, a custom hierarchy for domain errors, bare `raise` to re-throw, `raise X from e` to translate while keeping the cause, `add_note` for context.
- Python prefers **EAFP** (try it) over LBYL (check first) — race-free and fast when failures are rare.
- `ExceptionGroup` + `except*` handle several simultaneous errors (e.g. `asyncio.TaskGroup`).
- `assert` checks invariants and vanishes under `-O` — never for validation.
- `with` + context managers guarantee cleanup (`__enter__`/`__exit__`, `@contextmanager`, `ExitStack`).
- `open(path, mode, encoding="utf-8")` inside `with`; text vs binary; `w` truncates, `a` appends, `x` refuses to overwrite; iterate big files.
- `pathlib` for paths, `shutil` for trees and archives, `tempfile` for throwaway files, temp + `os.replace` for atomic writes.

- CSV (strings, `newline=""`), JSON (safe, portable, limited types), pickle (any object, Python-only, **unsafe on untrusted data**).

Chapter 10 — Object-Oriented Programming in Python

What you will learn. Classes and instances, `__init__` and `self`, class vs instance attributes, the three kinds of method, properties. The *data model* — "dunder" methods (`__repr__`, `__eq__`, `__hash__`, `__lt__`, `__len__`, `__iter__`, `__getitem__`, `__call__`, `__enter__` …) that make your classes work with `len()`, `for`, `==`, `+`, `with`, `sorted()`. Reuse: inheritance (a class reusing and extending another) and `super()`, the method resolution order (MRO), multiple inheritance, mixins, abstract base classes, duck typing and `Protocol`, composition vs inheritance. The modern toolbox — `dataclasses`, `enum`, `__slots__` — encapsulation (keeping internals behind methods) conventions, and Pythonic design patterns (named, reusable solutions to common design problems).

Prerequisites. Functions, basic types, exceptions. SOLID (five classic OO design principles) and the Gang-of-Four patterns (23 patterns from the 1994 book *Design Patterns*) are covered language-neutrally in the LLD Handbook (LLD = low-level design); this chapter is about *how Python does OOP*.

Code. Scripts are in `chapters/10-oop-in-python/code/` and run on Python 3.12+ with no dependencies:

```
python chapters/10-oop-in-python/code/data_model.py
```



10.1 Classes and instances, `__init__` and `self`

What it is

A **class** is a blueprint bundling data (**attributes**) and behaviour (**methods**). An **instance** (*object*) is one concrete thing built from it — cookie cutter vs cookies. In Python *everything* is an object — ints, functions, modules, and classes themselves (a class is an instance of `type`).

Why it matters

State and the code that changes it live together, so invariants (rules that must always hold: "balance never negative") are enforced in one place.

How it works

1. `class BankAccount: ...` runs the body once and creates a class object.
2. Calling it, `BankAccount("Asha", 100)`: `__new__` allocates an empty instance, then `__init__` initialises it. You rarely write `__new__` (immutable types, instance caching, metaclass tricks — a *metaclass* is the class of a class).
3. `__init__(self, ...)` receives the new instance first; `self` is a convention, not a keyword.
4. `self.owner = owner` stores an **instance attribute** in the instance's namespace dict, `obj.__dict__`.
5. `__init__` returns `None`; it *initialises*, not *constructs*.

Class attributes vs instance attributes

A **class attribute** is defined in the class body and shared; an **instance attribute** is set on `self`, one per object.

Attribute lookup for `obj.name` (simplified): `obj.__dict__`, then the class, then parents along the MRO (§10.5). Assigning `obj.name = ...` always writes to the *instance*, *shadowing* (hiding) any class attribute.

The three kinds of method

Kind	Decorator	First parameter	Typical use
Instance method	none	`self` (the instance)	Read/modify one object's state
Class method	`@classmethod`	`cls` (the class)	Alternative constructors (`from_csv`, `dict.fromkeys`, `datetime.fromtimestamp`); subclass-friendly because `cls` is the subclass
Static method	`@staticmethod`	none	Helper belonging to the class, needing neither instance nor class

`a.deposit` is a **bound method** — the function with `self` pre-filled — so `a.deposit(5)` ≡ `BankAccount.deposit(a, 5)`. (Mechanism: functions are *descriptors*, objects customising attribute lookup — Chapter 12.)

Properties

A **property** makes a method look like an attribute. Start with a plain public attribute; when you later need validation or computation, turn it into a property and **no caller changes** — so Python needs no Java-style getters/setters (`get_x()` / `set_x()`).

```
class BankAccount:
    bank_name = "PyBank"          # class attribute: shared by every instance
    _next_id = 1                  # class attribute used as a counter

    def __init__(self, owner: str, balance: float = 0.0) -> None:
        self.owner = owner        # instance attributes: one copy per object
        self._balance = balance   # leading underscore = "internal, please don't touch"
        self.id = BankAccount._next_id
        BankAccount._next_id += 1

    def deposit(self, amount: float) -> None:          # instance method
        if amount <= 0:
            raise ValueError("amount must be positive")
        self._balance += amount

    @property
    def balance(self) -> float:                        # read like an attribute
        return self._balance

    @balance.setter
    def balance(self, value: float) -> None:           # validated assignment
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value

    @classmethod
    def from_csv(cls, line: str) -> "BankAccount":     # alternative constructor
        owner, balance = line.split(",")
        return cls(owner, float(balance))

    @staticmethod
    def is_valid_amount(amount: float) -> bool:        # plain helper
        return amount > 0


class Buggy:
    items = []                    # MISTAKE: one list shared by all instances
    def add(self, x):
        self.items.append(x)
```

The demo builds two accounts (one via the class method), shadows `bank_name` on `a`, has the setter reject a negative value, proves `a.deposit` is bound, and shows the shared-mutable-class-attribute bug (`self.items.append` mutates the one class list — no assignment, so no shadowing).

Output (`classes_basics.py`):

```
a: Asha 125 id 1
b: Ben 50.0 id 2
static check: False
a.bank_name: PyBank
after assignment -> a: Shadow | b: PyBank | class: PyBank
a.__dict__: {'owner': 'Asha', '_balance': 125, 'id': 1, 'bank_name': 'Shadow'}
setter refused: balance cannot be negative
a.deposit is bound: True
after unbound call: 130
Buggy y.items: [1]
Fixed q.items: []
```

Trade-offs

Choice	Good for	Watch out
Plain public attribute	Simple data, zero overhead	Can't validate — but can become a property later without breaking callers
Property	Validation, computed values, read-only views	Hides work behind attribute syntax; keep it cheap
Class attribute	Constants, defaults, shared counters	Mutable class attributes are shared — a classic bug
`@classmethod` vs `@staticmethod`	`cls` makes alternative constructors subclass-friendly	A static method naming the class directly breaks for subclasses

When not to use a class

One function, no state → a function. Data, no behaviour → `dataclass`, `NamedTuple` or `dict`. A class with only `__init__` and one method is often a function in disguise (Jack Diederich, "Stop Writing Classes", PyCon).

Common mistakes

- Missing `self` → `TypeError: takes 0 positional arguments but 1 was given`.
- Mutable class attributes (`items = []`) — initialise in `__init__`.
- Mutable default arguments (`def __init__(self, items=[])`) — use `None` (§7.2).
- `self._next_id += 1` creates an instance attribute; update the class: `type(self)._next_id += 1`.
- Property and attribute with the same name (`self.balance = x` inside the `balance` setter) → infinite recursion; store in `_balance`.

Real-world examples

- Class-method constructors: `datetime.datetime.fromtimestamp`, `dict.fromkeys`, `int.from_bytes`.
- Properties: `pathlib.Path.name`, `.suffix`, `.parent`.
- Django model fields are class attributes; instances hold their own values.

In the interview

30-second answer: "A class is a blueprint; instances are built by `__new__` then initialised by `__init__`, which gets the instance as `self`. Class attributes live on the class and are shared; instance attributes live in each object's `__dict__`, and lookup checks the instance first, then the class and its parents. Instance methods take `self`, class methods take `cls` and are used for alternative constructors, static methods take neither. Properties let me add validation later without changing the public API."

- *Why is `self` explicit?* — "Explicit is better than implicit": a method is a function on the class; `obj.m()` passes `obj` first. It also makes instance vs local variables obvious.

- *Factory (object-creating function) as class or static method?* — Class method, so `SubClass.from_csv(...)` returns a `SubClass`.
- `__new__` *for?* — Controlling creation: subclassing immutables (`int`, `str`, `tuple`), caching instances, metaclasses.

TL;DR

Classes bundle state and behaviour; `__init__` initialises `self`. Instance attributes per object, class attributes shared (beware mutable ones). Instance/class/static methods take `self`/`cls`/nothing. Public attributes first; properties when needed.

10.2 The data model: dunder methods

What it is

The **Python data model** is the set of special methods — **double underscore** names like `__len__` — that the interpreter calls behind syntax and built-ins: `len(x)` → `x.__len__()`, `a + b` → `a.__add__(b)`, `for` → `__iter__`, `with` → `__enter__`/`__exit__`. *Fluent Python* calls it Python's "metaobject protocol" (the rules by which objects plug into the language).

Why it matters

Your classes get the familiar interface — `len(playlist)`, `playlist[0]`, `sorted(vectors)`, `v1 + v2`, `print(v)` — and plug into the standard library (`sorted`, `max`, `set`, `dict`, `random.choice`, `in`) for free.

How it works — the most important dunders

You write	Python calls	Notes
`repr(x)`, REPL echo, inside containers	`__repr__`	Unambiguous, for developers; ideally like the constructor call
`str(x)`, `print(x)`, `f"{x}"`	`__str__`	Readable, for users; **falls back to** `__repr__`
`x == y`, `x != y`	`__eq__`, `__ne__`	Default is identity; `__ne__` inverts `__eq__` automatically
`hash(x)`, dict key, set member	`__hash__`	Must agree with `__eq__`
`x < y` etc.	`__lt__`, `__le__`, `__gt__`, `__ge__`	`@functools.total_ordering` derives the rest from `__eq__` + one of them
`len(x)`	`__len__`	Must return an int ≥ 0
`bool(x)`, `if x:`	`__bool__`	Falls back to `__len__() != 0`, else `True`
`for i in x`, `iter(x)`	`__iter__`	Falls back to `__getitem__(0), (1), …` until `IndexError`
`x[k]`, `x[a:b]`	`__getitem__`	Receives an int, key, or `slice` object
`x[k] = v`, `del x[k]`	`__setitem__`, `__delitem__`	
`v in x`	`__contains__`	Falls back to iterating
`x(args)`	`__call__`	Makes instances callable
`with x as y:`	`__enter__`, `__exit__`	Chapter 12
`a + b`, `a * b`, `abs(a)`, `-a`	`__add__`, `__mul__`, `__abs__`, `__neg__`	Operator overloading (§10.3)

`__eq__` and `__hash__` — the contract

A **hash** is the integer dicts and sets use to pick a slot (§6.2). Rule: **if** `a == b` **then** `hash(a) == hash(b)` (collisions — different values sharing a hash — are allowed).

1. By default objects compare and hash by identity — consistent.
2. Defining `__eq__` without `__hash__` makes Python set `__hash__ = None` — instances become **unhashable**, protecting the rule.
3. To restore hashing, hash the *same* fields `__eq__` uses: `hash((self.x, self.y))`.
4. Hash only **immutable** state — a key mutated inside a dict sits in the wrong slot and is "lost". `frozen=True` dataclasses (§10.10) are the easy route.

`NotImplemented`

When a comparison or arithmetic method can't handle the other operand, **return** (don't raise) the singleton `NotImplemented`. Python then tries the reflected method on the other operand (`b.__radd__(a)`, or `b.__eq__(a)`) and raises `TypeError` only if both give up.

```python
@total_ordering
class Vector:
    __match_args__ = ("x", "y")

    def __init__(self, x: float, y: float) -> None:
        self.x, self.y = x, y

    def __repr__(self) -> str:
        return f"Vector({self.x!r}, {self.y!r})"

    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x, self.y) == (other.x, other.y)

    def __hash__(self) -> int:
        return hash((self.x, self.y))

    def __lt__(self, other: "Vector") -> bool:     # total_ordering adds <=, >, >=
        if not isinstance(other, Vector):
            return NotImplemented
        return abs(self) < abs(other)

    def __add__(self, other): ...                  # see §10.3
    def __abs__(self) -> float:
        return math.hypot(self.x, self.y)
    def __bool__(self) -> bool:
        return bool(self.x or self.y)
    def __iter__(self):                            # enables x, y = v
        yield self.x
        yield self.y


class Playlist:
    def __init__(self, *songs: str) -> None:
        self._songs = list(songs)
    def __len__(self) -> int:
        return len(self._songs)
    def __getitem__(self, index):                  # ints AND slices
        result = self._songs[index]
        return Playlist(*result) if isinstance(index, slice) else result
    def __contains__(self, song: str) -> bool:
        return song in self._songs


class Multiplier:
    def __init__(self, factor: int) -> None:
        self.factor, self.calls = factor, 0
    def __call__(self, x: int) -> int:             # instance used like a function
        self.calls += 1
        return x * self.factor


class Timer:
    def __enter__(self):
        print("  enter: start timer")
        return self                                 # bound to the `as` name
    def __exit__(self, exc_type, exc, tb) -> bool:
        print(f"  exit: stop timer (exception={exc_type.__name__ if exc_type else None})")
        return False                                # don't swallow exceptions
```

Output (`data_model.py`):

```
repr: Vector(3, 4) | str: (3, 4) | f-string: (3, 4)
v + w = (4, 5) | v * 2 = (6, 8) | 2 * v = (6, 8)
abs(v) = 5.0 | bool(Vector(0,0)) = False
v == Vector(3,4): True | v is Vector(3,4): False
w < v: True | v >= w: True (>= from total_ordering)
set dedups equal vectors: {Vector(1, 2)}
sorted: [Vector(1, 1), Vector(0, 2), Vector(3, 4)]
unpacked: 3 4
pattern matched, y = 4
TypeError: unsupported operand type(s) for +: 'Vector' and 'int'
len: 4 | p[0]: intro | p[-1]: outro | p[1:3]: Playlist('verse', 'chorus')
'chorus' in p: True | iterate via __getitem__: ['INTRO', 'VERSE', 'CHORUS', 'OUTRO']
callable: 15 21 calls = 2 | callable(): True
  enter: start timer
  inside block, t is a Timer: True
  exit: stop timer (exception=None)
  enter: start timer
  exit: stop timer (exception=KeyError)
  KeyError still propagated
__hash__ of class defining only __eq__: None
```

`Playlist` has no `__iter__`, yet `for` works via the legacy `__getitem__` fallback; negative indexes and slices work by delegating to the list. `__match_args__` lets `match v: case Vector(3, y)` destructure positionally. The last line proves the `__eq__`-without-`__hash__` rule. `print([v])` uses `__repr__` (containers show item reprs).

This teaching `Vector` is mutable *and* hashable — fine only if never mutated while used as a key; in production make it immutable (frozen dataclass or read-only properties).

Trade-offs

Approach	Pros	Cons
Dunders by hand	Full control, any semantics	Boilerplate; easy to make `__eq__` / `__hash__` inconsistent
`@dataclass`	Generates `__init__`, `__repr__`, `__eq__` (optionally ordering/hash)	Field-by-field semantics only
Inherit `collections.abc.Sequence` / `Mapping`	Implement 2–3 abstract methods, get `index`, `count`, `__contains__`, `__iter__`, `__reversed__` as mixin methods	Ties you to that ABC
Subclass `list` / `dict` directly	Quick	C methods often bypass your overrides (`dict.update` ignores your `__setitem__`); prefer `collections.UserDict` / `UserList`

When not to use it

No surprising operator meanings (`+` that deletes a file). No `__call__` for cleverness — only when the object genuinely *is* function-like (a model, strategy, validator).

Common mistakes

- Calling `x.__len__()` instead of `len(x)` (the built-in is faster for built-ins and adds checks).
- `raise NotImplementedError` in `__eq__` / `__add__` instead of `return NotImplemented`.
- Surprise at unhashability after defining `__eq__`.
- Useless `__repr__` like `<object>` — include key fields.
- Non-int/non-bool from `__len__` / `__bool__` → `TypeError`.

Real-world examples

- `pathlib.Path` overloads `/` (`__truediv__`): `Path("a") / "b"`.

- NumPy/pandas overload arithmetic, comparison and `__getitem__` (boolean masks — True/False arrays selecting elements — and slicing).
- `requests.Session`, files, `threading.Lock`, `sqlite3.Connection` implement `__enter__/__exit__`.
- PyTorch `nn.Module.__call__` runs hooks (user-registered callbacks) then `forward()` — hence `model(x)`.
- SQLAlchemy (maps classes to database tables) overloads `==`, `<` on columns to build SQL (`User.age > 18`).

In the interview

> **30-second answer:** "The data model is Python's protocol of special methods: built-ins and syntax call dunders — `len` calls `__len__`, `+` calls `__add__`, `for` calls `__iter__`. Implementing them makes custom classes behave like built-ins and work with the standard library. Key rules: `__repr__` for developers, `__str__` falls back to it; `__eq__` and `__hash__` must agree and defining `__eq__` alone makes the class unhashable; return `NotImplemented` for unsupported operands."

- *Why equal objects need equal hashes?* — The hash picks a bucket (hash-table slot), then `==` confirms; different hashes → different buckets → duplicates.
- *Minimum to be iterable?* — `__iter__` returning an iterator, or `__getitem__` for 0, 1, 2… raising `IndexError` at the end.
- *`__str__` vs `__repr__`?* — `str` falls back to `repr`, not vice versa — write `__repr__` first.

TL;DR

Dunders are hooks for syntax and built-ins. Always `__repr__`. Keep `__eq__`/`__hash__` consistent on immutable fields. Return `NotImplemented`. `__len__`/`__getitem__`/`__iter__`/`__contains__` for containers, `__call__` for callables, `__enter__/__exit__` for `with`.

10.3 Operator overloading

What it is

Operator overloading gives `+`, `-`, `*`, `@`, `==`, `<`, `[]` etc. meaning for your types via the matching dunder.

How it works

For `a + b`:

1. `a.__add__(b)`; a non-`NotImplemented` result wins.
2. Else the **reflected** `b.__radd__(a)`. (If `type(b)` is a *subclass* of `type(a)` that overrides `__radd__`, it's tried first.)
3. Both `NotImplemented` → `TypeError: unsupported operand type(s)`.

For `a += b`, the **in-place** `a.__iadd__(b)` is tried first (may mutate and `return self`); if missing, `a = a + b`. Hence `list += ...` mutates while `tuple += ...` builds a new tuple, and `lst += (1,)` works (`__iadd__` accepts any iterable, like `extend`) while `lst = lst + (1,)` fails (`__add__` wants a list).

Operator	Method	Reflected	In-place
`+`	`__add__`	`__radd__`	`__iadd__`
`-`	`__sub__`	`__rsub__`	`__isub__`
`*`	`__mul__`	`__rmul__`	`__imul__`
`@` (matrix mult)	`__matmul__`	`__rmatmul__`	`__imatmul__`
`/`, `//`, `%`, `**`	`__truediv__`, `__floordiv__`, `__mod__`, `__pow__`	`__r…__`	`__i…__`
`&`, `|`, `^`	`__and__`, `__or__`, `__xor__`	`__r…__`	`__i…__`
unary `-`, `+`, `~`, `abs()`	`__neg__`, `__pos__`, `__invert__`, `__abs__`	—	—

Comparisons have no `__r…__` versions; they reflect each other: if `a < b` is `NotImplemented`, Python tries `b > a`.

```
    def __add__(self, other: "Vector") -> "Vector":
        if not isinstance(other, Vector):
            return NotImplemented
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, k: float) -> "Vector":      # v * 3
        if not isinstance(k, (int, float)):
            return NotImplemented
        return Vector(self.x * k, self.y * k)

    __rmul__ = __mul__                            # 3 * v  (int.__mul__ returns NotImplemented)
```

`2 * v` works only via `__rmul__`: `int.__mul__(2, v)` returns `NotImplemented`, so Python calls `v.__rmul__(2)`.

Common mistakes

- No `__rmul__` → `v * 2` works, `2 * v` fails.
- `__iadd__` forgetting `return self` → the variable becomes `None`.
- Mutating operands inside `__add__` — return a *new* object.

Real-world examples

`Decimal`, `Fraction`, `datetime - timedelta`, `set | set`, `dict | dict` (3.9+), NumPy `@` (PEP 465), `pathlib` `/`.

In the interview

> **30-second answer:** "Each operator maps to a dunder. For `a + b` Python calls `a.__add__(b)`; if that returns `NotImplemented` it tries `b.__radd__(a)`, then raises `TypeError`. `+=` tries `__iadd__` for in-place update and falls back to `+`. Return `NotImplemented` for unsupported types so the other operand gets a chance."

TL;DR

Operators are dunders. Return `NotImplemented` for unknown types; `__r…__` for "other type on the left", `__i…__` for in-place updates.

10.4 Inheritance and `super()`

What it is

Inheritance lets a **subclass** (child) reuse and specialise a **base class** (parent): `class Manager(Employee)` gets all parent attributes and methods and can **override** them. It models **is-a**.

How it works

1. Lookup on a `Manager` searches `Manager`, `Employee`, then `object` (every class's ultimate base).
2. To extend a parent method, call **`super()`** — a proxy forwarding lookups to the *next* class in the MRO.
3. A child `__init__` does **not** auto-call the parent's; call `super().__init__(...)`.
4. `isinstance(obj, Cls)` checks the class and ancestors; `issubclass(A, B)` checks classes. Prefer `isinstance` over `type(x) == Cls`, which ignores subclasses.

```
class Employee:
    def __init__(self, name: str, salary: int) -> None:
        self.name, self.salary = name, salary
    def pay(self) -> int:
        return self.salary
    def __repr__(self) -> str:
        return f"{type(self).__name__}({self.name!r})"   # type(self) -> works for subclasses


class Manager(Employee):
    def __init__(self, name: str, salary: int, bonus: int) -> None:
        super().__init__(name, salary)      # reuse the parent's initialiser
        self.bonus = bonus
    def pay(self) -> int:                    # override, then extend
        return super().pay() + self.bonus
```

Output: `Manager('Mia') pays 120 | isinstance Employee: True | issubclass: True`

Common mistakes

- Forgetting `super().__init__()` → missing parent attributes → `AttributeError` later.
- Hard-coding `Employee.__init__(self, ...)` — breaks cooperative multiple inheritance (§10.5).
- Deep hierarchies (5+ levels) — prefer composition (§10.8).
- Overriding with an incompatible signature — violates the Liskov Substitution Principle (a subclass must work wherever its parent does; LLD Handbook ch. 4).

TL;DR

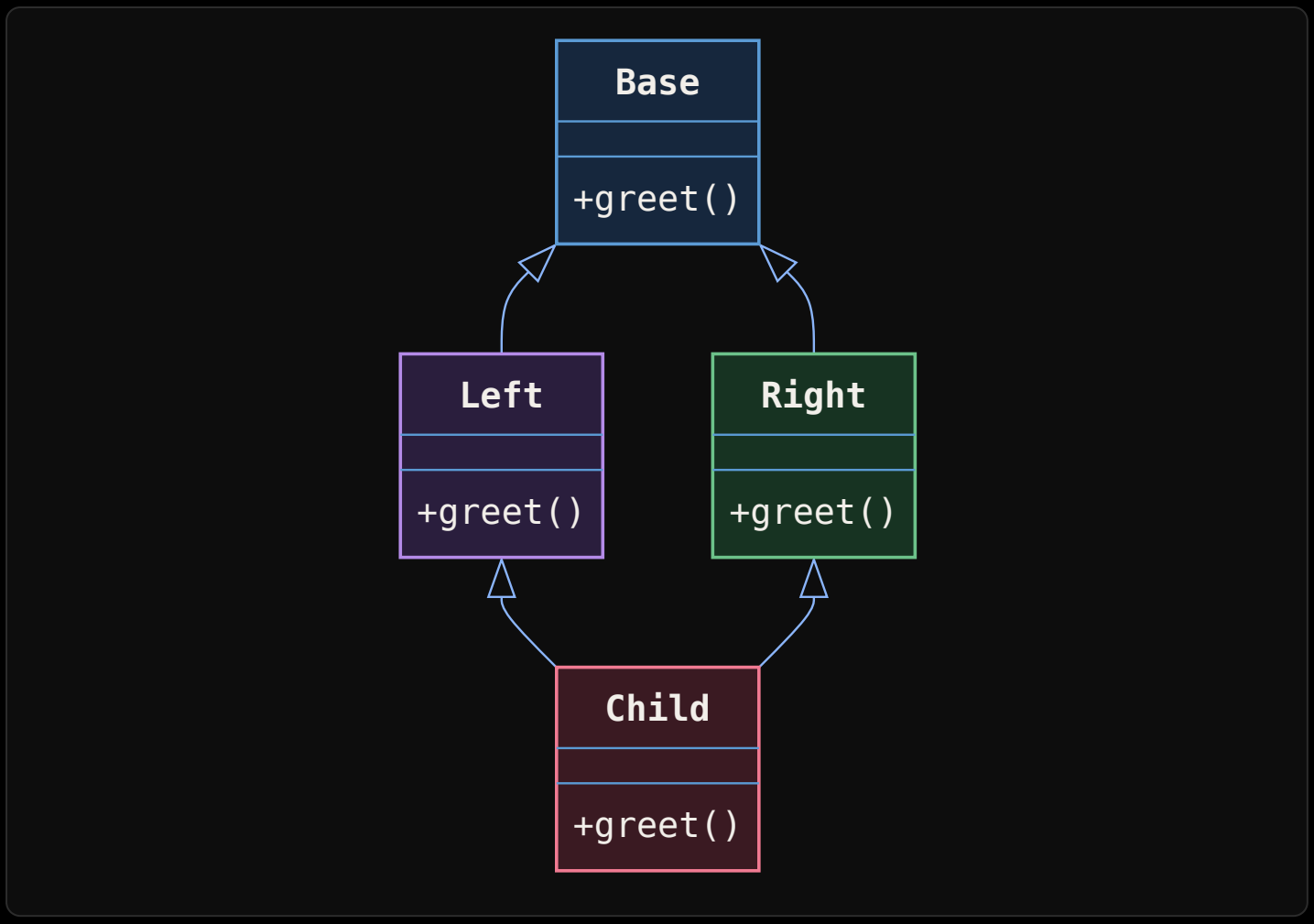
Subclass for true is-a; override to specialise; `super()` to extend; always call `super().__init__`.

10.5 Multiple inheritance and the MRO (C3 linearization)

What it is

`class Child(Left, Right)` has several bases. The **method resolution order (MRO)** is the single ordered list of classes searched for attributes (`Child.__mro__` / `Child.mro()`), computed by **C3 linearization** (since Python 2.3).

The problem it solves — the diamond



Which `greet` runs, and does `Base.greet` run once or twice? C3 gives one consistent answer with `Base` exactly once.

How it works — C3 in plain words

MRO(class) = the class + a **merge** of its parents' MROs and the parent list, obeying:

1. **Children before parents.**
2. **Local order kept** — bases in written order (`Left` before `Right`).
3. **Monotonic** — each parent's MRO order is preserved.

Worked example, `Child(Left, Right)`: lists [Left, Base, object], [Right, Base, object], [Left, Right]. Merge = repeatedly take the first head (first element) not in any other list's *tail* (everything after the head):

- `Left` — in no tail → take.
- `Base` — in the tail of [Right, Base, object] → skip; `Right` → take.
- `Base` now free → take; then `object`.
- Result: **Child → Left → Right → Base → object**.

No consistent order → `TypeError: Cannot create a consistent method resolution order`.

`super()` follows the MRO, not "the parent"

`super()` = "the next class after me **in the MRO of the instance's actual type**". In `Left.greet` on a `Child`, that's `Right` — a sibling. If every class calls `super()`, each implementation runs exactly once: **cooperative multiple inheritance**.

```python
class Base:
    def greet(self): return ["Base"]
class Left(Base):
    def greet(self): return ["Left"] + super().greet()
class Right(Base):
    def greet(self): return ["Right"] + super().greet()
class Child(Left, Right):
    def greet(self): return ["Child"] + super().greet()

class X: ...
class Y: ...
class A(X, Y): ...
class B(Y, X): ...
class C(A, B): ...        # A wants X before Y, B wants Y before X -> impossible
```

Output (`inheritance_mro.py`):

```
Child MRO: ['Child', 'Left', 'Right', 'Base', 'object']
Child().greet(): ['Child', 'Left', 'Right', 'Base']
C3 refuses: Cannot create a consistent method resolution order (MRO) for bases X, Y
```

Cooperative `__init__`: with differing arguments, take `**kwargs`, consume yours, pass the rest: `def __init__(self, *, color, **kw): super().__init__(**kw)`.

Trade-offs

Multiple inheritance	Pros	Cons
Mixins (small, stateless add-ons)	Reuse orthogonal (independent) behaviour cheaply	Name clashes, MRO surprises
Several "real" stateful bases	Rarely justified	Conflicting `__init__` signatures, fragile

Common mistakes

- Assuming `super()` means "the parent".
- Mixing `super()` with explicit `Parent.method(self)` → methods run twice or not at all.
- Listing a mixin *after* a base that already defines the method — the mixin is never reached.

Real-world examples

- Django class-based views: `class MyView(LoginRequiredMixin, ListView)` — mixin first so its `dispatch` runs before `ListView`'s.
- `socketserver.ThreadingMixIn` + `TCPServer` (TCP: the internet's reliable connection protocol — System Design Fundamentals) → `ThreadingTCPServer`.

In the interview

> **30-second answer:** "The MRO is the linear order Python searches for attributes; it's computed by C3 linearization, which keeps children before parents, preserves the order of bases, and is consistent with every parent's MRO — so in a diamond the shared base appears once, last. `super()` delegates to the next class in the MRO of the instance's type, which enables cooperative multiple inheritance where each method runs once."

- *MRO of `D(B, C)` with `B, C` extending `A`?* — D, B, C, A, object.
- *When does class creation fail?* — Conflicting base orders.

TL;DR

MRO = C3: children first, bases in declared order, consistent with parents. `super()` = "next in the MRO", so cooperating classes in a diamond each run once.

10.6 Mixins

What it is

A **mixin** is a small class providing one capability (serialise — convert to storable text/bytes — to JSON, a nice `__repr__`, logging), meant to be *combined*, not used alone; usually no `__init__` or state.

```
class JsonMixin:
    def to_json(self) -> str:
        return json.dumps(self.__dict__, sort_keys=True)

class ReprMixin:
    def __repr__(self) -> str:
        fields = ", ".join(f"{k}={v!r}" for k, v in self.__dict__.items())
        return f"{type(self).__name__}({fields})"

class Product(JsonMixin, ReprMixin):
    def __init__(self, sku: str, price: float) -> None:
        self.sku, self.price = sku, price
```

Output:

```
mixins: Product(sku='A-1', price=9.5) {"price": 9.5, "sku": "A-1"}
Product MRO: ['Product', 'JsonMixin', 'ReprMixin', 'object']
```

Rules of thumb

1. Name them `…Mixin`. 2. Stateless (or cooperative with `**kwargs`). 3. List **before** the main base. 4. One capability each.

When not to use it

Lots of state/configuration → compose an object (§10.8). A plain function `to_json(obj)` would do → use it.

Real-world examples

Django `LoginRequiredMixin`, `PermissionRequiredMixin`; `socketserver.ForkingMixIn` / `ThreadingMixIn`; `collections.abc` classes supply concrete (implemented) methods like `Sequence.index`.

TL;DR

Small, stateless, single-purpose classes mixed in via multiple inheritance, left of the main base.

10.7 Abstract base classes, duck typing and Protocols

Three ways to say "this object must be able to do X".

Duck typing — what it is

"If it walks and quacks like a duck, it's a duck": Python calls the method and lets it fail if absent — any object with `.speak()` works in `make_it_speak`. It pairs with **EAFP** (try, then catch) over **LBYL** (check first) — §9.5.

Abstract base classes (ABCs) — what it is

An **ABC** (`abc` module) declares `@abstractmethod`s that subclasses **must** implement; instantiating a class with any left raises `TypeError` — at *creation*, not at the call. ABCs can also share concrete methods.

`collections.abc` has the standard ones: `Iterable`, `Iterator`, `Sized`, `Container`, `Sequence`, `Mapping`, `MutableMapping`, `Callable`, `Hashable` … Several (`Sized`, `Iterable`, `Hashable`, `Callable`, `Container`) implement `__subclasshook__` (lets a class decide what counts as its subclass), so `isinstance(x, Sized)` is

`True` for any class with `__len__`, no inheritance needed. `Shape.register(SomeClass)` declares a *virtual subclass*.

Protocols — what it is

`typing.Protocol` (PEP 544, 3.8) formalises duck typing for **static type checkers** (mypy, pyright): a class matches if it has the right methods/attributes — **structural typing**, no inheritance. `@runtime_checkable` enables `isinstance`, which only checks that the members *exist* (not signatures).

```
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    def describe(self) -> str:                # concrete, shared
        return f"{type(self).__name__} with area {self.area():.2f}"

class Square(Shape):
    def __init__(self, side): self.side = side
    def area(self): return self.side ** 2

class Broken(Shape): pass                     # forgot area()

class Duck:
    def speak(self): return "Quack"
class Robot:
    def speak(self): return "Beep"

@runtime_checkable
class Speaker(Protocol):
    def speak(self) -> str: ...

def announce(s: Speaker) -> str:              # checked structurally by mypy/pyright
    return s.speak().upper()

class Deck:                                   # never inherits from Sized
    def __len__(self): return 52
```

Output (`abc_protocols.py`):

```
Square with area 9.00
TypeError: Can't instantiate abstract class Shape without an implementation for abstract method 'area'
TypeError: Can't instantiate abstract class Broken without an implementation for abstract method 'area'
['Quack', 'Beep']
BEEP | isinstance(Duck(), Speaker): True | isinstance(42, Speaker): False
Deck is Sized: True | Deck is Iterable: False
```

Protocols, generics and type checking in depth: Chapter 13 §13.8.

Comparison

	Duck typing	ABC	Protocol
Needs inheritance?	No	Yes (or `register`)	No
Checked when?	At call time (AttributeError)	At instantiation (TypeError)	By the static type checker; optionally `isinstance`
Shares code?	No	Yes, concrete methods	Not intended for sharing implementation
Best for	Small scripts, flexible APIs	Frameworks/plugins you own, enforced contracts	Library APIs, third-party types, typed codebases

When not to use them

No ABC for a single implementation "just in case" (YAGNI — "You Aren't Gonna Need It"). Don't rely on `@runtime_checkable` for correctness.

Common mistakes

- Not inheriting `ABC` (or `metaclass=ABCMeta`) — `@abstractmethod` has no effect.
- `type(x) is list` instead of `isinstance(x, collections.abc.Sequence)`.
- `isinstance(obj, SomeProtocol)` without `@runtime_checkable` → `TypeError`.

Real-world examples

`collections.abc.Mapping` for dict-like classes; `io.IOBase` ABC for files; `typing.SupportsInt`, `SupportsAbs`, `SupportsIndex` Protocols; the `numbers.Number` ABC hierarchy.

In the interview

> **30-second answer:** "Python is duck-typed: we care what an object can do, not its class. ABCs make that contract explicit and enforced — you can't instantiate a subclass until it implements every `@abstractmethod`. Protocols describe the same contract structurally, so a type checker accepts any class with matching methods, no inheritance needed. I use ABCs when I own the hierarchy and want shared code, Protocols for flexible, typed interfaces."

- *`isinstance(x, Sized)` checks?* — `__len__` on the type (via `__subclasshook__`), or registration/inheritance.

TL;DR

Duck typing = behaviour over type. ABC = nominal (declared inheritance), enforced at instantiation, shares code. Protocol = structural, for type checkers. `collections.abc` holds the standard interfaces.

10.8 Composition vs inheritance

What it is

Composition builds an object from parts it *has* (has-a: a `Car` *has an* `Engine`); **inheritance** says *is a*. Classic advice (Gang of Four, *Effective Python*): **"favour composition over inheritance"**.

Why it matters

Inheritance couples children to parent internals (the "fragile base class" problem: parent changes silently break children). Composition hides parts behind small interfaces, swappable at run time (a fake engine in tests, an electric motor in production).

```python
class Engine:
    def __init__(self, hp: int) -> None: self.hp = hp
    def start(self) -> str: return f"{self.hp}hp engine started"

class ElectricMotor:
    def start(self) -> str: return "silent motor started"

class Car:
    def __init__(self, engine) -> None:
        self.engine = engine                # injected part (dependency injection)
    def drive(self) -> str:
        return self.engine.start() + ", driving"
```

Output:

```
150hp engine started, driving
silent motor started, driving
```

Question	Inheritance	Composition
Relationship	is-a	has-a / uses-a
Coupling	Tight (sees parent internals)	Loose (only the part's interface)
Change at run time?	No	Yes (swap the part)
Code reuse	Automatic	Delegate explicitly (a few one-line methods)
Testing	Hard to replace parent behaviour	Inject a fake (passing parts in from outside = *dependency injection*)

When inheritance is right

Genuine is-a with substitutability (Liskov); framework extension points designed for subclassing (`unittest.TestCase`, `Exception` subclasses, Django `Model`, `nn.Module`); mixins.

Common mistakes

`class Stack(list)` — callers can `insert(0, x)` and break stack semantics; wrap a list. Inheriting just to reuse one helper.

In the interview

"I default to composition: it's looser coupling and easier to test and change at run time. I inherit when there's a real is-a relationship that satisfies Liskov, or when a framework is designed to be extended by subclassing."

TL;DR

Has-a → composition (default); is-a with substitutability → inheritance. Duck typing makes plugging in parts easy.

10.9 Encapsulation conventions: `_single` and `__double` underscores

What it is

No private keyword — encapsulation by convention ("we're all consenting adults"):

Name	Meaning	Enforced?
`name`	Public API	—
`_name`	Internal; may change. `from module import *` skips it	Convention (and linters)
`__name` (no trailing `__`)	**Name mangling**: inside class `Account`, `self.__secret` becomes `self._Account__secret` at compile time	Only prevents *accidental* subclass clashes; still reachable
`__name__`	Dunder: reserved for the language	Don't invent your own
`name_`	Avoids clashing with a keyword (`class_`, `type_`)	—

Why name mangling exists

To stop a subclass from accidentally overwriting a same-named parent attribute — **not** a security feature.

```python
class Account:
    def __init__(self) -> None:
        self._internal = "convention only"
        self.__secret = "mangled"            # stored as _Account__secret

class SubAccount(Account):
    def __init__(self) -> None:
        super().__init__()
        self.__secret = "child's own"         # stored as _SubAccount__secret
```

Output:

```
instance dict: {'_internal': 'convention only', '_Account__secret': 'mangled', '_SubAccount__secret': "child's
own"}
AttributeError: 'SubAccount' object has no attribute '__secret'
```

Common mistakes

`__x` everywhere to "make it private" — complicates debugging, pickling and subclass access. *Effective Python*: prefer `_protected`; reserve `__private` for clashes with subclasses you don't control.

TL;DR

`_x` = internal by convention (use this). `__x` = mangled to `_Class__x` against subclass clashes. Nothing is truly private; use properties to control access.

10.10 dataclasses

What it is

`@dataclass` (`dataclasses`, Python 3.7, PEP 557) reads **type-annotated** class attributes and generates `__init__`, `__repr__`, `__eq__`; flags add ordering, hashing, immutability and `__slots__`. Hand-written versions are repetitive and bug-prone (forget a field in `__eq__` and equality is silently wrong).

How it works

1. Each annotated class attribute becomes a **field**, in order. Un-annotated attributes (`x = 5`) and `ClassVar[...]` (true class-level variables) are ignored.
2. Defaults become `__init__` defaults; non-default fields can't follow default ones (unless `kw_only`).
3. `field(...)` customises: `default_factory=list` (fresh object per instance), `repr=False`, `compare=False` (excluded from `__eq__`/ordering), `init=False`, `kw_only=True`.
4. `__post_init__` runs after the generated `__init__` — validation, derived fields.

`@dataclass(...)` flag	Effect
`eq=True` (default)	`__eq__` comparing fields as a tuple — so the class is unhashable unless `frozen` / `unsafe_hash`
`order=True`	`<`, `<=`, `>`, `>=` comparing field tuples in declaration order
`frozen=True`	Field assignment raises `FrozenInstanceError`; with `eq=True` also generates `__hash__`
`slots=True` (3.10)	Generates `__slots__` → less memory, no new attributes
`kw_only=True` (3.10)	All fields keyword-only
`unsafe_hash=True`	Forces `__hash__` even when mutable — rarely right

Helpers: `asdict()`, `astuple()`, `replace(obj, **changes)` (copy with changes — how you "modify" a frozen instance), `fields()`.

```python
@dataclass
class Item:
    name: str
    price: float
    qty: int = 1
    tags: list[str] = field(default_factory=list)          # never `tags: list = []`
    _cache: dict = field(default_factory=dict, repr=False, compare=False)

    def __post_init__(self) -> None:
        if self.price < 0:
            raise ValueError("price must be >= 0")

    @property
    def total(self) -> float:
        return self.price * self.qty


@dataclass(frozen=True, order=True)
class Version:
    major: int
    minor: int
    patch: int = 0


@dataclass(slots=True, kw_only=True)
class Config:
    host: str
    port: int = 8080
```

Output (dataclass part of `dataclasses_enums_slots.py`):

```
Item(name='pen', price=1.5, qty=4, tags=['office']) | total: 6.0
eq ignores _cache: True
asdict: {'name': 'pen', 'price': 1.5, 'qty': 4, 'tags': ['office'], '_cache': {}}
post_init: price must be >= 0
mutable default: mutable default <class 'list'> for field tags is not allowed: use default_factory
order compares field-by-field: True | sorted: [Version(major=0, minor=9, patch=0), Version(major=1, minor=4,
patch=0), Version(major=1, minor=10, patch=0)]
frozen is hashable: stable
FrozenInstanceError: cannot assign to field 'major'
replace(): Version(major=1, minor=4, patch=7)
Config(host='localhost', port=8080) | has __dict__: False
```

`Version(1, 4) < Version(1, 10)` because tuples `(1, 4, 0) < (1, 10, 0)` compare numerically — unlike strings `"1.4" < "1.10"`.

Trade-offs — data-holder options

Option	Mutable	Validation	Speed/memory	Use when
`dict`	Yes	None	Fine	Truly dynamic keys, JSON
`typing.NamedTuple`	No	None	Tuple-light	Immutable records that should also unpack/index like tuples
`@dataclass`	Configurable	`__post_init__`	Good; `slots=True` for lean	Default for internal data
`attrs` (3rd party)	Configurable	Validators/converters	Good	More features; inspired dataclasses
Pydantic `BaseModel` (3rd party)	Configurable	Full parsing/validation of untrusted input	Heavier	API payloads, config files

Common mistakes

- `tags: list = []` → `ValueError`; use `field(default_factory=list)`.
- Missing annotation — `x = 5` is a class attribute, not a field.
- Expecting runtime type validation — annotations aren't enforced.
- `frozen=True` is shallow: a contained list can still mutate.
- Base field with default + subclass field without → `TypeError` (fix with `kw_only=True`).

Real-world examples

Configs, DTOs (data-transfer objects: plain records passed between system parts), events; Hugging Face `TrainingArguments` is a dataclass; FastAPI/Pydantic accept dataclasses.

In the interview

> **30-second answer:** "`@dataclass` generates `__init__`, `__repr__` and `__eq__` from annotated fields. `order=True` adds comparisons by field tuple, `frozen=True` makes it immutable and hashable, `slots=True` saves memory. Use `field(default_factory=list)` for mutable defaults and `__post_init__` for validation. For untrusted input I'd use Pydantic instead because dataclasses don't validate types."

TL;DR

Dataclasses remove data-class boilerplate. Know `field(default_factory=…)`, `__post_init__`, `frozen`, `order`, `slots`, `kw_only`, `replace`, `asdict`.

10.11 Enums

What it is

An **enumeration** (`enum.Enum`) is a set of named, unique constant members (`Color.RED`) — singletons with `.name` and `.value`, iterable, compared by identity. They replace magic strings/ints (unexplained literals: `status == 3`, the `"acitve"` typo) with autocomplete, typo-proof names, a closed set of legal values and readable reprs.

Class	Behaves like	Use for
`Enum`	Its own type; `Color.RED == 1` is `False`	Default
`IntEnum`	Also an `int`	Interop with int-expecting code (`http.HTTPStatus`)
`StrEnum` (3.11)	Also a `str`; `auto()` gives the lower-case name	Values stored in JSON/DB as strings
`Flag` / `IntFlag`	Bit flags combinable with `|`, tested with `in`	Permissions, options (`re.RegexFlag`)

`auto()` assigns values (1, 2, 3 … for `Enum`); `@enum.unique` forbids aliases (two names, same value). Look up by value `Color(3)` or name `Color["GREEN"]`.

```
class Color(Enum):
    RED = auto(); GREEN = auto(); BLUE = auto()
class HttpStatus(IntEnum):
    OK = 200; NOT_FOUND = 404
class Env(StrEnum):
    DEV = auto(); PROD = auto()
class Perm(Flag):
    READ = auto(); WRITE = auto(); EXEC = auto()
```

Output:

```
Color.RED RED 1 Color.GREEN Color.BLUE
members: [<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>] | identity compare: True
IntEnum == int: True | Enum == int: False
StrEnum: prod True
Flag: Perm.READ|WRITE | WRITE in rw: True | EXEC in rw: False
```

Common mistakes

- `member == "red"` with plain `Enum` → always `False`; compare to `Color.RED` or use `StrEnum`.
- `IntEnum` everywhere — unrelated int comparisons succeed silently.
- Modifying/adding members at run time — enums are fixed.

Real-world examples

`http.HTTPStatus`, `re.RegexFlag` (`re.IGNORECASE`), `signal.Signals`, Django `TextChoices`/`IntegerChoices`.

TL;DR

`Enum` for closed constant sets, `StrEnum`/`IntEnum` for str/int interop, `Flag` for combinable options; `auto()` for values.

10.12 `__slots__`

What it is

Instances normally keep attributes in a per-instance `__dict__` — flexible but memory-hungry. `__slots__ = ("x", "y")` reserves fixed storage for exactly those attributes and creates **no** `__dict__`. With millions of small objects (points, graph nodes, events) this cuts memory, speeds attribute access slightly, and stops typos from silently creating attributes.

```python
class PointDict:
    def __init__(self, x, y): self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")
    def __init__(self, x, y): self.x, self.y = x, y
```

Output (100 000 instances measured with `tracemalloc`, the stdlib allocation tracker, including the holding list):

```
100k objects: dict-based 12.8 MB vs slots 8.8 MB
slots block new attrs: 'PointSlots' object has no attribute 'z' and no __dict__ for setting new attributes
```

(Version-dependent — recent CPythons store instance dicts compactly, shrinking the saving.)

Caveats

- Only listed attributes; no `__dict__`, so no `vars(obj)`.
- No weak references (references that don't keep an object alive) unless `"__weakref__"` is listed.
- Every class in the hierarchy must declare `__slots__` (`()` if nothing new), or the subclass regains a `__dict__`.
- A slot can't have a same-named class-attribute default; `@dataclass(slots=True)` handles this.
- `functools.cached_property` needs `__dict__`, so it fails on slotted classes.

TL;DR

`__slots__` = fixed attribute set, no per-instance dict → less memory, no accidental attributes. Use for many small objects.

10.13 Pythonic design patterns (brief)

Full SOLID and all 23 GoF patterns are in the LLD Handbook: SOLID, creational incl. Singleton, structural incl. Decorator, behavioral incl. Strategy/Observer. First-class functions and modules shrink several of them:

Pattern	Classic (Java-style)	Pythonic
Strategy (swap algorithms)	Interface + one class per strategy	Pass a **function**
Singleton (exactly one instance)	Private constructor + `getInstance()`	A **module** (imported once, cached in `sys.modules`) or a `@functools.cache` factory
Command (action as an object)	Command class with `execute()`	A function, `functools.partial`, or a callable object (`__call__`)
Iterator	Class with `hasNext/next`	A generator function (Chapter 11)
Decorator (wrap to add behaviour)	Wrapper class implementing the same interface	`@decorator` functions (Chapter 12) — or a wrapper class for objects
Factory (creates other objects)	Factory class	`@classmethod` alternative constructor, or a dict mapping names to classes
Template method (base fixes steps, subclasses fill some)	Abstract base with hook methods	Same (ABC), or pass hook functions
Observer (notify subscribers)	Listener interface	A list of callbacks

```python
def no_discount(total): return total
def ten_percent(total): return total * 0.9
def bulk(total): return total - 20 if total > 200 else total

def checkout(total: float, strategy=no_discount) -> float:
    return round(strategy(total), 2)

@functools.cache
def get_settings() -> _Settings:        # lazily created, then always the same object
    return _Settings()
```

Output (`pythonic_patterns.py`):

```
no_discount  -> 250
ten_percent  -> 225.0
bulk         -> 230
<lambda>     -> 125.0
same object: True | debug: True
module cached once: True | in sys.modules: True
```

A Singleton is still global state and hurts testing — prefer passing dependencies; keep singletons for truly process-wide things (config, logging — `logging.getLogger(name)` returns the same logger per name).

In the interview

"In Python, a Strategy is usually just a function argument, like `sorted(key=...)`. A Singleton is usually a module-level object, because modules are imported once. I'd still use classes when the strategy needs its own state or several related methods."

TL;DR

Functions for Strategy/Command, modules for Singleton, generators for Iterator, `@decorators` for Decorator.

10.14 Chapter self-check

1. Two steps when calling `MyClass(...)`? → `__new__` creates, `__init__` initialises.
2. Why does `a.items.append(1)` affect `b` with class attribute `items = []`? → One shared list; mutation creates no instance attribute.
3. `class D(B, C)`, both subclass `A` — MRO? → D, B, C, A, object.
4. `__eq__` defined — instances in a set? → No, `__hash__` is `None`; hash the same immutable fields.
5. `v * 3` works, `3 * v` fails? → Implement `__rmul__`.
6. ABC vs Protocol? → Nominal, enforced at instantiation, shares code vs structural, checked by type checkers.
7. List default in a dataclass? → `field(default_factory=list)`.
8. `self.__token` inside class `Api`? → `self._Api__token`.
9. When do `__slots__` help? → Many small instances (no per-instance `__dict__`).
10. Pythonic Strategy? → Pass a function.

Chapter TL;DR

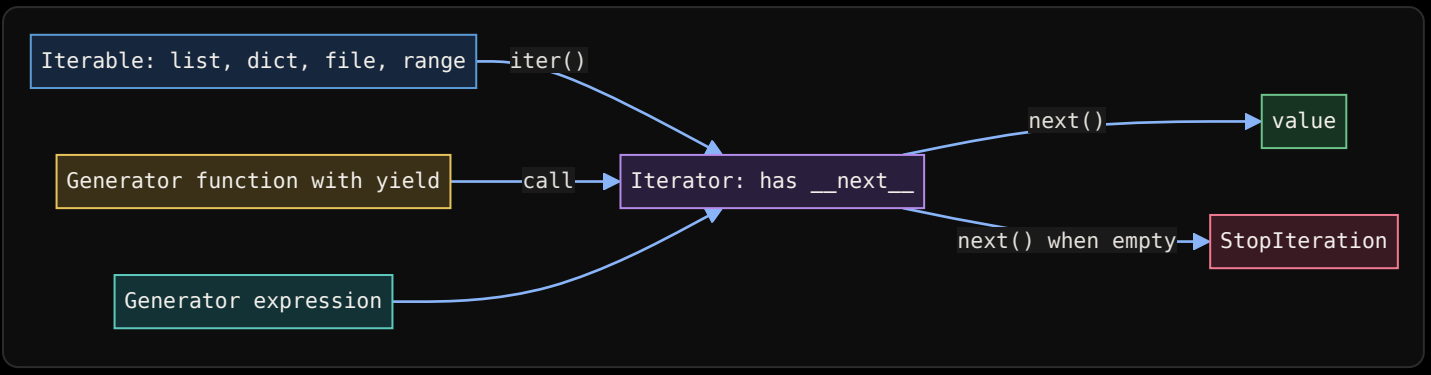
- **Classes** bundle state + behaviour; `__init__` initialises `self`. Instance attributes live in `obj.__dict__`; class attributes are shared (never a mutable one as per-instance state).
- **Methods:** instance (`self`), `@classmethod` (`cls`, alternative constructors), `@staticmethod` (neither). **Properties** add validation without changing the API.
- **Data model:** dunders connect your class to syntax and built-ins. Always `__repr__`; keep `__eq__`/`__hash__` consistent; return `NotImplemented`; `__r…__` for reflected operators; `total_ordering` for comparisons.
- **Inheritance:** `super()` = next in the **MRO** (C3: children first, bases in order, consistent). Cooperative `super()` makes diamonds safe. **Mixins** add small capabilities.
- **Contracts:** duck typing by default, **ABCs** to enforce, **Protocols** for structural static typing. **Prefer composition** unless it's a real is-a.
- **Toolbox:** `@dataclass` (`field`, `frozen`, `order`, `slots`, `kw_only`), `enum` (`Enum`, `IntEnum`, `StrEnum`, `Flag`, `auto`), `__slots__` for memory.
- **Encapsulation:** `_x` by convention; `__x` is name-mangled, not private.
- **Patterns:** functions for Strategy, modules for Singleton; details in the LLD Handbook.

Chapter 11 — Iterators, Generators and Functional Tools

What you will learn. The protocol behind every `for` loop (iterable vs iterator, `iter()`, `next()`), generator functions and expressions that produce values lazily, `yield from`, `send()` / `close()` basics, why laziness saves memory and time, the `itertools` toolkit (`count`, `cycle`, `chain`, `islice`, `groupby`, `product`, `permutations`, `combinations`, `accumulate`, `pairwise`, `batched` …), comprehension best practices, `map` / `filter` / `reduce` vs comprehensions, `any` / `all`, `zip(strict=True)`.

Prerequisites. Chapter 10's data-model idea: syntax calls dunder ("double underscore") methods such as `__iter__` and `__len__` for you.

Code. `chapters/11-iterators-generators-functional/code/` — `iter_protocol.py`, `generators.py`, `itertools_tour.py`, `comprehensions_functional.py`, `lazy_memory.py` (makes the plot in `img/`). Python 3.12+ (`itertools.batched`).



11.1 Iterable vs iterator: the iteration protocol

What it is

- An **iterable** can be looped over: it has `__iter__()` returning an iterator (or, as a legacy fallback still supported, a `__getitem__` accepting 0, 1, 2 …). Lists, tuples, strings, dicts, sets, `range`, files.
- An **iterator** walks the items: `__next__()` returns the next item or raises `StopIteration` ("no more items"), **and** `__iter__()` returns **itself**.

A book is an *iterable*; a bookmark is an *iterator* — many readers can each have one, and a bookmark only moves forward until used up.

Why it matters

This one protocol powers `for`, comprehensions, unpacking (`a, b = x`), `sum`, `list()`, `sorted`, `zip`, `in` — and explains "why is my generator empty the second time?". (Overview in §3.5.)

How it works — what `for x in obj:` really does

1. `it = iter(obj)` → `obj.__iter__()`.
2. Repeatedly `x = next(it)` → `it.__next__()`, run the body.
3. On `StopIteration`, stop quietly.

```python
class Countdown:                        # ITERABLE: returns a fresh iterator each time
    def __init__(self, start): self.start = start
    def __iter__(self): return CountdownIterator(self.start)

class CountdownIterator:                # ITERATOR: holds position, returns itself
    def __init__(self, current): self.current = current
    def __iter__(self): return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration         # "no more items"
        self.current -= 1
        return self.current + 1

def manual_for(iterable):               # what a for loop does
    out, it = [], iter(iterable)
    while True:
        try:
            x = next(it)
        except StopIteration:
            break
        out.append(x)
    return out
```

Output (`iter_protocol.py`):

```
for loop: [3, 2, 1] | again: [3, 2, 1]
manual_for: [3, 2, 1]
next: 10 20 30 | default: done
iter(it) is it: True
first pass: [0, 1, 4] | second pass: []
iter(callable, sentinel): ['a', 'b']
RuntimeError: dictionary changed size during iteration
```

`Countdown` is re-iterable (new iterator per `for`); `next(it, default)` returns the default instead of raising; an iterator's `iter()` is itself; a generator is **exhausted after one pass**; `iter(callable, sentinel)` calls a function until it returns the sentinel (a "stop here" marker — e.g. `iter(lambda: f.read(4096), b"")`); resizing a dict while iterating raises.

	Iterable	**Iterator**
Required methods	`__iter__`	`__iter__` (returns self) + `__next__`
Can loop over it twice?	Yes (new iterator each time)	No — one pass, then empty
Remembers position?	No	Yes
`len()`?	Often (`list`, `range`)	Usually no
Examples	`list`, `str`, `dict`, `range`, `set`	generator, file object, `map`, `zip`, `enumerate`, `iter(list)` (a `list_iterator`)

`range` is a lazy **sequence** (iterable, supports `len` and indexing), not an iterator: `next(range(3))` is a `TypeError`.

Common mistakes

- Looping twice over a generator/`map`/`zip`/file — second pass empty. Materialise (store as a real collection) with `list()` or write a re-iterable class.
- A class whose `__iter__` returns `self`, expected to be re-iterable.
- Modifying a list/dict while iterating — iterate a copy (`for k in list(d):`) or rebuild.
- `raise StopIteration` inside a **generator**: since 3.7 (PEP 479 — PEPs, *Python Enhancement Proposals*, are the numbered design documents for language changes) it becomes `RuntimeError`. Use `return`.

Real-world examples

Files iterate line by line without loading everything; `csv.reader` and `os.scandir` return iterators; database cursors (objects walking a query's result rows; DB-API, PEP 249, is Python's standard driver interface) are iterable; PyTorch's `DataLoader` yields batches (small groups of training examples) each epoch (one full pass over the data).

In the interview

> **30-second answer:** "An iterable has `__iter__` that returns an iterator; an iterator has `__next__`, which returns items or raises `StopIteration`, and its `__iter__` returns itself. A `for` loop calls `iter()` once then `next()` until `StopIteration`. Iterables like lists can be looped many times; iterators like generators are single-pass."

- *Generator empty the second time?* — It's an iterator positioned at the end.
- *Make a class iterable?* — `__iter__`, often as a generator: `def __iter__(self): yield from self._items`.

TL;DR

Iterable → `iter()` → iterator → `next()` … `StopIteration`. Iterables restart; iterators are one-shot.

11.2 Generators and `yield`

What it is

A **generator function** contains `yield`. Calling it does **not** run the body — it returns a **generator object** (an iterator). Each `next()` runs to the next `yield`, hands out the value and **pauses**, keeping local variables. When the function returns, `StopIteration` is raised (`return value` becomes `StopIteration.value`).

Why it matters

- **Memory:** one value at a time — a 50 GB log or an infinite stream in constant memory.
- **Simplicity:** a few lines instead of a class with `__iter__`/`__next__` and manual state.
- **Pipelines:** chain like Unix pipes (`cmd1 | cmd2`, output streaming into the next program): read → filter → parse → aggregate, each stage lazy.

How it works

1. `g = countdown(3)` creates the generator; the body hasn't started.
2. `next(g)` runs to the first `yield n`, returns 3, freezes the frame (the function's live state: locals and current line).
3. Next `next(g)` resumes after the `yield`, yields 2.
4. Body ends or `return` → `StopIteration`, handled by `for`/`list()`.

```python
def countdown(n: int):
    print("  (generator body starts)")
    while n > 0:
        yield n                     # pause here, hand n to the caller
        n -= 1
    return "liftoff"                # becomes StopIteration.value

def read_lines(lines):              # pipeline stage 1: strip, skip blanks
    for line in lines:
        line = line.strip()
        if line:
            yield line

def parse(lines):                   # pipeline stage 2: parse
    for line in lines:
        name, score = line.split(",")
        yield name, int(score)

def fib():                          # infinite but lazy
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

Output (first part of `generators.py`):

```
created: generator (nothing printed yet -> lazy)
  (generator body starts)
next: 3
rest: [2, 1]
  (generator body starts)
return value via StopIteration.value: liftoff
pipeline: [('alice', 90), ('bob', 72), ('cara', 85)]
...
first 10 fib: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

"(generator body starts)" appears only at the first `next()`. `fib()` never ends, yet `islice(fib(), 10)` (or `list(itertools.islice(gen, 5))` for the first five) takes only what's asked.

Generator expressions

A **generator expression** is the lazy list comprehension — parentheses: `(x * x for x in range(5))`; as a sole argument the extra parentheses drop: `sum(x * x for x in nums)`.

```
genexp sum: 30 | sum(x for x in ...) needs no extra parens: 10
```

Trade-offs

	List (comprehension)	Generator (function/expression)
Memory	Stores all n items	One item at a time (constant)
Multiple passes	Yes	No (one-shot)
`len()`, indexing, slicing	Yes	No (use `itertools.islice`)
First result available	After everything is computed	Immediately
Speed for small data used fully	Slightly faster	Slight per-item overhead
Infinite sequences	Impossible	Natural

When not to use it

Data needed more than once, `len()`/random access, or small data you'll use fully — a list is simpler. Laziness also moves errors to consumption time, which can confuse debugging.

Common mistakes

- Calling a generator function and expecting it to "do" something — nothing runs until iterated.
- Consuming a generator twice.
- `return some_list` inside a generator ends iteration (value → `StopIteration.value`), it isn't returned to the `for`.
- Holding resources (open files) in a never-finished generator — use `with` inside it and/or close it (`gen.close()`, `contextlib.closing`).

Real-world examples

`os.walk` is a generator; Scrapy spiders `yield` items and requests; pytest fixtures use `yield` to separate setup from teardown; Flask `Response(generator)` and FastAPI/Starlette `StreamingResponse` stream replies piece by piece; `pandas.read_csv(..., chunksize=N)` iterates chunks of too-big files.

In the interview

> **30-second answer:** "A generator function uses `yield`; calling it returns a generator object — an iterator — without running the body. Each `next()` runs to the next `yield`, returns that value and suspends with its local state intact. That gives lazy, constant-memory iteration, infinite sequences, and composable pipelines. Generator expressions are the inline form with parentheses."

- *Generator vs list comprehension?* — Lazy, one-shot vs eager, reusable, indexable.
- *Get a generator's return value?* — `StopIteration.value`, or the result of `yield from`.
- *`raise StopIteration` inside a generator?* — `RuntimeError` since 3.7; use `return`.

TL;DR

`yield` turns a function into a lazy iterator factory: pause/resume, constant memory, pipelines — but single-pass.

11.3 `yield from`, `send()` and `close()` (brief)

`yield from` — delegation

`yield from sub_iterable` (PEP 380, 3.3) yields every value of the sub-iterable and — for a sub-generator — evaluates to its `return` value. Replaces `for x in sub: yield x`; ideal for recursion.

```python
def flatten(nested):
    for item in nested:
        if isinstance(item, list):
            yield from flatten(item)   # delegate to a recursive sub-generator
        else:
            yield item

def inner():
    yield 1
    yield 2
    return "inner-done"

def outer():
    result = yield from inner()        # receives inner's return value
    yield f"got {result!r}"
```

Output:

```
flatten: [1, 2, 3, 4, 5]
outer: [1, 2, "got 'inner-done'"]
```

`yield from` also forwards `send()`/`throw()`/`close()` — the foundation of pre-`async` **coroutines** (functions that pause midway and resume later, letting one thread juggle many tasks).

`send()`, `throw()`, `close()` — two-way generators

`yield` is an **expression**: `value = yield out` hands `out` to the caller and receives what the caller passes via `gen.send(value)`.

1. **Prime** first with `next(gen)` (or `gen.send(None)`) to reach the first `yield`; sending non-`None` to a just-started generator is a `TypeError`.
2. `gen.send(v)` resumes; the paused `yield` evaluates to `v`; runs to the next `yield` and returns its value.
3. `gen.throw(exc)` raises at the paused `yield`.
4. `gen.close()` raises `GeneratorExit` there for cleanup (`finally`, or `except GeneratorExit` then re-raise/return). Generators also close when garbage-collected (freed once unreferenced).

```
def running_average():
    total, count, avg = 0.0, 0, None
    try:
        while True:
            value = yield avg          # send avg out, receive next value
            total += value
            count += 1
            avg = total / count
    except GeneratorExit:
        print("  closing running_average (cleanup here)")
        raise
```

Output:

```
send 10 -> 10.0
send 20 -> 15.0
send 60 -> 30.0
closing running_average (cleanup here)
```

Modern code uses `async def` / `await` **coroutines** (asyncio — stdlib framework for many waiting tasks on one thread, Chapter 16); `send` appears mostly in libraries and interviews. `contextlib.contextmanager` (Chapter 12) uses the same machinery.

TL;DR

`yield from` delegates and returns a sub-generator's return value. `send` pushes into a paused generator (prime with `next`), `throw` raises inside, `close` triggers `GeneratorExit` cleanup.

11.4 Lazy evaluation and memory

What it is

Lazy evaluation computes a value only when requested; **eager** computes everything up front. Lazy in Python 3: `range`, `map`, `filter`, `zip`, `enumerate`, `reversed`, dict views (`keys()/values()/items()`), file iteration, generators, all of `itertools`.

Why it matters — measured

`lazy_memory.py` compares peak memory of `sum([x*x for x in range(n)])` (builds a list) and `sum(x*x for x in range(n))` (streams):

```
getsizeof list of 1,000,000: 8.4 MB (pointers only)
getsizeof generator:      208 bytes
n=   10,000  sum(list comp) peak   0.40 MB | sum(gen expr) peak   0.4 KB
n=  100,000  sum(list comp) peak   4.00 MB | sum(gen expr) peak   0.4 KB
n=  300,000  sum(list comp) peak  12.20 MB | sum(gen expr) peak   0.4 KB
n=1,000,000  sum(list comp) peak  40.45 MB | sum(gen expr) peak   0.4 KB
any() list comp built all 1,000,000 items; gen stopped at item 12 -> gen faster: True
```

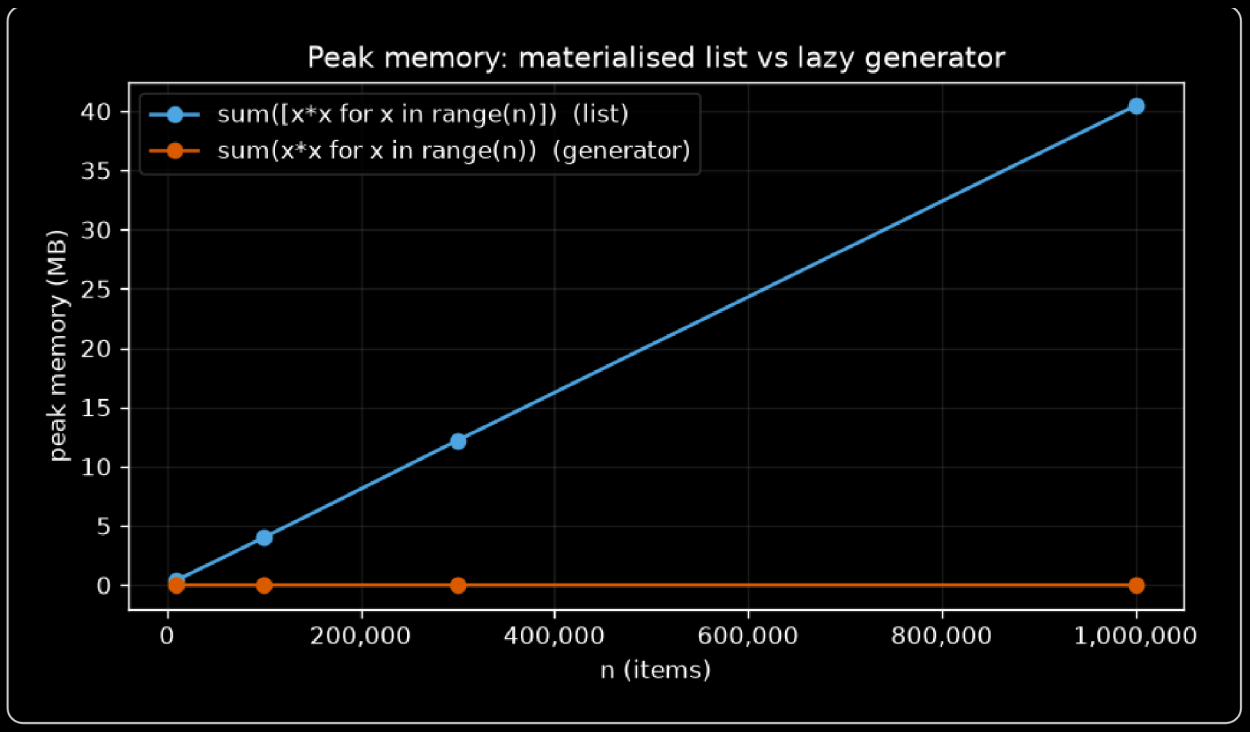


Figure — list peak memory grows linearly (≈40 bytes per item: 8-byte pointer + ~32-byte int) while the generator stays flat.

`sys.getsizeof(list)` counts only the pointer array (1,000,000 × 8 ≈ 8 MB); the int objects add ~32 MB, which `tracemalloc` (stdlib allocation tracker) sees. Sizes vary by version/platform.

Two benefits:

1. **Memory** — O(1) instead of O(n) for single-pass work (Big-O: O(1) constant regardless of *n*; O(n) proportional to *n*).
2. **Early exit** — `any`, `next`, `islice`, `takewhile` stop pulling once they have the answer.

Common mistakes

- `list(...)` everywhere "to be safe" — loses both benefits.
- Trusting Python 2 tutorials (`map` returned lists).
- Lazy code capturing a variable that changes before evaluation (closures bind late; §11.7).

TL;DR

Lazy = compute on demand. Generators and iterator tools for big/streaming/infinite data and early-exit searches; lists for reuse or random access.

11.5 `itertools` — the iterator toolkit

What it is

`itertools` is a stdlib module of fast (implemented in C), memory-efficient building blocks that take iterables and return **lazy iterators** — Lego for loops.

Infinite iterators

Function	Produces	Example → result
`count(start=0, step=1)`	start, start+step, … forever	`islice(count(10, 5), 4)` → 10, 15, 20, 25
`cycle(it)`	Repeats `it` forever (saves a copy)	`islice(cycle("AB"), 5)` → A B A B A
`repeat(x, n=∞)`	x, n times (or forever)	`repeat("x", 3)` → x x x

Terminating / selecting

Function	What it does	Output from the script
`chain(a, b, …)`	Concatenate iterables lazily	`[1, 2, 3, 'a', 'b']`
`chain.from_iterable(x)`	Flatten one level	`[1, 2, 3]`
`islice(it, start, stop, step)`	Slice any iterator (no negatives)	`islice(range(10), 2, 8, 2)` → `[2, 4, 6]`
`takewhile(pred, it)`	Yield while the predicate (True/False function) holds, then stop	`[1, 3]`
`dropwhile(pred, it)`	Skip while true, then yield the rest	`[6, 2]`
`filterfalse(pred, it)`	Items where the predicate is false	`[1, 3, 5]`
`compress(data, selectors)`	Keep items whose selector is truthy	`['A', 'C']`
`starmap(f, it)`	`f(*args)` for each args tuple	`[8, 9]`
`zip_longest(*its, fillvalue)`	`zip` padding the shorter ones	`[('a', 1), ('b', 2), ('-', 3)]`
`tee(it, n=2)`	n independent iterators from one (buffers in memory)	`[1, 2, 3] [1, 2, 3]`
`accumulate(it, func=add, initial=None)`	Running totals (or max, product …)	`[3, 4, 8, 9, 14]`
`pairwise(it)` (3.10)	Overlapping pairs (a,b), (b,c), …	`[(1, 4), (4, 9), (9, 16)]`
`batched(it, n)` (3.12)	Tuples of size n (last may be shorter; `strict=True` in 3.13+ raises instead)	`[(0, 1, 2), (3, 4, 5), (6,)]`

`groupby` — the one everybody gets wrong

`groupby(iterable, key)` groups **consecutive** equal-key items and does **not** sort — non-adjacent equal keys form separate groups (`groupby([1,1,2,1])` → three groups). **Sort by the same key first.**

```
words = ["apple", "avocado", "banana", "blueberry", "cherry", "apricot"]
{k: list(g) for k, g in groupby(words, key=lambda w: w[0])}
# 'a' appears twice as a group; the dict keeps only the last one -> 'apple', 'avocado' lost
{k: list(g) for k, g in groupby(sorted(words), key=lambda w: w[0])}
```

```
groupby UNSORTED (apricot lost/overwritten): {'a': ['apricot'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
groupby sorted: {'a': ['apple', 'apricot', 'avocado'], 'b': ['banana', 'blueberry'], 'c': ['cherry']}
```

Each group `g` is a lazy view invalidated when you advance — `list(g)` immediately to keep it. For simple grouping into a dict, `collections.defaultdict(list)` needs no sorting.

Combinatorics

Function	Meaning	Count for n items, pick r	`"abc"` , r=2
`product(A, B)` / `product(A, repeat=r)`	Cartesian product (every pairing; nested loops)	n^r	`aa ab ac ba …` (9)
`permutations(it, r)`	Ordered arrangements, no repeats	n!/(n−r)! = 6	`ab ac ba bc ca cb`
`combinations(it, r)`	Unordered selections, no repeats	n!/(r!(n−r)!) = 3	`ab ac bc`
`combinations_with_replacement(it, r)`	Unordered, repeats allowed	(n+r−1)!/(r!(n−1)!) = 6	`aa ab ac bb bc cc`

```
product: [('a', 1), ('a', 2), ('b', 1), ('b', 2)]
product repeat=2 of 01: ['00', '01', '10', '11']
permutations(abc, 2): ['ab', 'ac', 'ba', 'bc', 'ca', 'cb']
combinations(abc, 2): ['ab', 'ac', 'bc']
combinations_with_replacement(ab, 2): ['aa', 'ab', 'bb']
accumulate max: [3, 3, 4, 4, 5]
accumulate product, initial=1: [1, 2, 6, 24]
differences via pairwise: [3, 5, 7]
```

Output explodes (`permutations(range(12))` = 479,001,600 items) — iterate lazily, never `list()` blindly.

Real-world examples

`product` for hyperparameter grids (settings chosen before training an ML model; scikit-learn's `ParameterGrid` iterates the Cartesian product); `batched` for API/database writes in chunks of 100; `pairwise` for deltas between timestamps; `accumulate` for running balances; `chain.from_iterable` to flatten paged API results; `islice` for "first N" of a stream. `more-itertools` adds recipes (`chunked`, `windowed`, …).

In the interview

> **30-second answer:** "itertools provides lazy, C-speed building blocks: infinite ones like `count` and `cycle`; selectors like `islice`, `takewhile`, `compress`; combiners like `chain` and `zip_longest`; aggregators like `accumulate`, `groupby`, `pairwise`, `batched`; and combinatorics — `product`, `permutations`, `combinations`. The classic trap is that `groupby` only groups consecutive equal keys, so you sort first."

- *Permutations vs combinations?* — Order matters (ab ≠ ba) vs doesn't.
- *Batches of 100?* — `itertools.batched(items, 100)` (3.12+); earlier, `zip(*[iter(items)]*100)` drops the remainder, so loop with `islice`.
- *Why no `gen[:5]`?* — Generators lack `__getitem__`; use `islice`.

TL;DR

Reach for `itertools` before index-juggling loops: lazy results, `groupby` needs sorted input, combinatorics explode, `pairwise` (3.10) and `batched` (3.12) are modern.

11.6 Comprehensions — best practices

What it is

A **comprehension** builds a collection from an iterable in one expression (introduced in §5.6 and §6.5):

Kind	Syntax	Result
List	`[expr for x in it if cond]`	`list`
Dict	`{k: v for x in it if cond}`	`dict`
Set	`{expr for x in it if cond}`	`set`
Generator expression	`(expr for x in it if cond)`	lazy iterator

How to read one

`[n * n for n in nums if n % 2 == 0]` = "for each `n` in `nums`, if even, collect `n * n`". Multiple `for`s nest **left to right** like written loops.

```python
nums = [1, 2, 3, 4, 5, 6]
[n * n for n in nums if n % 2 == 0]                  # filter: `if` AFTER the for
["even" if n % 2 == 0 else "odd" for n in nums[:4]]  # choose: if-else BEFORE the for
[x for row in matrix for x in row]                   # flatten: outer loop first
[[row[i] for row in matrix] for i in range(3)]       # nested comprehension: transpose
[v for s in data if s.isdigit() and (v := int(s)) > 5]  # walrus: compute once
{v: k for k, v in prices.items()}                    # invert a dict
```

Output (`comprehensions_functional.py`):

```
list: [4, 16, 36]
dict: {4: 16, 5: 25, 6: 36}
set: {0, 1, 2}
gen : 91
labels: ['odd', 'even', 'odd', 'even']
flatten: [1, 2, 3, 4, 5, 6]
transpose: [[1, 4], [2, 5], [3, 6]]
walrus: [10, 7]
invert: {3: 'apple', 2: 'pear'}
```

Best practices

1. **One line of logic**: at most two `for`s, or one `for` + one condition (*Effective Python*); beyond that, a loop or named generator function.
2. **No side effects** (printing, writing files, changing globals) — `[print(x) for x in xs]` builds a list of `None`s.
3. **Generator expression** when consumed once by `sum`, `max`, `any`, `"".join`.
4. **Loop variables don't leak** — each comprehension has its own scope (3.12's PEP 709 inlines them for speed without changing that).
5. **Walrus** `:=` (3.8) to compute an expensive value once for filter and output.
6. **`dict` / `set` comprehensions** over `dict([(k, v) for …])`.

Common mistakes

- `[x for x in xs if c else y]` → `SyntaxError`: filter `if` goes after, choice `if-else` before.
- Wrong order of nested `for`s.
- Building a huge list to take `[0]` — `next(genexp, default)`.

TL;DR

Comprehensions map + filter into a new collection: short, side-effect free, generator form when consumed once.

11.7 `map` / `filter` / `reduce`, `any` / `all`, `zip(strict=True)` — the functional toolbox

What it is

Functional programming favours pure functions (no side effects) and functions as values. Python supports it via first-class functions, `lambda` (§7.5) and helpers:

- `map(f, it, ...)` — lazily applies `f` to each item (or items from several iterables in parallel).
- `filter(pred, it)` — lazily keeps truthy `pred(item)`; `filter(None, it)` keeps truthy items.
- `functools.reduce(f, it, initial)` — folds left to right: `f(f(f(init, a), b), c)`. (Moved out of built-ins in Python 3; empty input without `initial` → `TypeError`.)
- `any(it)` / `all(it)` — short-circuiting (stop once the answer is known).
- `zip(*its, strict=False)` — pairs items, stops at the shortest; `strict=True` (3.10, PEP 618) raises `ValueError` on unequal lengths.

Comprehension or `map` / `filter`?

Situation	Prefer	Why
Transform/filter with an expression	Comprehension	Readable; no `lambda`
Apply an existing named function	`map(str.upper, xs)` either is fine	No lambda needed
Need `lambda` to use `map` / `filter`	Comprehension	`[x*2 for x in xs]` beats `map(lambda x: x*2, xs)`
Reduce to one value	`sum`, `min`, `max`, `any`, `all`, `math.prod`, `"".join`	Clearer than `reduce`
Custom fold with no built-in	`reduce` or a loop	Loop is often more readable

Guido van Rossum (and *Effective Python*) advise comprehensions over `map`/`filter` with lambdas, and loops or built-ins over most `reduce` calls.

`any` / `all` and `zip` details

- `any` stops at the first truthy item, `all` at the first falsy — pass a **generator expression**, not a list (`any([expensive(x) for x in xs])` computes everything).
- `all([])` is `True` (vacuous truth: "every item passes" holds with no items); `any([])` is `False`.
- Plain `zip` silently truncates — a classic data bug; use `strict=True` when lists must align. `zip(*pairs)` unzips; `dict(zip(keys, values))` builds a dict; `zip` is lazy.

```
print("map:", list(map(str.upper, ["a", "b"])))
print("filter:", list(filter(None, [0, 1, "", "hi", None])))
print("reduce product:", reduce(operator.mul, nums, 1))
result = any(check(x) for x in [1, 3, 5])          # stops at 3
list(zip(names, scores, strict=True))              # ValueError on mismatch
letters, numbers = zip(*pairs)                     # unzip
funcs = [lambda: i for i in range(3)]              # late binding trap
funcs = [lambda i=i: i for i in range(3)]          # fix: bind now via default
sorted(people, key=lambda p: (-p[1], p[0]))        # multi-key sort
```

Output:

```
map: ['A', 'B'] | comp: ['A', 'B']
filter: [1, 'hi'] (filter(None, ...) keeps truthy)
reduce sum: 21 | sum(): 21
reduce product: 720
map over two iterables: [11, 22]
  check(1)  check(3)
  any -> True (stopped after 3; 5 never checked)
all([]) = True | any([]) = False
zip truncates silently: [('a', 90), ('b', 80)]
strict=True: zip() argument 2 is shorter than argument 1
unzip: ('a', 'b') (1, 2)
late binding: [2, 2, 2]
fixed with default arg: [0, 1, 2]
sorted by age desc, then name: [('ann', 31), ('cy', 31), ('bob', 25)]
max by age (first wins on tie): ('ann', 31)
```

Late binding — why `[2, 2, 2]`?

A closure (function using a variable from an enclosing scope) looks the variable up **when called**; all three lambdas share `i`, which is 2 by then. `lambda i=i: i` or `functools.partial(f, i)` captures the current value (§7.4).

Other functional helpers worth knowing

- `operator.itemgetter(1)`, `attrgetter("age")`, `methodcaller("strip")` — fast, self-describing replacements for small `key=` lambdas.
- `functools.partial(f, x)` — pre-fill arguments.
- `sorted` / `min` / `max` take `key=`; `sorted` is **stable**, enabling multi-pass or tuple-key sorts.
- `enumerate(it, start=1)` instead of `range(len(it))`.

Common mistakes

- Forgetting `map` / `filter` / `zip` are one-shot iterators in Python 3.
- `reduce` with no initial value on empty input.
- `zip` on lists assumed equal length — `strict=True`.

Real-world examples

`sorted(records, key=itemgetter("created_at"))`; `all(field in payload for field in REQUIRED)` for validation; `dict(zip(header, row))` for CSV rows; PySpark (Python API to Apache Spark, distributed data processing) and pandas (Chapter 18) expose `rdd.map`, `Series.map` from this vocabulary.

In the interview

> **30-second answer:** "`map` and `filter` are lazy in Python 3; I prefer comprehensions when I'd otherwise need a lambda, and built-ins like `sum`, `max`, `any`, `all` over `reduce`. `any` / `all` short-circuit, so I pass generator expressions. `zip` truncates silently, so for data that must align I use `strict=True`. And closures bind late — capture loop variables with a default argument or `partial`."

- *`all([])`? —* `True`.
- *Why `any(x > 10 for x in range(10**9))` is instant?* — Stops at 11.

TL;DR

Comprehensions over `map` / `filter` + lambda; built-ins over `reduce`; generator expressions in `any` / `all`; `zip(strict=True)` when lengths must match; closures bind late.

11.8 Chapter self-check

1. Two methods making an iterator? → `__iter__` (returning self) and `__next__`.
2. Why is `list(gen)` empty the second time? → One-shot iterator.
3. When does a generator body start? → First `next()`.
4. Get a sub-generator's `return` value? → `result = yield from sub()`.
5. Before `gen.send(value)`? → Prime with `next(gen)`.
6. Why sort before `groupby`? → It groups only consecutive equal keys.
7. `permutations("abc", 2)` vs `combinations("abc", 2)` counts? → 6 vs 3.
8. Memory of `sum(x*x for x in range(n))`? → Constant.
9. Where does the `if` go in a comprehension? → Filter after `for`; `a if c else b` before.
10. Make `zip` fail on mismatched lengths? → `strict=True`.

Chapter TL;DR

- **Protocol:** iterable (`__iter__`) → iterator (`__next__` + `__iter__` returning self) → `StopIteration`. `for` = `iter()` + `next()` loop. Iterators are single-pass.
- **Generators:** `yield` makes a lazy iterator with paused state; generator expressions `( … )` are the inline form. `yield from` delegates; `send` / `throw` / `close` make two-way generators (mostly replaced by `async`).

- **Laziness:** constant memory and early exit; materialise with `list()` only when you need reuse or indexing.
- **itertools:** `count` / `cycle` / `repeat`, `chain` / `islice` / `takewhile` / `dropwhile`, `groupby` (sort first!), `product` / `permutations` / `combinations`, `accumulate`, `pairwise`, `batched`, `zip_longest`, `tee`.
- **Comprehensions:** short, no side effects, filter-if after / choose-if-else before; generator form when consumed once.
- **Functional:** comprehensions over `map` / `filter` + lambda; `sum` / `max` / `any` / `all` over `reduce`; `zip(strict=True)`; closures bind late.